Everything You Need To Know About MAT309

— ***EYNTKA*** **Series** —

Nathanael Chwojko-Srawley

October 12, 2026

Contents

Contents **2**

1 Languages **6**

1.1 First Order Formal Languages . . . 7
1.2 Sentences . . . 14
1.3 Structure . . . 16
1.4 Truth in a Structure . . . 17
1.5 Substitutions and Substitutability . . . 24
1.6 Logical Implication . . . 27
1.7 *Propositional Language . . . 28
1.8 Summary . . . 32

2 Deductions **34**

2.1 Deductions . . . 35
2.2 The Logical Axioms . . . 37
2.2.1 Equality Axioms . . . 37
2.2.2 Quantifier Axioms . . . 37
2.3 Rules of Inference . . . 38
2.3.1 Quantifier Rules . . . 40
2.4 Soundness . . . 41
2.5 Propositional Logic: a Warm-Up . . . 43
2.5.1 Soundness and Completeness For Propositional Logic . . . 51

2.6 A second look at Deductions . . . 56
2.6.1 Soundness and Completeness . . . 65
2.7 Applications of Compactness . . . 75
2.8 Summary . . . 76

3 Computability **79**
3.1 Turing Machines . . . 80
3.2 Computable Functions and Relations . . . 84
3.2.1 Primitive Recursion . . . 90
3.2.2 Unbounded Minimalization . . . 101
3.3 Characterizing Computability . . . 105
3.3.1 Universal Turing Machine and Halting Problem . . . 112
3.3.2 Recursively enumerable sets . . . 116
3.4 Summary . . . 118

4 Incompleteness **121**
4.1 Deductions are Recursive . . . 121
4.2 Recursive Functions are Representable in $\mathcal{A}$ . . . 133
4.3 Incompleteness Theorems . . . 149
4.4 Summary . . . 159

A Algebrization **161**

Bibliography **162**

0.0.1 Reading the numbering A star marks *bonus material*: a chapter, section or subsection that belongs in the book but that nothing later depends on. Skipping it costs nothing downstream. The star sits on the number, as in §2.7⋆; in divisions written before this convention it sits at the front of the title instead, as in **Uniform Spaces*. The two mean the same thing.

Abstract

In these notes, we will cover the language of first-order and predicate logic, define a new mathematical object known as a *model* and use it to define notions of truth, satisfiability, implication, and so forth. Then, we shall take a moment to detour from looking at "truth" and instead look at "deducibility" or "provability". This will culminates in linking "truth" and "provable" via the soundness and completeness theorems. After having covered the notion of provability, we shall start looking into computability (which, if you think about it, is the natural way in which we think), and show that something being "computable" is equivalent to it being "recursive", thus giving us a model for

working with computable processes. Finally, we build up towards showing our two mains theorems: Gödels Incompleteness Theorem I and II. Essentially, if we have a system of axioms strong enough to be used to prove and work with the natural numbers, then there is no way of finding and proving all possible theorems, and there is no way of showing the system of axioms has a contradiction; these both are the foundation of the crisis in mathematics spearheaded by Gödel.

Some Philosophical Musings

Fundamentally, in any reality you might imagine (social, mental, physical), there is always an ontology and an epistemology, and no matter what we give symbols and sounds meaning through language to represent these, arguably even embody these. In math, the objects are usually called the *structures* or the *models*. We want to study as close as possible what is the "first principle" of objects. This naturally leads to many classical philosophical problems: what's the relation between epistemology and ontology, is there a first principle, what's the relation between logos and language, where do we derive meaning, is it possible to make analytic a-priori claims, and so forth.

Very often the same underlying object has many different interpretations depending on it's purpose, and it is suited for some purposes over other purposes. For example, let's take $\mathbb{R}^n$. Then

1. It is a vector-space: given an understanding of what a function is, we have that $+$ and $\mathbb{R} \cdot \mathbb{R}^n$ is well-defined. In the right number of dimensions, it is even an algebra (usually in $n = 1, 2, 4, 8$ we can impose an interesting multiplication structure)
2. It is a topological space: Given an understanding of subsets and closure, we have topology on it. There are many topologies that can even be put on it (from the obvious three, to many other possible one using weak topologies, order topologies, an so forth)
3. It is a metric space. There is a notion of distance. We can go further and say it's a normed and an inner product space, telling us that there is a notion of size and "orientation"
4. It is a complete space: we may freely use sequences without the worry of their limits not existing. Though it's not important, the space also complete through in terms of nets.
5. If is a smooth manifold: It is (trivially) locally Euclidean. The notion of differentiability is well-defined.
6. It is a flat space: The gaussian curvature is always 0, which also shows that $\mathbb{R}^n$ does not exhibit the geometric properties of hyperbolic space or positively curved space.
7. It is classical Euclidean space (by eliminating the notion of an origin)
8. it is *not* a discrete space: results requiring the lack of continuity might not work on it
9. it is *not* a complex manifold of n is odd.
10. it is an ordered space (in the classical ordering) if $n = 1$

Thus, it is important to understand what epistemological structure we are considering on the object. As in the examples above, that often involves literally upgrading the object to have that extra information (ex.a vector-space, a topological space), however this is in my mind because we are trying

to make all objects part of the universal epistemology of set theory, and so we would distinguish objects by adding structures to them. In a more "early mathematics" or pre-set theory of looking at it, we can think of $\mathbb{R}^n$ having all these properties (or not having for the false cases), and using the right epistemology to distinguish them.

Overall, we see how important it is to look at the epistemology. Importantly, we need a clear way of communicating and expressing this epistemology, i.e. a *language*. We usually will say *formal language*, to mean an invented set of symbols to which we prescribe some meaning. We will take some time in the first chapter to describe the formal language we will use in these notes

(symbolic logic is when you block up concepts and look at the relations between them; Aristotle did this and tried to get rid of some Ethos and Pathos from reasoning)

1

Languages

Throughout history, the study of reason (and epistemology more generally, so source of meaning, mode of reasoning, nature of truth etc.) took on many forms. We will say that what we are studying when studying reason and all that surrounds reason is that we are studying *logos*. Though all of the following were happening at the same time, we can roughly say that there is a chronology in what philosophers believed to be the focus for the study of logos is:

1. Ontological focus: This started with the works of Thales (if not in a yet-to-be-found work in South Asia) who believed that "natural phenomena have natural causes", moving away from mythos based arguments and onto logos based arguments. These philosophers of the western tradition (essentially up to Descartes) focused on finding the meaning, the truth, the nature of valid reasoning (i.e. the logos) in the "real" world (what I would call the physical world, or physical cosmology). The word *ontology* comes from *ontos* meaning "that which is", "that has happened", or "that is true", and *logos* meaning "account of", or "understanding of". Key here is that language was used to express *opinions* of the world.

2. epistemological focus: We can say that this period starting with Descartes and continuing on to Frege was the start of the questioning of the validity of knowledge-claims. We may ask questions like "how do we know" or "how are we sure this is knowledge". This leads to philosophical movements like *rationalism*, and more interestingly and sophisticatedly *idealism*. Some examples of the consequence of this way of thinking is to have *methods* (ex. the scientific method) be used as the authority of knowledge, or to believe that the source of knowledge. While this project was a good project, the problem was that it is actually quite difficult to find a model for which to analyze such a system of knowledge. It turned out that the emphasis had to be on language, that is language naturally inherits.

3. Linguistic Focus: Starting with Frege's PhD thesis in the 1870s, it has become evident that language is deeply tied to any metaphysic – one simply does not have a non-metaphysical language. Hence, much more care must be taken in the words we use and pin down: their exact

meaning, the exact way to put them together, the exact scope of them, the exact rule to on how to use them, and so forth. The professor of this course also points out how logos no longer has any grounding in an authority, be it divine or other. There is also now a correlation with the loss of authority in Europe in general at this time, with the *crisis of meaning* and *nihilism* movements spreading.

1.1 First Order Formal Languages

The language we shall create will be made to ask about the validity or "truth" of structures that we may put on sets. For example, for groups we will want that $a * (b * c) = (a * b) * c$ for all elements, or for an ordered field we will want one of $x > 0$ or $x < 0$ or $x = 0$, but not any two at once, or that there is a special constant symbol 1 such that $1 * c = c$. First order language is designed to talk about such statements. The key features of the first order language will be:

1. constant: symbols that represent some value
2. relation: something that relates two symbols (the verbs if you will)
3. functions: something that can combine symbols together (the nouns or compound nouns if you will)
4. variables: symbols that can be substituted with some elements of your "set"

We have yet to bring sets in our discussion: this will happen once we will find *models* on that will let us use our language.

Definition 1.1.1: First-Order Language

The set $\mathcal{L}$ is an infinite collection of distinct symbols, no one of which is properly contained in another, separated into the following categories:

1. Parenthesis: which are the symbols $(,)$
2. Connectives: which are the symbols $\neg$ and $\rightarrow$
3. Quantifier: which is the symbol $\forall$
4. Equality symbol: $=$[a]
5. **Variables**, one for each natural number n: $v_0, v_1, v_2, ..., v_n, ...$. The set of variable symbols will be denoted *Vars*
6. **Constant symbols**: Some set of zero or more symbols
7. **Function symbols**: For each positive integer n, some set of zero or more n-ary *function symbols*
8. **Relation symbols**: For each positive integer n, some set of zero or more n-ary *relation symbols*

where the word *function* and *relation* are currently simply formal add-ons to the word symbol. Symbols of the form 1-5 are called *logical symbols* while 6-8 are called *non-logical symbols*, and are dependent on the particular language.

[a] Some authors do not include this symbol as a mandatory symbol and leave it as an optional 2-ary function.

Note that we should treat this set as simply a collection of symbols: they do not yet have any *semantic* meaning, they are simply *syntax*. The arity of a function or relation is the number of variables it "takes in". So, $+$ has 2-arity, $(_)^{-1}$ has 1-arity, 0 as 0-arity, and so forth. In terms of relation, an n-ary relation can be thought of as a n-tuple. Note too the importance of "no one of which is properly contained in another". Thinking of symbols as something that is written down, let's say for example we had a constant symbol $\triangle$ and a constant symbol $\triangle\triangle$, i.e. the second symbol contains the first. Then when we later construct sentences, the sentence $\triangle\triangle$ has the vagueness of being interpretable as either two $\triangle$ or one $\triangle\triangle$.

We will often collect the constant, functions, and relations (the constant, nouns, and verbs if you will), like so:

$$\{c_1, c_2, ..., c_n, f_1^{a_1}, f_2^{a_2}, ..., f_m^{a_m}, R_1^{b_1}, R_2^{b_2}, ..., R_l^{b_l}\}$$

where the superscript of the functions and relations represent the arity of them. Note that any collection of constants, functions, or relations, can also be infinite, and we may still denote them like in the above equation, however it is less common. The above set can be thought of as a generating set for the language, as we will shortly see. Overall, to specify a language, we will need to specify the constant, function, and relation symbols that are used.

Example 1.1.2: Formal Languages

When defining a language, we will take the symbols that are always part of any first-order language as implicitly part of the set so as to not clutter.

1. The langue $\mathcal{L}$ with no extra non-logical symbols is a language. Sometimes, this language is called the language of equality since $=$ is the only relationship.

2. Let's say we want to work with groups and are trying to come up with the language to work with. We know a group is a set along with a binary operation $+$ and a special symbol 0 that interacts with $+$ in a consistent way for all groups. Thus, one language for groups can be

$$\mathcal{L}_G \qquad \text{is} \qquad \{0, +\}$$

   Another possible one would be to write it in terms of multiplication:

$$\mathcal{L}_G \qquad \text{is} \qquad \{1, \cdot\}$$

   It is also reasonable to think of inverting an element to be a 1-ary operation

$$\mathcal{L}_G \qquad \text{is} \qquad \{1, (_)^{-1}, \cdot\}$$

   Thus, we can have much flexibility when choosing a language.

3. As an example of a language with a constant, functions, and a relation: an ordered field can have the language of:

$$\mathcal{L}_F \qquad \text{is} \qquad \{0, 1, (_)^{-1}, +, \cdot, <\}$$

4. We might think that the language of set theory has many possible symbols (ex. $\subseteq, \in, \notin, \subsetneq$, etc.). Though this is a choice, we can also define the language of set theory as simply being:

$$\mathcal{L}_{\mathbf{Set}} \qquad \text{is} \qquad \{\in\}$$

   In other words, the fundamental idea in the language of set theory is that we have containment, which is a binary relation. The use of this simplification of language will be in many proofs being easier if we only have this one [extra] symbol (similar to how computer-architecture is simpler with a reduced instruction set computer, RISC).

5. The language of field $\mathcal{L}_F$ consists of constant symbols $0, 1$ and 2 binary functions symbols, usually denoted $+$ and $\cdot$

6. The language of linear order $\mathcal{L}_O$ consists of a binary relation symbol $<$

Now that for a collection of symbols of a language, we will want to make meaningful combinations of the symbols. For example, if our language is $\{0, +, <\}$ (remember to think of this set as the generating set) and we use it to describe the order theory of the naturals (i.e. giving in that meaning), then we can reasonably say that:

$$(v_1 + 0) < v_1$$

is a sentence with meaning[1], while

$$= \exists((+ + + + 0((\forall = v_{42}$$

is a meaningless combination of symbols. Thus, we will work towards finding subsets of $P(\mathcal{L})$ that have "meaning". Like with language, we will split the most common types of subsets with meanings in categories (similarly to how in english we have nouns, verbs, adjectives, and so forth). For math,

[1] and it has the truth value false, see section 1.4 for how a formula gains meaning

the two most common categories that have meanings are *terms* and *formulas*. We define each and give examples:

Definition 1.1.3: Term

Let $\mathcal{L}$ be a language. Then a *term* of $\mathcal{L}$ is a nonempty finite string t of symbols from $\mathcal{L}$ such that that either:

1. t is a variable
2. t is a constant symbol
3. $t :\equiv ft_1t_2...t_n$ where f is any n-ary function symbol of $\mathcal{L}$ and each of the t_i is a term of $\mathcal{L}$.

The symbol $:\equiv$ is a meta-linguistic symbol that means that the symbol on the left hand side represents the same thing as on the right hand side. It is *not* part of the language $\mathcal{L}$. Note too that the 3rd point in the above definition is recursive, that is we can recursively define terms by inputting them into functions. Given countably many variable, constant symbols, and function symbols, there are only countably many terms in a language.

Example 1.1.4: Terms In Language

1. Let $\mathcal{L} = \{\overline{0}, \overline{1}, ..., \overline{n}, ..., +, \cdot\}$ where we have one constant symbol for each natural number, and two binary function symbols. In the following set of examples, anything with a bar over it is a term, and we are using the polish notation (which will make it easier to consistently identify a unique reading of a sentence, especially once the symbols are not familiar to us):
   (a) $\overline{714}$
   (b) $+\overline{32}$
   (c) $\cdot + \overline{123}$

   Note that $\overline{123}$ would not be a term of $\mathcal{L}$, but a sequence of three terms.

Note that any term with no function symbols will be of length 1.

Overall, terms in a language $\mathcal{L}$ play the role of nouns. We will now show how to combine nouns with the "statements":

Definition 1.1.5: Formulas

Let $\mathcal{L}$ be a first-order language. Then a *formula* of $\mathcal{L}$ is a nonempty finite string φ of symbols from $\mathcal{L}$ such that either *atomic formulas*:

1. $\varphi :\equiv= t_1 t_2$ where t_1 and t_2 are terms of $\mathcal{L}$
2. $\varphi :\equiv Rt_1 t_2 ... t_n$ where R is an n-ary relation symbol of $\mathcal{L}$ and $t_1, t_2, ..., t_n$ are all terms of $\mathcal{L}$

and *recursive formulas*, or just simply *formulas*:

1. $\varphi :\equiv (\neg\alpha)$ where α is a formula of $\mathcal{L}$
2. $\varphi :\equiv (\alpha \to \beta)$ where α and β are formulas of $\mathcal{L}$
3. $\varphi :\equiv (\forall v)(\alpha)$ where v is a variable and α is a formula of $\mathcal{L}$

The connectives $\vee$, $\wedge$, $\leftrightarrow$ and the quantifier $\exists$ are not symbols of $\mathcal{L}$. They abbreviate formulas of $\mathcal{L}$: for formulas α, β and a variable v,

$$\begin{aligned}(\alpha \vee \beta) &:\equiv ((\neg\alpha) \to \beta)\\ (\alpha \wedge \beta) &:\equiv (\neg(\alpha \to (\neg\beta)))\\ (\alpha \leftrightarrow \beta) &:\equiv ((\alpha \to \beta) \wedge (\beta \to \alpha))\\ (\exists v)(\alpha) &:\equiv (\neg(\forall v)(\neg\alpha))\end{aligned}$$

Be sure to realise that in the formula $(\forall v)(\alpha)$, the symbol v is a *variable*, not a term or another formula. If we have if for a formula φ, we have the string $(\forall v)(\alpha)$ as a substring, then we will say that the *scope* of the quantifier $\forall$ is α. Any symbol within α will then be said to lie within the scope of the quantifier $\forall$. Note that a formula φ may have many quantifiers, some lying within each other's scope. Note that terms are *not* formulas. Going by the linguistic analogy, terms are the nouns and the formulas are the statements. Nouns' do not have truth values, but formulas will have truth values once we get to it.

Example 1.1.6: Formulas In A Language

1. The language of set theory has one binary relation symbol $\in$, written between its arguments. Read with $\in$ as membership, the formula $\exists x \forall y \neg(y \in x)$ says that some set has no element, and the following formulas say:
   (a) $\forall x \exists y \forall z \big(z \in y \leftrightarrow \exists w(w \in x \wedge z \in w)\big)$: every set x has a union, a set whose elements are the elements of the elements of x
   (b) $\forall x \exists y \forall z \big(z \in y \leftrightarrow \forall w(w \in x \to z \in w)\big)$: every set x has an intersection, a set whose elements are the sets that belong to every element of x
   (c) $\forall x \neg(x \in x)$: no set is an element of itself
   (d) $\forall x \forall y \exists z \forall w \big(w \in z \leftrightarrow (w \in x \wedge w \in y)\big)$: any two sets x and y have an intersection
2. The language of groups has a binary function symbol $*$, written between its arguments, and a constant symbol e. Read with $*$ as the group operation and e as the identity, the following formulas are the axioms of a group:

(a) $\forall a \forall b \forall c\ a * (b * c) = (a * b) * c$

(b) $\forall a\,(e * a = a \wedge a * e = a)$

(c) $\forall a \exists b\,(a * b = e \wedge b * a = e)$

No formula states that the operation is closed: a structure for this language interprets $*$ by a function from pairs of its elements to its elements (definition 1.3.1).

These are the basic terms we will use. I will stress a final time to not yet interpret any symbol: $\forall$ does not yet mean "for all", $\rightarrow$ does not yet mean "if ... then". Assigning meaning to symbols will come in due time. The abbreviations of definition 1.1.5 are shorthand for strings built from $\neg$, $\rightarrow$ and $\forall$. Statements about all formulas cover them, and a definition or a proof by induction on formulas has exactly three recursive cases, for $\neg$, $\rightarrow$ and $\forall$. Proposition 1.4.6 and proposition 1.4.13 show that each abbreviation receives its expected meaning. To make life easier there are a couple of conventions we will adopt:

1. we will usually use infix notation instead of polish notation
2. we will occasionally change the size of parenthesis, or the symbol for the parenthesis, to make strings more legible
3. we will sometimes add or drop parenthesis to make strings more legible
4. If we write $\neg\alpha \rightarrow \beta$, we mean $((\neg\alpha) \rightarrow \beta)$, that is $\neg$ takes precedence
5. When parentheses are missing, the order of precedence is $\forall$, $\exists$, $\neg$, $\wedge$, $\vee$, $\rightarrow$, $\leftrightarrow$, so $\forall x \alpha \rightarrow \beta$ means $((\forall x)(\alpha) \rightarrow \beta)$
6. If we have $\alpha \rightarrow \beta \rightarrow \gamma$, we will group towards the right, meaning that could be read as $(\alpha \rightarrow (\beta \rightarrow \gamma))$, and likewise for $\vee$, $\wedge$ and $\leftrightarrow$

For future purposes we define the following concept

Definition 1.1.7: Subformulas

Let $\mathcal{L}$ be a language and φ a formula. Then the set of *subformulas* of φ, denoted $S(\varphi)$ is defined as follows:

1. If φ is an atomic formula, then $S(\varphi) = \{\varphi\}$
2. if φ is $(\neg\alpha)$, then $S(\varphi) = S(\alpha) \cup \{(\neg\alpha)\}$
3. If φ is $(\alpha \rightarrow \beta)$ then $S(\varphi) = S(\alpha) \cup S(\beta) \cup \{(\alpha \rightarrow \beta)\}$
4. If φ is $(\forall v)(\alpha)$ then $S(\varphi) = S(\alpha) \cup \{(\forall v)(\alpha)\}$

It should be mentioned that any formula can only have one way of being read: there cannot be confusion with whether a symbol is a symbol or not:

Theorem 1.1.8: Unique Readability

Let $\mathcal{L}$ be a language and let α be a formula of $\mathcal{L}$. Then α can only satisfy one of the conditions of the definition of a formula, and it satisfies that condition in only one way: the terms, formulas and variable that the condition names are uniquely determined by α. A string that satisfies none of the conditions is not a formula.

***Proof*:**
Terms and formulas are finite strings of symbols. A string σ is an *initial segment* of a string τ if τ is σ followed by a possibly empty string. Of two initial segments of the same string, the shorter is an initial segment of the longer.

Step 1: terms. Give a variable or a constant symbol the weight 1, and an n-ary function symbol the weight $1-n$. The weight $w(\sigma)$ of a string σ is the sum of the weights of its symbols. We show by induction on terms (definition 1.1.3) that every term t has $w(t)=1$, and that every nonempty initial segment of t other than t has weight at most 0. If t is a variable or a constant symbol, then $w(t)=1$ and t has no nonempty initial segment other than itself. Let $t :\equiv ft_1\cdots t_n$ with f an n-ary function symbol, $n\geq 1$, and terms $t_1,\ldots,t_n$ for which the claim holds. Then $w(t)=(1-n)+n=1$. A nonempty initial segment $\sigma\neq t$ of t is either the one-symbol string f, of weight $1-n\leq 0$, or $ft_1\cdots t_{i-1}\rho$ with $1\leq i\leq n$ and ρ a nonempty initial segment of t_i, where $\rho\neq t_n$ if $i=n$. In the second case $w(\sigma)=i-n+w(\rho)$. If ρ is t_i, then $i<n$ and $w(\sigma)=i-n+1\leq 0$. Otherwise $w(\rho)\leq 0$ by the induction hypothesis, and $w(\sigma)\leq i-n\leq 0$.

Since a term is nonempty and has weight 1, no initial segment of a term t other than t is a term. It follows by induction on n that if the strings $t_1\cdots t_n\sigma$ and $u_1\cdots u_n\tau$ coincide, where the t_i and u_i are terms, then t_i is u_i for every i and σ is τ. For $n\geq 1$, t_1 and u_1 are initial segments of the same string, so one is an initial segment of the other, and therefore they coincide. Deleting them leaves two coinciding strings with $n-1$ leading terms each.

Step 2: no initial segment of a formula other than the formula itself is a formula. By definition 1.1.5, a formula begins with $=$, with a relation symbol, or with (, and these are distinct symbols. The second symbol of $(\neg\beta)$ is $\neg$, that of $(\forall v)(\beta)$ is $\forall$, and that of $(\beta\to\gamma)$ is the first symbol of β, which is $=$, a relation symbol or (. These are again distinct symbols, so the first two symbols of a formula decide whether it is an equation, a relation formula, a negation, a quantified formula, or an implication $(\beta\to\gamma)$.

Let φ and ψ be formulas with ψ an initial segment of φ. We show ψ is φ by strong induction on the length of φ. The formula ψ shares the first symbol of φ, and if that symbol is (then ψ has length at least 2 and shares the second symbol as well, so φ and ψ are of the same kind.

- If $\varphi :\equiv =t_1t_2$, then $\psi :\equiv =u_1u_2$ for terms u_1,u_2, and the string t_1t_2 is $u_1u_2\sigma$ for some string σ. By Step 1, u_1 is t_1, u_2 is t_2 and σ is empty, so ψ is φ. If $\varphi :\equiv Rt_1\cdots t_n$ with R an n-ary relation symbol, then $\psi :\equiv Ru_1\cdots u_n$ and the string $t_1\cdots t_n$ is $u_1\cdots u_n\sigma$, so Step 1 again gives $u_i=t_i$ for every i and σ empty.
- If $\varphi :\equiv(\neg\alpha)$, then $\psi :\equiv(\neg\beta)$ for a formula β, and $\beta)$ is an initial segment of $\alpha)$. An initial

segment is no longer than the string it begins, so β is no longer than α, and therefore β is an initial segment of α. As α is shorter than φ, the induction hypothesis gives that β is α, and ψ is φ.

- If $\varphi :\equiv (\alpha \to \beta)$, then $\psi :\equiv (\gamma \to \delta)$ for formulas γ, δ. After the leading (, both α and γ are initial segments of the string $\alpha \to \beta)$, so one of them is an initial segment of the other, and both are shorter than φ. The induction hypothesis applied to the longer of the two gives that γ is α. Deleting the common string ($\alpha \to$ then leaves $\delta)$ as an initial segment of $\beta)$, so δ is β as in the previous case.
- If $\varphi :\equiv (\forall v)(\alpha)$, then $\psi :\equiv (\forall u)(\beta)$ for a variable u and a formula β. The third symbols give $u = v$, so after the common string $(\forall v)($ the string $\beta)$ is an initial segment of $\alpha)$, and β is α as before.

Step 3: unique readability. By definition 1.1.5 a formula is a string satisfying one of its conditions, so a string satisfying none of them is not a formula, and the formula α satisfies at least one. By the first paragraph of Step 2, the first two symbols of α decide which kind of condition it satisfies. Within that kind the named parts are determined by α. If α is both $= t_1 t_2$ and $= u_1 u_2$, or both $Rt_1 \cdots t_n$ and $Ru_1 \cdots u_n$, then Step 1 gives $t_i = u_i$ for every i. If $\alpha :\equiv (\neg\beta)$, then β is α with its first two symbols and its last symbol deleted. If α is both $(\beta \to \gamma)$ and $(\beta' \to \gamma')$, then β and β' are initial segments of the same string, so one is an initial segment of the other, and Step 2 gives that β is β'. Then γ is γ', both being the rest of α after $(\beta \to$ and before its last symbol. If $\alpha :\equiv (\forall v)(\beta)$, then v is the third symbol of α and β is α with its first five symbols and its last symbol deleted. Hence α satisfies exactly one condition of definition 1.1.5, in exactly one way, as we sought to show.

For future reference, it is important to think of extensions of language:

Definition 1.1.9: Language Extension

Let $\mathcal{L}$ and $\mathcal{L}'$ be first order languages. Then $\mathcal{L}'$ is an *extension* of $\mathcal{L}$ if $\mathcal{L} \subseteq \mathcal{L}'$, that is if every non-logical symbol of $\mathcal{L}$ is a non-logical symbol of the same kind in $\mathcal{L}'$.

As an example, when we defined group with $\in$, that language was an extension of $\mathcal{L}_{\mathbf{Set}}$.

1.2 Sentences

Given a language $\mathcal{L}$, we can create formulas. The formulas we are most interested in are those which require no choice, or assignment of value. For example, if I write $\forall x\ x = y$, once we define how to give semantic meaning to this formula, we will see that the "value" of the formula depends on the choice of y. However, if we write $\forall x \forall y\ x = y$ or $\forall x \exists y\ x = y$ then y is no bound to the given logic. A formula with no "free" variables will be called a sentence.

Before continuing to define these concepts more rigorously, we will make a special definition for the language of number theory as it is extremely valuable for future discussions:

Definition 1.2.1: Number Theory Language

Let $\mathcal{L}_{NT}$ be the language generated by $\{0, S, +, \cdot, E, <\}$, where 0 is a constant, S is a unary function (the "successor function"), $+$, $\cdot$, and E are binary function (E for exponential) and $<$ is a binary relation. Then we usually call $\mathcal{L}_{NT}$ the language of *number theory*

This language will become important for us later, but for now let's just use it to demonstrate the idea of a sentence. Take the following two formulas[2]:

$$\neg(\forall x)[(y < x) \vee (y = x)] \tag{1.1}$$
$$(\forall x)(\forall y)[(x < y) \vee (x = y) \vee (y < x)] \tag{1.2}$$

The first sentence reads "not for all x is y less than or equal to x", while the second sentence is the trichotomy law of $<$. The second formula is true about the collection of natural numbers as we already understand them. The first formula has the problem of y not being quantities: are we picking particular value of y, any value of y, all possible values of y? In other words, y is a *free variable*. We may choose what y to put into that formula. Think of integration:

$$F(x) = \int_0^x f(t)dt$$

In this case, the value of x is the free variable because we are free to choose what it is. This makes a formula dependent upon a choice, and hence not holding an inherent truth value. We will give variables that bring in an element of choice into a formula a name:

Definition 1.2.2: free variable

Let v be a variable and φ a formula. We will say that v is *free* in φ if:

1. φ is an atomic formula and v occurs in (as a symbol in) φ
2. $\varphi :\equiv (\neg\alpha)$ and v is free in α
3. $\varphi :\equiv (\alpha \to \beta)$ and v is free in at least one of α or β
4. $\varphi :\equiv (\forall u)(\alpha)$ and v is not u and v is free in α

Example 1.2.3: Free Variables

1. In the formula
$$\forall v_2 \neg(\forall v_3)(v_1 = S(v_2) \vee v_3 = v_2)$$
the variable v_1 is free whereas the variables v_2 and v_3 are not free
2. A more subtle example is the following
$$(\forall v_1 \forall v_2(v_1 + v_2 = 0)) \vee v_1 = S(0)$$
Notice that v_1 here is free while v_2 is not free; in other words, be weary of parenthesis and the scope of quantifiers in parenthesis.

[2]Notice that we are already abusing notation here

3. An example of a variable being both free and not free, depending on the scope, is v_3 in the following formula:
$$\forall v_0(= v_1 f_3^1 v_3 \to (\neg \forall v_3 P_9^1 v_3))$$

Definition 1.2.4: Sentence

Let $\mathcal{L}$ a language. Then a *sentence* in $\mathcal{L}$ is a formula of $\mathcal{L}$ that contains no free variables.

Example 1.2.5: Sentences
The following uses $\mathcal{L}_{NT}$ as a language

1. the formula $1 + 1 = 2$ is a sentence
2. the formula $(\forall x)(x + 1 = x)$ is a sentence

As a rule of thumb, if it seems like you can determine if a statement is true or false without any choice then it's a sentence. Intuitively, the first statement is true while the second statement is false. We will explore exactly what is the nature of this in section 1.4.

1.3 Structure

It was stressed many times that the symbols in a language do not come with any meaning attached to them. The goal of this section is to show how we attach a particular meaning to the symbols

Definition 1.3.1: Structure (Model)

Let $\mathcal{L}$ be a language. Then an $\mathcal{L}$-structure $\mathfrak{A}$ is a nonempty set A, called the *universe of* $\mathfrak{A}$ together with:

1. For each constant symbol c of $\mathcal{L}$, an element $c^{\mathfrak{A}}$ in A
2. For each n-ary function symbol f of $\mathcal{L}$, a function $f^{\mathfrak{A}} : A^N \to A$
3. For each n-ary relation symbol R of $\mathcal{L}$, an n-ary relation $R^{\mathfrak{A}}$ on A (i.e. a subset of A^n)

Often, we write $|\mathfrak{A}| := A$.

Note that all functions will be total functions (the entire domain maps to something in the co-domain) unless otherwise specified. Note too that $c^{\mathfrak{A}}$ need not be the same symbol as c: for example, c might be \$ while $c^{\mathfrak{A}}$ is "Friends". Often, a structure will be written as an ordered pair, for example:
$$\mathfrak{A} = (A, c_1^{\mathfrak{A}}, c_2^{\mathfrak{A}}, f_1^{\mathfrak{A}}, R_1^{\mathfrak{A}}, R_2^{\mathfrak{A}})$$

Naturally, we will immediately start simplifying notation as long as it's not ambiguous. For example, the standard structure on $\mathcal{L}_{NT}$ will be denoted
$$\mathfrak{N} = (\mathbb{N}, 0, S, +, \cdot, E, <)$$

where we have dropped the superscript and take $\mathbb{N} := \{0, 1, 2, ...\}$. In set theory, 0 is usually considered a natural number. Naturally, we can construct different $\mathcal{L}_{NT}$-structures, for example we can just as well construct a (mod 2) structure with the language $\mathcal{L}_{NT}$, and label this $\mathcal{L}_{NT}$-structure $\mathfrak{T}$.

Example 1.3.2: Structure

1. $\mathfrak{Q} = (\mathbb{Q}, <)$ is a structure with the language of linear order on $\mathbb{Q}$. Other structures with the language $<$ maybe be $(\mathbb{R}, <)$, $((\{0\}, \emptyset), <)$, $((\mathbb{N}, \mathbb{N}), <)$ are other examples, given appropriate definition of $<$.

2. Let $\mathcal{L}_F$ be the language of fields we defined earlier. Then we may certainly make a structure for $\mathcal{L}_F$ that will be a field, however there is no restriction to make it follows the axioms of a field: we may also map the two field operations to the same function and have a group structure, or really almost any other structures.

3. An important structure that will be useful to us later on is the *Henkin structure* of a language. Let $\mathcal{L}$ be $\{0, f, g, R\}$, with constant symbol 0, unary function symbol f, binary function symbol g, and 3-ary relation symbol R. We now define a $\mathcal{L}$ structure $\mathfrak{B}$: the universe, B, is the set of all variable-free $\mathcal{L}$-terms; the universe contains all terms that do not have a variable. The constant $0^{\mathfrak{B}}$ is the term 0. The functions $f^{\mathfrak{B}}$ and $g^{\mathfrak{B}}$ are defined by concatenation like so:

$$f^{\mathfrak{B}}(t) = ft \qquad g^{\mathfrak{B}}(t, s) = gts$$

Note that the function symbols f, g are indeed variable free, and hence this is a valid function. Finally, we may let the relation $R^{\mathfrak{B}}$ be anything we want. As a random example:

$$R^{\mathfrak{B}} = \left\{(r, s, t \in B^3) \;\middle|\; \text{the number of function symbols in } r \text{ is even}\right\}$$

The model constructed in theorem 2.6.28 on the way to the completeness theorem is a quotient of a structure of this kind: its universe consists of classes of closed terms.

1.4 Truth in a Structure

We know get to assigning truth value's to sentences. We will do so by having an assignment function. There are many assignment functions we will use at varying levels: we will start by having a *variable* assignment function which gives to each variable in $Vars$ an element of A. We then build on this by defining a *term assignment function*, which uses the variable assignment function and then also allows us to give to each term of $\mathcal{L}$ an element of A. Finally, we will link formulas to an assignment function

Definition 1.4.1: Variable Assignment Function

Let $\mathcal{L}$ be a language and $\mathfrak{A}$ be a $\mathcal{L}$-structure. Then a *variable assignment function* into $\mathfrak{A}$ is a function s that assigned to each variable an element of the universe A. So a variable assignment function into $\mathfrak{A}$ is any function with domain $Vars$ and codomain A, $s : Vars \to A$.

Note variable assignment function need not be injective or surjective.

Example 1.4.2: Variable Assignment Function

Take $\mathcal{L}_{NT}$ with the standard structure $\mathfrak{N}$. Then the function s mapping $s(v_i) = i$ is a variable assignment function. Another example would be s' where

$$s'(v_i) = \text{the smallest prime number that does not divide } i+1$$

Definition 1.4.3: Modification Of The Assignment Function

Let s be a variable assignment function into $\mathfrak{A}$, x is a variable, and $a \in A$. Then $s[x|a]$ is the variable assignment function into $\mathfrak{A}$ defined as:

$$s[x|a](v) = \begin{cases} s(v) & \text{if } v \text{ is a variable other than } x \\ a & \text{if } v \text{ is the variable } x \end{cases}$$

We call the function $s[x|a]$ an *x-modification of the assignment function s.*

In other words, an x-modification of an assignment function overrides what $s(x)$ is; it force it to map to a. Next, we will extend the variable assignment function s to a term assignment function $\overline{s}$. This function will assign an element of the universe to each term of the language $\mathcal{L}$, and is "auto-generated" based off of the variable-assignment function and the functions of a given $\mathcal{L}$-structure:

Definition 1.4.4: Term Assignment Function

Let $\mathcal{L}$ be a language, $\mathfrak{A}$ an $\mathcal{L}$-structure, and s a variable assignment function into $\mathfrak{A}$. Then the function $\overline{s}$, called the *term assignment function generated by s*, is the function with domain consisting of the set of $\mathcal{L}$-terms and codomain A defined recursively as follows:

1. if t is a variables, $\overline{s}(t) = s(t)$
2. If $t :\equiv c$ is a constant symbol, then $\overline{s}(t) = c^{\mathfrak{A}}$
3. if $t :\equiv ft_1t_2...t_n$, then $\overline{s}(t) = f^{\mathfrak{A}}(\overline{s}(t_1), \overline{s}(t_2), ..., \overline{s}(t_n))$

We will mostly be interested in truth of sentences, but for future purposes we will first describe truth (or "satisfaction") for arbitrary formulas, relative to an assignment function

Definition 1.4.5: Structure Satisfying Assignment Formula ($\mathfrak{M} \models \varphi[s]$)

Let $\mathfrak{A}$ be an $\mathcal{L}$-structure, φ an $\mathcal{L}$-formula, and $s : Vars \to A$ an assignment function. Then we will say that $\mathfrak{A}$ *satisfies* φ *with assignment* s, and write $\mathfrak{A} \models \varphi[s]$ in the following circumstances:

1. Atomic formulas:
   (a) if $\varphi :\equiv= t_1 t_2$ and $\overline{s}(t_i)$ is the same element of the universe A as $\overline{s}(t_2)$ (i.e. $\overline{s}(t_1) = a = \overline{s}(t_2)$)
   (b) If $\varphi :\equiv Rt_1 t_2 ... t_n$ and $(\overline{s}(t_1), \overline{s}(t_2), ..., \overline{s}(t_n)) \in R^{\mathfrak{A}}$
2. recursive formulas:
   (a) $\varphi :\equiv (\neg\alpha)$ and $\mathfrak{A} \not\models \alpha[s]$ (where $\not\models$ means "does not satisfy")
   (b) if $\varphi :\equiv (\alpha \to \beta)$ and $\mathfrak{A} \not\models \alpha[s]$ or $\mathfrak{A} \models \beta[s]$, or both
   (c) If $\varphi :\equiv (\forall x)(\alpha)$ and, for each a of A, $\mathfrak{A} \models \alpha[s(x|a)]$

If Γ is a set of $\mathcal{L}$-formulas, we say that $\mathfrak{A}$ satisfies Γ with assignment s if for each $\gamma \in \Gamma$, $\mathfrak{A} \models \gamma[s]$, and write $\mathfrak{A} \models \Gamma[s]$.

Note that the symbol $\models$ is *not* part of the language $\mathcal{L}$. Instead, $\models$ is a metalinguistic symbol that we use to talk about formulas in the language and structures of the language. Notice too that we finally have our usual interpretation of $\neg$ as negation, $\forall$ as for all, and $\to$ as "if ... then": $\mathfrak{A} \models (\alpha \to \beta)[s]$ holds exactly when $\mathfrak{A} \models \beta[s]$ whenever $\mathfrak{A} \models \alpha[s]$.

Proposition 1.4.6: Truth Conditions Of The Abbreviations

Let $\mathfrak{A}$ be an $\mathcal{L}$-structure, s an assignment into $\mathfrak{A}$, and α, β formulas of $\mathcal{L}$. Then

1. $\mathfrak{A} \models (\alpha \vee \beta)[s]$ if and only if $\mathfrak{A} \models \alpha[s]$ or $\mathfrak{A} \models \beta[s]$
2. $\mathfrak{A} \models (\alpha \wedge \beta)[s]$ if and only if $\mathfrak{A} \models \alpha[s]$ and $\mathfrak{A} \models \beta[s]$
3. $\mathfrak{A} \models (\alpha \leftrightarrow \beta)[s]$ if and only if $\mathfrak{A} \models \alpha[s]$ exactly when $\mathfrak{A} \models \beta[s]$

***Proof*:**

1. The formula $\alpha \vee \beta$ is $((\neg\alpha) \to \beta)$. By the implication clause of definition 1.4.5, $\mathfrak{A} \models ((\neg\alpha) \to \beta)[s]$ holds exactly when $\mathfrak{A} \not\models (\neg\alpha)[s]$ or $\mathfrak{A} \models \beta[s]$. By the negation clause, $\mathfrak{A} \not\models (\neg\alpha)[s]$ holds exactly when $\mathfrak{A} \models \alpha[s]$.
2. The formula $\alpha \wedge \beta$ is $(\neg(\alpha \to (\neg\beta)))$. By the negation clause, $\mathfrak{A} \models (\alpha \wedge \beta)[s]$ holds exactly when $\mathfrak{A} \not\models (\alpha \to (\neg\beta))[s]$. By the implication clause, this holds exactly when $\mathfrak{A} \models \alpha[s]$ and $\mathfrak{A} \not\models (\neg\beta)[s]$, and by the negation clause $\mathfrak{A} \not\models (\neg\beta)[s]$ holds exactly when $\mathfrak{A} \models \beta[s]$.
3. The formula $\alpha \leftrightarrow \beta$ is $((\alpha \to \beta) \wedge (\beta \to \alpha))$. By item (2), $\mathfrak{A} \models (\alpha \leftrightarrow \beta)[s]$ holds exactly when $\mathfrak{A} \models (\alpha \to \beta)[s]$ and $\mathfrak{A} \models (\beta \to \alpha)[s]$. By the implication clause, the first says that $\mathfrak{A} \models \beta[s]$ whenever $\mathfrak{A} \models \alpha[s]$, and the second that $\mathfrak{A} \models \alpha[s]$ whenever $\mathfrak{A} \models \beta[s]$. Together they say that $\mathfrak{A} \models \alpha[s]$ holds exactly when $\mathfrak{A} \models \beta[s]$ does, completing the proof.

Example 1.4.7: Truth Assignment

1. If $\mathcal{L} = \{\}$ is the empty language, $\mathfrak{A} = (\{1, 2\})$ a $\mathcal{L}$-structure, then take $s(v_1) = 1$ and $s(v_2) = 2$. Let $\varphi :\equiv v_1 = v_2$. Then $\mathfrak{A} \not\models \varphi[s]$, that is, $\mathfrak{A}$ does not satisfy φ with s. On the other hand, if $s(v_2) = 1$, then $\mathfrak{A} \models \varphi[s]$. It should be easy to come up with other assignment functions s and or other $\mathcal{L}$-structures that will give the formula $v_1 = v_2$ (or in formal notation $= v_1 v_2$) a different true value.

We have determined whether a general formula satisfies a $\mathcal{L}$-structure given an assignment function, but what about the special case of sentences? Since they do not have free variables, they should in a sense be independent of any choice function. That is indeed the case. For now, we will take the choice of assignment function for sentences too, and show that the choice of assignment function is inconsequential. From there, we will conclude that σ is a true sentence in $\mathfrak{A}$ if and only if $\mathfrak{A} \models \sigma[s]$ for all possible variable assignment function s.

Lemma 1.4.8: Equal On Variables Then Equal On Terms

Let s_1, s_2 be variable-assignment functions into a structure $\mathfrak{A}$, and let t be a term such that

$$s_1(v) = s_2(v) \qquad \text{for every variable } v \text{ occurring in } t$$

Then $\overline{s_1}(t) = \overline{s_2}(t)$.

***Proof*:**
We use induction on terms (definition 1.1.3) and the clauses of definition 1.4.4. If t is a variable, it occurs in t, so $\overline{s_1}(t) = s_1(t) = s_2(t) = \overline{s_2}(t)$. If t is a constant symbol c, then $\overline{s_1}(t) = c^{\mathfrak{A}} = \overline{s_2}(t)$. If $t :\equiv f t_1 t_2 \cdots t_n$, then every variable occurring in some t_i occurs in t, so the induction hypothesis gives $\overline{s_1}(t_i) = \overline{s_2}(t_i)$ for all $1 \leq i \leq n$, and therefore

$$\overline{s_1}(t) = f^{\mathfrak{A}}(\overline{s_1}(t_1), \ldots, \overline{s_1}(t_n)) = f^{\mathfrak{A}}(\overline{s_2}(t_1), \ldots, \overline{s_2}(t_n)) = \overline{s_2}(t)$$

as we sought to show.

Proposition 1.4.9: Agree On Variables Then Models The Same

Let s_1, s_2 be variables assignment functions into a structure $\mathfrak{A}$ such that $s_1(v) = s_2(v)$ for every *free variable* v in the formula φ. Then

$$\mathfrak{A} \models \varphi[s_1] \qquad \text{iff} \qquad \mathfrak{A} \models \varphi[s_2]$$

***Proof*:**
We again use structural induction. If $\varphi :\equiv t_1 = t_2$ then the free variables of φ are exactly the variables that occur in φ. Thus, by lemma 1.4.8 we have $\overline{s_1}(t_1) = \overline{s_2}(t_1)$ and $\overline{s_1}(t_2) = \overline{s_2}(t_2)$ meaning they are the same element of the universe A, and so $\mathfrak{A} \models (t_1 = t_2)[s_1]$ if and only if $\mathfrak{A} \models (t_1 = t_2)[s_2]$. If $\varphi :\equiv R t_1 t_2 \cdots t_n$, the free variables of φ are again the variables occurring in it, so lemma 1.4.8 gives $\overline{s_1}(t_i) = \overline{s_2}(t_i)$ for every i, and the tuple $(\overline{s_1}(t_1), \ldots, \overline{s_1}(t_n))$ lies in $R^{\mathfrak{A}}$ exactly when $(\overline{s_2}(t_1), \ldots, \overline{s_2}(t_n))$ does.

For $\varphi :\equiv \neg\alpha$, notice that $\neg\alpha$ and α have exactly the same free-variables, so s_1 and s_2 agree on the free-variables of α. Thus, by the inductive hypothesis $\mathfrak{A} \models \alpha[s_1]$ if and only if $\mathfrak{A} \models \alpha[s_2]$, and by the negation clause $\mathfrak{A} \models \varphi[s_1]$ if and only if $\mathfrak{A} \models \varphi[s_2]$. For $\varphi :\equiv (\alpha \to \beta)$, a variable free in α or in β is free in φ by definition 1.2.2, so the induction hypothesis applies to α and to β, and the implication clause gives $\mathfrak{A} \models \varphi[s_1]$ if and only if $\mathfrak{A} \models \varphi[s_2]$.

Finally, for $\varphi :\equiv (\forall x)(\alpha)$, the only variable that might be free in α that is not free in $(\forall x)(\alpha)$ is x. Thus, for each $a \in A$, the assignments $s_1[x|a]$ and $s_2[x|a]$ agree on all free variables of α, and so by the induction hypothesis $\mathfrak{A} \models \alpha[s_1[x|a]]$ if and only if $\mathfrak{A} \models \alpha[s_2[x|a]]$. As this holds for every $a \in A$, the quantifier clause of definition 1.4.5 gives

$$\mathfrak{A} \models \varphi[s_1] \qquad \text{if and only if} \qquad \mathfrak{A} \models \varphi[s_2]$$

Thus, by the principle of induction, this holds true for all of φ, completing the proof.

The following corollary is important as it shows that sentences are "independent" of variable assignment functions

Corollary 1.4.10: Satisfiability For Sentences

Let σ be a sentence in the language $\mathcal{L}$ and $\mathfrak{A}$ an $\mathcal{L}$-structure. Then $\mathfrak{A} \models \sigma[s]$ for all assignment functions s, or $\mathfrak{A} \models \sigma[s]$ for no assignment functions s

***Proof*:**
By definition, σ has *no* free variables. And so if s_1 and s_2 are two assignment functions, they (vacuously) agree on free variables. Thus, $\mathfrak{A} \models \sigma[s_1]$ if and only if $\mathfrak{A} \models \sigma[s_2]$. Since this is true in general, we see that either $\mathfrak{A} \models \sigma[s]$ for all assignments or no assignments, as we sought to show.

A consequence of the above corollary is that if σ is a sentence, $\mathfrak{M} \models \sigma$ if and only if there is some assignment $v : V \to |\mathfrak{M}|$ such that $\mathfrak{M} \models \sigma[s]$.

Definition 1.4.11: Modelled Formulas and True Sentences ($\mathfrak{M} \models \varphi$)

Let φ be a formula in a language $\mathcal{L}$ and $\mathfrak{A}$ be a $\mathcal{L}$-structure. Then

1. we say that $\mathfrak{A}$ is a *model* for φ if and only if $\mathfrak{A} \models \varphi[s]$ for every assignment function s, and we write $\mathfrak{A} \models \varphi$.
2. If Φ is a set of $\mathcal{L}$-formulas, we will say that $\mathfrak{A}$ models Φ, and write $\mathfrak{A} \models \Phi$ if and only if $\mathfrak{A} \models \varphi$ for each $\varphi \in \Phi$.
3. The set Φ is *satisfiable* if some structure $\mathfrak{M}$ has $\mathfrak{M} \models \Phi$, that is, $\mathfrak{M} \models \varphi[s]$ for every $\varphi \in \Phi$ and every assignment function s.
4. For the case of φ being a *sentence*, then we have $\mathfrak{A} \models \sigma$ if and only if $\mathfrak{A} \models \sigma[s]$ for any assignment function s. In this case we will say that the sentence φ is *true* in $\mathfrak{A}$.

Example 1.4.12: True Sentences
Let's take the number theory structure with language $\mathcal{L}_{NT}$ and standard structure

$$\mathfrak{N} = (\mathbb{N}, 0, S, +, \cdot, E, <)$$

Let s be the variable assignment function $s(v_i) = 2i$ (for example $s(v_3) = 6$). As an example of the term-assignment function that comes from it:

$$\begin{aligned} \overline{s}(v_1 + v_1) \quad &\text{is} \quad +^{\mathfrak{N}}(\overline{s}(v_1), \overline{s}(v_1)) \\ &\text{is} \quad +^{\mathfrak{N}}(2, 2) \\ &\text{is} \quad 4 \end{aligned}$$

or similarly:

$$\begin{aligned} \overline{s}(SSSS0) \quad &\text{is} \quad S^{\mathfrak{N}}(S^{\mathfrak{N}}(S^{\mathfrak{N}}(S^{\mathfrak{N}}(0)))) \\ &\text{is} \quad 4 \end{aligned}$$

Let's now look for true sentences. Consider the sentence:

$$(\forall v_1)\neg(\forall v_2)\neg(v_1 = v_2 + v_2)$$

where $\neg(\forall v_2)\neg$ is usually shorthanded to $(\exists v_2)$. Parsing this into a english, we see that the sentence is asking whether all numbers are the sum of two identical numbers; if all numbers are even. This is intuitively wrong, so let's start parsing the sentence by the rules we have been given to come to the same conclusion:

$$\begin{aligned} \mathfrak{A} \models \sigma[s] \quad &\text{iff} \quad \text{For every } a \in \mathbb{N},\ \mathfrak{A} \models \neg(\forall v_2)\neg(v_1 = v_2 + v_2)s[v_1|a] \\ &\text{iff} \quad \text{For every } a \in \mathbb{N},\ \mathfrak{A} \not\models (\forall v_2)\neg(v_1 = v_2 + v_2)s[v_1|a] \\ &\text{iff} \quad \text{For every } a \in \mathbb{N}, \text{there exists a } b \in \mathbb{N},\ \mathfrak{A} \models (v_1 = v_2 + v_2)s[v_1|a][v_2|b] \end{aligned}$$

Taking $a = 1$, no $b \in \mathbb{N}$ satisfies $1 = b + b$, so the last condition fails and $\mathfrak{A} \not\models \sigma[s]$. Thus $\mathfrak{A}$ does not model σ, and σ is *false*.

Proposition 1.4.13: Existential Quantifier

Let $\mathfrak{M}$ be a structure of a first order language$\mathcal{L}$, s an assignment for $\mathfrak{M}$, x a variable, and φ a formula of $\mathcal{L}$. Then $\mathfrak{M} \models \exists x, \varphi[s]$ if and only if $\mathfrak{M} \models \varphi[s(x|m)]$ for some $m \in |\mathfrak{M}|$

***Proof*:**
By the abbreviation $(\exists x)(\alpha) :\equiv (\neg(\forall x)(\neg\alpha))$ of definition 1.1.5, the formula $\exists x\varphi$ is $(\neg(\forall x)(\neg\varphi))$. By the negation clause of definition 1.4.5, $\mathfrak{M} \models (\neg(\forall x)(\neg\varphi))[s]$ holds exactly when $\mathfrak{M} \not\models (\forall x)(\neg\varphi)[s]$. By the quantifier clause, $\mathfrak{M} \models (\forall x)(\neg\varphi)[s]$ holds exactly when $\mathfrak{M} \models (\neg\varphi)[s(x|m)]$ for every $m \in |\mathfrak{M}|$, and by the negation clause again this holds exactly when $\mathfrak{M} \not\models \varphi[s(x|m)]$ for every $m \in |\mathfrak{M}|$. Hence $\mathfrak{M} \models (\exists x\varphi)[s]$ holds exactly when it is not the case that $\mathfrak{M} \not\models \varphi[s(x|m)]$ for every m, that is, exactly when $\mathfrak{M} \models \varphi[s(x|m)]$ for some $m \in |\mathfrak{M}|$, as claimed.

Proposition 1.4.14: Important Shorthands

Let $\mathfrak{M}$ be a structure for $\mathcal{L}$ and α, β be sentences of $\mathcal{L}$. Then:

1. $\mathfrak{M} \models \neg\alpha$ if and only if $\mathfrak{M} \not\models \alpha$
2. $\mathfrak{M} \models \alpha \to \beta$ if and only if $\mathfrak{M} \models \beta$ whenever $\mathfrak{M} \models \alpha$
3. $\mathfrak{M} \models \alpha \vee \beta$ if and only if $\mathfrak{M} \models \alpha$ or $\mathfrak{M} \models \beta$
4. $\mathfrak{M} \models \alpha \wedge \beta$ if and only if $\mathfrak{M} \models \alpha$ and $\mathfrak{M} \models \beta$
5. $\mathfrak{M} \models \alpha \leftrightarrow \beta$ if and only if $\mathfrak{M} \models \alpha$ exactly when $\mathfrak{M} \models \beta$
6. $\mathfrak{M} \models \forall x\alpha$ if and only if $\mathfrak{M} \models \alpha$

Moreover, let Σ be a set of formulas, ψ a sentence and ρ a formula, and for a set of formulas Γ write $\Gamma \models \rho$ when every structure that satisfies every member of Γ (definition 1.4.11) satisfies ρ. Then

$$\Sigma \cup \{\psi\} \models \rho \qquad \text{if and only if} \qquad \Sigma \models (\psi \to \rho)$$

***Proof*:**
The universe $|\mathfrak{M}|$ is nonempty (definition 1.3.1), so we may fix an assignment s_0 into $\mathfrak{M}$. For a sentence σ, corollary 1.4.10 gives that $\mathfrak{M} \models \sigma[s]$ holds for every assignment s or for none, so by definition 1.4.11 we have $\mathfrak{M} \models \sigma$ exactly when $\mathfrak{M} \models \sigma[s_0]$. If α and β are sentences, then so are $\neg\alpha$, $\alpha \to \beta$ and $\forall x\alpha$, because by definition 1.2.2 a variable free in one of these formulas is free in α or in β. The formulas $\alpha \vee \beta$, $\alpha \wedge \beta$ and $\alpha \leftrightarrow \beta$ are built from α and β using only $\neg$ and $\to$ (definition 1.1.5), so they are sentences as well. Each item therefore reduces to a statement about s_0, which we unwind with the clauses of definition 1.4.5 and with proposition 1.4.6.

1. $\mathfrak{M} \models \neg\alpha$ holds exactly when $\mathfrak{M} \models (\neg\alpha)[s_0]$, which by the negation clause holds exactly when $\mathfrak{M} \not\models \alpha[s_0]$, that is, when $\mathfrak{M} \not\models \alpha$.
2. By the implication clause, $\mathfrak{M} \models (\alpha \to \beta)[s_0]$ holds exactly when $\mathfrak{M} \not\models \alpha[s_0]$ or $\mathfrak{M} \models \beta[s_0]$. Hence $\mathfrak{M} \models \alpha \to \beta$ exactly when $\mathfrak{M} \not\models \alpha$ or $\mathfrak{M} \models \beta$, that is, exactly when $\mathfrak{M} \models \beta$ whenever $\mathfrak{M} \models \alpha$.
3. By proposition 1.4.6(1), $\mathfrak{M} \models (\alpha \vee \beta)[s_0]$ holds exactly when $\mathfrak{M} \models \alpha[s_0]$ or $\mathfrak{M} \models \beta[s_0]$, that is, when $\mathfrak{M} \models \alpha$ or $\mathfrak{M} \models \beta$.
4. By proposition 1.4.6(2), $\mathfrak{M} \models (\alpha \wedge \beta)[s_0]$ holds exactly when $\mathfrak{M} \models \alpha[s_0]$ and $\mathfrak{M} \models \beta[s_0]$, that is, when $\mathfrak{M} \models \alpha$ and $\mathfrak{M} \models \beta$.
5. By proposition 1.4.6(3), $\mathfrak{M} \models (\alpha \leftrightarrow \beta)[s_0]$ says that $\mathfrak{M} \models \alpha[s_0]$ holds exactly when $\mathfrak{M} \models \beta[s_0]$ does, that is, that $\mathfrak{M} \models \alpha$ holds exactly when $\mathfrak{M} \models \beta$ does.
6. If $\mathfrak{M} \models \forall x\alpha$, then $\mathfrak{M} \models (\forall x\alpha)[s_0]$, so by the quantifier clause $\mathfrak{M} \models \alpha[s_0(x|m)]$ for every $m \in |\mathfrak{M}|$. Taking $m = s_0(x)$, for which $s_0(x|m)$ is s_0, gives $\mathfrak{M} \models \alpha[s_0]$, so $\mathfrak{M} \models \alpha$. Conversely, if $\mathfrak{M} \models \alpha$, then $\mathfrak{M} \models \alpha[s]$ for every assignment s by definition 1.4.11, in particular for every $s_0(x|m)$, so $\mathfrak{M} \models (\forall x\alpha)[s_0]$ and $\mathfrak{M} \models \forall x\alpha$.

For the final claim, suppose first that $\Sigma \cup \{\psi\} \models \rho$. Let $\mathfrak{M} \models \Sigma$ and let s be an assignment into $\mathfrak{M}$. If $\mathfrak{M} \not\models \psi[s]$, then $\mathfrak{M} \models (\psi \to \rho)[s]$ by the implication clause. If $\mathfrak{M} \models \psi[s]$, then $\mathfrak{M} \models \psi$ by the first paragraph, since ψ is a sentence, so $\mathfrak{M} \models \Sigma \cup \{\psi\}$. The hypothesis then gives $\mathfrak{M} \models \rho$, in particular $\mathfrak{M} \models \rho[s]$, and again $\mathfrak{M} \models (\psi \to \rho)[s]$. As s was arbitrary, $\mathfrak{M} \models \psi \to \rho$, and so $\Sigma \models (\psi \to \rho)$. Conversely, suppose $\Sigma \models (\psi \to \rho)$ and $\mathfrak{M} \models \Sigma \cup \{\psi\}$. For every assignment s we have $\mathfrak{M} \models (\psi \to \rho)[s]$ and $\mathfrak{M} \models \psi[s]$, so the implication clause gives $\mathfrak{M} \models \rho[s]$. Thus $\mathfrak{M} \models \rho$, and $\Sigma \cup \{\psi\} \models \rho$, completing the proof.

Note that α and β are sentences in the above. If they were formulas, we must be weary of particular assignment functions.

1.5 Substitutions and Substitutability

we now tackle the problem of when we can substitute a variable by a term. For example, if we have a term $(\forall x)\varphi(x)$ which is true in a particular structure $\mathfrak{A}$, we see that if we replace the variable x by the constant c (recall that $c^{\mathfrak{A}}$ in a structure is an element of A), then we would expect $\varphi(c)$ will also be true. However, things are not always so simple. Take for example $\mathfrak{A} \models \forall x \exists y \neg(x = y)$. This sentence is true in any structure $\mathfrak{A}$ if A has at least 2 elements. Then if we replace the variable x with the variable u, then we would expect that the truth values stays the same. However, if we replace x with y, then we see that the sentence is no longer true, for any structure! In other words, our bad choice of substitution changed the truth value of the formula. We will thus be more careful with how we proceed with substituting terms inside a quantifier that binds a variable involved with the term

Definition 1.5.1: Term Substitution

let u be a term, x a variable, and t a term. Then we define the term u_t^x (read "u with x replaced by t") as follows:

1. If u is a variable not equal to x, then u_t^x is u
2. if u is x, then u_t^x is t
3. if u is a constant symbol, then u_t^x is u
4. if $u :\equiv f u_1 u_2 ... u_n$, where f is an n-ary function symbol and the u_i are terms, then[a]

$$u_t^x \text{ is } f(u_1)_t^x (u_2)_t^x ... (u_n)_t^x$$

[a] The parenthesis technically should not be there, but they were added for readability

Essentially, all this definition is saying is that we should find the variable x (if it's in the term u) and replace it with t.

Example 1.5.2: Term Substitution

Let's say u is $f(x, y) + h(z, x, g(x))$, and we want to replace x with t which is $g(c)$. Processing the definition, we get u_t^x is:

$$f(g(c), y) + h(z, g(c), g(g(c)))$$

We define it similarly for a formula

Definition 1.5.3: Formula Substitution

Suppose that φ is an $\mathcal{L}$-formula, t is a term, and x is a variable. Then we define the formula φ_t^x (read "φ with x replaced by t") as:

1. if $\varphi :\equiv u_1 = u_2$ then φ_t^x is $(u_1)_t^x = (u_2)_t^x$
2. If $\varphi :\equiv Ru_1u_2...u_n$, then φ_t^x is $R(u_1)_t^x(u_2)_t^x...(u_n)_t^x$
3. if $\varphi :\equiv \neg(\alpha)$ then φ_t^x is $\neg(\alpha_t^x)$
4. If $\varphi :\equiv (\alpha \to \beta)$, then φ_t^x is $(\alpha_t^x \to \beta_t^x)$
5. If $\varphi :\equiv (\forall y)(\alpha)$, then
$$\varphi_t^x = \begin{cases} \varphi & \text{if } x \text{ is } y \\ (\forall y)(\alpha_t^x) & \text{otherwise} \end{cases}$$

Example 1.5.4: Formula Substitution

Suppose that we have the formula φ

$$P(x, y) \to [(\forall x)(Q(g(x), z)) \vee (\forall y)(R(x, h(x)))]$$

then if t is the term $g(c)$, we get that φ_t^x is

$$P(g(c), y) \to [(\forall x)(Q(g(g(c)), z)) \vee (\forall y)(R(g(c), h(g(c))))]$$

With the preliminary definitions done, we can now talk about when we can substitute a term for a variable without running into the problems mentioned at the beginning of this section; we will not substitute a term in such a way that a variable contained in that term is inadvertently bound by a quantifier

Definition 1.5.5: Substitutable

Let φ be an $\mathcal{L}$-formula, t a term, and x a variable. Then we say that t is *substitutable* for x in φ, or x can be *substituted* by t, if:

1. φ is atomic
2. $\varphi :\equiv \neg(\alpha)$ and t is substitutable for x in α
3. $\varphi :\equiv (\alpha \to \beta)$ and t is substitutable for x in both α and β
4. $\varphi :\equiv (\forall y)(\alpha)$ and either
   (a) x is not free in φ (i.e. x is bound or does not exist in φ)
   (b) y does not occur in t and t is substitutable for x in α

Note that φ_t^x is defined whether or not t is substitutable for x in φ. Usually we will want to do substitutions only if the terms are substitutable, but we can do a substitution even if it will change the truth value. On the other hand, we will often only substitute if it is substitutable.

Example 1.5.6: Substitutability

1. x is always substitutable for x in any formula φ, namely φ_x^x is simply φ
2. Let $\varphi :\equiv \forall y\ x = y$. Then the term y is not substitutable for x, since the new y would be bound by the quantifier $\forall y$

Lemma 1.5.7: Syntax Of Substitution

Let φ be a formula, x a variable and t a term.

1. If x is not free in φ, then φ_t^x is φ, and t is substitutable for x in φ.
2. If x is free in φ, then every symbol occurring in t occurs in φ_t^x.
3. If no variable occurs in t, then x is not free in φ_t^x, and every variable free in φ_t^x is free in φ.
4. The formula φ_t^x has as many occurrences of $\neg$, of $\rightarrow$ and of $\forall$ as φ.

***Proof*:**
For a term u, the term u_t^x is u with every occurrence of x replaced by t, by induction on u through definition 1.5.1. So for an atomic formula φ, the formula φ_t^x of definition 1.5.3 is φ with every occurrence of x replaced by t, and by definition 1.2.2 the variables free in an atomic formula are those occurring in it. Each item is proved by induction on φ (definition 1.1.5).

1. If φ is atomic, then x does not occur in φ, so φ_t^x is φ, and t is substitutable for x in φ by the atomic clause of definition 1.5.5. If $\varphi :\equiv (\neg\alpha)$ or $\varphi :\equiv (\alpha \rightarrow \beta)$, then by definition 1.2.2 the variable x is free in none of the immediate subformulas α, β of φ. The induction hypothesis gives that substituting t for x leaves each of them unchanged and that t is substitutable for x in each. So φ_t^x, which is $(\neg\alpha_t^x)$ or $(\alpha_t^x \rightarrow \beta_t^x)$, is φ, and t is substitutable for x in φ. Let $\varphi :\equiv (\forall y)(\alpha)$. Then t is substitutable for x in φ by the first alternative of the quantifier clause of definition 1.5.5. If y is x, then φ_t^x is φ by definition 1.5.3. Otherwise x is not free in α, so α_t^x is α by the induction hypothesis, and φ_t^x is $(\forall y)(\alpha_t^x)$, which is φ.
2. If φ is atomic, then x occurs in φ, and that occurrence is replaced by t in φ_t^x. If $\varphi :\equiv (\neg\alpha)$ or $\varphi :\equiv (\alpha \rightarrow \beta)$, then by definition 1.2.2 the variable x is free in one of the immediate subformulas α, β of φ, and φ_t^x, which is $(\neg\alpha_t^x)$ or $(\alpha_t^x \rightarrow \beta_t^x)$, contains the result of substituting t for x in it, to which the induction hypothesis applies. If $\varphi :\equiv (\forall y)(\alpha)$, then x is free in φ only if y is not x and x is free in α, and then φ_t^x is $(\forall y)(\alpha_t^x)$, to which the induction hypothesis applies.
3. If φ is atomic, then every variable occurring in φ_t^x occurs in φ and is not x, since t contributes no variable, so every variable free in φ_t^x is free in φ and is not x. If $\varphi :\equiv (\neg\alpha)$ or $\varphi :\equiv (\alpha \rightarrow \beta)$, then φ_t^x is $(\neg\alpha_t^x)$ or $(\alpha_t^x \rightarrow \beta_t^x)$, so by definition 1.2.2 a variable free in φ_t^x is free in one of α_t^x, β_t^x, and the induction hypothesis applies. Let $\varphi :\equiv (\forall y)(\alpha)$. If y is x, then φ_t^x is φ, in which x is not free. Otherwise φ_t^x is $(\forall y)(\alpha_t^x)$, and a variable v free in it is not y and is free in α_t^x. By the induction hypothesis v is not x and is free in α, hence free in φ.

4. In an atomic formula only terms change, and terms contain no $\neg$, $\to$ or $\forall$. The clauses of definition 1.5.3 for $\neg$ and $\to$ keep the connective, and the quantifier clause keeps the quantifier $(\forall y)$, so the counts agree by the induction hypothesis.

1.6 Logical Implication

The last bit of language we will gain is that of formula implication. In mathematics, it is very common to have answers to questions of the form: If these statements are true, it is necessarily the case that the other statement(s) is/are true.

Definition 1.6.1: Logical Implication ($\Delta \models \Gamma$)

Suppose that Δ and Γ are sets of $\mathcal{L}$-formulas. Then we will say that Δ *logically implies* Γ and write $\Delta \models \Gamma$ if for every $\mathcal{L}$-structure $\mathfrak{A}$, if $\mathfrak{A} \models \Delta$, then $\mathfrak{A} \models \Gamma$

To be pedantic, this definition does not necessarily mean that there is a causal relation between Δ and Γ, it means that it so happens that if Δ is true, it must be that Γ is true. Notice too that it is independent of any $\mathcal{L}$-structure, meaning the implication should be "universally" valid.

Example 1.6.2: Logical Implication

1. Let $\mathfrak{M}$ be any structure. Then $\mathfrak{M} \models \{\varphi\}$ if and only if $\mathfrak{M} \models \varphi$, then $\{\varphi\} \models \varphi$
2. Let α, β be formulas. Show that $\{\alpha \to \beta, \alpha\} \models \beta$

Definition 1.6.3: Valid Formula (Tautology and Contradiction) ($\models \varphi$)

Let φ be an $\mathcal{L}$-formula. Then we say that φ is valid (or that it is a tautology) if $\emptyset \models \varphi$, in other words φ is true in every $\mathcal{L}$-structure with every assignment function s. If $\models \neg\varphi$, we say that φ is a *contradiction*.

Example 1.6.4: Valid Formula and Models

1. $\models \forall x\ x = x$ is a true sentence in all models and in all term assignment functions. On the other hand $\exists x\ \neg x = x$ is a contradiction
2. $\models \varphi \to \varphi$ is a tautology, while $\models \neg(\varphi \to \varphi)$ is a contradiction.

Proposition 1.6.5: Modelling And Contradiction

Suppose φ is a contradiction. Then $\mathfrak{M} \models \varphi[s]$ is false for every structure $\mathfrak{M}$ and assignment s. As a consequence, Σ is satisfiable if and only if there is no contradiction χ such that $\Sigma \models \chi$.

***Proof*:**
Suppose φ is a contradiction, so that $\models \neg\varphi$ by definition 1.6.3: every $\mathcal{L}$-structure $\mathfrak{M}$ and every assignment s have $\mathfrak{M} \models (\neg\varphi)[s]$. By the negation clause of definition 1.4.5, this holds exactly when $\mathfrak{M} \not\models \varphi[s]$. Hence $\mathfrak{M} \models \varphi[s]$ fails for every structure $\mathfrak{M}$ and every assignment s.

For the consequence, suppose first that Σ is satisfiable, and fix a structure $\mathfrak{M}$ with $\mathfrak{M} \models \Sigma$. If χ were a contradiction with $\Sigma \models \chi$, then $\mathfrak{M} \models \chi$ by definition 1.6.1. The universe A of $\mathfrak{M}$ is nonempty by definition 1.3.1, so there is at least one assignment $s : Vars \to A$, and then $\mathfrak{M} \models \chi[s]$, contradicting the first part. Hence no contradiction χ has $\Sigma \models \chi$.

Suppose instead that Σ is not satisfiable, and let $\chi :\equiv \neg(\forall x)(x = x)$. For any $\mathfrak{M}$, any s, and any $a \in A$ we have $\overline{s[x|a]}(x) = a$, so $\mathfrak{M} \models (x = x)[s[x|a]]$ by the equality clause of definition 1.4.5, and therefore $\mathfrak{M} \models (\forall x)(x = x)[s]$. Applying the negation clause twice, $\mathfrak{M} \models (\neg\chi)[s]$. Thus $\models \neg\chi$, and χ is a contradiction. Since no structure models Σ, the condition defining $\Sigma \models \chi$ in definition 1.6.1 holds vacuously, so $\Sigma \models \chi$. An unsatisfiable Σ therefore implies a contradiction, which is the contrapositive of the remaining direction.

Finally, we require the following terminology for later

Definition 1.6.6: Theory and Axioms

If Σ is a set of sentences then the theory of Σ is

$$\mathrm{Th}(\Sigma) = \{\tau \mid \tau \text{ is a sentence and } \Sigma \models \tau\}$$

If Δ is a set of sentences and $\mathcal{S}$ is a collection of structures, then Δ is a set of (non-logical) axioms for $\mathcal{S}$ if for every structure $\mathfrak{M}$, $\mathfrak{M} \in \mathcal{S}$ if and only if $\mathfrak{M} \models \Delta$

Example 1.6.7: Theory And Axiom

1. example of theory
2. Let $\{\exists x \exists y((\neg(x = y)) \wedge \forall z(z = x \vee z = y))\}$. Every structure over the language $\mathcal{L} = \{\}$ satisfying this sentence must have exactly two elements in the universe, and so the above set is a set of non-logical axioms for the collection of sets of cardinality 2.

1.7 *Propositional Language

We will take a moment to focus on another type language. This language will be purely focused on determining where a given formula φ is either *True* or *False*. The language will feel restrictive as compared to first-order language, and it indeed has limitations: if not careful with our definition of atomic formulas we may raise a Russel-like contradiction. However, its simplicity also has it's advantages: this language will capture the intuition that a given statement can either be assignment *true* (T) or *false* (F). Given a model $\mathcal{M}$ with a language $\mathcal{L}$, we may convert many first-order questions into propositional questions. This will come in handy in the next chapter where we will link "deductibility" (the ability to prove or deduce a statement) to "satisfiability" ($\mathcal{M} \models \varphi$). Due to this link, one of my professors called propositional logic "0th order logic".

Definition 1.7.1: Propositional Language

The symbols of $\mathcal{L}_P$ are

1. Parenthesis: (and)
2. Connectives: $\neg$ and $\rightarrow$
3. Atomic formulas $A_0, A_1, ..., A_n, ...$

A *formula* of $\mathcal{L}_P$ are finite sequences or strings of symbols from $\mathcal{L}_P$ satisfying one of the following rules:

1. Every atomic formula is a formula
2. If α is a formula, $(\neg\alpha)$ is a formula
3. if α and β are formulas, $(\alpha \rightarrow \beta)$ is a formula
4. No other sequence of symbols is a formula

Like first-order languages, we have shorthand's:

1. $(\alpha \wedge \beta)$ is $(\neg(\alpha \rightarrow (\neg\beta)))$
2. $(\alpha \vee \beta)$ is $((\neg\alpha) \rightarrow \beta)$
3. $(\alpha \leftrightarrow \beta)$ for $(\alpha \rightarrow \beta) \wedge (\beta \rightarrow \alpha)$

Definition 1.7.2: Propositional Sub formula

Let φ be a formula of $\mathcal{L}_P$. Then the subformulas of φ, $S(\varphi)$ is defined as follows:

1. if φ is an atomic formula, $S(\varphi) = \{\varphi\}$
2. if φ is $(\neg\alpha)$ then $S(\varphi) = S(\alpha) \cup \{(\neg\alpha)\}$
3. If φ is $(\alpha \rightarrow \beta)$ then $S(\varphi)$ is $S(\alpha) \cup S(\beta) \cup \{(\alpha \rightarrow \beta)|\}$

Just like for first order language, we have unique readability for propositional language. Instead of having structure and a variable-assignment function, we define a truth assignment function directly on the formulas:

Definition 1.7.3: Truth Assignment Function

A *truth assignment* is a function v whose domain is the set of all formulas of $\mathcal{L}_P$ and whose range is any binary set, say $\{T, F\}$ such that

1. $v(A_n)$ is defined for every atomic formula A_n
2. For any formula α,
$$v((\neg\alpha)) = \begin{cases} T & \text{if } v(\alpha) = F \\ F & \text{if } v(\alpha) = T \end{cases}$$
3. For any α, β:
$$v((\alpha \to \beta)) = \begin{cases} F & \text{if } v(\alpha) = T \text{ and } v(\beta) = F \\ T & \text{Otherwise} \end{cases}$$

Induction on δ through the clauses of definition 1.7.3 shows that if δ is any formula and u, v are truth assignments such that $u(A_n) = v(A_n)$ for all atomic formulas which occur in δ, then $u(\delta) = v(\delta)$.

Proposition 1.7.4: Relating Our Shorthand With Our Intuition

Let α, β be formulas and v a truth assignment function. Then:

1. $v(\neg\alpha) = T$ if and only if $v(\alpha) = F$
2. $v(\alpha \to \beta) = T$ if and only if $v(\beta) = T$ whenever $v(\alpha) = T$
3. $v(\alpha \wedge \beta) = T$ if and only if $v(\alpha) = T$ and $v(\beta) = T$
4. $v(\alpha \vee \beta) = T$ if and only if $v(\alpha) = T$ or $v(\beta) = T$
5. $v(\alpha \leftrightarrow \beta) = T$ if and only if $v(\alpha) = v(\beta)$

***Proof*:**
Here $\neg$ and $\to$ are the connectives of definition 1.7.1, and $\wedge, \vee, \leftrightarrow$ are the shorthands listed after it: $(\alpha \wedge \beta)$ is $(\neg(\alpha \to (\neg\beta)))$, $(\alpha \vee \beta)$ is $((\neg\alpha) \to \beta)$, and $(\alpha \leftrightarrow \beta)$ is $((\alpha \to \beta) \wedge (\beta \to \alpha))$. The two clauses of definition 1.7.3 compute v on a negation or an implication from the values of v on its parts. Applying them to the parts of the shorthands, for each of the four possible pairs $(v(\alpha), v(\beta))$, gives the two tables

α	β	$\neg\beta$	$\alpha \to \neg\beta$	$\alpha \wedge \beta$	$\neg\alpha$	$\alpha \vee \beta$
T	T	F	F	T	F	T
T	F	T	T	F	F	T
F	T	F	T	F	T	T
F	F	T	T	F	T	F

α	β	$\alpha \to \beta$	$\beta \to \alpha$	$\alpha \leftrightarrow \beta$
T	T	T	T	T
T	F	F	T	F
F	T	T	F	F
F	F	T	T	T

in which each entry is the value of v on the formula heading its column. In the first table the column for $\neg\beta$ comes from that for β by the negation clause, the column for $\alpha \to \neg\beta$ from those for α and $\neg\beta$ by the implication clause, the column for $\alpha \wedge \beta$, which is $\neg(\alpha \to \neg\beta)$, from the previous column by the negation clause, the column for $\neg\alpha$ from that for α, and the column for $\alpha \vee \beta$, which is $\neg\alpha \to \beta$, from those for $\neg\alpha$ and β by the implication clause. In the second table

the two implications come from the implication clause.

1. This is the negation clause of definition 1.7.3.
2. By the implication clause, $v(\alpha \to \beta) = F$ exactly when $v(\alpha) = T$ and $v(\beta) = F$. So $v(\alpha \to \beta) = T$ exactly when $v(\beta) = T$ whenever $v(\alpha) = T$.
3. In the first table, $v(\alpha \wedge \beta) = T$ in the first row only, which is the row with $v(\alpha) = v(\beta) = T$.
4. In the first table, $v(\alpha \vee \beta) = T$ in the first three rows, which are the rows where $v(\alpha) = T$ or $v(\beta) = T$.
5. The formula $\alpha \leftrightarrow \beta$ is the conjunction of the formulas $\alpha \to \beta$ and $\beta \to \alpha$, so by item (3) applied to these two formulas, its value is T exactly when both of the two preceding columns of the second table are T, which gives its last column. That column is T in the first and last rows, which are the rows with $v(\alpha) = v(\beta)$, completing the proof.

Note that $\to$ is *not* associative, that is $\alpha \to (\beta \to \gamma)$ does not model $(\alpha \to \beta) \to \gamma$, and vice-versa.

Definition 1.7.5: Satisfied Formula

Let v be a truth assignment and φ a formula. Then we say that v *satisfies* φ if $v(\varphi) = T$. If Σ is a set of formulas, we will say v satisfies Σ if $v(\sigma) = T$ for all $\sigma \in \Sigma$.

Definition 1.7.6: Satisfiable, Tautology, Contradiction

We say that φ (resp. Σ) is *satisfiable* if there is a truth assignment such that $v(\varphi) = T$ (or $v(\sigma) = T$ for all Σ), is a *tautology* if it is satisfied by every truth assignment, and a *contradiction* if there is not truth assignment satisfying it

A natural tautology is $\alpha \to \alpha$ and a natural contradiction is $\neg(\alpha \to \alpha)$. β More generally, β a tautology if and only if $\neg\beta$ is a contradiction.

Definition 1.7.7: Propositional Implication

A set of formulas Σ *implies* a formula φ if every truth assignment v which satisfies Σ also satisfies φ. We write $\Sigma \models \varphi$, and $\Sigma \not\models \varphi$ in the opposite case. If Σ is empty, we usually write $\models \varphi$ instead of $\emptyset \models \varphi$.
If Δ and Γ are sets of formulas, then Δ implies Γ if every truth assignment v which satisfies Δ also satisfies Γ

Proposition 1.7.8: Properties of Propositional Implication

1. If $\Gamma \subseteq \Sigma$ is a subset of formulas then $\Sigma \models \Gamma$
2. $\Sigma \cup \{\psi\} \models \rho$ if and only if $\Sigma \models \psi \to \rho$
3. Σ is satisfiable if and only if there is no contradiction χ such that $\Sigma \models \chi$

***Proof*:**

1. Let v be a truth assignment satisfying Σ. Each $\gamma \in \Gamma$ lies in Σ, so $v(\gamma) = T$ by definition 1.7.5. Hence v satisfies Γ, and $\Sigma \models \Gamma$ by definition 1.7.7.

2. Suppose $\Sigma \cup \{\psi\} \models \rho$, and let v satisfy Σ. If $v(\psi) = F$, then $v(\psi \to \rho) = T$ by the implication clause of definition 1.7.3. If $v(\psi) = T$, then v satisfies $\Sigma \cup \{\psi\}$, so $v(\rho) = T$ and again $v(\psi \to \rho) = T$. Hence $\Sigma \models \psi \to \rho$. Conversely, suppose $\Sigma \models \psi \to \rho$, and let v satisfy $\Sigma \cup \{\psi\}$. Then $v(\psi \to \rho) = T$ and $v(\psi) = T$. The implication clause rules out $v(\rho) = F$, since $v(\psi) = T$ together with $v(\rho) = F$ would force $v(\psi \to \rho) = F$. Hence $v(\rho) = T$, and $\Sigma \cup \{\psi\} \models \rho$.

3. Suppose Σ is satisfiable, say v satisfies Σ, and let χ be any formula with $\Sigma \models \chi$. Then $v(\chi) = T$, so χ is satisfied by some truth assignment and is not a contradiction by definition 1.7.6. Conversely, suppose Σ is not satisfiable. Then no truth assignment satisfies Σ, so the defining condition of $\Sigma \models \chi$ holds vacuously for every formula χ, in particular for $\chi :\equiv \neg(A_0 \to A_0)$. For any v, the value $v(A_0 \to A_0) = F$ would require $v(A_0) = T$ and $v(A_0) = F$ at once, so $v(A_0 \to A_0) = T$ and $v(\neg(A_0 \to A_0)) = F$. Hence $\neg(A_0 \to A_0)$ is a contradiction implied by Σ. Taking the contrapositive completes the equivalence.

1.8 Summary

1. **terms and formulas**: terms are the nouns which can either be a variable, a constant, or a function taking in a term. A formula can either be $t_1 = t_2$ for some two terms t_1, t_2, some n-relation $Rt_1...t_n$, or recursively built from negations, implications and universal quantifiers: $\neg\varphi$, $\alpha \to \beta$, $(\forall v)(\varphi)$

2. **infix notation and shortcuts**: $\alpha \vee \beta$, $\alpha \wedge \beta$, $\alpha \leftrightarrow \beta$ and $\exists v\varphi$ abbreviate $\neg\alpha \to \beta$, $\neg(\alpha \to \neg\beta)$, $(\alpha \to \beta) \wedge (\beta \to \alpha)$ and $\neg\forall v\neg\varphi$.

3. **subformulas**: recursively go through and compile all the subformulas within a formula.

4. **free variables**: within a formula, we may have a variable v_i which is not quantified by a $\forall$.

5. **sentence**: a formula with no free variables.

6. **structure (model)**: a set A (called the universe) along with a collection of functions, relations, and a subset of A which be used for associated functions to function symbols, relations to relations symbols, and constants to constant symbols.

7. **variable and term assignment function**: If $Vars \subseteq \mathcal{L}$ is the collection of all variables, then $s : Vars \to A$ is a variable assignment function. This can then be extended to be a term assignment function. We can also override a particular assignment by doing $s[x|a](v)$, which will on input of variable x give a instead of $s(x)$.

8. **Modelling** ($\mathfrak{M} \models \varphi[s]$): Given an assignment function s, after we evaluate it and interpret all the relation symbols as well as $\neg$, $\to$ and $\forall$ semantically, is the resulting value true. If two assignment functions that agree on the free variables of a formula φ, then $\mathfrak{M} \models \varphi[s_1]$ if and

only if $\mathfrak{M} \models \varphi[s_1]$. Consequently, if φ is a sentence $\mathfrak{M} \models \varphi[s]$ for any variable assignment function s, and so it is meaningful to drop the $[s]$ and just say $\mathfrak{M} \models \varphi$ if $\mathfrak{M} \models \varphi[s]$ for all s.

9. **(Models, satisfiable, truth)**: if $\mathfrak{M} \models \varphi$, w say that $\mathfrak{M}$ models φ. If φ is a formula in $\mathfrak{M}$, we will say that it's *satisfiable* if there is a $\mathfrak{M}$ such that $\mathfrak{M} \models \varphi$. If φ is a sentence, then then if $\mathfrak{M} \models \varphi$, we shall say that φ is *true*.

10. **Existential Quantifier and Shorthand's** Let φ be a formula. Then $\mathfrak{M} \models (\exists x,\ \varphi)[s]$ if and only if there is some $m \in |\mathfrak{M}|$ such that $\mathfrak{M} \models \varphi[s(x|m)]$. Furthermore, for sentences, all the usual rules of logic works for modelling, for example $\mathfrak{M} \models \neg\alpha$ if and only if $\mathfrak{M} \not\models \alpha$. If we were dealing with formulas, we would have to keep track of an assignment function. For a set of formulas Σ, a sentence ψ and a formula ρ, we also have:

$$\Sigma \cup \{\psi\} \models \rho \qquad \text{if and only if} \qquad \Sigma \models (\psi \to \rho)$$

11. **Substitution and Substitutability**: replace the free occurrences of a variable x by a term t. The term t is substitutable for x when no variable of t becomes bound by the replacement.

12. **Logical Implications** ($\Delta \models \Gamma$) Let Δ, Γ be two sets of $\mathcal{L}$-formula. Then Δ logically implies Γ if for any structure where $\mathfrak{M} \models \Delta$ then $\mathfrak{M} \models \Gamma$.

13. **Tautology and Contradiction**: If $\emptyset \models \varphi$, then φ is a tautology. If $\emptyset \models \neg\varphi$,then φ is a *contradiction*.

14. **Modelling Contradiction**: If φ is a contradiction, $\mathfrak{M} \models \varphi[s]$ is false for every structure $\mathfrak{M}$ and assignment s. Hence, a set Σ is satisfiable if and only if there is no contradiction χ such that $\Sigma \models \chi$.

15. **Propositional formulas**: no functions, relations, constants. Just variables, connectives, parenthesis, and recursive relations.

16. **Truth Assignment et al**: assign T, F to every variable. Then recursively go through formulas and deal with $\neg$ and $\to$. All the same shorthand's apply for $\vee, \leftrightarrow, \wedge$, and so forth. Same notion of satisfiable, tautology, and contradiction, and logical implication (which we will call *propositional implication* if the context requires specificity)

2

Deductions

We have now firmly established the language of logic, with the notion of a language having nouns and statements, structures in which we may wonder if those formulas are satisfiable, and we can ask whether a formula or sentences are valid, that is model any structure with any variable-assignment function. We know learn how to string together deductions that will let us step-by-step deduce more complicated results by incrementally showing each step of the way a true statement that let's us deduce the next. In the beginning, we will start by defining the language of deductions (just like how language have yet to have any truth values). In general, a deductive system need not be *sound*, meaning that a deduction will be equivalent to a *logical implication* (i.e. if we deduce B from A, $A \vdash B$, then $A \models B$). This will be called the *soundness theorem*. We will see that the deductive systems we will create are sound: deductions give logical implications. Later in this chapter, theorem 2.6.31 gives the converse for sentences and the axiom system A1 to A8 with Modus Ponens: logical implication gives deduction. This is the *completeness theorem*.

One of the key properties of proof's is that they are (relatively) easy to follow, as compared to the usual difficulty there is in producing them. We would want confidence in being able to know that at the end of a proof, we know it's true and was easily checkable (if at least easy mechanically) to see that it's true. Hence, we be following these restrictions on our logical axioms and rules of inference:

1. There will be an algorithm (i.e., a mechanical procedure) that will decide, given a formula θ, whether or not θ is a logical axiom.
2. There will be an algorithm that will decide, given a finite set of formulas Γ and a formula θ, whether or not (Γ, θ) is a rule of inference.
3. For each rule of inference (Γ, θ), Γ will be a finite set of formulas.
4. Each logical axiom will be valid.
5. Our rules of inference will preserve truth. In other words, for each rule of inference (Γ, θ), $\Gamma \models \theta$.

2.1 Deductions

Fix a language $\mathcal{L}$. A deduction will be a finite sequence of formula's which we will consider to be legal steps to take letting us go from our assumptions (if any) to our conclusion. If each step of the sequence follows the rule of deduction, we will get a valid deduction. The legal actions we can take may come from one of three sources:

1. the *logical axioms*: these are a collection of formulas which are part of our deductive system that we may use in any deduction (note that the validity of them is not in question yet, they are simply syntactical formulas)
2. the *nonlogical axioms*: These are a collection of formulas which are given *for a given deduction*. They are the assumptions that are made for a particular deduction
3. the *rules of inference*: These are a collection of rules that will tell us when we are allowed to combine two pieces of information together to get the next step in our deduction.

For now, assume we have a fixed set of $\mathcal{L}$-formula Λ called the *logical axioms*, and set of ordered pairs (Γ, φ) called the *rules of inferences*. In the following sections, we will specify the formulas that belong to the logical axioms and the rules of inferences. The logical axioms are will be formulas that will be deemed to be "justified" in any deduction. Sometimes, we require more context, and so we will sometimes need a context-dependent set Σ to have the justification (thus, a deduction will be dependent on Σ). The rules of inferences will be a way of deducing the next step in a deduction given what was already deduced.

Definition 2.1.1: Deduction

Let Σ be a collection of $\mathcal{L}$-formulas and D a finite sequence of $\mathcal{L}$-formulas $(\varphi_1, \varphi_2, ..., \varphi_n)$. Then we will say that D is a *deduction* from Σ if for each i, $1 \leq i \leq n$, either

1. $\varphi_i \in \Lambda$ (that is φ_i is a logical axiom)
2. $\varphi_i \in \Sigma$ (that is φ_i is a nonlogical axiom)
3. there is a rule of inference (Γ, φ_i) such that $\Gamma \subseteq \{\varphi_1, \varphi_2, ..., \varphi_{i-1}\}$

I there is a deduction from Σ, the last line of which is the formula φ, we will call this a *deduction form* Σ *of* φ and write $\Sigma \vdash \varphi$

Depending on Λ and the rules of inferences, there may be many different types of deductive systems. Often, there is either many logical axioms with few rules of inferences, or many rules of inference with few logical axioms, though this is a rule of thumb.

Example 2.1.2: Deductions

1. Let $\mathcal{L} = \{P\}$ where P is a binary relation symbol. Let Σ be our set of axioms

$$\Sigma = \{\forall x P(x, x), P(u, v), P(u, v) \to P(v, u), P(v, u) \to P(u, u)\}$$

and let $\Lambda = \emptyset$ for now. For the rules of inferences, let it be

$$\{(\{\alpha, \alpha \to \beta\}, \beta) \mid \alpha \text{ and } \beta \text{ are formulas of } \mathcal{L}\}$$

which is the rule of modus ponens, that is the given the formulas α and $\alpha \to \beta$, we may conclude β (we will abbreviate this to **MP**). Now we can write a deduction from Σ of the formula $P(u,u)$, $\Sigma \vdash P(u,u)$ as follows

$$\begin{aligned} &P(u,v) && \in \Sigma \\ &P(u,v) \to P(v,u) && \in \Sigma \\ &P(v,u) && \textbf{MP},\ (1,2) \\ &P(v,u) \to P(u,u) && \in \Sigma \\ &P(u,u) && \textbf{MP},\ (3,4) \end{aligned}$$

and so, we have deduced $P(u,u)$. Note that we could not have deduce $\forall x P(x,x) \vdash P(u,u)$ since the symbol $\forall$ is still a formality here! We are very restricted in what symbols means, deductions at this stage are still linguistic manipulation rules.

2. Let $\Lambda = \Sigma = \emptyset$. Then given any language $\mathcal{L}$, is it possible to deduce something? In fact, it is not[a]! This captures intuitively the idea that we cannot make deductions without any assumptions.

[a]Once we get to rules of inference, if we allow any tautology for any formula φ once converted into propositional logic φ_p to be in a rule of inference ,then we can actually deduce more. This will be explored when we talk about rules of inference

Proposition 2.1.3: Universal set of Deduction for Σ

Let $\mathcal{L}$ be a language, fix sets of $\mathcal{L}$-formulas Σ and Λ, and a collection of rules of inference (Γ, θ), each with Γ finite as required at the start of this chapter. Then the set $Thm_\Sigma = \{\varphi \mid \Sigma \vdash \varphi\}$ is the smallest set C of $\mathcal{L}$-formulas such that

1. $\Sigma \subseteq C$
2. $\Lambda \subseteq C$
3. If (Γ, θ) is a rule of inference and $\Gamma \subseteq C$ then $\theta \in C$.

***Proof*:**
Deductions are as in definition 2.1.1. First, a formula $\sigma \in \Sigma$ has the one-line deduction (σ) from Σ, and a formula $\lambda \in \Lambda$ has the one-line deduction (λ). So $\Sigma \subseteq Thm_\Sigma$ and $\Lambda \subseteq Thm_\Sigma$. Now let (Γ, θ) be a rule of inference with $\Gamma \subseteq Thm_\Sigma$, and write $\Gamma = \{\gamma_1, \ldots, \gamma_k\}$, which is possible as Γ is finite. Choose deductions $D_1, \ldots, D_k$ from Σ whose last lines are $\gamma_1, \ldots, \gamma_k$, and form the sequence $D_1 D_2 \cdots D_k \theta$. Each entry of D_j keeps its justification in definition 2.1.1: that definition constrains an entry only through Λ, Σ, and the set of entries preceding it, and in the longer sequence an entry of D_j is preceded by every entry that preceded it in D_j. The last entry θ is justified by the rule (Γ, θ), because each γ_j is the last line of D_j and so precedes θ. If $k = 0$, the sequence is the single line θ, and the rule $(\emptyset, \theta)$ justifies it since $\emptyset$ is a subset of the (empty) set of preceding entries. Hence $\Sigma \vdash \theta$ and $\theta \in Thm_\Sigma$, so Thm_Σ satisfies (1) to (3).

Now let C be any set satisfying (1) to (3), let $\varphi \in Thm_\Sigma$, and let $(\varphi_1, \ldots, \varphi_n)$ be a deduction of φ from Σ. We show $\varphi_i \in C$ for every i, by strong induction on i. If $\varphi_i \in \Lambda$ or $\varphi_i \in \Sigma$, then

$\varphi_i \in C$ by (2) or (1). Otherwise there is a rule of inference (Γ, φ_i) with $\Gamma \subseteq \{\varphi_1, \ldots, \varphi_{i-1}\}$, and $\Gamma \subseteq C$ by the induction hypothesis, so $\varphi_i \in C$ by (3). In particular $\varphi = \varphi_n \in C$, so $Thm_\Sigma \subseteq C$. Thus Thm_Σ is the smallest set satisfying (1) to (3), as claimed.

Example 2.1.4: Theorem
Take the sentence $\varphi :\equiv \exists x \exists y((\neg x = y) \wedge \forall z(z = x \vee z = y))$. Every $\mathcal{L}$ structure satisfying this sentence must have exactly two elements in the universe. Hence $\{\varphi\}$ is a set of non-logical axioms for the collection of sets of cardinality two:

$$\{\mathfrak{M} \mid \mathfrak{M} \text{ is an } \mathcal{L}\text{-structure with exactly two elements}\}$$

2.2 The Logical Axioms

We know figure out what our logical axioms will be. There will be an infinite amount of them, but we can they will be decidable (a computer in a finite amount of time can decide whether a given formula φ is a logical axiom or not). These will be the axioms we will use for all our structures. For a particular structure, we will amend some more axioms via Σ. We will talk about those and rules of inference later, concentrating for now on logical axioms

2.2.1 Equality Axioms

In these notes, we have taken the approach for $=$ being part of any language. We will have three equality axioms. These axioms (in particular the last 2) are designed to allow substitution of equal for equals

1. The first simply says that any object is equal to itself

$$x = x \text{ for each variable } x \tag{2.1}$$

2. Assume that $x_1, x_2, ..., x_n$ are variable and $y_1, y_2, ..., y_n$ are variables and f is an n-ary function symbol. Then

$$[(x_1 = y_1) \wedge (x_2 = y_2) \wedge \cdots \wedge (x_n = y_n)] \to (f(x_1, x_2, ..., x_n) = f(y_1, y_2, ..., y_n)) \tag{2.2}$$

3. Similarly to the above except replace the function f with the relation R

$$[(x_1 = y_1) \wedge (x_2 = y_2) \wedge \cdots \wedge (x_n = y_n)] \to (R(x_1, x_2, ..., x_n) \to R(y_1, y_2, ..., y_n)) \tag{2.3}$$

Sometimes, $=$ is seen as a part of this part

2.2.2 Quantifier Axioms

The Quantifier axioms allow for easy substitution. In other words, if we have $\forall x P(x)$, and t is a term in our language, we would like to be able to state $P(t)$, or if we have formula where we can substitute t by x φ_t^x, then we want to say $\exists x \varphi$. Hence the following axioms:

1. (universal substantiation)

$$(\forall x\varphi) \to \varphi_t^x, \text{ if } t \text{ is substitutable for } x \text{ in } \varphi \tag{2.4}$$

2. (existential generalization)

$$\varphi_t^x \to (\exists x\varphi), \text{ if } t \text{ is substitutable for } x \text{ in } \varphi \tag{2.5}$$

2.3 Rules of Inference

There are two rules of inference: one for propositional consequences and one for quantifiers. The first rests on *propositional logic*, in the language $\mathcal{L}_P$ of definition 1.7.1: countably many propositional variables $A_0, A_1, \ldots$, also written $A, B, C, \ldots$, the connectives $\neg$ and $\to$, and no quantifiers, relation symbols, function symbols or constants. A truth assignment function v gives each propositional variable one of the two truth values T and F, and it extends to every formula by

$$\overline{v}(\varphi) = \begin{cases} v(\varphi) & \text{if } \varphi \text{ is a propositional variable} \\ F & \text{if } \varphi :\equiv (\neg\alpha) \text{ and } \overline{v}(\alpha) = T \\ F & \text{if } \varphi :\equiv (\alpha \to \beta),\ \overline{v}(\alpha) = T \text{ and } \overline{v}(\beta) = F \\ T & \text{otherwise} \end{cases}$$

The extension $\overline{v}$ satisfies the clauses of definition 1.7.3, and $\vee$, $\wedge$ and $\leftrightarrow$ are the abbreviations listed after definition 1.7.1.

With this, we have the following definition:

Definition 2.3.1: Tautology And Contradiction

Let φ be a propositional formula. Then φ Is a *tautology* if $\overline{v}(\varphi) = T$ for any truth assignment v, and a *contradiction* if $\overline{v}(\varphi) = F$ for any truth assignment v.

Propositional logic will be a useful simplification of first-order logic. We will show how to convert a $\mathcal{L}$-formula β into a (propositional) formula β_P

1. Find all subformulas of β of the form $\forall x\alpha$ that are not in the scope of another quantifier. Replace them with propositional variables in a systematic fashion. This means that if $\forall y Q(y, c)$ appears twice in β, it is replaced by the same letter both times, and distinct subformulas are replaced by distinct letters
2. Find all atomic formulas that remain, and replace them systematically with new propositional variables

For example, take

$$(\forall x P(x) \wedge Q(c, z)) \to (Q(c, z) \vee \forall x P(x))$$

Then this would become

$$(B \wedge A) \to (A \vee B)$$

Note that if β_P is a tautology, then β is valid, but the converse is false

Example 2.3.2: Valid First Order Formula Not Tautology

Let

$$\beta :\equiv [(\forall x)(\theta) \wedge (\forall x)(\theta \to \rho)] \to (\forall x)(\rho)$$

Then β is valid, but β_P would be $[A \wedge B] \to C$, which his certainly not a tautology.

Definition 2.3.3: Propositional Consequence for Propositional Logic

Let Γ_P be a set of propositional formulas and φ_P a propositional formula. Then φ_P is a *propositional consequence* of Γ_P if every truth assignment that makes each propositional formula in Γ_P true also makes φ_P true. Notice that φ_P is a tautology if and only if φ_P is a propositional consequence of $\emptyset$

Lemma 2.3.4: propositional Consequence Equivalent Formulation

If $\Gamma_P = \{(\gamma_1)_P, ..., (\gamma_n)_P\}$ is a nonempty finite set of propositional formulas and φ_P is a propositional formula, then φ_P is a propositional consequence of Γ_P if and only if

$$[(\gamma_1)_P \wedge (\gamma_2)_P \wedge \cdots \wedge (\gamma_n)_P] \to \varphi_P$$

is a tautology

***Proof*:**

Truth values are computed by the extension $\overline{v}$ of a truth assignment v displayed at the start of this section. By its third case, *(i)* $\overline{v}(\alpha \to \beta) = T$ exactly when $\overline{v}(\alpha) = F$ or $\overline{v}(\beta) = T$. The formula $\alpha \wedge \beta$ is $(\neg(\alpha \to (\neg\beta)))$, so by the second case $\overline{v}(\alpha \wedge \beta) = T$ exactly when $\overline{v}(\alpha \to (\neg\beta)) = F$, by the third case exactly when $\overline{v}(\alpha) = T$ and $\overline{v}(\neg\beta) = F$, and by the second case again exactly when $\overline{v}(\alpha) = T$ and $\overline{v}(\beta) = T$. So *(ii)* $\overline{v}(\alpha \wedge \beta) = T$ exactly when $\overline{v}(\alpha) = \overline{v}(\beta) = T$.

Write γ for the conjunction $[(\gamma_1)_P \wedge \cdots \wedge (\gamma_n)_P]$, grouped in any way, for instance to the right. We claim $\overline{v}(\gamma) = T$ exactly when $\overline{v}((\gamma_i)_P) = T$ for every $i \leq n$, by induction on $n \geq 1$. For $n = 1$ the conjunction is $(\gamma_1)_P$. For $n \geq 2$ it is $(\gamma' \wedge \gamma'')$, where γ' and γ'' are groupings of the first j and of the last $n - j$ conjuncts for some $1 \leq j < n$, and (ii) with the induction hypothesis for γ' and γ'' gives the claim. By (i), $\overline{v}(\gamma \to \varphi_P) = T$ exactly when $\overline{v}(\gamma) = F$ or $\overline{v}(\varphi_P) = T$.

Hence $\gamma \to \varphi_P$ is a tautology (definition 2.3.1) exactly when every truth assignment v has $\overline{v}(\gamma) = F$ or $\overline{v}(\varphi_P) = T$, that is, exactly when every truth assignment v with $\overline{v}((\gamma_i)_P) = T$ for all $i \leq n$ has $\overline{v}(\varphi_P) = T$. By definition 2.3.3, this last condition says that φ_P is a propositional consequence of Γ_P, as claimed.

Definition 2.3.5: Propositional Consequence For 1st Order Logic

Let Γ be a finite set of $\mathcal{L}$-formulas and φ is an $\mathcal{L}$-formula. Then φ is a *propositional consequence* of Γ if φ_P is a propositional consequence of Γ_P, where the formulas of $\Gamma \cup \{\varphi\}$ are translated together, with one system of letters.

Example 2.3.6: Propositional Consequence For 1st Order Logic
Let $\mathcal{L}$ be a language that contains two relations symbols P and Q. Let Γ be the set

$$\{\forall x P(x) \to \exists y Q(y), \exists y Q(y) \to P(x), \neg P(x) \leftrightarrow (y = z)\}$$

Let φ be the formula $\forall x P(x) \to \neg(y = z)$. The conversion procedure, applied to $\exists y Q(y)$ in its unabbreviated form $\neg\forall y \neg Q(y)$ from definition 1.1.5, replaces $\forall x P(x)$ by A, $\forall y \neg Q(y)$ by B, $P(x)$ by C and $y = z$ by D, which gives

$$\Gamma_P = \{A \to \neg B, \neg B \to C, \neg C \leftrightarrow D\}$$

and

$$\varphi_P :\equiv A \to \neg D$$

Let a truth assignment make every member of Γ_P true. If it gives A the value F, then $A \to \neg D$ has the value T. If it gives A the value T, then $A \to \neg B$ gives $\neg B$ the value T, then $\neg B \to C$ gives C the value T, so $\neg C$ has the value F, and $\neg C \leftrightarrow D$ gives D the value F by proposition 1.7.4(5). Then $\neg D$ and $A \to \neg D$ have the value T. Hence φ is a propositional consequence of Γ.

Definition 2.3.7: Rule Of Inference Of Type (PC)

Let Γ be a finite set of $\mathcal{L}$-formulas, φ is an $\mathcal{L}$-formula, and φ is a propositional consequence of Γ, then (Γ, φ) is a *rule of inference of type (PC)*.

The point behind the formalism is that we managed to prove, say γ_1, γ_2, and we proved that

$$[\gamma_1 \wedge \gamma_2 \to \varphi]_P$$

is a tautology, then we may conclude φ. Furthermore, if φ_P is a tautology, rule (PC) allows us to assert φ in any deduction by setting $\Gamma = \emptyset$.

2.3.1 Quantifier Rules

The idea behind Quantifier rules is simples: If we managed to prove "x is this property" without quantifying $x<$ then it is reasonable to say that we have proved "$(\forall x)x$ is this property". Similarly, if we proved something with the assumption of "x did this thing", then it is safe to say that we assume "$(\exists x)x$ did this thing", then we can still prove the same thing

Definition 2.3.8: Rules Of Inference Of Type (QR)

Suppose that the variable x is not free in the formula ψ. Then both of the following are *rules of inference of type (QR)*:

$$(\{\psi \to \varphi\}, \psi \to (\forall x \varphi))$$
$$(\{\varphi \to \psi\}, (\exists x \varphi) \to \psi)$$

What we are essentially saying is

1. From the formula $\psi \to \varphi$, we may deduce $\psi \to (\forall x \varphi)$

2. From the formula $\varphi \to \psi$, we may deduce $(\exists x\varphi) \to \psi$

2.4 Soundness

This deductive system is *sound*: every deduction records a genuine logical implication. Each logical axiom is valid and each rule of inference preserves truth, and soundness follows by induction on the length of a deduction.

Lemma 2.4.1: Substitution Lemma

Let φ be an $\mathcal{L}$-formula, x a variable, and t a term substitutable for x in φ. Then for every $\mathcal{L}$-structure $\mathfrak{A}$ and every assignment s into $\mathfrak{A}$,

$$\mathfrak{A} \models \varphi_t^x[s] \qquad \text{if and only if} \qquad \mathfrak{A} \models \varphi[s(x|\overline{s}(t))]$$

***Proof*:**
Write $s' = s(x|\overline{s}(t))$. We first record the corresponding term identity: for every term u, $\overline{s}(u_t^x) = \overline{s'}(u)$. This is a structural induction on u through definition 1.5.1 and definition 1.4.4. If u is x, then u_t^x is t and $\overline{s}(t) = s'(x) = \overline{s'}(u)$. If u is a variable other than x, then u_t^x is u and $\overline{s}(u) = s(u) = s'(u) = \overline{s'}(u)$. If u is a constant symbol c, then u_t^x is c and both sides are $c^{\mathfrak{A}}$. If $u :\equiv f u_1 \cdots u_n$, then the induction hypothesis gives

$$\overline{s}(u_t^x) = f^{\mathfrak{A}}(\overline{s}((u_1)_t^x), \ldots, \overline{s}((u_n)_t^x)) = f^{\mathfrak{A}}(\overline{s'}(u_1), \ldots, \overline{s'}(u_n)) = \overline{s'}(u)$$

Now the formula statement, by induction on the number of connectives and quantifiers of φ, for all assignments s at once.

- If φ is $u_1 = u_2$, then φ_t^x is $(u_1)_t^x = (u_2)_t^x$. By the equality clause and the term identity, $\mathfrak{A} \models \varphi_t^x[s]$ holds exactly when $\overline{s}((u_1)_t^x) = \overline{s}((u_2)_t^x)$, that is, when $\overline{s'}(u_1) = \overline{s'}(u_2)$, that is, when $\mathfrak{A} \models \varphi[s']$. If φ is $Ru_1 \cdots u_n$, then by the relation clause and the term identity $\mathfrak{A} \models \varphi_t^x[s]$ holds exactly when $(\overline{s'}(u_1), \ldots, \overline{s'}(u_n)) \in R^{\mathfrak{A}}$, that is, when $\mathfrak{A} \models \varphi[s']$.
- If $\varphi :\equiv (\neg\alpha)$, then φ_t^x is $(\neg\alpha_t^x)$ and t is substitutable for x in α. By the negation clause of definition 1.4.5 and the induction hypothesis, $\mathfrak{A} \models \varphi_t^x[s]$ holds exactly when $\mathfrak{A} \not\models \alpha_t^x[s]$, that is, when $\mathfrak{A} \not\models \alpha[s']$, that is, when $\mathfrak{A} \models \varphi[s']$.
- If $\varphi :\equiv (\alpha \to \beta)$, then φ_t^x is $(\alpha_t^x \to \beta_t^x)$ and t is substitutable for x in α and in β. By the implication clause and the induction hypothesis, $\mathfrak{A} \models \varphi_t^x[s]$ holds exactly when $\mathfrak{A} \not\models \alpha_t^x[s]$ or $\mathfrak{A} \models \beta_t^x[s]$, that is, when $\mathfrak{A} \not\models \alpha[s']$ or $\mathfrak{A} \models \beta[s']$, that is, when $\mathfrak{A} \models \varphi[s']$.
- Let $\varphi :\equiv (\forall y)(\alpha)$. If x is not free in φ, which includes the case that x is y, then φ_t^x is φ by lemma 1.5.7(1). The assignments s and s' differ at most at x, so they agree on the free variables of φ, and the claim is proposition 1.4.9. Otherwise x is free in φ, so x is not y, and the substitutability of t gives that y does not occur in t and that t is substitutable for x in α. Then φ_t^x is $(\forall y)(\alpha_t^x)$. Let $a \in A$. As y does not occur in t, lemma 1.4.8 gives $\overline{s(y|a)}(t) = \overline{s}(t)$, so the induction hypothesis for α at the assignment $s(y|a)$ gives

$$\mathfrak{A} \models \alpha_t^x[s(y|a)] \qquad \text{if and only if} \qquad \mathfrak{A} \models \alpha[s(y|a)(x|\overline{s}(t))]$$

As x is not y, the assignment $s(y|a)(x|\overline{s}(t))$ is $s'(y|a)$. Since this holds for every $a \in A$, the quantifier clause gives $\mathfrak{A} \models \varphi_t^x[s]$ if and only if $\mathfrak{A} \models \varphi[s']$, completing the proof.

Theorem 2.4.2: Soundness Of The Logical Axioms

Every logical axiom is valid.

***Proof*:**
Fix an $\mathcal{L}$-structure $\mathfrak{A}$ and an assignment s into $\mathfrak{A}$. Each logical axiom other than $x = x$ is an implication, and by the implication clause of definition 1.4.5 it holds under s as soon as its consequent holds under s whenever its antecedent does.

- The axiom $x = x$ holds under s by the equality clause, as both sides have the value $s(x)$.
- Suppose $\mathfrak{A} \models [(x_1 = y_1) \wedge \cdots \wedge (x_n = y_n)][s]$. By proposition 1.4.6(2), applied $n - 1$ times, and the equality clause, $s(x_i) = s(y_i)$ for every i. Then

  $$\overline{s}(f(x_1, \ldots, x_n)) = f^{\mathfrak{A}}(s(x_1), \ldots, s(x_n)) = f^{\mathfrak{A}}(s(y_1), \ldots, s(y_n)) = \overline{s}(f(y_1, \ldots, y_n))$$

  so the function axiom holds under s. The tuples $(s(x_1), \ldots, s(x_n))$ and $(s(y_1), \ldots, s(y_n))$ coincide, so one lies in $R^{\mathfrak{A}}$ exactly when the other does, and the relation axiom holds under s by the relation clause and the implication clause.
- Let t be substitutable for x in φ, and write $s' = s(x|\overline{s}(t))$. If $\mathfrak{A} \models (\forall x\varphi)[s]$, then $\mathfrak{A} \models \varphi[s(x|a)]$ for every $a \in A$, in particular $\mathfrak{A} \models \varphi[s']$, which by lemma 2.4.1 is $\mathfrak{A} \models \varphi_t^x[s]$. If $\mathfrak{A} \models \varphi_t^x[s]$, then lemma 2.4.1 gives $\mathfrak{A} \models \varphi[s(x|m)]$ for $m = \overline{s}(t)$, and proposition 1.4.13 gives $\mathfrak{A} \models (\exists x\varphi)[s]$. So both quantifier axioms hold under s.

As $\mathfrak{A}$ and s were arbitrary, every logical axiom is valid.

Theorem 2.4.3: Soundness Of The Rules Of Inference

If (Γ, θ) is a rule of inference, then $\Gamma \models \theta$.

***Proof*:**
By definition 1.6.1 and definition 1.4.11, the claim is that a structure $\mathfrak{A}$ with $\mathfrak{A} \models \gamma[s]$ for every $\gamma \in \Gamma$ and every assignment s has $\mathfrak{A} \models \theta[s]$ for every assignment s. Fix such an $\mathfrak{A}$ and an assignment s.

Suppose first that (Γ, θ) is of type (PC), so θ_P is a propositional consequence of Γ_P, the formulas of $\Gamma \cup \{\theta\}$ being translated with one system of letters. Let v give a letter the value T exactly when $\mathfrak{A} \models \chi[s]$ for the first-order formula χ that the letter replaces, and give the unused propositional variables the value F. The translation of a formula is computed recursively: a formula that is atomic or of the form $(\forall y)(\alpha)$ becomes its letter, $(\neg\alpha)$ becomes $(\neg\alpha_P)$, and $(\alpha \to \beta)$ becomes $(\alpha_P \to \beta_P)$. Induction along this recursion gives $\overline{v}(\chi_P) = T$ exactly when $\mathfrak{A} \models \chi[s]$. For a letter this is the definition of v, and for $\neg$ and $\to$ the cases of $\overline{v}$ displayed at the start of section 2.3 match the negation and implication clauses. Hence $\overline{v}(\gamma_P) = T$ for every $\gamma \in \Gamma$, so $\overline{v}(\theta_P) = T$,

that is, $\mathfrak{A} \models \theta[s]$.

Suppose instead that (Γ, θ) is of type (QR), with x not free in ψ. For every $a \in A$ the assignment $s(x|a)$ agrees with s on the free variables of ψ, so $\mathfrak{A} \models \psi[s(x|a)]$ exactly when $\mathfrak{A} \models \psi[s]$, by proposition 1.4.9.

- Let $\Gamma = \{\psi \to \varphi\}$ and $\theta :\equiv \psi \to (\forall x\varphi)$, and suppose $\mathfrak{A} \models \psi[s]$. For every $a \in A$, $\mathfrak{A} \models \psi[s(x|a)]$ and $\mathfrak{A} \models (\psi \to \varphi)[s(x|a)]$, so the implication clause gives $\mathfrak{A} \models \varphi[s(x|a)]$. Hence $\mathfrak{A} \models (\forall x\varphi)[s]$, and $\mathfrak{A} \models \theta[s]$ by the implication clause of definition 1.4.5.
- Let $\Gamma = \{\varphi \to \psi\}$ and $\theta :\equiv (\exists x\varphi) \to \psi$, and suppose $\mathfrak{A} \models (\exists x\varphi)[s]$. By proposition 1.4.13 there is $a \in A$ with $\mathfrak{A} \models \varphi[s(x|a)]$. As $\mathfrak{A} \models (\varphi \to \psi)[s(x|a)]$, the implication clause gives $\mathfrak{A} \models \psi[s(x|a)]$, so $\mathfrak{A} \models \psi[s]$, and $\mathfrak{A} \models \theta[s]$ by the implication clause.

In each case $\mathfrak{A} \models \theta[s]$, completing the proof.

Theorem 2.4.4: Soundness Theorem

If $\Sigma \vdash \varphi$ then $\Sigma \models \varphi$.

***Proof*:**
Let $(\varphi_1, \ldots, \varphi_n)$ be a deduction of $\varphi = \varphi_n$ from Σ, and fix a structure $\mathfrak{A}$ with $\mathfrak{A} \models \Sigma$, that is, $\mathfrak{A} \models \sigma[s]$ for every $\sigma \in \Sigma$ and every assignment s (definition 1.4.11). We show $\mathfrak{A} \models \varphi_i$ for every i, by strong induction on i. If φ_i is a logical axiom, it is valid by theorem 2.4.2. If $\varphi_i \in \Sigma$, then $\mathfrak{A} \models \varphi_i$ by assumption. If φ_i arises from a rule (Γ, φ_i) with $\Gamma \subseteq \{\varphi_1, \ldots, \varphi_{i-1}\}$, then $\mathfrak{A} \models \gamma$ for every $\gamma \in \Gamma$ by the induction hypothesis, so $\mathfrak{A} \models \varphi_i$ by theorem 2.4.3. In particular $\mathfrak{A} \models \varphi$, and as $\mathfrak{A}$ was arbitrary, $\Sigma \models \varphi$ by definition 1.6.1.

2.5 Propositional Logic: a Warm-Up

We take a moment to develop a different deductive system that is still sound. This deductive system will be for propositional logic. For any language $\mathcal{L}$, we will label $\mathcal{L}_p$ the propositional language induced by $\mathcal{L}$. in this section, the term formula will refer to *propositional formula*.

Definition 2.5.1: Axiom Schema

For any formulas' $\alpha, \beta, \gamma \in \mathcal{L}_P$, the following are valid axioms:

A1 $(\alpha \to (\beta \to \alpha))$

A2 $((\alpha \to (\beta \to \gamma)) \to ((\alpha \to \beta) \to (\alpha \to \gamma)))$

A3 $(((\neg\beta) \to (\neg\alpha)) \to (((\neg\beta) \to \alpha) \to \beta))$

These three are called the *axiom schemas*, and any particular choice of formulas gives an *axiom* of $\mathcal{L}_P$

this is the equivalent to the logical axioms Λ. Every axiom of $\mathcal{L}_P$ is a tautology and Modus Ponens preserves truth (proposition 2.5.3, proved below), which together make this system sound.

The rule of inference we will have is simply Modus Ponens

Definition 2.5.2: Modus Ponens (MP)

Given any formula φ and $(\varphi \to \psi)$, then we may infer ψ, i.e.

$$(\{\varphi, (\varphi \to \psi)\}, \psi)$$

is our rule of inference.

In this section a deduction is as in definition 2.1.1, with the instances of A1, A2, A3 of definition 2.5.1 as the logical axioms and Modus Ponens as the rule of inference.

It is important for future reference to box the following result:

Proposition 2.5.3: MP and axiom Satisfies Logical Implication

The logical axioms are tautologies, and

$$\{\varphi, (\varphi \to \psi)\} \models \psi$$

Do not forget that as of yet, MP is just a rule by which we can add further formulas to our deduction and does not have a semantic meaning. If you wish, you can think that it simply happens that MP satisfies the above formula

***Proof*:**
The logical axioms here are the instances of A1, A2, A3 in definition 2.5.1, and truth assignments are those of definition 1.7.3, under which $v(\alpha \to \beta) = F$ exactly when $v(\alpha) = T$ and $v(\beta) = F$. Each axiom is a tautology in the sense of definition 1.7.6, because a truth assignment v giving it the value F leads to a contradiction in each case.

- A1: $v(\alpha \to (\beta \to \alpha)) = F$ requires $v(\alpha) = T$ and $v(\beta \to \alpha) = F$, and the latter requires $v(\alpha) = F$.
- A2: v of the axiom is F only if $v(\alpha \to (\beta \to \gamma)) = T$ and $v((\alpha \to \beta) \to (\alpha \to \gamma)) = F$. The latter requires $v(\alpha \to \beta) = T$ and $v(\alpha \to \gamma) = F$, so $v(\alpha) = T$ and $v(\gamma) = F$, and then $v(\alpha \to \beta) = T$ forces $v(\beta) = T$. Now $v(\beta \to \gamma) = F$, and with $v(\alpha) = T$ this gives $v(\alpha \to (\beta \to \gamma)) = F$.
- A3: v of the axiom is F only if $v((\neg\beta) \to (\neg\alpha)) = T$ and $v(((\neg\beta) \to \alpha) \to \beta) = F$. The latter requires $v((\neg\beta) \to \alpha) = T$ and $v(\beta) = F$, so $v(\neg\beta) = T$. Then $v((\neg\beta) \to \alpha) = T$ forces $v(\alpha) = T$, while $v((\neg\beta) \to (\neg\alpha)) = T$ forces $v(\neg\alpha) = T$, that is, $v(\alpha) = F$.

For the second claim, let v satisfy $\{\varphi, (\varphi \to \psi)\}$. If $v(\psi) = F$, then $v(\varphi) = T$ would give $v(\varphi \to \psi) = F$, which is not the case. So $v(\psi) = T$, and $\{\varphi, (\varphi \to \psi)\} \models \psi$ by definition 1.7.7, completing the proof.

Example 2.5.4: Deductions In Propositional Logic

1. We will show $\vdash \varphi \to \varphi$

$(\varphi \to ((\varphi \to \varphi) \to \varphi)) \to ((\varphi \to (\varphi \to \varphi)) \to (\varphi \to \varphi))$	A2
$\varphi \to ((\varphi \to \varphi) \to \varphi)$	A1
$(\varphi \to (\varphi \to \varphi)) \to (\varphi \to \varphi)$	1, 2 MP
$\varphi \to (\varphi \to \varphi)$	A1
$\varphi \to \varphi$	3,4 MP

2. We will next show transitivity: $\{\alpha \to \beta, \beta \to \gamma\} \vdash \alpha \to \gamma$:

$(\beta \to \gamma) \to (\alpha \to (\beta \to \gamma))$	A1
$\beta \to \gamma$	Premise
$\alpha \to (\beta \to \gamma)$	1,2 MP
$(\alpha \to (\beta \to \gamma)) \to ((\alpha \to \beta) \to (\alpha \to \gamma))$	A2
$(\alpha \to \beta) \to (\alpha \to \gamma)$	4,3 MP
$\alpha \to \beta$	Premise
$\alpha \to \gamma$	5,6 MP

   Hence, $\{\alpha \to \beta, \beta \to \gamma\} \vdash \alpha \to \gamma$, completing the proof.

3. Since we have shown the above two deductions, which is simply a finite length of sequences, we may always simply add in the deduction of them into any other deduction. For example, we will show $\vdash (\neg\alpha \to \alpha) \to \alpha$

$(\neg\alpha \to \neg\alpha) \to ((\neg\alpha \to \alpha) \to \alpha)$	A3
$(\neg\alpha \to \neg\alpha)$	first example
$(\neg\alpha \to \alpha) \to \alpha$	1,2 MP

   completing the proof.

4. Another useful one to have is $\{\alpha \to (\beta \to \gamma), \beta\} \vdash \alpha \to \gamma$:

$(\alpha \to (\beta \to \gamma)) \to ((\alpha \to \beta) \to (\alpha \to \gamma))$	A2
$\alpha \to (\beta \to \gamma)$	Premise
$(\alpha \to \beta) \to (\alpha \to \gamma)$	1,2 MP
$\beta \to (\alpha \to \beta)$	A1
β	Premise
$\alpha \to \beta$	4,5 MP
$\alpha \to \gamma$	3,6 MP

   completing the proof.

5. For $\vdash \alpha \vee \neg\alpha$, remember to first convert to $\neg a \to b$, which in this case is $(\neg\alpha) \to \neg\alpha$, which we have shown is deducible in example 1.

6. A result that combines many of the above results is the following: $\vdash \neg\neg\beta \to \beta$

$(\neg\beta \to \neg\neg\beta) \to ((\neg\beta \to \neg\beta) \to \beta)$	A3
$(\neg\neg\beta \to (\neg\beta \to \neg\neg\beta))$	A1
$\neg\neg\beta \to ((\neg\beta \to \neg\beta) \to \beta)$	1,2 and example 2
$\neg\beta \to \neg\beta$	example 1
$\neg\neg\beta \to \beta$	3,4 example 4

completing the proof.

There are many important shortcuts that one ought to know

Lemma 2.5.5: Deduction Shortcuts

1. If $(\varphi_1, \varphi_2, ..., \varphi_n)$ is a deduction of $\mathcal{L}_P$, then $\varphi_1, \varphi_2, ..., \varphi_l$ is a deduction of $\mathcal{L}_P$, $1 \leq l \leq n$
2. If $\Gamma \vdash \delta$, $\Gamma \vdash \delta \to \beta$, then $\Gamma \vdash \beta$
3. If $\Gamma \subseteq \Delta$ and $\Gamma \vdash \alpha$, then $\Delta \vdash \alpha$
4. If $\Gamma \vdash \Delta$, $\Delta \vdash \sigma$, then $\Gamma \vdash \sigma$

***Proof*:**

1. Let $(\varphi_1, \ldots, \varphi_n)$ be a deduction from a set Γ and let $1 \leq l \leq n$. The condition of definition 2.1.1 on an entry φ_i mentions only the logical axioms, Γ, and the entries $\varphi_1, \ldots, \varphi_{i-1}$. For $i \leq l$ these are the same in $(\varphi_1, \ldots, \varphi_l)$ as in the full sequence, so $(\varphi_1, \ldots, \varphi_l)$ is a deduction from Γ.
2. Let $(\alpha_1, \ldots, \alpha_p)$ and $(\beta_1, \ldots, \beta_q)$ be deductions from Γ with $\alpha_p :\equiv \delta$ and $\beta_q :\equiv \delta \to \beta$. In the sequence $(\alpha_1, \ldots, \alpha_p, \beta_1, \ldots, \beta_q, \beta)$ each α_i and each β_j keeps its justification, because every entry that preceded it in its own deduction still precedes it. The last entry β follows by Modus Ponens from the earlier entries δ and $\delta \to \beta$. So the sequence is a deduction of β from Γ, and $\Gamma \vdash \beta$.
3. The only clause of definition 2.1.1 that mentions the nonlogical axioms is $\varphi_i \in \Gamma$, and $\Gamma \subseteq \Delta$ turns it into $\varphi_i \in \Delta$. So a deduction of α from Γ is a deduction of α from Δ.
4. Here $\Gamma \vdash \Delta$ means $\Gamma \vdash \delta$ for every $\delta \in \Delta$. Let $(\varphi_1, \ldots, \varphi_n)$ be a deduction of σ from Δ. We show $\Gamma \vdash \varphi_i$ for every i, by strong induction on i. If φ_i is a logical axiom, the one-line sequence (φ_i) is a deduction from Γ. If $\varphi_i \in \Delta$, then $\Gamma \vdash \varphi_i$ by hypothesis. If φ_i follows by Modus Ponens from φ_u and $\varphi_w :\equiv \varphi_u \to \varphi_i$ with $u, w < i$, then $\Gamma \vdash \varphi_u$ and $\Gamma \vdash \varphi_u \to \varphi_i$ by the induction hypothesis, and $\Gamma \vdash \varphi_i$ by item (2). For $i = n$ this gives $\Gamma \vdash \sigma$, completing the proof.

An important result is that two opposite formulas can deduce anything:

Theorem 2.5.6: Contradiction Deduces Anything

For any δ and γ

$$\{\delta, \neg\delta\} \vdash \gamma$$

showing that having (intuitively) a premise and the negation of the premise can imply anything

***Proof*:**

(1)	$\neg\delta \to (\neg\gamma \to \neg\delta)$	A1
(2)	$\neg\delta$	Premise
(3)	$\neg\gamma \to \neg\delta$	1,2 MP
(4)	$\delta \to (\neg\gamma \to \delta)$	A1
(5)	δ	Premise
(6)	$\neg\gamma \to \delta$	4,5 MP
(7)	$(\neg\gamma \to \neg\delta) \to ((\neg\gamma \to \delta) \to \gamma)$	A3
(8)	$(\neg\gamma \to \delta) \to \gamma$	3,7 MP
(9)	γ	6, 8 MP

One important simplification that is important but more complicated to prove is the following

Theorem 2.5.7: Deduction Theorem

If Σ is any set of formulas and α and β are any formulas, then $\Sigma \vdash \alpha \to \beta$ if and only if $\Sigma \cup \{\alpha\} \vdash \beta$

***Proof*:**
Suppose first that $\Sigma \vdash \alpha \to \beta$. Then $\Sigma \cup \{\alpha\} \vdash \alpha \to \beta$ by lemma 2.5.5(3), and $\Sigma \cup \{\alpha\} \vdash \alpha$ by the one-line deduction (α), so $\Sigma \cup \{\alpha\} \vdash \beta$ by lemma 2.5.5(2).

Conversely, let $(\varphi_1, \ldots, \varphi_n)$ be a deduction of β from $\Sigma \cup \{\alpha\}$. We show $\Sigma \vdash \alpha \to \varphi_i$ for every i, by strong induction on i.

- If φ_i is a logical axiom or a member of Σ, then $(\varphi_i,\ \varphi_i \to (\alpha \to \varphi_i),\ \alpha \to \varphi_i)$ is a deduction from Σ, the middle formula being an instance of A1 and the last following from the first two by Modus Ponens.
- If φ_i is α, then $\vdash \alpha \to \alpha$ by example 2.5.4(1), so $\Sigma \vdash \alpha \to \alpha$ by lemma 2.5.5(3).
- If φ_i follows by Modus Ponens from φ_j and $\varphi_k :\equiv (\varphi_j \to \varphi_i)$ with $j, k < i$, then $\Sigma \vdash \alpha \to \varphi_j$ and $\Sigma \vdash \alpha \to (\varphi_j \to \varphi_i)$ by the induction hypothesis. The formula

  $$(\alpha \to (\varphi_j \to \varphi_i)) \to ((\alpha \to \varphi_j) \to (\alpha \to \varphi_i))$$

  is an instance of A2, so it has a one-line deduction from Σ, and two applications of lemma 2.5.5(2) give $\Sigma \vdash \alpha \to \varphi_i$.

For $i = n$ this is $\Sigma \vdash \alpha \to \beta$, completing the proof.

Using this theorem, we can make some results easier:

Example 2.5.8: Using Deduction Theorem

1. By theorem 2.5.7, $\vdash \varphi \to \varphi$ if and only if $\{\varphi\} \vdash \varphi$, and the one-line sequence (φ) is a deduction of φ from $\{\varphi\}$.

2. By example 2.5.4(6), $\vdash \neg\neg\alpha \to \alpha$ for every formula α. With the deduction given there for $\alpha :\equiv (\neg\beta)$ inserted in place of line (1), the following is a deduction of $\neg\neg\beta$ from $\{\beta\}$, so $\beta \vdash \neg\neg\beta$, and $\vdash \beta \to \neg\neg\beta$ by theorem 2.5.7.

(1) $\neg\neg(\neg\beta) \to \neg\beta$	example 2.5.4(6)
(2) $(\neg\neg(\neg\beta) \to \neg\beta) \to ((\neg\neg\neg\beta \to \beta) \to \neg\neg\beta)$	A3
(3) $(\neg\neg\neg\beta \to \beta) \to \neg\neg\beta$	1,2 MP
(4) $\beta \to (\neg\neg\neg\beta \to \beta)$	A1
(5) β	Premise
(6) $(\neg\neg\neg\beta \to \beta)$	4,5 MP
(7) $\neg\neg\beta$	3,6 MP

3. $\neg\neg a \to b \vdash a \to b$. By the deduction theorem, it suffices to show $\{\neg\neg a \to b, a\} \vdash b$, and item (2) gives $a \vdash \neg\neg a$, so

a	premise
$\neg\neg a$	$a \vdash \neg\neg a$
$\neg\neg a \to b$	premise
b	MP

completing the proof. In the other direction, $a \to b \vdash \neg\neg a \to b$. By the deduction theorem, it suffices to show $\{a \to b, \neg\neg a\} \vdash b$. With the deduction of $\neg\neg a \to a$ from example 2.5.4(6) inserted in place of its second line,

$\neg\neg a$	premise
$\neg\neg a \to a$	example 2.5.4(6)
a	MP
$a \to b$	premise
b	MP

is such a deduction.

Another important result is Modus tollens

Lemma 2.5.9: Modus Tollens (MT)

$$\neg\beta \to \neg\alpha \vdash \alpha \to \beta \qquad \text{and} \qquad \alpha \to \beta \vdash \neg\beta \to \neg\alpha$$

***Proof*:**
For the first, by the deduction theorem (theorem 2.5.7) it suffices to show

$$\{\neg\beta \to \neg\alpha, \alpha\} \vdash \beta$$

and the following is such a deduction.

(1) $(\neg\beta \to \neg\alpha) \to ((\neg\beta \to \alpha) \to \beta)$	A3
(2) $(\neg\beta \to \neg\alpha)$	Premise
(3) $((\neg\beta \to \alpha) \to \beta)$	1,2 MP
(4) $\alpha \to (\neg\beta \to \alpha)$	A1
(5) α	Premise
(6) $\neg\beta \to \alpha$	4,5 MP
(7) β	3,6 MP

For the second, by the deduction theorem it suffices to show

$$\{\alpha \to \beta, \neg\beta\} \vdash \neg\alpha$$

The following is such a deduction once the deductions of example 2.5.8(3), which pass from $a \to b$ to $\neg\neg a \to b$, are inserted at lines (4) and (8).

(1)	$\neg\beta \to (\alpha \to \neg\beta)$	A1
(2)	$\neg\beta$	Premise
(3)	$\alpha \to \neg\beta$	1,2 MP
(4)	$(\neg\neg\alpha \to \neg\beta)$	3, example 2.5.8(3)
(5)	$(\neg\neg\alpha \to \neg\beta) \to ((\neg\neg\alpha \to \beta) \to \neg\alpha)$	A3
(6)	$(\neg\neg\alpha \to \beta) \to \neg\alpha$	4,5 MP
(7)	$(\alpha \to \beta)$	Premise
(8)	$(\neg\neg\alpha \to \beta)$	7, example 2.5.8(3)
(9)	$\neg\alpha$	6,8 MP

By the deduction theorem, $\alpha \to \beta \vdash \neg\beta \to \neg\alpha$, completing the proof.

Corollary 2.5.10: Consequences Of MT

1. $\alpha \vdash \beta$ if and only if $\neg\beta \vdash \neg\alpha$
2. $\{\alpha \wedge \beta\} \vdash \beta \wedge \alpha$
3. $(\beta \to \neg\alpha) \vdash (\alpha \to \neg\beta)$
4. $(\neg\beta \to \alpha) \vdash (\neg\alpha \to \beta)$ and $(\neg\alpha \to \beta) \vdash (\neg\beta \to \alpha)$
5. $\sigma \vdash (\sigma \vee \tau)$
6. $\{\alpha \wedge \beta\} \vdash \beta$
7. $\{\alpha \wedge \beta\} \vdash \alpha$

***Proof*:**

1. By the deduction theorem, $a \vdash b$ if and only if $\vdash a \to b$. By MT $a \to b \vdash \neg b \to \neg a$. Thus, by transitivity, we have $\vdash \neg b \to \neg a$. By the deduction theorem $\vdash \neg b \to \neg a$ if and only if $\neg b \vdash \neg a$, showing one side of the implication. For the other side, we use the other side of MT to get $\neg b \to \neg a \vdash a \to b$. Hence:

$$a \vdash b \;\Leftrightarrow \vdash a \to b \;\Leftrightarrow \vdash \neg b \to \neg a \;\Leftrightarrow\; \neg b \vdash \neg a$$

   Giving us that $a \vdash b$ if and only if $\neg b \vdash \neg a$, completing the lemma.

2. Recall that $\alpha \wedge \beta :\equiv \neg(\alpha \to (\neg\beta))$, so the claim is $\neg(\alpha \to (\neg\beta)) \vdash \neg(\beta \to (\neg\alpha))$. By item (1), with $\beta \to \neg\alpha$ in place of α and $\alpha \to \neg\beta$ in place of β, it suffices to show $(\beta \to \neg\alpha) \vdash (\alpha \to \neg\beta)$, which is item (3). We prove item (3) here. The second deduction of lemma 2.5.9 gives

$$\beta \to \neg\alpha \vdash \neg\neg\alpha \to \neg\beta$$

   and example 2.5.8(3) gives $\neg\neg\alpha \to \neg\beta \vdash \alpha \to \neg\beta$. By lemma 2.5.5(4), $\beta \to \neg\alpha \vdash \alpha \to \neg\beta$, as we sought to show.

3. This is the deduction obtained in the proof of item (2).

4. The second deduction of lemma 2.5.9, with $\neg\beta$ in place of α and α in place of β, gives $(\neg\beta \to \alpha) \vdash (\neg\alpha \to \neg\neg\beta)$. The transitivity of example 2.5.4(2) gives $\{\neg\alpha \to \neg\neg\beta, \neg\neg\beta \to \beta\} \vdash \neg\alpha \to \beta$, and $\vdash \neg\neg\beta \to \beta$ by example 2.5.4(6). So $(\neg\beta \to \alpha)$ deduces both members of $\{\neg\alpha \to \neg\neg\beta, \neg\neg\beta \to \beta\}$, using lemma 2.5.5(3) for the second, and lemma 2.5.5(4) gives $(\neg\beta \to \alpha) \vdash (\neg\alpha \to \beta)$. The second deduction is the first with α and β exchanged.

5. The formula $\sigma \vee \tau$ is $(\neg\sigma) \to \tau$. By theorem 2.5.6, $\{\sigma, \neg\sigma\} \vdash \tau$, so $\{\sigma\} \vdash (\neg\sigma) \to \tau$ by theorem 2.5.7.

6. The formula $\alpha \wedge \beta$ is $\neg(\alpha \to \neg\beta)$. The instance $\neg\beta \to (\alpha \to \neg\beta)$ of A1 and Modus Ponens give $\neg\beta \vdash \alpha \to \neg\beta$, so item (1) gives $\neg(\alpha \to \neg\beta) \vdash \neg\neg\beta$. As $\vdash \neg\neg\beta \to \beta$ by example 2.5.4(6), Modus Ponens and lemma 2.5.5(4) give $\neg(\alpha \to \neg\beta) \vdash \beta$.

7. By theorem 2.5.6, $\{\neg\alpha, \alpha\} \vdash \neg\beta$, so $\neg\alpha \vdash \alpha \to \neg\beta$ by theorem 2.5.7. Item (1) gives $\neg(\alpha \to \neg\beta) \vdash \neg\neg\alpha$, and as $\vdash \neg\neg\alpha \to \alpha$ by example 2.5.4(6), Modus Ponens and lemma 2.5.5(4) give $\neg(\alpha \to \neg\beta) \vdash \alpha$, completing the proof.

2.5.1 Soundness and Completeness For Propositional Logic

With that established, we may ask about the soundness of this deductive system, in particular if Δ is a set of formulas and α is a formula such that $\Delta \vdash \alpha$, then $\Delta \models \alpha$. The soundness theorem is relatively easy to prove

Theorem 2.5.11: Soundness Theorem

Let Δ be a set of formulas and α a formula such that $\Delta \vdash \alpha$. Then $\Delta \models \alpha$

***Proof*:**
We will proceed by induction on the shortest length deduction from Δ to α. If $n = 1$, then that either means α is an axiom or $\alpha \in \Delta$. An axiom is a tautology by proposition 2.5.3, and a member of Δ is satisfied by every truth assignment that satisfies Δ. Thus, if $v(\Delta) = T$, then $v(\alpha) = T$, and so $\Delta \models \alpha$

Assume now that $\Delta \vdash \beta$ then $\Delta \models \beta$ for any deduction of length $k < n$, and consider $\Delta \vdash \alpha$ has shortest deduction of length $n > 1$. Then it must be that we conclude α by Modus Ponens, since if it were by an axiom or if $\alpha \in \Delta$, the deduction would be of length 1. We thus must have

$$\varphi_i :\equiv \gamma \qquad \varphi_j :\equiv \gamma \to \alpha \qquad \text{for some } i, j < n$$

Then by modus ponens we'd get $\varphi_n :\equiv \alpha$. Now, since $\Delta \vdash \gamma$ and $\Delta \vdash \gamma \to \alpha$, by our induction hypothesis we know that $\Delta \models \gamma$ and $\Delta \models \gamma \to \alpha$. Since:

$$v(\gamma \to \alpha) = \begin{cases} F & \text{if } v(\gamma) = T \text{ and } v(\alpha) = F \\ T & \text{otherwise} \end{cases}$$

and $v(\gamma) = v(\gamma \to \alpha) = T$, it must be that $v(\alpha) = T$, implying $\Delta \models \alpha$. Thus, by the principle of induction, $\Delta \vdash \alpha$ then $\Delta \models \alpha$, completing the proof.

We start working towards the completeness theorem. We require some more definitions and lemma

Definition 2.5.12: Inconsistent

Let Γ be a set of formulas. Then Γ is *inconsistent* if $\Gamma \vdash \neg(\alpha \to \alpha)$ for some formula α. Γ is *consistent* if it is not inconsistent

The main result we will show is that satisfiability and consistency are "equivalent" for propositional logic which will lead us to the completeness theorem:

Lemma 2.5.13: Satisfiable, then Consistent

Let Σ be a set of satisfiable formulas (so that there is a v such that $v(\Sigma) = T$). Then Σ is consistent (so $\Sigma \nvdash \neg(\alpha \to \alpha)$)

***Proof*:**
Let v be a truth assignment satisfying Σ, and suppose $\Sigma \vdash \neg(\alpha \to \alpha)$ for some formula α. By theorem 2.5.11, $\Sigma \models \neg(\alpha \to \alpha)$, so $v(\neg(\alpha \to \alpha)) = T$, and $v(\alpha \to \alpha) = F$ by the negation clause of definition 1.7.3. By the implication clause, this requires $v(\alpha) = T$ and $v(\alpha) = F$ at once, which is impossible. Hence Σ is consistent.

For the converse, we would require to show that if Σ is consistent, we can find a truth assignment that satisfies it. We will build up to exactly this result, which will be they key to the soundness theorem. We first explore some property of inconsistent and consistent sets:

Lemma 2.5.14: Inconsistent Can Deduce Anything

Suppose that Δ is an inconsistent set of formulas. Then $\Delta \vdash \psi$ for any formula ψ

***Proof*:**
Since Δ is inconsistent, $\Delta \vdash \neg(\alpha \to \alpha)$ for some formula α. Let ψ be any formula. As $\vdash \alpha \to \alpha$ by example 2.5.4(1), $\Delta \vdash \alpha \to \alpha$ by lemma 2.5.5(3). By theorem 2.5.6, with $\delta :\equiv (\alpha \to \alpha)$ and $\gamma :\equiv \psi$, we have $\{\alpha \to \alpha, \neg(\alpha \to \alpha)\} \vdash \psi$. Since Δ deduces both members of this set, lemma 2.5.5(4) gives $\Delta \vdash \psi$, completing the proof.

Proposition 2.5.15: Inconsistent, Then Only Requires Finitely Many Formulas

Suppose Σ is an inconsistent set of formulas. Then there is a finite subset Δ of Σ such that Δ is inconsistent

***Proof*:**
Suppose that Σ is inconsistent so that $\Sigma \vdash \neg(\alpha \to \alpha)$. Then by definition of a deduction a finite set $\Delta \subseteq \Sigma$ is used in the deduction. But then $\Delta \vdash \neg(\alpha \to \alpha)$, completing the proof.

Corollary 2.5.16: Finite Subcondition For Consistency

A set of formulas Γ is consistent if and only if every finite subset of Γ is consistent

***Proof*:**
If Γ is inconsistent, then a finite subset is inconsistent by proposition 2.5.15. Conversely, suppose one subset $\Delta \subseteq \Gamma$ were inconsistent, meaning $\Delta \vdash \neg(\alpha \to \alpha)$. Then $\Gamma \vdash \neg(\alpha \to \alpha)$ by lemma 2.5.5(3), so Γ is inconsistent. The two directions together give the claim, completing the proof.

Finally, we have one more definition needed which will allow us to make consistency imply satisfiability

Definition 2.5.17: Maximally Consistent

A set of formulas Σ is *maximally consistent* if Σ is consistent but $\Sigma \cup \{\varphi\}$ is inconsistent for any $\varphi \notin \Sigma$

Such sets exist: if v is a truth assignment, then $\Sigma = \{\varphi \mid v(\varphi) = T\}$ is maximally consistent. Lemma 2.5.13 shows that it is consistent. If $\varphi \notin \Sigma$, then $v(\varphi) = F$, so $v(\neg\varphi) = T$ and $\neg\varphi \in \Sigma$. Then $\Sigma \cup \{\varphi\}$ contains φ and $\neg\varphi$, and theorem 2.5.6 with lemma 2.5.5(3) gives $\Sigma \cup \{\varphi\} \vdash \neg(\varphi \to \varphi)$.

Proposition 2.5.18: Properties Of Maximally Consistent Set

Suppose that Σ is a maximally consistent set of formulas and φ and ψ are formulas.

1. If $\Sigma \vdash \varphi$, then $\varphi \in \Sigma$
2. $\neg\varphi \in \Sigma$ if and only if $\varphi \notin \Sigma$
3. $\varphi \to \psi \in \Sigma$ if and only if $\varphi \notin \Sigma$ or $\psi \in \Sigma$

***Proof*:**
The set Σ is consistent by definition 2.5.17.

1. Suppose $\Sigma \vdash \varphi$ but $\varphi \notin \Sigma$. Then $\Sigma \cup \{\varphi\}$ is inconsistent, so $\Sigma \cup \{\varphi\} \vdash \neg(\chi \to \chi)$ for some formula χ, and $\Sigma \vdash \varphi \to \neg(\chi \to \chi)$ by theorem 2.5.7. Then lemma 2.5.5(2) gives $\Sigma \vdash \neg(\chi \to \chi)$, contradicting the consistency of Σ. Hence $\varphi \in \Sigma$.
2. If φ and $\neg\varphi$ were both in Σ, then theorem 2.5.6 and lemma 2.5.5(3) would give $\Sigma \vdash \neg(\varphi \to \varphi)$, contradicting the consistency of Σ. So $\neg\varphi \in \Sigma$ implies $\varphi \notin \Sigma$. Conversely, suppose $\varphi \notin \Sigma$. As in (1), $\Sigma \vdash \varphi \to \neg(\chi \to \chi)$ for some formula χ. Also $\Sigma \vdash \neg\neg\varphi \to \varphi$, by example 2.5.4(6) and lemma 2.5.5(3). The transitivity of example 2.5.4(2) with lemma 2.5.5(4) gives $\Sigma \vdash \neg\neg\varphi \to \neg(\chi \to \chi)$. The first deduction of lemma 2.5.9, with $\neg\varphi$ in place of β and $\chi \to \chi$ in place of α, gives $\neg\neg\varphi \to \neg(\chi \to \chi) \vdash (\chi \to \chi) \to \neg\varphi$, so $\Sigma \vdash (\chi \to \chi) \to \neg\varphi$ by lemma 2.5.5(4). As $\vdash \chi \to \chi$ by example 2.5.4(1), items (3) and (2) of lemma 2.5.5 give $\Sigma \vdash \neg\varphi$, and $\neg\varphi \in \Sigma$ by (1).
3. Suppose $\varphi \to \psi \in \Sigma$. If $\varphi \in \Sigma$, then $\Sigma \vdash \psi$ by lemma 2.5.5(2), so $\psi \in \Sigma$ by (1). Hence $\varphi \notin \Sigma$ or $\psi \in \Sigma$. Conversely, suppose $\psi \in \Sigma$. The instance $\psi \to (\varphi \to \psi)$ of A1 and Modus Ponens give $\Sigma \vdash \varphi \to \psi$. Now suppose $\varphi \notin \Sigma$, so that $\neg\varphi \in \Sigma$ by (2). By theorem 2.5.6, $\{\varphi, \neg\varphi\} \vdash \psi$, so $\{\neg\varphi\} \vdash \varphi \to \psi$ by theorem 2.5.7, and $\Sigma \vdash \varphi \to \psi$ by lemma 2.5.5(3). In both cases $\varphi \to \psi \in \Sigma$ by (1), completing the proof.

This proofs are very exciting: if we interpret this in terms of truth assignments and Σ being satisfied, then $\varphi \to \psi \in \Sigma$ if either φ is false or ψ is true, which essentially translates to the inclusion relation just proved above. Similarly, if φ is true, certainly $\neg\varphi$ is not true and hence not in Σ. This allows us to define the following:

Proposition 2.5.19: Truth Assignment Induced By Maximally Consistent Set

Let Σ be a maximally consistent set. Then define a function v by:

$$v(\varphi) = \begin{cases} T & \varphi \in \Sigma \\ F & \text{Otherwise} \end{cases}$$

Then v is a truth assignment.

***Proof*:**
By definition 1.7.3, v must be defined on every formula, which it is, and satisfy the negation and implication clauses. For a formula α, proposition 2.5.18(2) gives $v(\neg\alpha) = T$ exactly when $\neg\alpha \in \Sigma$, exactly when $\alpha \notin \Sigma$, that is, exactly when $v(\alpha) = F$. For formulas α, β, proposition 2.5.18(3) gives $v(\alpha \to \beta) = F$ exactly when $\alpha \to \beta \notin \Sigma$, exactly when $\alpha \in \Sigma$ and $\beta \notin \Sigma$, that is, exactly when $v(\alpha) = T$ and $v(\beta) = F$. Hence v is a truth assignment.

Proposition 2.5.20: Maximally Consistent, Then Satisfiable

Let Σ be maximally consistent. Then Σ is satisfiable

***Proof*:**
The function v of proposition 2.5.19 is a truth assignment, and $v(\sigma) = T$ for every $\sigma \in \Sigma$ by its definition, so v satisfies Σ.

Thus, we know have that if Σ is maximally consistent, it is satisfiable. Certainly if $\Delta \subseteq \Sigma$ and Σ is maximally consistent, then Δ is satisfiable. We want to show that any consistent formula is satisfiable. For this, we require the following important result showing us that any consistent set of formula belongs to a maximally consistent set of formulas:

Proposition 2.5.21: Embedding consistent set into Maximally consistent set

Suppose Γ is a consistent set of formulas. Then there is a maximally consistent set of formulas Σ such that $\Gamma \subseteq \Sigma$

***Proof*:**
The symbols of $\mathcal{L}_P$ are countably many, so there are countably many finite strings of them, and in particular countably many formulas. Enumerate the formulas as $\varphi_0, \varphi_1, \varphi_2, \ldots$. Put $\Sigma_0 = \Gamma$, and given Σ_n put

$$\Sigma_{n+1} = \begin{cases} \Sigma_n \cup \{\varphi_n\} & \text{if } \Sigma_n \vdash \varphi_n \\ \Sigma_n \cup \{\neg\varphi_n\} & \text{otherwise} \end{cases}$$

and let $\Sigma = \bigcup_{n\in\mathbb{N}} \Sigma_n$.

Step 1: each Σ_n is consistent. By induction on n, the case $n = 0$ being the hypothesis. Let Σ_n be consistent. If $\Sigma_n \vdash \varphi_n$ and $\Sigma_n \cup \{\varphi_n\} \vdash \neg(\chi \to \chi)$ for some formula χ, then $\Sigma_n \vdash \varphi_n \to \neg(\chi \to \chi)$ by theorem 2.5.7, and lemma 2.5.5(2) gives $\Sigma_n \vdash \neg(\chi \to \chi)$, which is impossible. If $\Sigma_n \nvdash \varphi_n$

and $\Sigma_n \cup \{\neg\varphi_n\} \vdash \neg(\chi \to \chi)$ for some formula χ, then $\Sigma_n \vdash \neg\varphi_n \to \neg(\chi \to \chi)$ by theorem 2.5.7. The first deduction of lemma 2.5.9 with lemma 2.5.5(4) gives $\Sigma_n \vdash (\chi \to \chi) \to \varphi_n$. As $\vdash \chi \to \chi$ by example 2.5.4(1), items (3) and (2) of lemma 2.5.5 give $\Sigma_n \vdash \varphi_n$, which is not the case. So Σ_{n+1} is consistent.

Step 2: Σ is consistent. Suppose $(\psi_1, \ldots, \psi_m)$ is a deduction of some $\neg(\chi \to \chi)$ from Σ. Each ψ_i that belongs to Σ belongs to some Σ_{n_i}. The sets Σ_n increase with n, so all of these ψ_i belong to Σ_k, where k is the largest n_i, or $k = 0$ if there are none. The same sequence is then a deduction from Σ_k, contradicting Step 1.

Step 3: Σ is maximally consistent. Let $\varphi \notin \Sigma$, and let n be the index with $\varphi = \varphi_n$. Then $\varphi_n \notin \Sigma_{n+1}$, so the second case of the construction applied and $\neg\varphi \in \Sigma_{n+1} \subseteq \Sigma$. By theorem 2.5.6 and lemma 2.5.5(3), $\Sigma \cup \{\varphi\} \vdash \neg(\varphi \to \varphi)$, so $\Sigma \cup \{\varphi\}$ is inconsistent. With Step 2 and $\Gamma = \Sigma_0 \subseteq \Sigma$, this completes the proof.

Theorem 2.5.22: Consistent If And Only If Satisfiable

A set Σ is consistent if and only if it is satisfiable

***Proof*:**
If Σ is satisfiable, it is consistent by lemma 2.5.13. Conversely, if Σ is consistent, proposition 2.5.21 gives a maximally consistent set $\Sigma' \supseteq \Sigma$, which some truth assignment v satisfies by proposition 2.5.20. Then v satisfies the subset Σ as well, completing the proof.

We finally got the Completeness theorem

Theorem 2.5.23: Completeness Theorem

Let Σ be a set of formulas. Then if:

$$\Sigma \models \varphi \qquad \text{then} \qquad \Sigma \vdash \varphi$$

***Proof*:**
Suppose $\Sigma \models \varphi$ but $\Sigma \nvdash \varphi$. Then $\Sigma \cup \{\neg\varphi\}$ is consistent. For if $\Sigma \cup \{\neg\varphi\} \vdash \neg(\chi \to \chi)$ for some formula χ, then $\Sigma \vdash \neg\varphi \to \neg(\chi \to \chi)$ by theorem 2.5.7, the first deduction of lemma 2.5.9 with lemma 2.5.5(4) gives $\Sigma \vdash (\chi \to \chi) \to \varphi$, and $\vdash \chi \to \chi$ (example 2.5.4(1)) with items (3) and (2) of lemma 2.5.5 gives $\Sigma \vdash \varphi$, which is not the case. By theorem 2.5.22, some truth assignment v satisfies $\Sigma \cup \{\neg\varphi\}$. Then v satisfies Σ, so $v(\varphi) = T$ because $\Sigma \models \varphi$, while $v(\neg\varphi) = T$ gives $v(\varphi) = F$ by the negation clause of definition 1.7.3. This contradiction shows $\Sigma \vdash \varphi$.

One result we get from this that might seem innocuous but is really important is that satisfiability is really a finite condition:

Theorem 2.5.24: Compactness Theorem

A set of formulas Γ is satisfiable if and only if every finite subset of Γ is satisfiable.

***Proof*:**
By theorem 2.5.22, Γ is satisfiable if and only if it is consistent. By corollary 2.5.16, this holds if and only if every finite subset of Γ is consistent, and by theorem 2.5.22 again, if and only if every finite subset of Γ is satisfiable.

2.6 A second look at Deductions

When we first studied first-order deduction, we have taken a inference heavy approach. In this section, we will take a axiom heavy approach similar to what we have done for propositional deductions. A lot of the work is the same, what will differ is that we will define a large axiom schema and only have modus ponens. Since we are using axioms, we will require the following when it comes to working with quantifiers

Definition 2.6.1: Generalization

If φ is any formula and $x_1, x_2, ..., x_n$ is any variables, then $\forall x_1, ..., \forall x_n \varphi$ is a *generalization* of φ

Example 2.6.2: Generalizations
Let $\varphi :\equiv x = y \to fx = fy$. Then $\forall z(x = y \to fx = fy)$ and $\forall x \forall y(x = y \to fx = fy)$ are generalizations of φ. $\forall x \exists y \varphi$ is not a generalization.

Lemma 2.6.3: Generalization Of Tautology

Let φ be a tautology (so that $\models \varphi$). Then any generalization of a tautology is a tautology

***Proof*:**
A generalization $\forall x_1 \cdots \forall x_n \varphi$ arises from φ by adding one quantifier at a time, so by induction on n it suffices to show that $\models \forall x \varphi$ whenever $\models \varphi$. Let $\mathfrak{A}$ be a structure, s an assignment into $\mathfrak{A}$ and x a variable. For every $a \in A$, $\mathfrak{A} \models \varphi[s(x|a)]$ because φ is valid (definition 1.6.3), so $\mathfrak{A} \models (\forall x \varphi)[s]$ by the quantifier clause of definition 1.4.5. Hence $\models \forall x \varphi$.

Definition 2.6.4: First Order Axiom Schema

Every first-order language $\mathcal{L}$ has eight *logical axiom schemas*:

1. **A1** $(\alpha \to (\beta \to \alpha))$
2. **A2** $((\alpha \to (\beta \to \gamma)) \to ((\alpha \to \beta) \to (\alpha \to \gamma)))$
3. **A3** $(((\neg\beta) \to (\neg\alpha)) \to (((\neg\beta) \to \alpha) \to \beta))$
4. **A4** $(\forall x\alpha \to \alpha_t^x)$ if t is substitutable for x in α
5. **A5** $(\forall x(\alpha \to \beta) \to (\forall x\alpha \to \forall x\beta))$
6. **A6** $(\alpha \to \forall x\alpha)$ if x does not occur free in α (i.e. if x is bounded or does not exist)
7. **A7** $x = x$
8. **A8** $(x = y \to (\alpha \to \beta))$ if α is atomic and β is obtained by replacing some occurrences (possibly all or none) of x in α by y.

all of these are called the *logical axioms* of $\mathcal{L}$ by replacing for formulas α, β, γ of $\mathcal{L}$. Furthermore, all generalizations of logical axioms of $\mathcal{L}$ are logical axioms of $\mathcal{L}$.

Notice that for **A4**, we may write $\forall x\alpha \to \alpha_x^x$ which is $\forall x\alpha \to \alpha$. All axioms are tautologies:

Proposition 2.6.5: Logical Axiom And Tautology

Every logical axiom is a tautology

***Proof*:**
A tautology here is a valid formula in the sense of definition 1.6.3, one that every $\mathcal{L}$-structure $\mathfrak{A}$ satisfies under every assignment s. Fix $\mathfrak{A}$ and s. By the implication clause of definition 1.4.5, $\mathfrak{A} \models (\varphi \to \psi)[s]$ fails exactly when $\mathfrak{A} \models \varphi[s]$ and $\mathfrak{A} \not\models \psi[s]$. We show that no instance of A1 to A8 of definition 2.6.4 fails under s.

- A1: if $(\alpha \to (\beta \to \alpha))$ failed, then $\mathfrak{A} \models \alpha[s]$ and $\mathfrak{A} \not\models (\beta \to \alpha)[s]$, and the latter gives $\mathfrak{A} \not\models \alpha[s]$.
- A2: if the instance failed, then $\mathfrak{A} \models (\alpha \to (\beta \to \gamma))[s]$ and $\mathfrak{A} \not\models ((\alpha \to \beta) \to (\alpha \to \gamma))[s]$. The latter gives $\mathfrak{A} \models (\alpha \to \beta)[s]$ and $\mathfrak{A} \not\models (\alpha \to \gamma)[s]$, so $\mathfrak{A} \models \alpha[s]$, $\mathfrak{A} \not\models \gamma[s]$, and then $\mathfrak{A} \models \beta[s]$. Hence $\mathfrak{A} \not\models (\beta \to \gamma)[s]$, which with $\mathfrak{A} \models \alpha[s]$ gives $\mathfrak{A} \not\models (\alpha \to (\beta \to \gamma))[s]$.
- A3: if the instance failed, then $\mathfrak{A} \models ((\neg\beta) \to (\neg\alpha))[s]$, $\mathfrak{A} \models ((\neg\beta) \to \alpha)[s]$ and $\mathfrak{A} \not\models \beta[s]$. Then $\mathfrak{A} \models (\neg\beta)[s]$, so the two implications give both $\mathfrak{A} \models \alpha[s]$ and $\mathfrak{A} \models (\neg\alpha)[s]$, which the negation clause forbids.
- A4: let t be substitutable for x in α, and suppose $\mathfrak{A} \models (\forall x\alpha)[s]$. Then $\mathfrak{A} \models \alpha[s(x|a)]$ for every $a \in A$, in particular for $a = \overline{s}(t)$, and lemma 2.4.1 turns this into $\mathfrak{A} \models \alpha_t^x[s]$.
- A5: suppose $\mathfrak{A} \models (\forall x(\alpha \to \beta))[s]$ and $\mathfrak{A} \models (\forall x\alpha)[s]$. For each $a \in A$ we have $\mathfrak{A} \models (\alpha \to \beta)[s(x|a)]$ and $\mathfrak{A} \models \alpha[s(x|a)]$, so $\mathfrak{A} \models \beta[s(x|a)]$. Hence $\mathfrak{A} \models (\forall x\beta)[s]$, and the instance does not fail.

- A6: let x not be free in α, and suppose $\mathfrak{A} \models \alpha[s]$. For each $a \in A$, the assignment $s(x|a)$ agrees with s on every variable other than x, hence on the free variables of α, so $\mathfrak{A} \models \alpha[s(x|a)]$ by proposition 1.4.9. Hence $\mathfrak{A} \models (\forall x\alpha)[s]$.
- A7: both sides of $x = x$ have the value $\overline{s}(x) = s(x)$, so $\mathfrak{A} \models (x = x)[s]$ by the equality clause.
- A8: suppose $\mathfrak{A} \models (x = y)[s]$, that is, $s(x) = s(y)$. Let u be a term and u' a string obtained from u by replacing some occurrences of x by y. By induction on u (definition 1.1.3), u' is a term with $\overline{s}(u') = \overline{s}(u)$. If u is x, then u' is x or y, and $s(x) = s(y)$. If u is another variable or a constant symbol, then u' is u. If $u :\equiv fu_1 \cdots u_n$, then the occurrences of x lie in the u_i, so $u' :\equiv fu'_1 \cdots u'_n$ with each u'_i obtained from u_i in the same manner, and definition 1.4.4 with the induction hypothesis gives $\overline{s}(u') = f^{\mathfrak{A}}(\overline{s}(u'_1), \ldots, \overline{s}(u'_n)) = f^{\mathfrak{A}}(\overline{s}(u_1), \ldots, \overline{s}(u_n)) = \overline{s}(u)$. Now the atomic formula α is $= u_1u_2$ or $Ru_1 \cdots u_n$, and β is $= u'_1u'_2$ or $Ru'_1 \cdots u'_n$ with each u'_i obtained from u_i in this manner. The atomic clauses then give $\mathfrak{A} \models \alpha[s]$ exactly when $\mathfrak{A} \models \beta[s]$, so $\mathfrak{A} \models (\alpha \to \beta)[s]$ and the instance does not fail.

Every instance of A1 to A8 is therefore valid. A generalization of a valid formula is valid by lemma 2.6.3, so every logical axiom of definition 2.6.4 is a tautology, as claimed.

This time, instead of having many rules of inference, we will only have one:

Definition 2.6.6: First Order Modus Ponens

Given a formula φ and $(\varphi \to \psi)$, we may infer ψ.

The definition of deduction is identical

Definition 2.6.7: Deduction Reiteration

Let Σ be a collection of $\mathcal{L}$-formulas and D a finite sequence of $\mathcal{L}$-formulas $(\varphi_1, \varphi_2, ..., \varphi_n)$. Then we will say that D is a *deduction* from Σ if for each i, $1 \leq i \leq n$, either

1. $\varphi_i \in \Lambda$ (that is φ_i is a logical axiom)
2. $\varphi_i \in \Sigma$ (that is φ_i is a nonlogical axiom)
3. there is a rule of inference (Γ, φ_i) such that $\Gamma \subseteq \{\varphi_1, \varphi_2, ..., \varphi_{i-1}\}$

I there is a deduction from Σ, the last line of which is the formula φ, we will call this a *deduction form* Σ *of* φ and write $\Sigma \vdash \varphi$

Lemma 2.6.8: Tautological Consequences Are Deducible

Let $\gamma_1, \ldots, \gamma_n, \varphi$ be formulas of the propositional language of definition 1.7.1 with $\{\gamma_1, \ldots, \gamma_n\} \models \varphi$ (definition 1.7.7), and let $A_0, \ldots, A_r$ include their atomic formulas. Let $\chi_0, \ldots, \chi_r$ be $\mathcal{L}$-formulas, and for a propositional formula δ write δ^* for the $\mathcal{L}$-formula obtained by replacing each A_j with $j \leq r$ by χ_j and every other atomic formula by χ_0. Then $\{\gamma_1^*, \ldots, \gamma_n^*\} \vdash \varphi^*$ in the deductive system of definition 2.6.7.

***Proof*:**
By theorem 2.5.23 there is a deduction $(\delta_1, \ldots, \delta_m)$ of φ from $\{\gamma_1, \ldots, \gamma_n\}$ in the propositional system of section 2.5, whose logical axioms are the instances of A1, A2, A3 of definition 2.5.1 and whose rule is Modus Ponens. The replacement $\delta \mapsto \delta^*$ satisfies $(\neg\delta)^* :\equiv (\neg\delta^*)$ and $(\delta \to \delta')^* :\equiv (\delta^* \to \delta'^*)$, where on the right $\neg$ and $\to$ are the connectives of definition 1.1.5. So it sends an instance of A1, A2 or A3 to an instance of the same schema of definition 2.6.4, a premise γ_i to γ_i^*, and an application of Modus Ponens to δ_u and $(\delta_u \to \delta_w)$ to an application of Modus Ponens to δ_u^* and $(\delta_u^* \to \delta_w^*)$. Hence $(\delta_1^*, \ldots, \delta_m^*)$ is a deduction of φ^* from $\{\gamma_1^*, \ldots, \gamma_n^*\}$ as in definition 2.6.7, as claimed.

Example 2.6.9: First Order Deductions

1. We will show $\{\alpha\} \vdash \exists x\alpha$

$(\forall x\neg\alpha \to \neg\alpha) \to (\alpha \to \neg\forall x\neg\alpha)$	lemma 2.6.8
$\forall x\ \neg\alpha \to \neg\alpha$	A4
$\alpha \to \neg\forall x\neg\alpha$	MP
α	Assumption
$\neg\forall x\neg\alpha$	MP
$\exists x\alpha$	definition 1.1.5

   The first line is the replacement of A_0 by $\forall x\neg\alpha$ and A_1 by α in the tautology $(A_0 \to \neg A_1) \to (A_1 \to \neg A_0)$. A truth assignment giving it the value F would give $A_0 \to \neg A_1$, A_1 and A_0 the value T, and then $\neg A_1$ the value T although A_1 has the value T, which is impossible.

2. Show that $\vdash \forall x\varphi \to \forall y\varphi_y^x$ if y does not occur at all in φ

3. $\vdash \alpha \vee \neg\alpha$. By definition 1.1.5, the formula $\alpha \vee \neg\alpha$ is $(\neg\alpha) \to (\neg\alpha)$, the replacement of A_0 by $\neg\alpha$ in $A_0 \to A_0$. Every truth assignment gives $A_0 \to A_0$ the value T, since the value F would need A_0 to be both T and F, so lemma 2.6.8 gives $\vdash \alpha \vee \neg\alpha$.

4. $\{c = d\} \vdash \forall z Qazc \to Qazd$

5. $\vdash x = y \to y = x$

6. $\{\exists x\alpha\} \vdash \alpha$ if x does not occur free in α (i.e. if x is bound in α)

We get many of the same results as before:

Proposition 2.6.10: Properties Of Deduction

1. Let $\varphi_1, \varphi_2, ..., \varphi_n$ be a deduction of $\mathcal{L}$, then $\varphi_1, \varphi_2, ..., \varphi_l$ be a deduction of $\mathcal{L}$ for any l such that $1 \leq l \leq n$.
2. If $\Gamma \vdash \delta$ and $\Gamma \vdash \delta \to \beta$ then $\Gamma \vdash \beta$
3. if $\Gamma \subseteq \Delta$ and $\Gamma \vdash \alpha$ then $\Delta \vdash \alpha$
4. if $\Gamma \vdash \Delta$ and $\Delta \vdash \sigma$ then $\Gamma \vdash \sigma$

***Proof*:**
Throughout, deductions are as in definition 2.6.7, with Modus Ponens (definition 2.6.6) as the rule of inference.

1. Let $(\varphi_1, \varphi_2, \ldots, \varphi_n)$ be a deduction from a set of formulas Γ and let $1 \leq l \leq n$. The condition of definition 2.6.7 constrains each index i separately, and for a given i it mentions only Λ, Γ, and the formulas $\varphi_1, \ldots, \varphi_{i-1}$. For $i \leq l$ these data are the same in $(\varphi_1, \ldots, \varphi_l)$ as in the full sequence. Hence $(\varphi_1, \ldots, \varphi_l)$ is a deduction from Γ.
2. Let $(\alpha_1, \ldots, \alpha_p)$ be a deduction from Γ with $\alpha_p :\equiv \delta$, and $(\beta_1, \ldots, \beta_q)$ a deduction from Γ with $\beta_q :\equiv \delta \to \beta$. Consider the sequence

   $$(\alpha_1, \ldots, \alpha_p, \beta_1, \ldots, \beta_q, \beta)$$

   Each α_i keeps its original justification. Each β_j that is a logical axiom or a member of Γ keeps its justification as well, and each β_j obtained by Modus Ponens from β_u and β_w with $u, w < j$ still has both premises among the strictly earlier entries of the new sequence. Finally, β follows by Modus Ponens from δ and $\delta \to \beta$, which occur earlier. Hence the displayed sequence is a deduction from Γ with last line β, and $\Gamma \vdash \beta$.
3. A deduction from Γ is verbatim a deduction from Δ: the only clause of definition 2.6.7 that mentions the set of nonlogical axioms is $\varphi_i \in \Sigma$, read here with Γ for Σ, and $\Gamma \subseteq \Delta$ turns it into $\varphi_i \in \Delta$.
4. Here $\Gamma \vdash \Delta$ means $\Gamma \vdash \delta$ for every $\delta \in \Delta$. Let $(\varphi_1, \ldots, \varphi_n)$ be a deduction of $\sigma :\equiv \varphi_n$ from Δ. By induction on i we show $\Gamma \vdash \varphi_i$ for every $i \leq n$. If $\varphi_i \in \Lambda$, the one term sequence (φ_i) is a deduction from Γ. If $\varphi_i \in \Delta$, then $\Gamma \vdash \varphi_i$ by hypothesis. If φ_i follows by Modus Ponens from φ_u and $\varphi_w :\equiv \varphi_u \to \varphi_i$ with $u, w < i$, then $\Gamma \vdash \varphi_u$ and $\Gamma \vdash \varphi_u \to \varphi_i$ by the induction hypothesis, so $\Gamma \vdash \varphi_i$ by item (2). Taking $i = n$ gives $\Gamma \vdash \sigma$.

Theorem 2.6.11: Deduction Theorem

Let Σ be any set of formulas and α, β any formulas. Then $\Sigma \vdash \alpha \to \beta$ if and only if $\Sigma \cup \{\alpha\} \vdash \beta$

***Proof*:**
The proof of theorem 2.5.7 uses only that logical axioms and members of $\Sigma \cup \{\alpha\}$ have one-line deductions, the schemas A1 and A2, Modus Ponens as the only rule of inference, the deduction of $\alpha \to \alpha$ in example 2.5.4(1), which consists of instances of A1, A2 and applications of Modus Ponens, and items (2) and (3) of lemma 2.5.5. For the deductions of definition 2.6.7, A1 and A2 are schemas of definition 2.6.4, Modus Ponens is the rule of definition 2.6.6, and the two items are proposition 2.6.10(2) and (3). The same argument therefore proves the theorem.

We know get a result is a new result due to our axiomatic approach

Theorem 2.6.12: Generalization Theorem

Suppose x is a variable, Γ is a set of formulas in which x does not occur free (i.e. x is either bound or does not exist), and φ is a formula such that $\Gamma \vdash \varphi$. Then $\Gamma \vdash \forall x\varphi$

***Proof*:**
Let $(\varphi_1, \ldots, \varphi_n)$ be a deduction of φ from Γ (definition 2.6.7), with Modus Ponens as the rule of inference. We show $\Gamma \vdash \forall x\varphi_i$ for every i, by strong induction on i.

- If φ_i is a logical axiom, then $\forall x\varphi_i$ is a generalization of a logical axiom, hence a logical axiom by definition 2.6.4, and the one-line deduction $(\forall x\varphi_i)$ suffices.
- If $\varphi_i \in \Gamma$, then x is not free in φ_i, so $\varphi_i \to \forall x\varphi_i$ is an instance of **A6**, and $(\varphi_i,\ \varphi_i \to \forall x\varphi_i,\ \forall x\varphi_i)$ is a deduction from Γ.
- If φ_i follows by Modus Ponens from φ_j and $\varphi_k :\equiv (\varphi_j \to \varphi_i)$ with $j, k < i$, then $\Gamma \vdash \forall x\varphi_j$ and $\Gamma \vdash \forall x(\varphi_j \to \varphi_i)$ by the induction hypothesis. The instance $\forall x(\varphi_j \to \varphi_i) \to (\forall x\varphi_j \to \forall x\varphi_i)$ of **A5** and two applications of proposition 2.6.10(2) give $\Gamma \vdash \forall x\varphi_i$.

For $i = n$ this is $\Gamma \vdash \forall x\varphi$, completing the proof.

Lemma 2.6.13: Replacing A Constant By A Variable

Let c be a constant symbol and x a variable. For a formula ψ, write ψ^c_x for the string obtained from ψ by replacing every occurrence of c by x. Let $(\varphi_1, \ldots, \varphi_n)$ be a deduction (definition 2.6.7) from a set Γ of formulas in which c does not occur, and suppose that x occurs in none of $\varphi_1, \ldots, \varphi_n$. Then $((\varphi_1)^c_x, \ldots, (\varphi_n)^c_x)$ is a deduction from the set of those φ_i that belong to Γ, and c does not occur in it.

***Proof*:**
By induction on definition 1.1.3 and then on definition 1.1.5, the replacement turns terms into terms and formulas into formulas, and it passes through the connectives and the quantifier:

$$(\neg\psi)^c_x :\equiv (\neg\psi^c_x) \qquad (\psi \to \chi)^c_x :\equiv (\psi^c_x \to \chi^c_x) \qquad ((\forall y)(\psi))^c_x :\equiv (\forall y)(\psi^c_x)$$

the last because the quantified variable y is not the constant c. Write ψ' for ψ^c_x, and let Φ be the set of those φ_i that belong to Γ.

Step 1: three facts about the replacement. Let y be a variable other than x, let α be a formula and t a term.

(a) Every variable other than x that is free in α' is free in α. This is an induction on α through definition 1.2.2: the replacement introduces only occurrences of x, and it leaves each quantifier $(\forall u)$ of α unchanged.

(b) $(\alpha_t^y)' :\equiv (\alpha')_{t'}^y$. The corresponding identity $(u_t^y)' :\equiv (u')_{t'}^y$ for a term u holds by induction through definition 1.5.1. If u is y, both sides are t'. If u is c, both sides are x, because x is not y. If u is another variable or constant symbol, both sides are u. If $u :\equiv f u_1 \cdots u_n$, both operations act argument by argument, so the induction hypothesis for the u_i gives the identity. For formulas we induct through definition 1.5.3. The atomic case is the term identity applied to each argument, and the cases of $\neg$ and $\to$ follow from the induction hypothesis. If $\alpha :\equiv (\forall z)(\beta)$, then both sides are α' when z is y, and both are $(\forall z)((\beta')_{t'}^y)$ by the induction hypothesis when z is not y.

(c) If x does not occur in α and t is substitutable for y in α, then t' is substitutable for y in α'. By induction through definition 1.5.5, the atomic case is automatic and the cases of $\neg$ and $\to$ follow from the induction hypothesis. Let $\alpha :\equiv (\forall z)(\beta)$. If y is not free in α, then y is not free in α' by (a). Otherwise z does not occur in t and t is substitutable for y in β. As x does not occur in α, the variable z is not x, so z does not occur in t', whose variables are those of t and possibly x, and t' is substitutable for y in β' by the induction hypothesis.

Step 2: $(\varphi_1', \ldots, \varphi_n')$ is a deduction from Φ in which c does not occur. No φ_i' contains c. If $\varphi_i \in \Phi$, then φ_i' is φ_i, since c does not occur in Γ. If φ_i follows by Modus Ponens from φ_j and $\varphi_k :\equiv (\varphi_j \to \varphi_i)$ with $j, k < i$, then $\varphi_k' :\equiv (\varphi_j' \to \varphi_i')$, so φ_i' follows from φ_j' and φ_k' by Modus Ponens. It remains to show that ψ' is a logical axiom whenever ψ is a logical axiom in which x does not occur. By definition 2.6.4, such a ψ is a generalization $\forall y_1 \cdots \forall y_j \theta$ of an instance θ of one of A1 to A8, and $\psi' :\equiv \forall y_1 \cdots \forall y_j \theta'$, so it suffices that θ' is an instance of the same schema.

- A1, A2, A3, A5: the replacement passes through $\neg$, $\to$ and $\forall$, so θ' is the instance of the same schema with α', β', γ' in place of α, β, γ.
- A4: $\theta :\equiv \forall y \alpha \to \alpha_t^y$ with t substitutable for y in α. The variable x does not occur in α and is not y, which occurs in θ. By (b), $\theta' :\equiv \forall y \alpha' \to (\alpha')_{t'}^y$, and by (c), t' is substitutable for y in α'.
- A6: $\theta :\equiv \alpha \to \forall y \alpha$ with y not free in α, and $\theta' :\equiv \alpha' \to \forall y \alpha'$. The variable y occurs in θ, so it is not x, and it is not free in α' by (a).
- A7: θ is $y = y$, which does not contain c, so θ' is θ.
- A8: $\theta :\equiv (y = z \to (\alpha \to \beta))$ with α atomic and β obtained from α by replacing some occurrences of y by z. Then $\theta' :\equiv (y = z \to (\alpha' \to \beta'))$, the formula α' is atomic, and β' is obtained from α' by replacing the same occurrences of y by z, because the two replacements act on different positions, c not being the variable y.

Hence $(\varphi_1', \ldots, \varphi_n')$ is a deduction from Φ in which c does not occur, completing the proof.

Theorem 2.6.14: Generalization On Constants

Suppose that c is a constant symbol, Γ is a set of formulas in which c does not occur, and φ is a formula such that $\Gamma \vdash \varphi$. Then there is a variable x which does not occur in φ such that $\Gamma \vdash \forall x \varphi_x^c$. Moreover, there is a deduction of $\forall x \varphi_x^c$ from Γ in which c does not occur

What this theorem is saying is that if we prove a property of a constant, say x_0 is a constant and $\vdash P(x_0)$, then we in fact have $\vdash \forall x P(x)$

***Proof*:**
Fix a deduction $(\varphi_1, \ldots, \varphi_n)$ of φ from Γ as in definition 2.6.7, with Modus Ponens as the rule of inference, and let Φ be the finite set of those φ_i that belong to Γ. Only finitely many variables occur in $\varphi_1, \ldots, \varphi_n$, so we may choose a variable x occurring in none of them. In particular x does not occur in φ or in any member of Φ. Write ψ' for ψ_x^c. By lemma 2.6.13, $(\varphi_1', \ldots, \varphi_n')$ is a deduction of φ' from Φ in which c does not occur.

The variable x occurs in no member of Φ, so it is free in none. We show by strong induction on i that $\forall x \varphi_i'$ has a deduction from Φ in which c does not occur. If φ_i' is a logical axiom, then $\forall x \varphi_i'$ is a generalization of it, hence a logical axiom, and the one-line deduction suffices. If $\varphi_i' \in \Phi$, then $(\varphi_i',\ \varphi_i' \to \forall x \varphi_i',\ \forall x \varphi_i')$ is a deduction, the middle formula being an instance of A6 since x is not free in φ_i'. If φ_i' follows by Modus Ponens from φ_j' and $\varphi_k' :\equiv (\varphi_j' \to \varphi_i')$ with $j, k < i$, we concatenate the deductions of $\forall x \varphi_j'$ and $\forall x(\varphi_j' \to \varphi_i')$ given by the induction hypothesis, in which each entry keeps its justification since the entries before it in its own deduction still precede it, and append the three formulas

$$\forall x(\varphi_j' \to \varphi_i') \to (\forall x \varphi_j' \to \forall x \varphi_i') \qquad \forall x \varphi_j' \to \forall x \varphi_i' \qquad \forall x \varphi_i'$$

an instance of A5 followed by two applications of Modus Ponens. In each case the new formulas do not contain c. For $i = n$ this is a deduction of $\forall x \varphi_x^c$ from Φ, hence from Γ by proposition 2.6.10(3), in which c does not occur. Since x does not occur in φ, this completes the proof.

Lemma 2.6.15: Alphabetic Variants

Let φ be a formula and x, y variables such that y does not occur in φ. Then

$$\vdash \forall x \varphi \to \forall y \varphi_y^x \qquad \text{and} \qquad \vdash \forall y \varphi_y^x \to \forall x \varphi$$

***Proof*:**
Deductions are those of definition 2.6.7. If y is x, then x does not occur in φ, so φ_y^x is φ by lemma 1.5.7(1). Both claims then read $\vdash \forall x \varphi \to \forall x \varphi$, which lemma 2.6.8 gives from $\emptyset \models A_0 \to A_0$. Suppose from now on that y is not x. Each of the following facts is proved by induction on φ through definition 1.5.1, definition 1.5.3 and definition 1.5.5, using that y does not occur in φ.

(i) y is substitutable for x in φ. The atomic case is automatic and the cases of $\neg$ and $\to$ follow from the induction hypothesis. Let $\varphi :\equiv (\forall z)(\alpha)$. If x is not free in φ, the first alternative of the quantifier clause applies. Otherwise z occurs in φ while y does not, so z does not

occur in the term y, and y is substitutable for x in α by the induction hypothesis.

(ii) x is not free in φ^x_y. In an atomic formula every occurrence of x is replaced by y, and the cases of $\neg$ and $\to$ follow from the induction hypothesis. If $\varphi :\equiv (\forall z)(\alpha)$, then φ^x_y is φ when z is x, in which x is not free, and φ^x_y is $(\forall z)(\alpha^x_y)$ otherwise, in which x is not free by the induction hypothesis.

(iii) x is substitutable for y in φ^x_y. The atomic case is automatic and the cases of $\neg$ and $\to$ follow from the induction hypothesis. Let $\varphi :\equiv (\forall z)(\alpha)$. If z is x, then φ^x_y is φ, in which y does not occur, so y is not free in it. Otherwise φ^x_y is $(\forall z)(\alpha^x_y)$, the variable z is not x and so does not occur in the term x, and x is substitutable for y in α^x_y by the induction hypothesis.

(iv) $(\varphi^x_y)^y_x :\equiv \varphi$. For a term, x becomes y and then x again, the variable y does not occur, other variables and constant symbols are left unchanged, and compound terms are treated argument by argument. For formulas the atomic case follows, and the cases of $\neg$ and $\to$ follow from the induction hypothesis. If $\varphi :\equiv (\forall z)(\alpha)$ and z is x, then φ^x_y is φ, and substituting for y, which does not occur in φ, leaves φ unchanged by lemma 1.5.7(1). Otherwise z is neither x nor y, the variable y not occurring in φ, so $(\varphi^x_y)^y_x$ is $(\forall z)((\alpha^x_y)^y_x)$, which is φ by the induction hypothesis.

(v) If $\vdash \psi \to \chi$ and the variable u is not free in ψ, then $\vdash \psi \to \forall u\chi$. Indeed theorem 2.6.12, with the empty set of hypotheses, gives $\vdash \forall u(\psi \to \chi)$, and the instance $\forall u(\psi \to \chi) \to (\forall u\psi \to \forall u\chi)$ of **A5** with proposition 2.6.10(2) gives $\vdash \forall u\psi \to \forall u\chi$. The formula $\psi \to \forall u\psi$ is an instance of **A6**. Applied to $\{A_0 \to A_1, A_1 \to A_2\} \models A_0 \to A_2$, lemma 2.6.8 gives $\{\psi \to \forall u\psi, \forall u\psi \to \forall u\chi\} \vdash \psi \to \forall u\chi$, and proposition 2.6.10(4) gives $\vdash \psi \to \forall u\chi$.

By (i), $\forall x\varphi \to \varphi^x_y$ is an instance of **A4**. The variable y is not free in $\forall x\varphi$, since it does not occur in φ, so (v) gives $\vdash \forall x\varphi \to \forall y\varphi^x_y$. By (iii) and (iv), $\forall y\varphi^x_y \to \varphi$ is an instance of **A4**. By (ii), x is not free in $\forall y\varphi^x_y$, so (v) gives $\vdash \forall y\varphi^x_y \to \forall x\varphi$, completing the proof.

Example 2.6.16: More Deductions

1. We will show $\vdash \varphi \to \psi$ then $\vdash \forall x\varphi \to \forall x\psi$. Since x does not occur free in any formula $\emptyset$, it follows from $\vdash \varphi \to \psi$ by the generalization theorem that $\vdash \forall x(\varphi \to \psi)$. But then

$$\begin{array}{ll} \forall x(\varphi \to \psi) & \\ \forall x(\varphi \to \psi) \to (\forall x\varphi \to \forall x\psi) & \textbf{A5} \\ \forall x\varphi \to \forall x\psi & \textbf{MP} \end{array}$$

2. show $\vdash \forall x \forall y \forall z(x = y \to (y = z \to x = z))$ (Hint: apply Deduction theorem twice, and show $x = y \vdash y = z$. Now think of the derivation, and finish with Generalization theorem)

3. $\vdash \forall x\alpha \to \exists x\alpha$

4. $\vdash \exists x\gamma \to \forall x\gamma$ if x does not occur free in γ

5. (Using generalization on constants) Let $\mathcal{L}$ be the language of number theory and take $\forall x \exists y\ x < y$. Then we can say given an n, we have $n < n + 1$, so $\exists y\ n < y$, and so we conclude $\forall x \exists y\ x < y$

2.6.1 Soundness and Completeness

Let $\mathcal{L}$ be a fixed first-order language. All formulas will be considered as part of $\mathcal{L}$. First, a set Γ of sentences is inconsistent if $\Gamma \vdash \neg(\psi \to \psi)$ for some formula ψ, and consistent otherwise. A set Σ of sentences is maximally consistent if Σ is consistent and $\Sigma \cup \{\tau\}$ is inconsistent whenever τ is a sentence such that $\tau \notin \Sigma$. Now some review:

Proposition 2.6.17: Results From Earlier

1. (Soundness Theorem) If α is a sentence and Δ is a set of sentences such that $\Delta \vdash \alpha$, then $\Delta \models \alpha$
2. If a set of sentences Γ is satisfiable, then it is consistent
3. Suppose Δ Is an inconsistent set of sentences. Then $\Delta \vdash \psi$ for any formula ψ
4. Supposer Σ is an inconsistent set of sentences. then there is a finite subset Δ of Σ such that Δ is inconsistent
5. A set of sentences Γ Is consistent if and only if every finite subset of Γ is consistent

***Proof*:**
Deductions are those of definition 2.6.7, with the logical axioms of definition 2.6.4 and Modus Ponens (definition 2.6.6) as the only rule of inference.

1. We prove more: if $(\varphi_1, \ldots, \varphi_n)$ is a deduction from a set of formulas Δ and $\mathfrak{M} \models \Delta$, then $\mathfrak{M} \models \varphi_i[s]$ for every i and every assignment s into $\mathfrak{M}$. We use strong induction on i. If φ_i is a logical axiom, it is valid by proposition 2.6.5. If $\varphi_i \in \Delta$, then $\mathfrak{M} \models \varphi_i[s]$ because $\mathfrak{M} \models \Delta$ (definition 1.4.11). If φ_i follows by Modus Ponens from φ_j and $\varphi_k :\equiv (\varphi_j \to \varphi_i)$ with $j, k < i$, then $\mathfrak{M} \models \varphi_j[s]$ and $\mathfrak{M} \models (\varphi_j \to \varphi_i)[s]$ by the induction hypothesis, and the implication clause of definition 1.4.5 gives $\mathfrak{M} \models \varphi_i[s]$. Now let Δ be a set of sentences and α a sentence with $\Delta \vdash \alpha$. Every structure $\mathfrak{M}$ with $\mathfrak{M} \models \Delta$ satisfies α under every assignment, so $\mathfrak{M} \models \alpha$, and $\Delta \models \alpha$ by definition 1.6.1.
2. Let $\mathfrak{M} \models \Gamma$, and suppose $\Gamma \vdash \neg(\psi \to \psi)$ for a formula ψ. Fix an assignment s into $\mathfrak{M}$, which exists since $|\mathfrak{M}|$ is nonempty. The claim proved in (1) gives $\mathfrak{M} \models (\neg(\psi \to \psi))[s]$, that is, $\mathfrak{M} \not\models (\psi \to \psi)[s]$ by the negation clause. By the implication clause this requires $\mathfrak{M} \models \psi[s]$ and $\mathfrak{M} \not\models \psi[s]$ at once, which is impossible. Hence Γ is consistent.
3. The deduction of $\varphi \to \varphi$ in example 2.5.4(1), and the deduction of γ from $\{\delta, \neg\delta\}$ in the proof of theorem 2.5.6, consist of instances of A1, A2, A3 and applications of Modus Ponens, for arbitrary formulas φ, γ, δ. These schemas and this rule are also those of definition 2.6.4 and definition 2.6.6, so the same sequences, with $\mathcal{L}$-formulas in place of φ, γ, δ, are deductions in the sense of definition 2.6.7. Now let $\Delta \vdash \neg(\chi \to \chi)$ for a formula χ, and let ψ be any formula. The first deduction gives $\vdash \chi \to \chi$, so $\Delta \vdash \chi \to \chi$ by proposition 2.6.10(3). The second, with $\delta :\equiv (\chi \to \chi)$ and $\gamma :\equiv \psi$, gives $\{\chi \to \chi, \neg(\chi \to \chi)\} \vdash \psi$. Since Δ deduces both members of this set, proposition 2.6.10(4) gives $\Delta \vdash \psi$.
4. Let $(\varphi_1, \ldots, \varphi_n)$ be a deduction of $\neg(\psi \to \psi)$ from Σ, and let Δ be the finite set of those φ_i that belong to Σ. The same sequence is a deduction from Δ, because each φ_i justified

by membership in Σ belongs to Δ, and the other justifications do not mention the set of nonlogical axioms. So $\Delta \vdash \neg(\psi \to \psi)$, and Δ is a finite inconsistent subset of Σ.

5. If Γ is consistent and $\Delta \subseteq \Gamma$ is finite, then Δ is consistent, since a deduction of some $\neg(\psi \to \psi)$ from Δ would give one from Γ by proposition 2.6.10(3). Conversely, if every finite subset of Γ is consistent, then Γ is consistent, since by (4) an inconsistent Γ would have a finite inconsistent subset. This completes the proof.

In propositional logic, we found maximally consistent sets given consistent sets. We may do the same thing in propositional logic, but we have one more tool at our disposal to find maximally consistent sets

Proposition 2.6.18: Relating Structures And Theorems

Let $\mathfrak{M}$ be a structure. Then $Th(\mathfrak{M})$ is a maximally consistent set of sentences

This is the mirror of $\Sigma = \{\varphi \mid v(\varphi) = T\}$

***Proof*:**
Here $Th(\mathfrak{M}) = \{\tau \mid \tau \text{ is a sentence and } \mathfrak{M} \models \tau\}$. Since $\mathfrak{M} \models Th(\mathfrak{M})$, the set $Th(\mathfrak{M})$ is satisfiable, hence consistent by proposition 2.6.17(2). Let τ be a sentence with $\tau \notin Th(\mathfrak{M})$. Then $\mathfrak{M} \not\models \tau$, so $\mathfrak{M} \models \neg\tau$ by proposition 1.4.14(1), and $\neg\tau \in Th(\mathfrak{M})$, as $\neg\tau$ is a sentence. No truth assignment satisfies $\{A_0, \neg A_0\}$, so $\{A_0, \neg A_0\} \models \neg(A_0 \to A_0)$, and lemma 2.6.8 gives $\{\tau, \neg\tau\} \vdash \neg(\tau \to \tau)$. By proposition 2.6.10(3), $Th(\mathfrak{M}) \cup \{\tau\} \vdash \neg(\tau \to \tau)$, so $Th(\mathfrak{M}) \cup \{\tau\}$ is inconsistent, and $Th(\mathfrak{M})$ is maximally consistent.

Example 2.6.19: Theory Of Structure
Let $\mathfrak{M} = (\{5\})$ be a structure for $\mathcal{L} = \{\}$. Then $Th(\mathfrak{M})$ is a maximally consistent set of sentences. It turns out that

$$Th(\mathfrak{M}) = Th(\{\forall x \forall y\ x = y\})$$

So we also get a set of sentences, $\Sigma = \{\forall x \forall y\ x = y\}$ such that $Th(\Sigma)$ is maximally consistent.

Theorem 2.6.20: Zorn's Lemma

Let $(P, \leq)$ be a partially ordered set in which every chain, that is every subset of P totally ordered by $\leq$, has an upper bound in P. Then P has a maximal element, an element m such that $m \leq p$ implies $p = m$.

Zorn's lemma is equivalent, over the remaining axioms of set theory, to the axiom of choice, and it is adopted here without proof.

As before, we may abuse maximality to get the same results as in first-order logic, mainly due to the fact we are working with sentences

Proposition 2.6.21: Operational Facts About Maximally Consistent Sets

1. If Σ is a maximally consistent set of sentences, τ is a sentence, and $\Sigma \vdash \tau$, then $\tau \in \Sigma$
2. Suppose Σ is maximally consistent set of sentences and τ is a sentence. Then $\neg\tau \in \Sigma$ if and only if $\tau \notin \Sigma$
3. Suppose Σ is a maximally consistent set of sentences and φ and ψ are any sentence. Then $\varphi \to \psi \in \Sigma$ if and only if $\varphi \notin \Sigma$ or $\psi \in \Sigma$
4. Suppose Γ is a consistent set of sentences. Then there is a maximally consistent set of sentences Σ with $\Gamma \subseteq \Sigma$

***Proof*:**
Deductions are those of definition 2.6.7, and a maximally consistent set is in particular consistent. Each use of lemma 2.6.8 below rests on a propositional implication checked with the clauses of definition 1.7.3.

1. Suppose $\Sigma \vdash \tau$ but $\tau \notin \Sigma$. Then $\Sigma \cup \{\tau\}$ is inconsistent, so $\Sigma \cup \{\tau\} \vdash \neg(\psi \to \psi)$ for some formula ψ, and $\Sigma \vdash \tau \to \neg(\psi \to \psi)$ by theorem 2.6.11. Then proposition 2.6.10(2) gives $\Sigma \vdash \neg(\psi \to \psi)$, contradicting the consistency of Σ. Hence $\tau \in \Sigma$.
2. The formulas τ and $\neg\tau$ are not both in Σ: no truth assignment satisfies $\{A_0, \neg A_0\}$, so $\{A_0, \neg A_0\} \models \neg(A_0 \to A_0)$, and lemma 2.6.8 with proposition 2.6.10(3) would give $\Sigma \vdash \neg(\tau \to \tau)$. Suppose now that $\tau \notin \Sigma$. As in (1), $\Sigma \vdash \tau \to \neg(\psi \to \psi)$ for some formula ψ. A truth assignment with $v(A_0) = T$ gives $A_0 \to \neg(A_1 \to A_1)$ the value F, because $v(A_1 \to A_1) = T$, so $\{A_0 \to \neg(A_1 \to A_1)\} \models \neg A_0$. By lemma 2.6.8 and proposition 2.6.10(4), $\Sigma \vdash \neg\tau$, and $\neg\tau \in \Sigma$ by (1), since $\neg\tau$ is a sentence. Hence exactly one of τ and $\neg\tau$ lies in Σ.
3. If $\varphi \to \psi \in \Sigma$ and $\varphi \in \Sigma$, then $\Sigma \vdash \psi$ by proposition 2.6.10(2), so $\psi \in \Sigma$ by (1). Conversely, $\varphi \to \psi$ is a sentence, and it lies in Σ by (1) as soon as $\Sigma \vdash \varphi \to \psi$. If $\psi \in \Sigma$, this follows from $\{A_1\} \models A_0 \to A_1$, lemma 2.6.8 and proposition 2.6.10(4). If $\varphi \notin \Sigma$, then $\neg\varphi \in \Sigma$ by (2), and it follows from $\{\neg A_0\} \models A_0 \to A_1$ in the same way.
4. Let P be the set of consistent sets of sentences of $\mathcal{L}$ containing Γ, partially ordered by inclusion. The empty chain has the upper bound $\Gamma \in P$. If $\mathcal{C} \subseteq P$ is a nonempty chain, then $\bigcup \mathcal{C}$ contains Γ, and a finite subset of $\bigcup \mathcal{C}$ lies in a single member of $\mathcal{C}$, the chain being totally ordered. That member is consistent, so the finite subset is consistent by proposition 2.6.17(5), and then $\bigcup \mathcal{C}$ is consistent by proposition 2.6.17(5) again. So $\bigcup \mathcal{C} \in P$ is an upper bound for $\mathcal{C}$. By theorem 2.6.20, P has a maximal element Σ. It is consistent and contains Γ, and if τ is a sentence not in Σ, then $\Sigma \cup \{\tau\}$ contains Γ and properly contains Σ, so it is not in P, that is, it is inconsistent. Hence Σ is maximally consistent, completing the proof.

Hence, we have the same relation between maximally consistent sets and "propositionally satisfiability". When we did this in propositional logic, we defined a suitable truth assignment function from a maximally consistent set of formulas. Now we need to construct a suitable structure from a maximally consistent set of sentences. The following definition will be key in our construction. The motivation

behind this definition is as follows: suppose we have $\Sigma \vdash \exists x\varphi$. Then recall that for $\mathfrak{M}$ to satisfy $\exists x\varphi$, we must have a constant c such that φ_c^x is satisfied by $\mathfrak{M}$. To have $\mathfrak{M} \models \varphi_c^x$, we need $\Sigma \vdash \varphi_c^x$. Thus, if we have $\Sigma \vdash \exists x\varphi$, we in a sense need a constant c such that $\Sigma \vdash \exists x\varphi \to \varphi_c^x$. These constant will be "witnesses" for the existential formulas that our maximally consistent set derives

Definition 2.6.22: Witness

Suppose σ is a set of sentences, and C is a set of (some of the) constants of $\mathcal{L}$. Then C is a *set of witnesses* for Σ in $\mathcal{L}$ if for every formula φ of $\mathcal{L}$ with at most one free variable x, there is a constant symbol $c \in C$ such that $\Sigma \vdash \exists x\varphi \to \varphi_c^x$

Essentially, every element of the universe of formulas for which Σ must exist is "named" or "witnessed" by some constant C, so we take the subset of constant symbols that we use that are associated with symbols $C^{\mathfrak{M}}$ for any structure $\mathfrak{M}$.

Proposition 2.6.23: Extension Of Witnesses

Let Γ and Σ be sets of sentences of $\mathcal{L}$, $\Gamma \subseteq \Sigma$ and C is a set of witnesses for Γ in $\mathcal{L}$. Then C Is a set of witnesses for Σ in $\mathcal{L}$

***Proof*:**
Let φ be a formula of $\mathcal{L}$ with at most one free variable x. Since C is a set of witnesses for Γ, there is a constant $c \in C$ with $\Gamma \vdash \exists x\varphi \to \varphi_c^x$. As $\Gamma \subseteq \Sigma$, every deduction from Γ is a deduction from Σ, so $\Sigma \vdash \exists x\varphi \to \varphi_c^x$. Hence C is a set of witnesses for Σ.

Example 2.6.24: Set Of Witnesses
Let $\mathcal{L}_O$ be the first order language with a signal 2-ary relation symbol $<$ and countably many constant symbols c_q for each $q \in \mathbb{Q}$. Let Σ include all sentences

1. $c_p < c_q$ for every $p, q \in \mathbb{Q}$ such that $p < q$
2. $\forall x(\neg x < x)$
3. $\forall x \forall y(x < y \vee x = y \vee y < x)$
4. $\forall x \forall y \forall z(x < y \to (y < z \to x < z))$
5. $\forall x \forall y(x < y \to \exists z(x < z \wedge z < y))$
6. $\forall x \exists y(x < y)$
7. $\forall x \exists y(y < x)$

Essentially σ asserts that $<$ is a linear order on the universe (points 2 to 4) which is dense (point 5) and has no endpoints (6,7) and has a suborder isomorphic to Q (1). Then $C = \{c_q \mid q \in Q\}$ is a set of Witnesses for Σ in $\mathcal{L}_O$.

With the above example, we can deduce the model for the set of sentences from the set of witnesses by simply letting the universe *be* the set of witnesses, and define the relation intepreting $<$ in a

very obvious way from Σ. However, this is due to the example being purposefully well-chosen, in particualr there are not constant symbols which are not witnesses, Σ proves that distinct constant symbols are not equal to each other, and Σ explicitly has everything you need to know about $<$. In general, we may not know whether Σ has a set of witnesses, for Σ may not even have constants. Furthermore, if Σ has a set of witnesses, say C, we usually cannot let the universe just be C (for example, what if $\Sigma \vdash c = d$ with dstinct witnesses c and d). What if have constant symbols whichare not in C? What if Σ doesnt' prove enough to define relation and functions for the given symbols? Even if Σ does prove enough, how would we be able to tell?

The main way we shall get aroundthis is by workign with maximally consistent sets of sentences in suitable langauges:

Lemma 2.6.25: Generalization on Sentences

Suppose Σ is a set of sentences, φ is any formula and x is any variable. Then $\Sigma \vdash \varphi$ if and only if $\Sigma \vdash \forall x\varphi$

Note that the key part of this lemma is that we are working with sentences.

***Proof*:**
The members of Σ are sentences, so x is free in none of them, and $\Sigma \vdash \varphi$ gives $\Sigma \vdash \forall x\varphi$ by theorem 2.6.12. Conversely, x is substitutable for x in φ, by induction through definition 1.5.5: in the quantifier clause either x is not free in $(\forall z)(\alpha)$, or z is not x and so does not occur in the term x. Also φ^x_x is φ, since replacing x by x changes nothing. So $\forall x\varphi \to \varphi$ is an instance of **A4**, and $\Sigma \vdash \forall x\varphi$ gives $\Sigma \vdash \varphi$ by proposition 2.6.10(2), completing the proof.

Theorem 2.6.26: Enough Witnesses

Suppose Γ is a consistent set of sentences of $\mathcal{L}$. Then there is a nonempty set C of constant symbols which are *not* symbols of $\mathcal{L}$ such that, for the language $\mathcal{L}' = \mathcal{L} \cup C$ obtained by adding the constant symbols in C to the symbols of $\mathcal{L}$, there is a maximally consistent set Σ of sentences of $\mathcal{L}'$ with $\Gamma \subseteq \Sigma$ for which C is a set of witnesses

***Proof*:**
Deductions are those of definition 2.6.7. Each use of lemma 2.6.8 rests on a propositional implication checked with the clauses of definition 1.7.3.

Step 1: the language $\mathcal{L}'$. Put $\mathcal{L}_0 = \mathcal{L}$. Given $\mathcal{L}_k$, let P_k be the set of pairs (φ, x) of an $\mathcal{L}_k$-formula φ and a variable x such that no variable other than x is free in φ, with $P_{-1} = \emptyset$. Let $\mathcal{L}_{k+1}$ be $\mathcal{L}_k$ together with a new constant symbol $c_{(\varphi,x)}$ for each pair $(\varphi, x) \in P_k \setminus P_{k-1}$, distinct pairs receiving distinct symbols. Let $\mathcal{L}'$ be the union of the $\mathcal{L}_k$ and C the set of new constant symbols, which is nonempty as $(x = x, x) \in P_0$. A formula has finitely many symbols, so every $\mathcal{L}'$-formula is an $\mathcal{L}_k$-formula for some k, and every pair (φ, x) of an $\mathcal{L}'$-formula with at most x free lies in some P_k. Call the least such k the level of the pair, and write $c_{(\varphi,x)}$ for its constant, which lies in $\mathcal{L}_{k+1}$ but not in $\mathcal{L}_k$. Let

$$\theta_{(\varphi,x)} :\equiv \left(\exists x\varphi \to \varphi^x_{c_{(\varphi,x)}}\right) \qquad \Theta := \Gamma \cup \left\{\theta_{(\varphi,x)} \;\middle|\; (\varphi, x) \text{ a pair of some level}\right\}$$

Each $\theta_{(\varphi,x)}$ is a sentence of $\mathcal{L}'$, since x is the only variable that may be free in φ.

Step 2: Γ *is consistent as a set of* $\mathcal{L}'$*-sentences.* Suppose D is a deduction in $\mathcal{L}'$ of $\neg(\psi \to \psi)$ from Γ. Let $d_1, \ldots, d_r$ be the members of C occurring in D, and choose distinct variables $y_1, \ldots, y_r$ occurring in no line of D. No d_i occurs in Γ, so applying lemma 2.6.13 r times, replacing d_i by y_i at the i-th time, gives a deduction D' from Γ in which no member of C occurs. Its last line is $\neg(\psi' \to \psi')$, where ψ' is ψ with each d_i replaced by y_i, because the replacement passes through the connectives. Every line of D' is an $\mathcal{L}$-formula, and a logical axiom of $\mathcal{L}'$ that is an $\mathcal{L}$-formula is a logical axiom of $\mathcal{L}$. Such an axiom is a generalization of an instance θ of one of **A1** to **A8**, and θ is an $\mathcal{L}$-formula. For the schemas other than **A4**, the shape of θ and the side condition mention only the formulas involved, so θ is an instance of the same schema in $\mathcal{L}$. Let $\theta :\equiv \forall x\alpha \to \alpha_t^x$ with t substitutable for x in α. If x is free in α, then every symbol of t occurs in α_t^x by lemma 1.5.7(2), so t is an $\mathcal{L}$-term and θ is an instance of **A4** in $\mathcal{L}$. If x is not free in α, then lemma 1.5.7(1) shows that α_t^x and α_x^x are both α and that x is substitutable for x in α, so θ is the instance $\forall x\alpha \to \alpha_x^x$ of **A4** in $\mathcal{L}$. So D' is a deduction in $\mathcal{L}$, and Γ would be inconsistent in $\mathcal{L}$, contrary to hypothesis.

Step 3: adding one witness sentence. Let Δ be a consistent set of $\mathcal{L}'$-sentences, let (φ, x) be a pair, write $c = c_{(\varphi,x)}$, and suppose that c occurs neither in Δ nor in φ. We show that $\Delta \cup \{\theta_{(\varphi,x)}\}$ is consistent. Suppose not. Then $\Delta \cup \{\theta_{(\varphi,x)}\} \vdash \neg(\psi \to \psi)$ for some formula ψ, so $\Delta \vdash \theta_{(\varphi,x)} \to \neg(\psi \to \psi)$ by theorem 2.6.11. From $\{A_0 \to \neg(A_1 \to A_1)\} \models \neg A_0$, then $\{\neg(A_0 \to A_1)\} \models A_0$ and $\{\neg(A_0 \to A_1)\} \models \neg A_1$, lemma 2.6.8 and proposition 2.6.10(4) give

$$\Delta \vdash \exists x\varphi \qquad \Delta \vdash \neg\varphi_c^x$$

By theorem 2.6.14, there is a variable y not occurring in $\neg\varphi_c^x$ with $\Delta \vdash \forall y(\neg\varphi_c^x)_y^c$, where $(\cdot)_y^c$ replaces every occurrence of c by y. As c does not occur in φ, the formula $(\varphi_c^x)_y^c$ is φ_y^x. This is an induction through definition 1.5.1 and definition 1.5.3. In a term, x becomes c and then y, while all other symbols are unchanged because c does not occur in φ. The atomic case and the cases of $\neg$ and $\to$ follow, and for $\varphi :\equiv (\forall z)(\alpha)$ both formulas are φ when z is x, and both are $(\forall z)(\alpha_y^x)$ by the induction hypothesis otherwise. So $\Delta \vdash \forall y(\neg\varphi)_y^x$. If y is x, this formula is $\forall x\neg\varphi$. Otherwise y does not occur in φ, since the symbols of φ other than the free occurrences of x all occur in φ_c^x, and lemma 2.6.15 with proposition 2.6.10(2) and (3) gives $\Delta \vdash \forall x\neg\varphi$ again. The formula $\exists x\varphi$ is $\neg\forall x\neg\varphi$, so $\{A_0, \neg A_0\} \models \neg(A_0 \to A_0)$ and lemma 2.6.8 give $\Delta \vdash \neg(\forall x\neg\varphi \to \forall x\neg\varphi)$, contradicting the consistency of Δ.

Step 4: Θ *is consistent.* By proposition 2.6.17(5) it suffices that every finite subset of Θ is consistent, and every finite subset lies in a set $\Gamma \cup \{\theta_1, \ldots, \theta_m\}$ with $\theta_i = \theta_{(\varphi_i,x_i)}$ for distinct pairs listed in order of nondecreasing level $k_1 \leq \cdots \leq k_m$. We show by induction on $j \leq m$ that $\Delta_j = \Gamma \cup \{\theta_1, \ldots, \theta_j\}$ is consistent. For $j = 0$ this is Step 2. Let $c = c_{(\varphi_{j+1},x_{j+1})}$. It does not occur in Γ, which consists of $\mathcal{L}$-sentences, nor in the $\mathcal{L}_{k_{j+1}}$-formula φ_{j+1}. For $i \leq j$ the sentence θ_i is built from the $\mathcal{L}_{k_i}$-formula φ_i, with $k_i \leq k_{j+1}$, and the constant $c_{(\varphi_i,x_i)}$, which differs from c because the pairs differ. So c does not occur in Δ_j, and Step 3 shows that Δ_{j+1} is consistent.

Step 5: conclusion. By proposition 2.6.21(4), applied in the language $\mathcal{L}'$, there is a maximally consistent set Σ of $\mathcal{L}'$-sentences with $\Gamma \subseteq \Theta \subseteq \Sigma$. If φ is an $\mathcal{L}'$-formula with at most one free variable x, then (φ, x) is a pair of some level, and $\theta_{(\varphi,x)} \in \Sigma$ gives $\Sigma \vdash \exists x\varphi \to \varphi_{c_{(\varphi,x)}}^x$ with $c_{(\varphi,x)} \in C$. Hence C is a set of witnesses for Σ in $\mathcal{L}'$, as we sought to show.

What this theorem shows is that we can always brute force a set of witnesses. If there is not a set of witnesses, we just add one, and if the set does not decide enough, we make it decide *everything*.

Lemma 2.6.27: Closed Terms And Equality

Let $\mathcal{L}$ be a first-order language, and let $t, u, w, t_1, \ldots, t_n, u_1, \ldots, u_n$ be closed terms of $\mathcal{L}$, that is terms in which no variable occurs. Let f be an n-ary function symbol and R an n-ary relation symbol of $\mathcal{L}$, and let $E = \{t_1 = u_1, \ldots, t_n = u_n\}$. Then, for deductions as in definition 2.6.7,

1. t is substitutable for every variable x in every formula φ, and $\vdash \varphi$ implies $\vdash \varphi^x_t$
2. $\vdash t = t$
3. $\{t = u\} \vdash u = t$
4. $\{t = u, u = w\} \vdash t = w$
5. $E \vdash ft_1 \cdots t_n = fu_1 \cdots u_n$
6. $E \cup \{Rt_1 \cdots t_n\} \vdash Ru_1 \cdots u_n$

***Proof*:**
Here $r = r'$ stands for the atomic formula $= rr'$ of definition 1.1.5, and **A1** to **A8** are the schemas of definition 2.6.4.

Step 1: item (1). Let r be a closed term. It is substitutable for x in φ, by induction through definition 1.5.5. The atomic case is automatic, and the cases of $\neg$ and $\to$ follow from the induction hypothesis. If $\varphi :\equiv (\forall y)(\alpha)$, then y does not occur in r, which contains no variable, and r is substitutable for x in α by the induction hypothesis. Now let $\vdash \varphi$. By theorem 2.6.12, applied with the empty set of hypotheses, $\vdash \forall x\varphi$. As $\forall x\varphi \to \varphi^x_r$ is an instance of **A4**, proposition 2.6.10(2) gives $\vdash \varphi^x_r$.

Step 2: closed instances. Let φ be a formula without quantifiers. By induction through definition 1.5.1 and definition 1.5.3, the formula φ^x_r is φ with every occurrence of x replaced by r, and it has no quantifiers. Let $z_1, \ldots, z_m$ be distinct variables and $r_1, \ldots, r_m$ closed terms, and suppose $\vdash \varphi$. Applying item (1) for z_1 and r_1, then for z_2 and r_2, and so on, gives $\vdash \varphi'$, where φ' is φ with every occurrence of each z_i replaced by r_i, because the closed terms inserted at one stage contain no variable that a later stage replaces.

*Step 3: chains of instances of **A8**.* Let $t_1, \ldots, t_n, u_1, \ldots, u_n$ be any closed terms, with E as in the statement, and let $x_1, \ldots, x_n, y_1, \ldots, y_n$ be distinct variables. For a formula φ without quantifiers, write φ' for φ with every occurrence of each x_i replaced by t_i and every occurrence of each y_i replaced by u_i. Let $\alpha_0, \ldots, \alpha_n$ be atomic formulas whose variables are among the x_i and y_i, such that for each $j < n$ the formula α_{j+1} is obtained from α_j by replacing some occurrences of x_{j+1} by y_{j+1}. We show $E \cup \{\alpha'_0\} \vdash \alpha'_n$. For $j < n$, the formula $x_{j+1} = y_{j+1} \to (\alpha_j \to \alpha_{j+1})$ is an instance of **A8**, and it has no quantifiers. Step 2 gives

$$\vdash t_{j+1} = u_{j+1} \to (\alpha'_j \to \alpha'_{j+1})$$

and proposition 2.6.10(3) with two applications of proposition 2.6.10(2) gives $E \cup \{\alpha'_j\} \vdash \alpha'_{j+1}$. By induction on j, it follows that $E \cup \{\alpha'_0\} \vdash \alpha'_j$ for every $j \leq n$. The step from j to $j+1$ is proposition 2.6.10(4), since $E \cup \{\alpha'_0\}$ deduces every member of $E \cup \{\alpha'_j\}$.

Step 4: items (2) to (6). As $x_1 = x_1$ is an instance of **A7**, item (1) with $\varphi :\equiv (x_1 = x_1)$ gives (2). Let P be the relation symbol R, or the symbol $=$ with $n = 2$, and take $\alpha_j :\equiv Py_1 \cdots y_j x_{j+1} \cdots x_n$ in Step 3. Then α'_0 is $Pt_1 \cdots t_n$ and α'_n is $Pu_1 \cdots u_n$, so

$$E \cup \{Pt_1 \cdots t_n\} \vdash Pu_1 \cdots u_n$$

For $P = R$ this is (6). For P the symbol $=$, taking (t_1, u_1, t_2, u_2) to be (t, u, t, t) gives $\{t = u, t = t\} \vdash u = t$, and taking it to be (t, t, u, w) gives $\{t = t, u = w, t = u\} \vdash t = w$. In both cases (2), with proposition 2.6.10(3) and (4), removes the hypothesis $t = t$, which proves (3) and (4). For (5), take $\alpha_j :\equiv (fx_1 \cdots x_n = fy_1 \cdots y_j x_{j+1} \cdots x_n)$ in Step 3, replacing the occurrence of x_{j+1} in the right-hand term. Then α'_0 is $ft_1 \cdots t_n = ft_1 \cdots t_n$, which (2) deduces, and α'_n is $ft_1 \cdots t_n = fu_1 \cdots u_n$. Step 3 with proposition 2.6.10(3) and (4) gives (5), completing the proof.

Theorem 2.6.28: Constructing Structure

Suppose Σ is a maximally consistent set of sentences of $\mathcal{L}'$ and C is a set of witnesses for Σ. Then there is a structure $\mathfrak{M}$ such that $\mathfrak{M} \models \Sigma$.

***Proof*:**
Deductions are those of definition 2.6.7, and proposition 2.6.21 and lemma 2.6.27 are applied in the language $\mathcal{L}'$.

Step 1: the structure. Applied to the formula $x = x$, definition 2.6.22 provides a member of C, so the set T of closed terms of $\mathcal{L}'$, the terms in which no variable occurs, contains C and is nonempty. For $t, u \in T$ write $t \sim u$ when $(t = u) \in \Sigma$. A sentence deduced from a set of sentences contained in Σ is deduced from Σ by proposition 2.6.10(3), and therefore lies in Σ by proposition 2.6.21(1). Formulas built from closed terms without quantifiers are sentences, so items (2) to (6) of lemma 2.6.27 give the following for $t, u, w, t_1, \ldots, t_n, u_1, \ldots, u_n \in T$.

- $(t = t) \in \Sigma$, if $(t = u) \in \Sigma$ then $(u = t) \in \Sigma$, and if $(t = u) \in \Sigma$ and $(u = w) \in \Sigma$ then $(t = w) \in \Sigma$. So $\sim$ is an equivalence relation on T.
- If $t_i \sim u_i$ for $1 \leq i \leq n$, then $(ft_1 \cdots t_n = fu_1 \cdots u_n) \in \Sigma$ for every n-ary function symbol f of $\mathcal{L}'$, and $(Rt_1 \cdots t_n) \in \Sigma$ implies $(Ru_1 \cdots u_n) \in \Sigma$ for every n-ary relation symbol R of $\mathcal{L}'$. As $u_i \sim t_i$ by the first item, the converse implication holds as well.

Write $[t]$ for the class of $t \in T$ and $M := T/\!\sim$ for the nonempty set of classes. Define the $\mathcal{L}'$-structure $\mathfrak{M}$ with universe M by

$$c^{\mathfrak{M}} := [c], \qquad f^{\mathfrak{M}}([t_1], \ldots, [t_n]) := [ft_1 \ldots t_n], \qquad ([t_1], \ldots, [t_n]) \in R^{\mathfrak{M}} \text{ iff } (Rt_1 \ldots t_n) \in \Sigma$$

for every constant symbol c, n-ary function symbol f and n-ary relation symbol R of $\mathcal{L}'$. By the second item, $f^{\mathfrak{M}}$ and $R^{\mathfrak{M}}$ do not depend on the terms chosen to represent the classes. Every element of M is $[t]$ for a closed term t, and $\overline{s}(t) = [t]$ for every closed term t and

every assignment s into $\mathfrak{M}$, by induction on t through definition 1.4.4: $\overline{s}(c) = c^{\mathfrak{M}} = [c]$, and $\overline{s}(ft_1 \cdots t_n) = f^{\mathfrak{M}}(\overline{s}(t_1), \ldots, \overline{s}(t_n)) = f^{\mathfrak{M}}([t_1], \ldots, [t_n]) = [ft_1 \cdots t_n]$.

Step 2: for every $\mathcal{L}'$-sentence τ, $\mathfrak{M} \models \tau$ if and only if $\tau \in \Sigma$. We induct on the number of connectives and quantifiers of τ. Fix an assignment s_0 into $\mathfrak{M}$. For a sentence σ, corollary 1.4.10 and definition 1.4.11 give $\mathfrak{M} \models \sigma$ exactly when $\mathfrak{M} \models \sigma[s_0]$.

- Let τ be atomic. Every variable occurring in τ is free in τ by definition 1.2.2, so the terms of τ are closed. If τ is $t = u$, then $\mathfrak{M} \models \tau[s_0]$ holds exactly when $[t] = [u]$, that is, when $\tau \in \Sigma$. If τ is $Rt_1 \cdots t_n$, then $\mathfrak{M} \models \tau[s_0]$ holds exactly when $([t_1], \ldots, [t_n]) \in R^{\mathfrak{M}}$, that is, when $\tau \in \Sigma$.
- Let $\tau :\equiv (\neg\alpha)$. Then α is a sentence, and $\mathfrak{M} \models \tau$ holds exactly when $\mathfrak{M} \not\models \alpha$ by proposition 1.4.14(1), by the induction hypothesis exactly when $\alpha \notin \Sigma$, and by proposition 2.6.21(2) exactly when $\tau \in \Sigma$.
- Let $\tau :\equiv (\alpha \to \beta)$. Then α and β are sentences by definition 1.2.2, and $\mathfrak{M} \models \tau$ holds exactly when $\mathfrak{M} \not\models \alpha$ or $\mathfrak{M} \models \beta$ by proposition 1.4.14(2), by the induction hypothesis exactly when $\alpha \notin \Sigma$ or $\beta \in \Sigma$, and by proposition 2.6.21(3) exactly when $\tau \in \Sigma$.
- Let $\tau :\equiv (\forall x)(\alpha)$, so that no variable other than x is free in α. For a closed term t, the formula α_t^x is a sentence by lemma 1.5.7(3), as x is the only variable that may be free in α, and it has as many connectives and quantifiers as α by lemma 1.5.7(4). So the induction hypothesis applies to it. By lemma 2.6.27(1), t is substitutable for x in α, so lemma 2.4.1 and $\overline{s_0}(t) = [t]$ give

  $$\mathfrak{M} \models \alpha_t^x \quad \text{exactly when} \quad \mathfrak{M} \models \alpha[s_0(x|[t])]$$

  Suppose $\tau \in \Sigma$. For a closed term t, the formula $\forall x\alpha \to \alpha_t^x$ is an instance of **A4**, so $\Sigma \vdash \alpha_t^x$ by proposition 2.6.10(2), and $\alpha_t^x \in \Sigma$ by proposition 2.6.21(1). By the induction hypothesis $\mathfrak{M} \models \alpha_t^x$, that is $\mathfrak{M} \models \alpha[s_0(x|[t])]$. Every element of M is $[t]$ for a closed term t, so $\mathfrak{M} \models \tau[s_0]$ by the quantifier clause of definition 1.4.5.

  Suppose $\tau \notin \Sigma$, so that $(\neg\tau) \in \Sigma$ by proposition 2.6.21(2). The formula $\exists x\neg\alpha$ is $\neg\forall x\neg\neg\alpha$. From $\emptyset \models \neg\neg A_0 \to A_0$, lemma 2.6.8 gives $\vdash \neg\neg\alpha \to \alpha$. Then theorem 2.6.12, with the empty set of hypotheses, gives $\vdash \forall x(\neg\neg\alpha \to \alpha)$, and the instance $\forall x(\neg\neg\alpha \to \alpha) \to (\forall x\neg\neg\alpha \to \forall x\alpha)$ of **A5** with proposition 2.6.10(2) gives $\vdash \forall x\neg\neg\alpha \to \forall x\alpha$. From $\{A_0 \to A_1\} \models \neg A_1 \to \neg A_0$, lemma 2.6.8 and proposition 2.6.10(4) give

  $$\vdash \neg\forall x\alpha \to \exists x\neg\alpha$$

  and proposition 2.6.10(2) and (3) give $\Sigma \vdash \exists x\neg\alpha$. No variable other than x is free in $\neg\alpha$, so definition 2.6.22 provides $c \in C$ with $\Sigma \vdash \exists x\neg\alpha \to (\neg\alpha)_c^x$, where $(\neg\alpha)_c^x$ is $\neg\alpha_c^x$ by definition 1.5.3. Hence $\Sigma \vdash \neg\alpha_c^x$ by proposition 2.6.10(2), the sentence $\neg\alpha_c^x$ lies in Σ by proposition 2.6.21(1), and $\alpha_c^x \notin \Sigma$ by (2) of the same proposition. By the induction hypothesis $\mathfrak{M} \not\models \alpha_c^x$, that is $\mathfrak{M} \not\models \alpha[s_0(x|[c])]$, so $\mathfrak{M} \not\models \tau$.

In particular $\mathfrak{M} \models \tau$ for every $\tau \in \Sigma$, so $\mathfrak{M} \models \Sigma$, completing the proof.

Combining the two preceding theorems gives model existence.

Corollary 2.6.29: Structure And Satisfaction First Order Logic

Suppose Γ is a consistent set of sentences of a first order language $\mathcal{L}$. Then there is a structure $\mathfrak{M}$ for $\mathcal{L}$ satisfying Γ.

***Proof*:**
By theorem 2.6.26 there is a maximally consistent set $\Sigma \supseteq \Gamma$ of sentences of $\mathcal{L}' = \mathcal{L} \cup C$ for which C is a set of witnesses, and by theorem 2.6.28 an $\mathcal{L}'$-structure $\mathfrak{M}'$ with $\mathfrak{M}' \models \Sigma$, hence $\mathfrak{M}' \models \Gamma$. Let $\mathfrak{M}$ be the $\mathcal{L}$-structure with the universe of $\mathfrak{M}'$ and the interpretations that $\mathfrak{M}'$ gives the symbols of $\mathcal{L}$, forgetting those of the constants in C. The two universes coincide, so an assignment into $\mathfrak{M}$ is an assignment into $\mathfrak{M}'$. For an $\mathcal{L}$-term t the clauses of definition 1.4.4 involve only symbols of $\mathcal{L}$, which $\mathfrak{M}$ and $\mathfrak{M}'$ interpret alike, so induction on t shows that $\overline{s}(t)$ is the same element whether computed in $\mathfrak{M}$ or in $\mathfrak{M}'$. Induction on an $\mathcal{L}$-formula φ through definition 1.4.5 then gives $\mathfrak{M} \models \varphi[s]$ exactly when $\mathfrak{M}' \models \varphi[s]$, for every assignment s. The atomic clauses compare these term values and use $R^{\mathfrak{M}} = R^{\mathfrak{M}'}$, the negation and implication clauses follow from the induction hypothesis, and the quantifier clause ranges over the common universe. Every member of Γ is an $\mathcal{L}$-sentence, so $\mathfrak{M} \models \Gamma$, completing the proof.

With model existence in hand, the remaining theorems follow exactly as in the propositional case.

Theorem 2.6.30: First Order Consistent Iff Satisfiable

A set of sentences Σ in $\mathcal{L}$ is consistent if and only if it is satisfiable

***Proof*:**
If Σ is satisfiable it is consistent by proposition 2.6.17(2). Conversely, if Σ is consistent it has a model by corollary 2.6.29, hence is satisfiable.

Theorem 2.6.31: Completeness Theorem

If α is a sentence and Δ is a set of sentences such that $\Delta \models \alpha$ then $\Delta \vdash \alpha$

***Proof*:**
Deductions are those of definition 2.6.7. We prove the contrapositive: if $\Delta \nvdash \alpha$ then $\Delta \not\models \alpha$. First, $\Delta \cup \{\neg\alpha\}$ is consistent. Otherwise proposition 2.6.17(3) gives $\Delta \cup \{\neg\alpha\} \vdash \alpha$, so $\Delta \vdash \neg\alpha \to \alpha$ by theorem 2.6.11. A truth assignment with $v(A_0) = F$ gives $\neg A_0 \to A_0$ the value F, so $\{\neg A_0 \to A_0\} \models A_0$, and lemma 2.6.8 gives $\{\neg\alpha \to \alpha\} \vdash \alpha$. Then proposition 2.6.10(4) gives $\Delta \vdash \alpha$, which is not the case. The set $\Delta \cup \{\neg\alpha\}$ consists of sentences, so by theorem 2.6.30 it has a model $\mathfrak{M}$. Then $\mathfrak{M} \models \Delta$ and $\mathfrak{M} \models \neg\alpha$, so $\mathfrak{M} \not\models \alpha$ by proposition 1.4.14(1), and therefore $\Delta \not\models \alpha$, as required.

Thus, in first order logic, just like for propositional logic, a sentence is implied by some set of premisses if and only if it has a proof from those premisses

Theorem 2.6.32: Compactness Theorem

A set of sentences Δ is satisfiable if and only if every finite subset of Δ is satisfiable

***Proof*:**
A set of sentences is satisfiable if and only if it is consistent (theorem 2.6.30), and a set of sentences is consistent if and only if every finite subset of it is consistent (proposition 2.6.17(5)). Hence Δ is satisfiable if and only if it is consistent, if and only if every finite subset is consistent, if and only if, again by theorem 2.6.30, every finite subset is satisfiable.

2.7 Applications of Compactness

In this chapter, we shall go over some Model theory and some application of the compactness theorem.

Definition 2.7.1: Model Morphism

Let $\mathcal{L}$ be a first-order language and $\mathfrak{M}, \mathfrak{N}$ two models of $\mathcal{L}$. Then:

1. $\mathfrak{M}$ and and $\mathfrak{N}$ are *isomorphic* if there is a funciont $\varphi : |\mathfrak{M}| - |\mathfrak{N}|$ such that
   (a) F is bijective
   (b) $F(c^{\mathfrak{M}}) = c^{\mathfrak{N}}$
   (c) $Ff^{\mathfrak{M}}(a_1, a_2, ..., a_k) = f^{\mathfrak{N}}(F(a_1), ..., F(a_k))$ for every k-placefunction f of $\mathcal{L}$ and $a_1, a_2, ..., a_k \in |\mathfrak{M}$
   (d) $P^{\mathfrak{M}}(a_1, a_2, ..., a_k)$ hodls if and only if $P^{\mathfrak{N}}(F(a_1), ..., F(a_k))$ for every k-place relation symbol of $\mathcal{L}$ and elements $a_1, a_2, ..., a_k \in |\mathfrak{M}$
2. $\mathfrak{M}$ and $\mathfrak{N}$ ar ei elementarily equivalent if $\mathrm{Th}(\mathfrak{M}) = \mathrm{Th}(\mathfrak{N})$, that is if $\mathfrak{M} \models \sigma$ if and only if $\mathfrak{N} \models \sigma$. We writeR $\mathfrak{M} \equiv \mathfrak{N}$

If $\mathfrak{M} \cong \mathfrak{N}$ then $\mathfrak{M} \equiv \mathfrak{N}$, however the converse need not be true:

Example 2.7.2: Elementary Equivalent, But Not Isomorphic
Let $\mathfrak{FC} = (\mathbb{N})$ be an infinite model of the basic languaage $\mathcal{L}$. Extend $\mathcal{L}$ to $\mathcal{L}_R$ by adding the constant symbol c_r for every real number r and elt Σ be the set of sentences of $\mathcal{L}$ including

1. every stnence τ of $\mathrm{Th}(\mathfrak{C})$
2. $\neg c_r = c_s$ for eery pair of real numbers r, s such that $r \neq s$

Then every fintie subset of Σ is satisraible. hence by the compatness theorem there is a model $\mathfrak{U}'$ of $\mathcal{L}_R$ satisfying Σ and hence $\mathrm{Th}(\mathfrak{C})$. Let $\mathfrak{U}$ be the model we get by dropping the interpretations of all the constant symbols c_r from $\mathfrak{U}'$ which is then a model of $\mathcal{L}$. Note that $|\mathfrak{U}| = |\mathfrak{U}'|$ since $\mathfrak{U}'$ requires a distinct element of the universe to itnerpret each constant symbol c_r of $\mathcal{L}_R$. Sicne $\mathrm{Th}(\mathfrak{C})$ is maximally consistent set of sentences of $\mathcal{L}$, it follows that $\mathfrak{C} \equiv \mathfrak{U}$. However, $\mathfrak{C}$ cannot be isomorphic to $\mathfrak{U}\mathfrak{K}$ since the cardinalities do not match.

(there are lots of cool examples to add here, especially the non-standard models of the real numbers)

(I would love to add Löwenheim–Skolem theorem and Lefschetz Principle to prove the Ax-Grothendieck Theorem, that is any injective regular function $f : V \to V$ between two varieties over an algebraically closed field (of characteristic zero) is surjective. This utilises those two theorems and compactness)

2.8 Summary

Due to how the course was run for me, I will be summarising this a bit differently for now. I will come back to this another time to add a more comprehensive summary another time

1. **Axiom Schema**: these are the logical axioms:

   A1 $(\alpha \to (\beta \to \alpha))$
   A2 $((\alpha \to (\beta \to \gamma)) \to ((\alpha \to \beta) \to (\alpha \to \gamma)))$
   A3 $(((\neg\beta) \to (\neg\alpha)) \to (((\neg\beta) \to \alpha) \to \beta))$

2. **Modus Ponens**: this is our one rule of inference: $(\{\varphi, (\varphi \to \psi)\}, \psi)$

3. **Deduction** Take a finite sequence of $\mathcal{L}$-formulas $(\varphi_1, \varphi_2, ..., \varphi_n)$. Then this is a *deduction* from Σ if:

   (a) φ_i is an axiom in the axiom schema (i.e. a logical axioms)
   (b) $\varphi_i \in \Sigma$ (i.e. a non-logical axiom)
   (c) φ_k follows from φ_i, φ_j $(i, j < k)$ and modus ponens.

4. **Examples** Look at example 2.5.4 for a list of important examples, and lemma 2.5.5 for important shortcuts

5. **Contradiction** $\{\delta, \neg\delta\} \vdash \gamma$

6. **Deduction Theorem**
$$\Sigma \vdash \alpha \to \beta \qquad \text{iff} \qquad \Sigma \cup \{\alpha\} \vdash \beta$$

7. **More examples** Look at example 2.5.8, Modus Tollens (lemma 2.5.9), and corollary 2.5.10 for more important deductions to have at your finger tips.

8. **Soundness Theorem** $\Delta \vdash \alpha$. Then $\Delta \models \alpha$. This one is easy to do with some good inductive reasoning. The rest of the results for propositional deductions builds towards the converse of the Soundness Theorem.

9. **Satisfiability and Consistency**. A set is *inconsistent* if $\Gamma \vdash \neg(\alpha \to \alpha)$ (i.e. it implies a contradiction). If Γ is satisfiable, then Γ is consistent (hence, if *a* truth assignment function works, Γ at least does not deduce a contradiction)

10. **Inconsistency**: If inconsistent, then can deduce anything, and only requires finitely many formulas to deduce the inconsistency. The converse of the last statement is that a set is consistent if and only if every finite subset is consistent.

11. **Maximally Consistent**: Σ is maximally consistent if when adding a new formula, Σ becomes inconsistent. Maximally consistent sets really behave like logical objects:

(a) $\Sigma \vdash \varphi$ then $\varphi \in \Sigma$

(b) $\neg\varphi \in \Sigma$ if and only if $\varphi \notin \Sigma$

(c) $\varphi \to \psi \in \Sigma$ if and only if $\varphi \notin \Sigma$ or $\psi \in \Sigma$ (i.e. if $\varphi \notin \Sigma$, it's the vacuous truth case, or if $\varphi \in \Sigma$ then (like in modus ponens) $\psi \in \Sigma$)

12. **Truth Assignment via Maximally Consistent Sets**: Simply do $v(\varphi) = T$ if $\varphi \in \Sigma$ and F otherwise. Then trivially, A maximally consistent set is satisfiable with the above truth assignment function. We next ensure that for any consistent set Γ, we can embed it in a maximally consistent set Σ, and since a set is satisfiable if and only if every subset is satisfiable, we got that *consistent implies satisfiable*. By our earlier result, we have that *consistent if and only if satisfiable*

13. **Completeness Theorem**: $\Sigma \models \varphi$ then $\Sigma \vdash \varphi$.

14. **Compactness Theorem** Γ is satisfiable if and only if every finite subset is satisfiable. This follows form the equivalence with consistency in which this holds.

15. **Generalization**: We know move on to deduction for first-order logic. If φ is any formula and $x_1, x_2, ..., x_n$ any variables, $\forall x_1 ... \forall x_n \varphi$ is the *generalization* of φ. Note that the generalization of a tautology is a tautology.

16. **First order Axiom Schema**:

(a) **A1** $(\alpha \to (\beta \to \alpha))$

(b) **A2** $((\alpha \to (\beta \to \gamma)) \to (\alpha \to \beta) \to (\alpha \to \gamma)))$

(c) **A3** $(((\neg\beta) \to (\neg\alpha)) \to ((\neg\beta) \to \alpha) \to \beta))$

(d) **A4** $(\forall x\alpha \to \alpha_t^x)$ if t is substitutable for x in α

(e) **A5** $(\forall x(\alpha \to \beta) \to (\forall x\alpha \to \forall x\beta))$

(f) **A6** $(\alpha \to \forall x\alpha)$ if x does not occur free in α (i.e. if x is bounded or does not exist)

(g) **A7** $x = x$

(h) **A8** $(x = y \to (\alpha \to \beta))$ if α is atomic and β is obtained by replacing some occurrences (possibly all or none) of x in α by y.

17. **Repeat**: deductions and MP are defined the same way. The properties of deductions carry over, and so does the Deduction Theorem

18. **Generalization Theorem**: If x is bound or does not exist in φ and $\Gamma \vdash \varphi$, then $\Gamma \vdash \forall x\varphi$.

19. **Generalization on Constants**: What this theorem is saying is that if we prove a property of a constant, say x_0 is a constant and $\vdash P(x_0)$, then we in fact have $\vdash \forall x P(x)$

20. **More Examples** Look at example 2.6.9 and example 2.6.16 for more good examples to have at your finger tips.

21. $Th(\mathfrak{M})$ **is maximally consistent**. Define

$$Th(\mathfrak{M}) = \{\tau \mid \tau \text{ is a sentence an } \mathfrak{M} \models \tau\}$$

Then it is easy to show it is maximally consistent, parallelising our earlier need in propositional deduction for showing that maximally consistent then satisfiable. However, this time we have constants to deal with. Hence

22. **Witness** In first order deductions, we have the existential quantifier, which means that we require a way to "witness" the existence. That's what the set of witnesses does. It is not too hard to show that there is enough witnesses, and that a maximally consistent set Σ along with a set of witnesses C will together be enough to show that there is a $\mathfrak{M}$ such that $\mathfrak{M} \models \Sigma$, that is Σ is satisfiable. The rest follows from the same results almost identical proofs from propositional deduction

3

Computability

In this chapter, we open a new discussion. We are going to characterize a type of function known as a *computable function*. These are functions that, roughly speaking, we can actually "work out" as human being. Since we can work it out, we may try to encode the process into an artificial thinking machine to grind the computation faster (a computer) and get an output for a given input. In other words, computable functions are those for which we can give an input and a set of transformation rules, and in a finite (or countably infinite) amount of time, we can deduce an output. Though it might seem like this is "obvious", mathematics as its now understood does have incomputable functions. For example, $\mathbb{R}$ an $\mathbb{R}^2$ have the same cardinality, which by definition means they are in bijection to each other: $f : \mathbb{R} \to \mathbb{R}^2$. However, it is impossible to define a function where when I give you an input $x \in \mathbb{R}$, and you can tell me $f(x)$ in a finite or even [countably] infinite amount of time. This is because the "construction" of the function used the [uncountable] axiom of choice, which instead of giving a way of choosing each element, it simply asserts that no matter how many sets there are (in our case, $\{x\}$ for each $x \in \mathbb{R}$), we may always choose an element from each set. Due to the existence of uncountable sets (like $\mathbb{R}$), we get some counter-intuitive results which spurred the development of much of logic for the first half of the 20th century.

We are thus going to look at functions in which we can actually find ways of getting values out of them. Our discussion will start with defining what it means to be able to "compute" values. We then use this definition to define computable functions, and then classify all possible computation functions in terms of a small set of initial function that we can use to build all computable functions using composition and two types of recursion. This discussion will lead us to show that in any model of first order logic that tries to capture the rules of arithmetic's, we can always find a contradiction.

3.1 Turing Machines

Fundamentally, a Turing machine tries to capture the idea that we may take an input and manipulate it in some way to get an output using rules of transformation. In essence, a Turing machine will require memory, rules of deduction and a way to take in inputs and give outputs. We start off with the input of a Turing machine. The input will be manipulated by the Turing machine to produce an output.

Definition 3.1.1: Tape

A *tape* is an infinite sequence

$$a = a_0 a_1 a_2 \cdots$$

such that for each integer i the *cell* $a_i \in \{0, 1\}$. The ith cell is said to be *blank* if a_i is zero and is *marked* if $a_i = 1$. A tape is *blank* if each cell is 0

As a convention, we will assume that all but finitely many cells of any given tape is blank, and that any cell not explicitly described is blank. We will omit as much of the tape as possible, and thus write

$$01011011011$$

to represent the tape 01011011011000..., and we will sometimes put superscript to abbreviate further, for example:

$$01^3 01^{25} 001$$

If σ is any finite sequence, σ^n is $n$copies of the sequence concatenated.

Definition 3.1.2: Tape Position

A *tape position* is a triple $(s, i, \mathbf{a})$ where s and i are natural numbers with $s > 0$, and $\mathbf{a}$ is a tape. Given a tape position $(s, i, \mathbf{a})$, we will refer to cell i as the *scanned cell* and s as the *state*.

We will usually display the tape position by underlining the scanned cell and put the state number to the left. For example, $(2, 4, \mathbf{a})$ can be represented as:

$$2 : 010\underline{1}1001011011$$

that is, it is a tape position at state two with the 4th cell being scanned.

Definition 3.1.3: Turing Machine

A *Turing machine* is a function M such that for some natural number n such that the domain is a subset of

$$\{1, ..., n\} \times \{0, 1\} = \{(s, b) \mid 1 \leq s \leq n,\ b \in \{0, 1\}\}$$

and the range is a subset of

$$\{0, 1\} \times \{-1, 1\} \times \{1, ..., n\}$$

or $\{(c, d, t) \mid c \in \{0, 1\},\ d \in \{-1, 1\},\ 1 \leq t \leq n\}$. We would write the domain and range as Dom and Range so $M : \text{Dom}(M) \to \text{Range}(M)$.

A possible source of confusion is the difference between the input of a Turing machine, the output of a Turing machine, and the tape position. The following summarizes these to diminish confusion:

1. (s, v) is the *state* and the *value* at the tape, so $M(s, v) = (o, d, s')$ maps to ($o$utput, $d$irection, $s$tate) with output values in $\{0, 1\}$, direction values $\{-1, 1\}$ (i.e. left or right) and state value $\{1, 2, ..., n\}$.
2. $(s, c, \mathbf{a})$ is the *state*, the *cell number*, and the *input tape* with $s \in \{1, 2, ..., n\}$ and $c \in \mathbb{N}$.

Often, we shall refer to a Turing machine simply as machine or TM. If $n \geq 1$ is least such that M satisfies the above, we call M an n-state Turing machine and $\{1, ..., n\}$ is the *states* of M. Here is how to interpret M. Let's say $M(s, c) = (b, d, t)$. What M will do is scan at the current cell, overwrite the symbol, move left or right, and execute the next instruction. In particular, if M is in state s (executing instruction number s) and the scanner is presently reading c in cell i, then the machine M should:

1. set $a_i = b$ (replace c with b in the scanned cell)
2. move the scanner to a_{i+d} (move it left if $d = -1$ and right if $d = 1$)
3. enter state t (go to instruction t)

If M is not equipped to handle input c at instruction s (so $M(s, c)$ is undefined), then the computation in progress will stop, or *halt*.

Example 3.1.4: Turing Machine
We usually visualize Turing machines using table. Take the following

M	0	1
1	$1R2$	$0R1$
2	$0L2$	

where L represents -1 and R represents 1. Then

$$M(1,0) = (1,1,2)$$
$$M(1,1) = (0,1,1)$$
$$M(2,0) = (0,-1,2)$$

and $M(2,1)$ is undefined. Thus we have

$$M\{(1,0),(1,1),(2,0)\} \to \{(1,1,2),(0,1,1),(0,-1,2)\}$$

Let's say the tap on which M was acting on was in position:

$$1 : 0100\underline{1}1111o$$

Then, since it was in state 1 while scanning a cell containing 0, it would:

1. write 1 in the scanned cell
2. more the scanner one cell to the right
3. go to state 2

Since M does not know what to do with input 1 in state 2, it would halt, ending the computation

This leads us to ask questions of *computability*. Informally, a computation is a sequence of actions of a machine M on a tape according to the given rules described above, starting with instruction 1 and the scanner at cell a_1 (or a_0 if you are a computer scientist) on the given tape. A computation ends (*halts*) when and if the machine encounters a tape position which it does not know what to do in. If it never halts, and does not *crash* by running the scanner off the left and of the tape[1], the computation never ends. We make this notion rigorous

Definition 3.1.5: Computation

Let M be a Turing machine. Then

1. if $p = (s, i, \mathbf{a})$ is a tape position and $M(s, a_i) = (b, d, t)$ is defined, then $\mathbf{M}(p) = (t, i+d, \mathbf{a}')$ is the *successor tape position* where $a'_i = b$ and $a'_j = a_j$ whenever $j \neq i$
2. A *partial computation* with respect to M is a sequence $p_1 p_2 ...$ of tape positions such that $p_{l+1} = \mathbf{M}(p_l)$ for each $l < k$.
3. A partial computation $p_1, p_2, ..., p_k$ with respect to M is a *computation* (with respect to M) with *input tape* $\mathbf{a}$ if $p_1 = (1, 0, \mathbf{a})$ and $\mathbf{M}(p_k)$ is undefined (and *not* because the scanner run off the end of the tape). The *output tape* of the computation is the tape of the final tape position p_k.

As we have defined things, a partial computation is a computation only if the Truing machine halts but does not crash in the final tape position. The requirement that it halts means that any computation can only have finitely many steps. Unless stated otherwise, all partial computations on

[1] some author's treat this as another way of haling

a given input begins in state 1.

Example 3.1.6: Computations

1. Let M be the same Turing machine as in example 3.1.4. Let $\mathbf{a} = 1100$. Then the initial starting point must be
$$1 : \underline{1}100$$
Then going trough the computations, we get:
$$\begin{aligned} 1 &: 0\underline{1}00 \\ 1 &: 00\underline{0}0 \\ 1 &: 001\underline{0} \\ 1 &: 0\underline{1}0 \end{aligned}$$
Since $M(2,1)$ is undefined, the process terminates, hence we have given a computation.
2. (look at homework and quiz for further examples)

It will be useful for future purposes to have a library of Turing machines that manipulate blocks of 1s in various ways and very useful to be able to combine machines performing simpler tasks to perform more complex ones. (give such Turing machines here)

As we have defined Turing machines, we have set ourselves many (potential) limitations. For example, why only input 0 and 1? Why have only one tape? Here are a list of possible additions and expansions we may wish to add to our definition:

1. The machines can move the scanner many steps at a time
2. any finite alphabet of at least two symbols can e use
3. Separate the read and write heads
4. have multiple heads
5. have the tape be two-way infinite
6. have multiple tapes
7. have two or higher dimensional tapes

We shall see that anything that is computable on any of the possible augmentations of Turing machines given from any combinations of the above features is in fact computable on a Turing machine, and so in terms of computability these augmentations are "redundant" (they do not add any extra computations).

(I skipped this cause its routine checks. I looked over it, and if I need to in the future I will write about it)

From now on, we shall assume we may have a Turing machine augmented with any of the above additions, since we have shown they are all simulated by Turing Machines.

3.2 Computable Functions and Relations

We shall discuss what it means to have a computable function and computable relations. From here on out, functions may be *partial functions*, in which case we shall explicitly specify what is the domain and codomain of f on which f becomes a function, or write $f : \text{Dom}(f) \to Y$. Note that any k-place function is a particular $k+1$-place relation:

$$R_f(n_1, n_2, ..., n_k, m) \quad \Leftrightarrow \quad f(n_1, n_2, ..., n_k) = m$$

and if R is a k-place relation, there is an induced function known as the *characteristic function* of R defined by:

$$\chi_R(n_1, n_2, ..., n_k) = \begin{cases} 1 & \text{if } R(n_1, n_2, ..., n_k) \text{ is true} \\ 0 & \text{if } R(n_1, n_2, ..., n_k) \text{ is false} \end{cases}$$

Natural numbers on tapes will be represented in unary notation: n will be represented as 1^{n+1} (the extra 1 serves as "memory" for the Turing machine to be aware of some of it's actions). If $(n_1, n_2, ..., n_k) \in \mathbb{N}^k$, then this will be represented as

$$1^{n_1+1}01^{n_2+1}...01^{n_k+1}$$

This way of defining natural numbers on a tape is not terribly space efficient (think of binary representations), but it will be Turing-machine-computation friendly.

Definition 3.2.1: [Turing] Computable Function

Let $f : \mathbb{N}^k \to \mathbb{N}$ be a partial function. Then f is said to be *[Turing] computable* (or just *computable*) if for any $(n_1, n_2, ..., n_k) \in \text{Dom}(f)$, the computation of M with input tape

$$\underline{0}1^{n_1+1}01^{n_2+1}...01^{n_k+1}$$

will halt with output tape

$$\underline{0}1^{f(n_1,n_2,...,n_k)+1}$$

A machine M that satisfies the above condition is said to *compute* f.

Example 3.2.2: Computable Functions

1. Show that $\iota : \mathbb{N} \to \mathbb{N}$ defined by $\iota(n) = n$ is computable with machine $M = \emptyset$.
2. Show that $f(n, m) = n + m$ is computable
3. show that $\pi_1 : \mathbb{N}^2 \to \mathbb{N}$ giving $\pi(n, m) = n$ is computable
4. show that $O : \mathbb{N} \to \mathbb{N}$ defined by $O(n) = 0$ is computable.
5. Show that

$$PRED(n) = \begin{cases} n-1 & n \geq 1 \\ 0 & n = 0 \end{cases}$$

is computable.

6. Show that
$$Diff(n) = \begin{cases} n-m & n \geq m \\ 0 & n < m \end{cases}$$
is computable.

7. show that any projection functions $\pi_i : \mathbb{N}^k \to \mathbb{N}$ is computable.

Note that not all functions $f : \mathbb{N}^k \to \mathbb{N}$ are computable, even when $k = 1$. To see this, note that there are $\mathbb{N}^{\mathbb{N}}$ number of functions from $\mathbb{N}$ to $\mathbb{N}$, i.e. uncountably many. By definition, a Turing machine has a finite domain and codomain. All possible Turing machines over a particular finite domain and codomain is countable, and the union of all such machines is a countable union which (by the countable axiom of choice) is still countable. Hence, there must be non-computable functions. Hence, there are more unique functions of the form $f : \mathbb{N} \to \mathbb{N}$ than there are unique Turing machines to show that each function is computable.

For a concrete example (which will be given, but not proven), consider the following:

Definition 3.2.3: Busy Beaver Competition

A machine M is an *n-state entry in the busy beaver competition* if:

1. M has a two-way infinite tape and alphabet $\{1\}$[a]
2. M has $n+1$ states, but state $n+1$ is used only for halting ($M(n+1,0)$ and $M(n+1,1)$ is undefined)
3. M eventually halts when given a blank input tape

The *score* of M in the competition is the number of 1's on the output tape of its computation from a blank input tape. The maximum of all possible scores for n-state entry machine is denoted $\Sigma(n)$

[a] Remember that such a machine can be converted to a Turing Machine

The function $\Sigma(n)$ is well-defined as there are only finitely many Turing $n+1$ state machines (with alphabet $\{1\}$) and since we are taking the subset which must halt, a maximum must be achieved.

Example 3.2.4: Busy Beaver Competition

1. The only 0-state entry is $M = \emptyset$, and so trivially $\Sigma(0) = 0$
2. Define P as

P	0	1
1	1R2	1L2
2	1L1	1L3

Then P is a 2-tate entry in the busy beaver competition with a score of 4, so $\Sigma(2) \geq 4$.

3. Show that $\Sigma(1) = 1$
4. Show that $\Sigma(3) \geq 6$
5. Show that $\Sigma(n) < \Sigma(n+1)$

The function Σ grows very quickly. We know that:

$$\Sigma(0) = 0 \quad \Sigma(1) = 1 \quad \Sigma(2) = 4 \quad \Sigma(3) = 6 \quad \Sigma(4) = 13$$

and $\Sigma(5) = 4098$ [5, Theorem 6.1]. The bbchallenge collaboration obtained the last value from a computer proof, completed in 2024 and checked in the Coq proof assistant, that decides for every 5-state machine started on a blank tape whether it halts. Up to renaming the states $2, \ldots, 5$, exchanging left with right, changing transitions that are never used, and the direction of the final move into the halting state, two 5-state entries score 4098. One of them, the machine with the longest run, halts after 47,176,870 steps [5, Theorem 1.1 and Appendix B].

Proposition 3.2.5: Non-Computable Function

The function Σ is *not* computable by any Turing machine

***Proof*:**
An entry in the busy beaver competition (definition 3.2.3) runs on a two-way infinite tape. We read such a run as in definition 3.1.5, except that a tape is a function $\mathbf{a}\colon \mathbb{Z} \to \{0,1\}$ with finitely many marked cells and a tape position $(s, i, \mathbf{a})$ allows any $i \in \mathbb{Z}$. The successor of $(s, i, \mathbf{a})$ is $(t, i+d, \mathbf{a}')$ when $M(s, a_i) = (b, d, t)$, where $a'_i = b$ and $a'_j = a_j$ for $j \neq i$. The run from the blank tape starts at $(1, 0, \mathbf{a})$ with $\mathbf{a}$ blank, and it halts at the first position $(s, i, \mathbf{a})$ at which $M(s, a_i)$ is undefined. Such a run cannot leave the tape.

Suppose, for a contradiction, that a k-state Turing machine M computes Σ in the sense of definition 3.2.1. For each $n \in \mathbb{N}$ we build an entry B_n from three parts, and we write $N := n+1$.

Step 1: the writer. For $1 \leq i \leq n+1$, the state w_i has the single instruction $B_n(w_i, 0) = (1, -1, w_{i+1})$, where w_{n+2} is the first state d_1 of the next part. From the blank tape, starting in w_1 at cell 0, the writer marks the cells $0, -1, \ldots, -n$ and stops in state d_1 at the blank cell $h := -(n+1)$. The marked cells are then exactly $h+1, \ldots, h+N$.

Step 2: the doubler. The twelve states $d_1, \ldots, d_{12}$ have the instructions below, in the notation of example 3.1.4, where m_1 is the first state of M and an empty entry is undefined.

	0	1
d_1	$0Rd_2$	
d_2		$0Rd_3$
d_3	$0Rd_9$	$1Rd_4$
d_4	$0Rd_5$	$1Rd_4$
d_5	$1Rd_6$	$1Rd_5$
d_6	$1Ld_7$	
d_7	$0Ld_8$	$1Ld_7$
d_8	$0Rd_2$	$1Ld_8$
d_9	$1Rd_{10}$	$1Rd_9$
d_{10}	$1Ld_{11}$	
d_{11}	$0Rd_{12}$	$1Ld_{11}$
d_{12}		$1Lm_1$

For $0 \leq j \leq N-1$ let I_j be the position in state d_2 at cell $h+1+j$ whose marked cells are exactly $h+1+j, \ldots, h+N$ and $h+N+2, \ldots, h+N+1+2j$. After Step 1, the state d_1 moves the head from h to $h+1$, which gives I_0. If $j \leq N-2$, then the run passes from I_j to I_{j+1} through the following moves.

- The state d_2 erases cell $h+1+j$ and moves right, and d_3 reads the marked cell $h+2+j$ and moves right into d_4.
- The state d_4 moves right across the marked cells up to $h+N$, reads the blank cell $h+N+1$ and moves right into d_5.
- The state d_5 moves right across the $2j$ marked cells $h+N+2, \ldots, h+N+1+2j$ and marks the blank cell $h+N+2+2j$, then d_6 marks $h+N+3+2j$ and moves left into d_7.
- The state d_7 moves left across the marked cells down to $h+N+2$, reads the blank cell $h+N+1$ and moves left into d_8 at cell $h+N$, which is marked because $j+2 \leq N$.
- The state d_8 moves left across the marked cells down to $h+2+j$, reads the blank cell $h+1+j$ and moves right into d_2 at cell $h+2+j$.

From I_{N-1}, the state d_2 erases cell $h+N$, then d_3 reads the blank cell $h+N+1$ and moves right into d_9, which crosses the $2N-2$ marked cells $h+N+2, \ldots, h+3N-1$ and marks $h+3N$. Then d_{10} marks $h+3N+1$ and moves left, d_{11} moves left across the marked cells down to $h+N+2$, reads the blank cell $h+N+1$ and moves right, and d_{12} reads the marked cell $h+N+2$ and moves left into m_1. The doubler therefore ends in state m_1 at the blank cell $c := h+N+1$, and the marked cells are exactly $c+1, \ldots, c+2N$. Relative to the cell c, this is the start of a computation on the input tape 01^{2N}, which represents the number $2N-1 = 2n+1$.

Step 3: the copy of M, and the halting state. The states of M are renamed $m_1, \ldots, m_k$ and keep their instructions. For every state m_s and symbol b at which M has no instruction, B_n gets the instruction $B_n(m_s, b) = (b, 1, H)$, for a new state H without instructions. Let $p_1, \ldots, p_r$ be the computation of M with input tape 01^{2N}, with $p_l = (s_l, i_l, \mathbf{a}^l)$. It exists because M computes Σ, and its output tape $\mathbf{a}^r$ is $01^{\Sigma(2n+1)+1}$. By induction on $l \leq r$, the run of B_n reaches the position in state m_{s_l} at cell $c+i_l$ in which cell $c+j$ holds a^l_j for every $j \geq 0$ and every cell below c is blank. The case $l = 1$ is the end of Step 2. For $l < r$, the scanned cell holds $a^l_{i_l}$ because $i_l \geq 0$,

and $M(s_l, a^l_{i_l})$ is defined because p_{l+1} exists (definition 3.1.5), so B_n follows the same instruction as M and reaches the position for $l+1$. The cell it writes is $c + i_l \geq c$, so the cells below c stay blank. At $l = r$, the instruction $M(s_r, a^r_{i_r})$ is undefined, so B_n writes back the scanned symbol, moves right and halts in H. Its final tape carries exactly the $\Sigma(2n+1)+1$ marked cells of $\mathbf{a}^r$.

Step 4: counting states. Number the states of B_n as $w_1, \ldots, w_{n+1}$, then $d_1, \ldots, d_{12}$, then $m_1, \ldots, m_k$, then H, so that w_1 is state 1 and H is state $n+k+14$. Every instruction refers to states among these. As M halts on the input tape 01^{2N}, it lacks an instruction somewhere, so H is the target of an instruction of B_n, while H has no instructions. By the convention following definition 3.1.3, B_n thus has $n+k+14$ states, the last of them used only for halting, and it halts on the blank tape. So B_n is an $(n+k+13)$-state entry in the busy beaver competition, with score $\Sigma(2n+1)+1$.

Step 5: the contradiction. Take $n = k+12$. Then $2n+1 = n+k+13$, so B_n is a $(2n+1)$-state entry whose score $\Sigma(2n+1)+1$ exceeds $\Sigma(2n+1)$, the greatest score of a $(2n+1)$-state entry. This contradiction shows that no Turing machine computes Σ, as claimed.

The function Σ and this proposition are due to Radó [3].

Coming back to computational functions, we have that they are very flexible:

Proposition 3.2.6: Composition Is Turing Computable

Let f and g be computable functions such that $g \circ f$ is well-defined. Then $g \circ f$ is computable

***Proof*:**
For simplicity, let's say $f : \mathbb{N}^1 \to \mathbb{N}$ and $g : \mathbb{N} \to \mathbb{N}$. Consider $(g \circ f)(x) = g(f(x))$. By definition of f, there exists a Turing machine, M_f such that when translating x into unary representation, M_f halts on

$$01^{f(x)+1}$$

Since $g \circ f$ is well-defined, $f(x)$ is a valid input for g. Then by the computability of g, there exists a Turing machine M_g such that when translating $f(x)$ into unary representation, M_g halts on

$$01^{g(f(x))+1} \tag{3.1}$$

Thus, we see that we can modify M_f so that instead of halting on the last input, it instead goes to the initial state of M_g. We may combine these two Turing machines and label it $M_{g\circ f}$. Thus, we have a Turing machine such that on input x in unary representation, we get that $M_{g\circ f}$ halts[a] on the tape at the position in equation (3.1). Since this is true for arbitrary input, we see that $g \circ f$ must be computable, completing the proof.

[a] in finite amount of time. For us, halting has to happen in a finite amount of time

Lemma 3.2.7: Composition Of Computable Functions

Let h be an m-place computable function and $g_1, \ldots, g_m$ be k-place computable functions. Then the k-place function $f = h \circ (g_1, \ldots, g_m)$, defined at $(n_1, \ldots, n_k)$ exactly when every $g_j(n_1, \ldots, n_k)$ is defined and h is defined at $(g_1(n_1, \ldots, n_k), \ldots, g_m(n_1, \ldots, n_k))$, is computable.

***Proof*:**
Let M_h and $M_{g_1}, \ldots, M_{g_m}$ compute h and the g_j (definition 3.2.1). We use a machine with $m+2$ tapes, one of the augmentations allowed after example 3.1.6, with input and output on tape 1. Started on the input $\underline{0}1^{n_1+1}0 \cdots 01^{n_k+1}$, it copies the input onto tapes $2, \ldots, m+1$ and runs M_{g_j} on tape $j+1$ for each j in turn. It then writes the resulting blocks one after another onto tape $m+2$ in the input format, and runs M_h on tape $m+2$. Finally it erases tape 1, copies the block of tape $m+2$ onto it, returns to cell 0 and halts.

Let $\vec{n} = (n_1, \ldots, n_k) \in \mathrm{Dom}(f)$. Each $g_j(\vec{n})$ is defined, so the run of M_{g_j} halts with tape $j+1$ reading $\underline{0}1^{g_j(\vec{n})+1}$, and tape $m+2$ then reads

$$\underline{0}1^{g_1(\vec{n})+1}0 \cdots 01^{g_m(\vec{n})+1}$$

As h is defined at $(g_1(\vec{n}), \ldots, g_m(\vec{n}))$, the run of M_h halts with tape $m+2$ reading $\underline{0}1^{f(\vec{n})+1}$. So the machine halts with output tape $\underline{0}1^{f(\vec{n})+1}$, and it computes f, as claimed.

With computable functions being closed under function composition, we may imagine there is a "basis" of computational functions:

Definition 3.2.8: Initial Functions

The following are called the *initial functions*

1. O, the 1-place function such that $O(n) = 0$ for all $\mathbb{N}$
2. S, the 1-place function such that $S(n) = n + 1$
3. $\pi_i^k : \mathbb{N}^k \to \mathbb{N}$ that projects onto the ith component (note that π_1^1 is the identity)

Note that we will be allowed to combine a finite number of inputs and still consider it a composition:

$$(O, S) : \mathbb{N}^2 \to \mathbb{N}^2 \qquad (a, b) \mapsto (0, b+1)$$

Though composing these functions gives us a good family of functions, we are nevertheless limited in the number of computable functions we have by this method as we can see by this next proposition:

Proposition 3.2.9: Composition Of Initial Functions Is Bounded

Let f be a 1-place function obtained from finitely many compositions of initial functions. Then there exists a constant $c \in \mathbb{N}$ such that $f(n) \leq n + c$

***Proof*:**
A composition step may combine several functions into a tuple, as noted after definition 3.2.8, so the functions met along the way may have several places. We therefore prove more: every k-place function g obtained from the initial functions by finitely many compositions has a constant $c_g \in \mathbb{N}$ with

$$g(n_1, \ldots, n_k) \leq \max(n_1, \ldots, n_k) + c_g \qquad \text{for all } (n_1, \ldots, n_k) \in \mathbb{N}^k$$

The induction is on the number of compositions used. For the initial functions, $O(n) = 0 \leq n + 0$, $S(n) = n + 1$, and $\pi_i^k(n_1, \ldots, n_k) = n_i \leq \max(n_1, \ldots, n_k) + 0$. For a composition $g = h \circ (g_1, \ldots, g_m)$, with h an m-place and each g_j a k-place function obtained with fewer compositions, the induction hypothesis gives

$$g(n_1, \ldots, n_k) \leq \max_j g_j(n_1, \ldots, n_k) + c_h \leq \max(n_1, \ldots, n_k) + \max_j c_{g_j} + c_h$$

so $c_g = c_h + \max_j c_{g_j}$ works. For a 1-place function f we have $\max(n) = n$, so $f(n) \leq n + c_f$, as claimed.

This shows us that there are in fact relatively few computable functions developed through composition. We thus will define 2 more methods by which we may construct computable functions which allows for a much greater class of them. We will show these 3 methods will allow us to construct all computable function on $\mathbb{N}$ from the initial functions.

3.2.1 Primitive Recursion

Primitive recursion is what we would usually call recursion in computer science or induction in mathematics. We call it primitive recursion to distinguish it with another recursive method to define functions which we will define in the next section (unbounded minimalization). Primitive recursion will be sufficient to build up essentially all familiar arithmetic functions from initial functions

Definition 3.2.10: Primitive Recursion

Let g be a k-place function ($k \geq 1$), and h a $(k+2)$-place function. Let f be a $(k+1)$-place function such that

1. $f(n_1, n_2, ..., n_k, 0) = g(n_1, n_2, ..., n_k)$
2. $f(n_1, n_2, ..., n_k, m+1) = h(n_1, n_2, ..., n_k, m, f(n_1, n_2, ..., n_k, m))$

for every $(n_1, n_2, ..., n_k) \in \mathbb{N}^k$ and $m \in \mathbb{N}$. Then f is said to be obtained from g and h by *primitive recursion.*

So f is defined inductively: its value at $m = 0$ is given by g, and its value at $m + 1$ by h, from the same inputs, m and the value at m. A 1-place function f with $f(0) = c$ and $f(n+1) = h(n, f(n))$, for a constant c and a 2-place function h, is not of this form, as definition 3.2.10 requires $k \geq 1$. With a variable u carried along unchanged, the 2-place function F with $F(u, 0) = c$ and $F(u, n+1) = h(n, F(u, n))$ is obtained by primitive recursion from the constant 1-place function with value c and the 3-place function $h \circ (\pi_2^3, \pi_3^3)$. By induction on n, $F(u, n) = f(n)$ for every u, so $f(n) = F(n, n)$.

Example 3.2.11: Primitive Recursion

1. $SUM(n,m) = n+m$ can be obtained by primitive recursion from the initial function π_1^1 and $S \circ \pi_3^3$ of initial functions as follows:
   (a) $SUM(n,0) = \pi_1^1(n)$
   (b) $SUM(n,m+1) = (S \circ \pi_3^3)(n,m,SUM(n,m))$

   We may prove this with induction, the base case is $SUM(n,0) = \pi_1^1(n) = n = n+0$. and for $m \geq 0$, assume $SUM(n,m) = n+m$ so that then

$$\begin{aligned} SUM(n,m+1) &= (S \circ \pi_3^3)(n,m,SUM(n,m)) \\ &= S(\pi_3^3(n,m,SUM(n,m))) \\ &= S(SUM(n,m)) \\ &= SUM(n,m)+1 \\ &= n+m+1 \end{aligned}$$

2. $h(n_1,n_2,...,n_k,m) = \sum_{i=0}^{m} g(n_1,n_2,...,n_k,i)$ for any $k \geq 0$ and each g is $(k+1)$-place primitive recursive. Here $h(\vec{n},0) = g(\vec{n},0)$ and $h(\vec{n},m+1) = SUM(h(\vec{n},m), g(\vec{n},m+1))$ for $\vec{n} = (n_1,\ldots,n_k)$, so for $k \geq 1$ the function h arises by primitive recursion from $g \circ (\pi_1^k,\ldots,\pi_k^k, O \circ \pi_1^k)$ and $SUM \circ (\pi_{k+2}^{k+2}, g \circ (\pi_1^{k+2},\ldots,\pi_k^{k+2}, S \circ \pi_{k+1}^{k+2}))$. For $k = 0$, the same recursion with an extra variable u carried along unchanged gives a 2-place function $H(u,m)$, and $h(m) = H(m,m)$.

3. $MULT(n,m) = nm$ is obtained via primitive recursion on O and $SUM \circ (\pi_1^3, \pi_3^3)$ by defining
   (a) $MULT(n,0) = O(n)$
   (b) $MULT(n,m+1) = (SUM \circ (\pi_1^3, \pi_3^3))(n,m,MULT(n,m))$

   Then:

$$\begin{aligned} MULT(n,m+1) &= (SUM \circ (\pi_1^3, \pi_3^3))(n,m,MULT(n,m)) \\ &= SUM(n,MULT(n,m)) \\ &= SUM(n,nm) \\ &= n+nm \\ &= (1+m)n \\ &= n(1+m) \end{aligned}$$

   giving us $MULT(n,m+1) = n(m+1)$.

4. $EXP(n,m) = n^m$ is obtained by primitive recursion from the constant function $S \circ O$ and $MULT \circ (\pi_1^3, \pi_3^3)$, with $MULT$ from (3), by defining $EXP(x,0) = (S \circ O)(x) = 1$ and

$$EXP(x,n+1) = (MULT \circ (\pi_1^3, \pi_3^3))(x,n,EXP(x,n))$$

Then $EXP(x,0) = 1 = x^0$, with $0^0 = 1$, and

$$\begin{aligned} EXP(x,n+1) &= MULT((\pi_1^3, \pi_3^3)(x,n,EXP(x,n))) \\ &= MULT(x, EXP(x,n)) \\ &= MULT(x, x^n) \\ &= xx^n \\ &= x^{n+1} \end{aligned}$$

given $EXP(x,n) = x^n$, so $EXP(x,n) = x^n$ for every n by induction.

5. For $FACT(n) = n!$, define:

$$Fact(n) = \begin{cases} 1 & n = 0 \\ MULT(n, Fact(n-1)) & n > 0 \end{cases}$$

With a variable u carried along unchanged, the 2-place function given by $F(u,0) = 1$ and $F(u,n+1) = MULT(S(n), F(u,n))$ arises by primitive recursion, and $Fact(n) = F(n,n)$ is primitive recursive.

6. The function $PRED$ with $PRED(0) = 0$ and $PRED(n) = n-1$ for $n \geq 1$ is primitive recursive. It satisfies $PRED(0) = 0$ and $PRED(n+1) = n$, so with a variable carried along unchanged, as described before this example, $PRED(n) = P(n,n)$ for the 2-place function P obtained by primitive recursion from O and π_2^3:

$$P(u,0) = O(u) = 0 \qquad P(u,n+1) = \pi_2^3(u,n,P(u,n)) = n$$

7. Recall

$$DIFF(n,m) = \begin{cases} n-m & n \geq m \\ 0 & \text{otherwise} \end{cases}$$

It is obtained by primitive recursion from π_1^1 and $PRED \circ \pi_3^3$, with $PRED$ from (6):

$$DIFF(n,0) = \pi_1^1(n) = n$$
$$DIFF(n,m+1) = (PRED \circ \pi_3^3)(n,m,DIFF(n,m)) = PRED(DIFF(n,m))$$

These equations give the displayed values, by induction on m. For $m = 0$ the value is n. Suppose $DIFF(n,m)$ has its displayed value. If $n \geq m+1$, then $DIFF(n,m) = n - m \geq 1$, and $PRED$ gives $n - (m+1)$. If $n \leq m$, then $DIFF(n,m) = 0$, the case $n = m$ giving $n - m = 0$, and $PRED(0) = 0$.

8. Define

$$AbsDiff(n,m) = |n-m|$$

We will write $n \dot{-} m$ for $DIFF(n,m)$. First, note that:

$$|n-m| = (n \dot{-} m) + (m \dot{-} n) = SUM((n \dot{-} m), (m \dot{-} n))$$

and that

$$m \dot{-} n = \pi_2^2(n,m) \dot{-} \pi_1^2(n,m)$$

Thus:

$$AbsDiff(n,m) = SUM(n \dot{-} m, \pi_2^2(n,m) \dot{-} \pi_1^2(n,m))$$

Proposition 3.2.12: Computable Functions Closed Under Primitive Recursion

Let g be a computable k-place function ($k \geq 1$), h a computable $(k+2)$-place function. Then if f is obtained from g and h by primitive recursion, then f is also Turing computable

***Proof*:**
Let M_g and M_h compute g and h (definition 3.2.1). We use a machine with four tapes, one of the augmentations allowed after example 3.1.6, with input and output on tape 1. Write $\vec{n} = (n_1, \ldots, n_k)$. Started on the input $\underline{0}1^{n_1+1}0 \cdots 01^{n_k+1}01^{m+1}$, the machine copies the first k blocks onto tape 4 and runs M_g there, copies the resulting block onto tape 3, and writes the counter $\underline{0}1$ on tape 2. It then repeats the following round. It compares the block on tape 2 with the last block of tape 1 by moving along both at once. If they have the same length, it erases tape 1, copies the block of tape 3 onto it, returns to cell 0 and halts. Otherwise it erases tape 4 and writes on it the first k blocks of tape 1, then the block of tape 2, then the block of tape 3, in the input format. It runs M_h on tape 4, replaces the block of tape 3 by the resulting block, marks one more cell at the end of the block of tape 2, and starts the next round.

Let $(\vec{n}, m) \in \mathrm{Dom}(f)$, so that $g(\vec{n})$ and every $f(\vec{n}, i)$ with $i \leq m$ are defined. We show by induction on $i \leq m$ that the i-th round starts with tape 2 reading $\underline{0}1^{i+1}$ and tape 3 reading $\underline{0}1^{f(\vec{n},i)+1}$. For $i = 0$, the run of M_g halts with output $\underline{0}1^{g(\vec{n})+1}$ and $g(\vec{n}) = f(\vec{n}, 0)$. If $i < m$, the comparison fails, tape 4 receives the input $\underline{0}1^{n_1+1}0 \cdots 01^{n_k+1}01^{i+1}01^{f(\vec{n},i)+1}$, and since h is defined there with value $f(\vec{n}, i+1)$, the run of M_h halts with output $\underline{0}1^{f(\vec{n},i+1)+1}$, which gives the claim for $i+1$. In the round $i = m$ the comparison succeeds, and the machine halts with output tape $\underline{0}1^{f(\vec{n},m)+1}$. Hence the machine computes f, as claimed.

Definition 3.2.13: Primitive Recursive Function

Let $f : \mathbb{N}^k \to \mathbb{N}$ be a function. Then f is said to be *primitively recursive* if it can be defined from the initial functions by finitely many compositions and primitive recursion.

Lemma 3.2.14: Important Primitive Recursive Functions

The following functions are primitive recursive.

1. For $k \geq 0$ and a primitive recursive $(k+1)$-place function g, the $(k+1)$-place function f given by
$$f(n_1, n_2, ..., n_k, m) = \prod_{i=0}^{m} g(n_1, n_2, ..., n_k, i)$$
2. For a constant $a \in \mathbb{N}$, the function
$$\chi_{\{a\}}(n) = \begin{cases} 0 & n \neq a \\ 1 & n = a \end{cases}$$
3. For a primitive recursive k-place function f and $a, c_1, c_2, ..., c_k \in \mathbb{N}$, the function
$$h(n_1, n_2, ..., n_k) = \begin{cases} f(n_1, n_2, ..., n_k) & (n_1, n_2, ..., n_k) \neq (c_1, c_2, ..., c_k) \\ a & (n_1, n_2, ..., n_k) = (c_1, c_2, ..., c_k) \end{cases}$$

***Proof*:**

Write $\vec{n}$ for $(n_1, \ldots, n_k)$, and for $a \in \mathbb{N}$ write c_a^k for the constant k-place function with value a, which is $S \circ \cdots \circ S \circ O \circ \pi_1^k$ with a applications of S, hence primitive recursive.

1. The function satisfies $f(\vec{n}, 0) = g(\vec{n}, 0)$ and $f(\vec{n}, m+1) = MULT(f(\vec{n}, m), g(\vec{n}, m+1))$, with $MULT$ from example 3.2.11(3). For $k \geq 1$, f arises by primitive recursion (definition 3.2.10) from the k-place function $g \circ (\pi_1^k, \ldots, \pi_k^k, O \circ \pi_1^k)$ and the $(k+2)$-place function $MULT \circ (\pi_{k+2}^{k+2}, g \circ (\pi_1^{k+2}, \ldots, \pi_k^{k+2}, S \circ \pi_{k+1}^{k+2}))$, both compositions of primitive recursive functions. For $k = 0$, the same recursion with an extra variable u carried along unchanged gives a 2-place function $F(u, m)$, and $f(m) = F(m, m)$, as in example 3.2.11(2).
2. The function $\chi_{\{0\}} = DIFF \circ (c_1^1, \pi_1^1)$, with $DIFF$ from example 3.2.11(7), is primitive recursive, and $\chi_{\{0\}}(n) = 1 \dot{-} n$ is 1 for $n = 0$ and 0 for $n \geq 1$. For $a \in \mathbb{N}$, the absolute difference $AbsDiff(n, a)$ of example 3.2.11(8) is 0 exactly when $n = a$, so $\chi_{\{a\}} = \chi_{\{0\}} \circ AbsDiff \circ (\pi_1^1, c_a^1)$, which is primitive recursive.
3. Let $\chi_c(\vec{n}) = \chi_{\{c_1\}}(n_1) \cdots \chi_{\{c_k\}}(n_k)$, the composition of $k-1$ applications of $MULT$ with the functions $\chi_{\{c_i\}} \circ \pi_i^k$, which are primitive recursive by (2). It takes the value 1 at $(c_1, \ldots, c_k)$ and 0 elsewhere. So
$$h = SUM \circ \big(MULT \circ (\chi_c, c_a^k),\ MULT \circ (\chi_{\{0\}} \circ \chi_c, f)\big)$$
with SUM from example 3.2.11(1): at $(c_1, \ldots, c_k)$ the first summand is a and the second is 0, and elsewhere the first is 0 and the second is $f(\vec{n})$. Hence h is primitive recursive.

Theorem 3.2.15: Primitive Recursive, Then Computable

Every primitive recursive function is computable.

***Proof*:**
By definition 3.2.13 a primitive recursive function is built from the initial functions by finitely many compositions and primitive recursions, so the theorem follows by induction on the number of constructions used once each initial function is computable and both constructions preserve computability.

For the initial functions of definition 3.2.8 we describe machines in the input format of definition 3.2.1. For O: on input $\underline{0}1^{n+1}$, move right onto the block of ones, leave the first one in place, erase every later one while moving right, then walk back to cell 0 and halt. The output tape is $\underline{0}1$, and $O(n) = 0$. For S: move right to the first blank cell after the block of ones, mark it, walk back to cell 0 and halt. The output tape is $\underline{0}1^{n+2}$, and $S(n) = n + 1$. For π_i^k: on input $\underline{0}1^{n_1+1}0 \dots 01^{n_k+1}$, if $i = 1$, erase every block after the first and halt at cell 0. If $i \geq 2$, erase the blocks other than the ith together with the zeros separating them, except keep the single one in cell 1 as an anchor marking the left end. Then repeat the following: walk left from the leftmost one of the surviving block; if the first cell to its left is marked, the block abuts the anchor; if it is blank, walk back, shift the block one cell to the left (erase its rightmost one and mark the cell just left of its leftmost one), and repeat. The anchor guarantees that the leftward walk always meets a marked cell before cell 0, so the machine never runs off the tape. Once the block abuts the anchor, erase the rightmost one of the combined block and halt at cell 0. The output tape is $\underline{0}1^{n_i+1}$, since the anchor and the shifted block together carry $n_i + 2$ ones.

A composition step may combine several functions into a tuple, as noted after definition 3.2.8, and computable functions are closed under such compositions by lemma 3.2.7. Computable functions are closed under primitive recursion by proposition 3.2.12. This completes the induction.

We may extend the notion of primitive recursive to relations using our earlier observation of how all relations can be represented through characteristic functions:

Definition 3.2.16: Primitive Recursive Relation

Let $k \geq 1$. Then a k-place relation $P \subseteq \mathbb{N}^k$ is *primitive recursive* if the characteristic function

$$\chi_P(n_1, n_2, ..., n_k) = \begin{cases} 1 & (n_1, n_2, ..., n_k) \in P \\ 0 & (n_1, n_2, ..., n_k) \notin P \end{cases}$$

is primitive recursive

Almost all relations we have seen are primitive recursive:

Example 3.2.17: Primitive Recursive Relations and further Functions
Below, a bounded sum or product whose index starts at a is read as the sum or product from 0 in which each term with index $i < a$ is replaced by 0 or by 1, using the factor $\chi_{\{0\}}(DIFF(a, i))$, which is 1 exactly when $a \leq i$. Bounded sums and products of primitive recursive functions are primitive recursive by example 3.2.11(2) and lemma 3.2.14(1), and so are their compositions with primitive recursive functions, also in the upper limit.

1. The relation $P = \{2\} \subseteq \mathbb{N}$ is primitive recursive, as its characteristic function $\chi_{\{2\}}$ is primitive recursive by lemma 3.2.14(2).

2. If P is primitive recursive, then $\neg P$ (i.e. $\mathbb{N}^k \setminus P$) is primitive recursive, since

$$\chi_{\neg P}(n_1, \ldots, n_k) = \chi_{\{0\}}(\chi_P(n_1, \ldots, n_k))$$

   In particular $\mathbb{N}^* = \mathbb{N} \setminus \{0\}$ has the primitive recursive characteristic function

$$\chi_{\mathbb{N}^*}(n) = \chi_{\{0\}}(\chi_{\{0\}}(n))$$

3. $P \vee Q$ (i.e. $P \cup Q$) is primitive recursive if P, Q are primitive recursive, since $\chi_{P \vee Q} = \chi_{\mathbb{N}^*} \circ SUM \circ (\chi_P, \chi_Q)$.

4. $P \wedge Q$ (i.e. $P \cap Q$) is primitive recursive if P, Q are primitive recursive, since $\chi_{P \wedge Q} = MULT \circ (\chi_P, \chi_Q)$.

5. $EQUAL$, where $EQUAL(n, m) \Leftrightarrow n = m$, is primitive recursive, since $\chi_{EQUAL}(n, m) = \chi_{\{0\}}(AbsDiff(n, m))$ with $AbsDiff$ from example 3.2.11(8). We also write $EQUAL$ for χ_{EQUAL}.

6. DIV, where $DIV(n, m) \Leftrightarrow n \big| m$, that is $nk = m$ for some $k \in \mathbb{N}$. Such a k can be taken with $k \leq m$: if $m = 0$ take $k = 0$, and if $m \geq 1$ then $n, k \geq 1$ and $k \leq nk = m$. Hence

$$\chi_{DIV}(n, m) = \chi_{\mathbb{N}^*}\Big(\sum_{k=0}^{m} EQUAL(MULT(n, k), m)\Big)$$

   is primitive recursive. We also write Div or div for χ_{DIV}.

7. $ISPRIME$, where $ISPRIME(n) \Leftrightarrow n$ is prime. For $n \geq 1$ the number of $k \leq n$ with $k \big| n$ is the number of divisors of n, which is 2 exactly when n is prime, and for $n = 0$ this number is 1, as $0 \big| 0$. Hence $\chi_{ISPRIME}(n) = \chi_{\{2\}}\big(\sum_{k=0}^{n} \chi_{DIV}(k, n)\big)$ is primitive recursive.

8. $Prime(n) = p_n$, also written $p(n)$, where $p_0 = 1$ and p_n is the nth prime for $n \geq 1$. For $x \geq 1$ some prime q has $x < q \leq x! + 1$: a prime factor q of $x! + 1 \geq 2$ cannot satisfy $q \leq x$, since then q would divide $x!$ and hence 1. So p_{n+1} is the least $z \leq p_n! + 1$ with $z > p_n$ and $ISPRIME(z)$. Let $\chi_Q(x, z) = \chi_{\mathbb{N}^*}(DIFF(z, x)) \cdot \chi_{ISPRIME}(z)$, the characteristic function of the relation $Q(x, z)$ that holds when $z > x$ and z is prime. If some $z \leq B$ satisfies $Q(x, z)$, then the least such z is

$$\sum_{y=0}^{B} \prod_{w=0}^{y} \chi_{\{0\}}(\chi_Q(x, w))$$

   because the product is 1 exactly when no $w \leq y$ satisfies $Q(x, w)$. Taking $B = FACT(x) + 1$, with $FACT$ from example 3.2.11(5), defines a primitive recursive function $N(x)$ with $p_{n+1} = N(p_n)$. With a variable u carried along unchanged, the 2-place function given by $F(u, 0) = 1$ and $F(u, n + 1) = N(F(u, n))$ arises by primitive recursion, and $Prime(n) = F(n, n)$ is primitive recursive.

9. $POWER(n, m)$ is the largest $k \leq m$ such that $n^k \big| m$. When $m \geq 1$ and $n \geq 2$ it is the largest k with $n^k \big| m$, because $n^k \big| m$ forces $k < 2^k \leq n^k \leq m$. Also $POWER(n, 0) = 0$. As

$n^j \big| n^k$ for $j \le k$, the $k \le m$ with $n^k \big| m$ are exactly $0, 1, \ldots, POWER(n,m)$, so

$$POWER(n,m) = DIFF\Big(\sum_{k=0}^{m} \chi_{DIV}(EXP(n,k),m),\ 1\Big)$$

is primitive recursive. We also write *Power* or *pow* for it.

10. $LENGTH(n)$ is the largest $\ell \le n$ such that $p_\ell \big| n$. For $n \ge 1$ it is the largest ℓ with $p_\ell \big| n$: the primes increase from $p_1 = 2$, so $p_\ell \ge \ell + 1$, and $p_\ell \big| n$ forces $\ell < p_\ell \le n$. Also $LENGTH(0) = 0$ and $LENGTH(1) = 0$. For $1 \le \ell \le n$, some j with $\ell \le j \le n$ has $p_j \big| n$ exactly when $\ell \le LENGTH(n)$, so

$$LENGTH(n) = \sum_{\ell=1}^{n} \chi_{\mathbb{N}^*}\Big(\sum_{j=\ell}^{n} \chi_{DIV}(Prime(j), n)\Big)$$

is primitive recursive. We also write *Length* for it.

11. $ELEMENT(n,i) = POWER(Prime(i), n)$, also written $Element(n,i)$. If $n = p_1^{n_1} \cdots p_k^{n_k}$ with $n \ge 1$ and $i \ge 1$, then $ELEMENT(n,i) = n_i$ by item (9) and unique factorization, where $n_i = 0$ if $i > k$. It is primitive recursive as a composition of $POWER$ and $Prime$.

12. $SUBSEQ(n,i,j) = p_i^{n_i} p_{i+1}^{n_{i+1}} \cdots p_j^{n_j}$ if $n = p_1^{n_1} \cdots p_k^{n_k}$ with $k = LENGTH(n)$ and $1 \le i \le j \le k$, and $SUBSEQ(n,i,j) = 0$ otherwise. Hence

$$\begin{aligned} SUBSEQ(n,i,j) = {} & \chi_{\mathbb{N}^*}(i) \cdot \chi_{\{0\}}(DIFF(i,j)) \cdot \chi_{\{0\}}(DIFF(j, LENGTH(n))) \\ & \cdot \prod_{t=i}^{j} EXP(Prime(t), ELEMENT(n,t)) \end{aligned}$$

so it is primitive recursive.

13. $CONCAT(n,m) = p_1^{n_1} \cdots p_k^{n_k} p_{k+1}^{m_1} \cdots p_{k+\ell}^{m_\ell}$ if $n = p_1^{n_1} \cdots p_k^{n_k}$ and $m = p_1^{m_1} \cdots p_\ell^{m_\ell}$ with $n, m \ge 1$, $k = LENGTH(n)$ and $\ell = LENGTH(m)$. No prime beyond p_k divides n, by item (10). So, with $E(n,m,t) = EXP(Prime(LENGTH(n) + t), ELEMENT(m,t))$,

$$CONCAT(n,m) = MULT\Big(n, \prod_{t=1}^{LENGTH(m)} E(n,m,t)\Big)$$

which is primitive recursive.

The above collection of primitive recursive functions and relation give us the tools for representing finite sequences of integers by a single integer, as well as tools for manipulating these representations. Thus, at least in principle, we may reduce all problems with primitive recursive functions and relations to problem involving only 1-place primitive recursive functions and relations

Theorem 3.2.18: K-Place To 1-Place Primitive Recursive Function

A k-place function g is primitive recursive if and only if the 1-place function h given by $h(n) = g(Power(p_1, n), \ldots, Power(p_k, n))$ is primitive recursive. In particular $h(n) = g(n_1, n_2, ..., n_k)$ if $n = p_1^{n_1} \cdots p_k^{n_k}$.

***Proof*:**
Here $Power$ is the primitive recursive function of example 3.2.17(9). For fixed k the primes $p_1, \ldots, p_k$ are constants, and a constant function is a composition of S with O, so each 1-place function $n \mapsto Power(p_i, n)$ is primitive recursive. By unique factorization, $Power(p_i, p_1^{n_1} \cdots p_k^{n_k}) = n_i$, which gives the last sentence of the statement.

If g is primitive recursive, then h is the composition of g with the k functions $n \mapsto Power(p_i, n)$, hence primitive recursive. If h is primitive recursive, then for every $(n_1, \ldots, n_k) \in \mathbb{N}^k$

$$g(n_1, \ldots, n_k) = h\Big(\prod_{i=1}^{k} EXP(p_i, n_i)\Big)$$

where EXP is example 3.2.11(4) and the product consists of $k-1$ applications of $MULT$ from example 3.2.11(3). This exhibits g as a composition of primitive recursive functions, so g is primitive recursive, completing the proof.

Lemma 3.2.19: Recognizing Prime Power Codes

Fix $k \geq 1$ and let

$$Code_k(n) = \chi_{\{0\}}\Big(AbsDiff\big(n, \prod_{i=1}^{k} EXP(p_i, Power(p_i, n))\big)\Big)$$

Then $Code_k$ is primitive recursive, $Code_k(n) = 1$ when $n = p_1^{n_1} \cdots p_k^{n_k}$ for some $n_1, \ldots, n_k \in \mathbb{N}$, and $Code_k(n) = 0$ otherwise. Moreover, for every k-place relation P, the 1-place relation $P' = \{p_1^{n_1} \cdots p_k^{n_k} \mid (n_1, \ldots, n_k) \in P\}$ satisfies, for every $n \in \mathbb{N}$,

$$\chi_{P'}(n) = MULT\big(Code_k(n),\ \chi_P(Power(p_1, n), \ldots, Power(p_k, n))\big)$$

***Proof*:**
For fixed k, the primes $p_1, \ldots, p_k$ are constants, and a constant function is a composition of S with O. With $Power$ from example 3.2.17(9), EXP, $MULT$ and $AbsDiff$ from example 3.2.11(4), (3) and (8), and $\chi_{\{0\}}$ from lemma 3.2.14(2), the function $Code_k$ is a composition of primitive recursive functions, the product being $k-1$ applications of $MULT$, so it is primitive recursive. It takes the value 1 exactly when n equals the product. If $n = p_1^{n_1} \cdots p_k^{n_k}$, then $Power(p_i, n) = n_i$ by unique factorization, so the product is n. If n equals the product, then n has this form. This includes $n = 0$, where $Power(p_i, 0) = 0$ by example 3.2.17(9), so the product is $1 \neq 0$ and $Code_k(0) = 0$.

For the identity, if $n = p_1^{n_1} \cdots p_k^{n_k}$, then the exponents of n are determined by unique factorization, so $n \in P'$ exactly when $(n_1, \ldots, n_k) = (Power(p_1, n), \ldots, Power(p_k, n))$ lies in P, and the right side is $\chi_P(n_1, \ldots, n_k) = \chi_{P'}(n)$. Otherwise $n \notin P'$, since every member of P' has this form, and both sides are 0, completing the proof.

Corollary 3.2.20: K-Place To 1-Place Primitive Recursive Relation

A k-place relation P is primitive recursive if and only if the 1-place relation $P' = \{p_1^{n_1} \cdots p_k^{n_k} \mid (n_1, \ldots, n_k) \in P\}$ is primitive recursive, so that

$$(n_1, n_2, ..., n_k) \in P \Leftrightarrow p_1^{n_1} \cdots p_k^{n_k} \in P'$$

***Proof*:**
By lemma 3.2.19, the primitive recursive function $Code_k$ satisfies, for every n,

$$\chi_{P'}(n) = MULT\big(Code_k(n),\ \chi_P(Power(p_1, n), \ldots, Power(p_k, n))\big)$$

If P is primitive recursive, the right side is a composition of primitive recursive functions, with $Power$ from example 3.2.17(9), so P' is primitive recursive. If P' is primitive recursive, then $\chi_P(n_1, \ldots, n_k) = \chi_{P'}\big(\prod_{i=1}^{k} EXP(p_i, n_i)\big)$ is a composition of primitive recursive functions, with EXP and $MULT$ from example 3.2.11(4) and (3), so P is primitive recursive, completing the proof.

Note that not all computable functions are primitive recursive function:

Definition 3.2.21: Ackerman's Function

Define the 2-place function A as follows:

1. $A(0, \ell) = S(\ell)$
2. $A(S(k), 0) = A(k, 1)$
3. $A(S(k), S(\ell)) = A(k, A(S(k), \ell))$

Given A, define the 1-place function α by $\alpha(n) = A(n, n)$. It is not too hard to see that A, and hence α are Turing computable.

Ackermann's function essentially stacks power towers. For example, with $\alpha_4(n) = A(4, n)$:

$$\begin{aligned}
\alpha_4(0) &= 2^{2^2} - 3 = 13 \\
\alpha_4(1) &= 2^{2^{2^2}} - 3 = 2^{16} - 3 = 65533 \\
\alpha_4(2) &= 2^{2^{2^{2^2}}} - 3 = 2^{65536} - 3 \\
&\vdots
\end{aligned}$$

In general $A(4, n)$ is 3 less than a tower of $n + 3$ twos.

Proposition 3.2.22: Ackerman's Functions Growth

Let α be Ackerman's function and f be any primitive recursive function. Then there is an $n \in \mathbb{N}$ such that for all $k > n$, $\alpha(k) > f(k)$. As a consequence, α is not primitive recursive.

***Proof*:**
Write A for the function of definition 3.2.21. Every value $A(k, \ell)$ is defined: each clause of the definition refers only to values $A(k', \ell')$ with $k' < k$, or with $k' = k$ and $\ell' < \ell$, and there is no infinite descending sequence in this lexicographic order on pairs.

The function A obeys the following growth estimates.

1. $A(k, \ell) > \ell$ for all k, ℓ. Induction on k, and for fixed k induction on ℓ. For $k = 0$: $A(0, \ell) = \ell + 1$. For $k + 1$ and $\ell = 0$: $A(k + 1, 0) = A(k, 1) > 1 > 0$. For $k + 1$ and $\ell + 1$: $A(k + 1, \ell + 1) = A(k, A(k + 1, \ell)) > A(k + 1, \ell) \geq \ell + 1$, the first inequality by the hypothesis for k and the second by the hypothesis for ℓ.

2. $A(k, \ell + 1) > A(k, \ell)$, so $A(k, \cdot)$ is strictly increasing in its second argument. For $k = 0$ this reads $\ell + 2 > \ell + 1$. For $k + 1$: $A(k + 1, \ell + 1) = A(k, A(k + 1, \ell)) > A(k + 1, \ell)$ by (1).

3. $A(k + 1, \ell) \geq A(k, \ell + 1)$. Induction on ℓ. For $\ell = 0$ both sides are $A(k, 1)$. For $\ell + 1$: $A(k + 1, \ell + 1) = A(k, A(k + 1, \ell)) \geq A(k, A(k, \ell + 1)) \geq A(k, \ell + 2)$, using the hypothesis for ℓ with (2) for the first step, and $A(k, \ell + 1) \geq \ell + 2$ from (1) with (2) for the second.

4. $A(k + 1, \ell) > A(k, \ell)$, by (3) followed by (2). Consequently $A(k', \ell) \geq A(k, \ell)$ whenever $k' \geq k$.

5. $A(1, \ell) = \ell + 2$ and $A(2, \ell) = 2\ell + 3$. Indeed $A(1, 0) = A(0, 1) = 2$ and $A(1, \ell + 1) = A(0, A(1, \ell)) = A(1, \ell) + 1$. Likewise $A(2, 0) = A(1, 1) = 3$ and $A(2, \ell + 1) = A(1, A(2, \ell)) = A(2, \ell) + 2$.

6. If $u \geq r$ and $u + 1 \geq s$, then $A(r, A(s, \ell)) \leq A(u + 2, \ell)$. Indeed $A(s, \ell) \leq A(u + 1, \ell)$ by (4), so by (2) and (4) again

$$A(r, A(s, \ell)) \leq A(u, A(u + 1, \ell)) = A(u + 1, \ell + 1) \leq A(u + 2, \ell)$$

where the equality is the defining recursion and the last step is (3).

Next, the domination lemma: for every primitive recursive j-place function f there is an m with

$$f(n_1, \ldots, n_j) < A(m, \max(n_1, \ldots, n_j)) \qquad \text{for all } (n_1, \ldots, n_j) \in \mathbb{N}^j$$

We induct on the construction of f from the initial functions in definition 3.2.13, writing $\vec{n}$ for the input and M for $\max(n_1, \ldots, n_j)$.

For the initial functions: $O(n) = 0 < n + 1 = A(0, n)$, next $S(n) = n + 1 < n + 2 = A(1, n)$ by (5), and $\pi^k_i(\vec{n}) = n_i \leq M < A(0, M)$.

For a composition $f = h \circ (g_1, \ldots, g_t)$ with $h(p_1, \ldots, p_t) < A(r, \max_i p_i)$ and $g_i(\vec{n}) < A(s_i, M)$: put $s = \max_i s_i$, so that $\max_i g_i(\vec{n}) < A(s, M)$ by (4). By (2),

$$f(\vec{n}) = h(g_1(\vec{n}), \ldots, g_t(\vec{n})) < A(r, A(s, M))$$

and (6) with $u = \max(r, s)$ gives $f(\vec{n}) < A(u + 2, M)$.

For a primitive recursion, let f be the $(k + 1)$-place function obtained from g and h as in definition 3.2.10, with $g(\vec{n}) < A(r, \max \vec{n})$ and $h(\vec{n}, t, p) < A(s, \max(\vec{n}, t, p))$, and put $q = \max(r, s) + 1$. We first show, by induction on t,

$$f(\vec{n}, t) < A(q, \max(\vec{n}) + t)$$

For $t = 0$ this is $g(\vec{n}) < A(r, \max \vec{n}) \leq A(q, \max \vec{n})$ by (4). For $t+1$, abbreviate $N = \max(\vec{n}) + t$. By the hypothesis for t, and since $\max(\vec{n}) \leq N$ and $t \leq N < A(q, N)$ by (1), every argument of h in $f(\vec{n}, t + 1) = h(\vec{n}, t, f(\vec{n}, t))$ is smaller than $A(q, N)$. By (2), then (4) with $s \leq q - 1$, then the defining recursion,

$$f(\vec{n}, t + 1) < A(s, A(q, N)) \leq A(q - 1, A(q, N)) = A(q, N + 1)$$

which is the bound for $t + 1$. Finally, $\max(\vec{n}) + t \leq 2 \max(\vec{n}, t) < A(2, \max(\vec{n}, t))$ by (5), so by (2) and then (6) with $u = q$,

$$f(\vec{n}, t) < A(q, A(2, \max(\vec{n}, t))) \leq A(q + 2, \max(\vec{n}, t))$$

This closes the induction and proves the domination lemma.

Now let f be a primitive recursive 1-place function and choose m with $f(k) < A(m, k)$ for all k. For every $k > m$ we have $k \geq m + 1$, so by (4)

$$\alpha(k) = A(k, k) \geq A(m + 1, k) > A(m, k) > f(k)$$

Hence $n = m$ witnesses the statement. If α itself were primitive recursive, the statement applied to $f = \alpha$ would produce an n with $\alpha(k) > \alpha(k)$ for all $k > n$, which is absurd. Hence α is not primitive recursive.

3.2.2 Unbounded Minimalization

Our final method will round off our tool box of possible ways of constructing computable functions. We shall show that using composition, primitive composition, and Unbounded Minimalization, we will have all Computable functions.

Definition 3.2.23: Unbounded Minimalization

Let g be a $(k + 1)$-place function $(k \geq 1)$. Then the *unbounded minimalization* of g is the k-place function f defined by

$$f(n_1, n_2, ..., n_k) = m \text{ where } m \text{ is least so that } g(n_1, n_2, ..., n_k, m) = 0$$

We often denote this as

$$f(n_1, n_2, ..., n_k) = \mu m[g(n_1, n_2, ..., n_k, m) = 0]$$

If no such m exists, then we say the unbounded minimalization of g is not defined on $(n_1, n_2, ..., n_k)$.

Note that it is not unbounded minimalization was not defined algorithmically, but logically. In other words, it is not obvious if unbounded minimalization is computable. If a give you a function g, can you determine in a finite amount if the unbounded minimalization of g is well-defined? We thus need to guarantee our function always can terminate

Definition 3.2.24: Regular Function

A $(k+1)$-place function g is said to be *regular* if for every $(n_1, n_2, ..., n_k) \in \mathbb{N}^k$, there is at least one $m \in \mathbb{N}$ such that $g(n_1, n_2, ..., n_k, m) = 0$.

Hence we may do the obvious algorithm of computing $g(n_1, n_2, ..., n_k, m)$ for $m = 0, 1...$, until we find an m so that $g(n_1, n_2, ..., n_k, m) = 0$.

Example 3.2.25: Unbounded Minimalization

1. $f(n)$ is the function that returns the smallest prime factor of n
2. If $p|n$, then $g(n) = k$ where $n = pk$.
3. $p(n)$ returns the nth prime number.

The reason why this is called unbounded minimization is due to the following

Proposition 3.2.26: Bounded Minimalization

Let g be a $(k+1)$-place primitive recursive regular function such that for some primitive recursive k-place function h,

$$\mu m[g(n_1, n_2, ..., n_k, m) = 0] \leq h(n_1, n_2, ..., n_k)$$

for all $(n_1, n_2, ..., n_k) \in \mathbb{N}^k$. Then $\mu m[g(n_1, n_2, ..., n_k, m) = 0]$ is primitive recursive

***Proof*:**
Write $\vec{n}$ for $(n_1, \ldots, n_k)$ and $\chi_{\mathbb{N}^*}$ for the function of example 3.2.17(2), so $\chi_{\mathbb{N}^*}(x) = 0$ if $x = 0$ and $\chi_{\mathbb{N}^*}(x) = 1$ if $x > 0$. It is primitive recursive, being $\chi_{\{0\}} \circ \chi_{\{0\}}$ with $\chi_{\{0\}}$ from lemma 3.2.14(2). The composition $\chi_{\mathbb{N}^*} \circ g$ is primitive recursive, hence so is the $(k+1)$-place function

$$t(\vec{n}, i) = \prod_{j=0}^{i} \chi_{\mathbb{N}^*}(g(\vec{n}, j))$$

by lemma 3.2.14(1). Since g is regular (definition 3.2.24), the value $\mu(\vec{n}) := \mu m[g(\vec{n}, m) = 0]$ is defined for every $\vec{n} \in \mathbb{N}^k$. If $i < \mu(\vec{n})$, every factor of $t(\vec{n}, i)$ equals 1, so $t(\vec{n}, i) = 1$. If $i \geq \mu(\vec{n})$, the factor with $j = \mu(\vec{n})$ equals $\chi_{\mathbb{N}^*}(0) = 0$, so $t(\vec{n}, i) = 0$. Hence for every m the sum $\sum_{i=0}^{m} t(\vec{n}, i)$ counts the indices $i \leq m$ with $i < \mu(\vec{n})$, so

$$\sum_{i=0}^{m} t(\vec{n}, i) = \min(\mu(\vec{n}),\ m+1)$$

The $(k+1)$-place function $u(\vec{n}, m) = \sum_{i=0}^{m} t(\vec{n}, i)$ is primitive recursive by example 3.2.11(2), and

therefore so is the k-place composition $F(\vec{n}) = u(\vec{n}, h(\vec{n}))$. By hypothesis $\mu(\vec{n}) \leq h(\vec{n}) < h(\vec{n})+1$, so $F(\vec{n}) = \mu(\vec{n})$. Hence $\mu m[g(n_1, n_2, ..., n_k, m) = 0]$ is primitive recursive.

Proposition 3.2.27: Unbounded Minimalization Is Computable

Let g be a Turing computable regular $(k+1)$-place function. Then the unbounded minimalization of g is Turing computable.

***Proof*:**
Let M_g be a machine computing g (definition 3.2.1), and write

$$f(n_1, \ldots, n_k) = \mu m[g(n_1, \ldots, n_k, m) = 0]$$

We use a machine with three tapes, one of the augmentations allowed after example 3.1.6. Started on the input $\underline{0}1^{n_1+1}0 \cdots 01^{n_k+1}$ on tape 1, it writes the counter $\underline{0}1$, representing $m = 0$, on tape 2, and then repeats the following round. It erases tape 3, copies onto it the input followed by the counter, in the form

$$\underline{0}1^{n_1+1}0 \cdots 01^{n_k+1}01^{m+1}$$

and runs M_g on tape 3. As g is a function on all of $\mathbb{N}^{k+1}$ and M_g computes it, this run halts with tape 3 reading $\underline{0}1^{g(n_1,\ldots,n_k,m)+1}$. The machine then reads cell 2 of tape 3, which is blank exactly when $g(n_1, \ldots, n_k, m) = 0$. If it is blank, the machine erases tape 1, copies the counter $\underline{0}1^{m+1}$ onto it, returns to cell 0 and halts. Otherwise it marks the cell after the last marked cell of tape 2, which replaces m by $m + 1$, and starts the next round.

The rounds test $m = 0, 1, 2, \ldots$ in increasing order, each round halts, and the machine halts in the round of the least m with $g(n_1, \ldots, n_k, m) = 0$. Such an m exists because g is regular (definition 3.2.24), so the machine halts after finitely many rounds with output tape $\underline{0}1^{f(n_1,\ldots,n_k)+1}$. Hence it computes f, and the unbounded minimalization of g is Turing computable, as claimed.

We now combine all our types of functions

Definition 3.2.28: Recursive Function

Let f be a k-place function. Then f is *recursive* if it can be defined from the initial functions by finitely many application of composition, primitive recursion, and unbounded minimalization of regular functions.

Similarly, a k-place partial function is *recursive* if it can be defined from initial functions by finitely many application of composition, primitive recursion, and unbounded minimalization of (possibly non-regular) functions.

Definition 3.2.29: Recursive Relation

A k-place relation P is said to be *recursive* (or Turing computable) if its characteristic function χ_P is recursive

Theorem 3.2.30: Recursive Function, Then Computable

Let f be a recursive function. Then f is computable

***Proof*:**
Here f is recursive in the first sense of definition 3.2.28, so there is a finite sequence $f_1, \ldots, f_r = f$ of functions in which each f_j is an initial function, a composition $h \circ (g_1, \ldots, g_m)$ of earlier functions of the sequence, obtained by primitive recursion from two earlier functions, or the unbounded minimalization of an earlier regular function. Each f_j is total, since the initial functions are total and composition, primitive recursion and the minimalization of a regular function produce total functions from total ones. We show by strong induction on j that each f_j is computable.

- An initial function is primitive recursive by definition 3.2.13, with no constructions applied, so it is computable by theorem 3.2.15.
- Let $f_j = h \circ (g_1, \ldots, g_m)$, where h and $g_1, \ldots, g_m$ are computable by the induction hypothesis. Then f_j is computable by lemma 3.2.7.
- Let f_j be obtained by primitive recursion from a k-place g with $k \geq 1$ and a $(k+2)$-place h (definition 3.2.10), both computable by the induction hypothesis. Then f_j is computable by proposition 3.2.12.
- Let f_j be the unbounded minimalization of a regular $(k+1)$-place function g that is computable by the induction hypothesis. Then f_j is computable by proposition 3.2.27.

For $j = r$ this shows that f is computable, as we sought to show.

In the next section, we shall show the following very important result

Theorem 3.2.31: Computable, Then Recursive

Let f be a total computable function. Then f is recursive

Hence, for total functions, recursive and Turing computable is synonymous. We end this section with some important technical results

Theorem 3.2.32: K-Place To 1-Place Recursive Function

A k-place function g is recursive if and only if the 1-place function h given by $h(n) = g(Power(p_1, n), \ldots, Power(p_k, n))$ is recursive. In particular $h(n) = g(n_1, n_2, ..., n_k)$ if $n = p_1^{n_1} \cdots p_k^{n_k}$.

***Proof*:**
Here $Power$ is the primitive recursive function of example 3.2.17(9). For fixed k the primes $p_1, \ldots, p_k$ are constants, and a constant function is a composition of S with O, so each 1-place function $n \mapsto Power(p_i, n)$ is primitive recursive. Primitive recursive functions are recursive, the build-up of definition 3.2.13 being a special case of that of definition 3.2.28. By unique

factorization, $Power(p_i, p_1^{n_1} \cdots p_k^{n_k}) = n_i$, which gives the last sentence of the statement.

If g is recursive, then h is the composition of g with the k functions $n \mapsto Power(p_i, n)$, hence recursive. If h is recursive, then for every $(n_1, \ldots, n_k) \in \mathbb{N}^k$

$$g(n_1, \ldots, n_k) = h\Big(\prod_{i=1}^{k} EXP(p_i, n_i)\Big)$$

where EXP is example 3.2.11(4) and the product consists of $k-1$ applications of $MULT$ from example 3.2.11(3). This exhibits g as a composition of recursive functions, so g is recursive, completing the proof.

Corollary 3.2.33: K-Place To 1-Place Recursive Relation

A k-place relation P is recursive if and only if the 1-place relation $P' = \{p_1^{n_1} \cdots p_k^{n_k} \mid (n_1, \ldots, n_k) \in P\}$ is recursive, so that

$$(n_1, ..., n_k) \in P \qquad \Leftrightarrow \qquad p_1^{n_1} \cdots p_k^{n_k} \in P'$$

***Proof*:**
By lemma 3.2.19, the function $Code_k$ defined there is primitive recursive, hence recursive, and for every n

$$\chi_{P'}(n) = MULT\big(Code_k(n),\ \chi_P(Power(p_1, n), \ldots, Power(p_k, n))\big)$$

If P is recursive, then so is χ_P (definition 3.2.29). By theorem 3.2.32 applied to $g = \chi_P$, the second argument of $MULT$ in the last display is a recursive function of n, and therefore $\chi_{P'}$ is recursive. If P' is recursive, then the identity

$$\chi_P(n_1, \ldots, n_k) = \chi_{P'}\Big(\prod_{i=1}^{k} EXP(p_i, n_i)\Big)$$

exhibits χ_P as a composition of recursive functions, so P is recursive, as claimed.

3.3 Characterizing Computability

In this section, we work towards showing that all total computable functions are recursive. We will show that any Turing machine can be simulated by some recursive function. Since the functions we worked with input natural numbers, we will have to codify the tape and the Turing machine as natural numbers.

Definition 3.3.1: Code of Position

Let $(s, i, \mathbf{a})$ be a tape position such that all but finitely many cells of $\mathbf{a}$ are blank. Let $n \in \mathbb{N}$ such that $a_k = 0$ for all $k > n$. Then the *code* of $(s, i, \mathbf{a})$ is

$$[(s, i, \mathbf{a})] = 2^s 3^i 5^{a_0} 7^{a_1} 11^{a_2} \cdots p_{n+3}^{a_n}$$

For example,

Example 3.3.2: Code of Position

1. the code of $(2, 1, 1001)$ is:
$$[(2, 1, 1001)] = 2^2 3^1 5^1 7^0 11^0 13^1 = 780$$
2. Let $(1, 0, \mathbf{a})$ where $\mathbf{a}$ is blank. Then it's code is 2
3. Let $(4, 3, \mathbf{a})$ where $\mathbf{a}$ is 1011100101. What is its code?
4. What tape position do we get from 10314720. What is the tape position?

Definition 3.3.3: Code Of Sequence Of Tape Positions

Let $t_1, t_2, ..., t_n$ be a sequence of tape positions. Then the code of the sequence is

$$[t_1, t_2, ..., t_n] = 2^{[t_1]} 3^{[t_2]} \cdots p_n^{[t_n]}$$

Note that both the tape positions and the sequence of tape positions have unique codes, and hence they are injective. However, they are *not* surjective, for example there is no code $2^{2^1 3^1 5^2}$ since there cannot be 2 "1"'s in the first cell. Hence, detecting when a number is a tape position or a sequence of tape positions is important, and for our purposes it has to be done in a recursive way:

Proposition 3.3.4: Tapepos And Tapeposseq

Define the following two relations:

1. **TapePos**, where **TapePos**(n) holds when n is the code of a tape position
2. **TapePosSeq**, where **TapePosSeq**(n) holds when n is the code of a finite sequence of tape positions

Then these two relations are primitive recursive.

***Proof*:**
We use *Element*, *Length*, *Prime* and the conventions on bounded sums and products from example 3.2.17, and $\chi_{\{a\}}$ from lemma 3.2.14(2).

1. By definition 3.3.1 and definition 3.1.2, n is the code of a tape position exactly when $n = 2^s 3^i \prod_{t \geq 3} p_t^{e_t}$ with $s \geq 1$ and every $e_t \in \{0, 1\}$, the cell c corresponding to the prime

p_{c+3}. For $n \geq 1$ we have $Element(n,1) = s$, and $Element(n,t) = 0$ for $t > Length(n)$. Hence

$$\chi_{\mathbf{TapePos}}(n) = \chi_{\mathbb{N}^*}(n) \cdot \chi_{\mathbb{N}^*}(Element(n,1)) \cdot \prod_{t=3}^{Length(n)} \big(\chi_{\{0\}}(Element(n,t)) + \chi_{\{1\}}(Element(n,t))\big)$$

where the two summands are never 1 at once. This is primitive recursive.

2. The code $2^{[t_1]}3^{[t_2]} \cdots p_r^{[t_r]}$ of definition 3.3.3 has positive exponents, as tape position codes are at least 2, so for it $Length(n) = r$ and $Element(n,l) = [t_l]$. Conversely, if $n \geq 1$ and $Element(n,l)$ codes a tape position for every $1 \leq l \leq Length(n)$, then $n = \prod_{l=1}^{Length(n)} p_l^{Element(n,l)}$ codes the sequence of these positions, since no prime beyond $p_{Length(n)}$ divides n. Hence

$$\chi_{\mathbf{TapePosSeq}}(n) = \chi_{\mathbb{N}^*}(n) \cdot \prod_{l=1}^{Length(n)} \chi_{\mathbf{TapePos}}(Element(n,l))$$

which is primitive recursive by (1), the case $n = 1$ being the empty sequence. This completes the proof.

Proposition 3.3.5: Important Primitive Recursive Functions

The following are primitive recursive

1. The 4-place function ENTRY (or NEXT) such that

$$ENTRY(j,w,t,n) = \begin{cases} [(t, i+w-1, \mathbf{a}')] & n = [(s,i,\mathbf{a})],\ j \in \{0,1\} \\ & w \in \{0,2\}, i+w-1 \geq 0 \\ & t \geq 1, \text{ where } a'_k = a_k \\ & \text{for } k \neq i \text{ and } a'_i = j. \\ 0 & \text{otherwise} \end{cases}$$

2. For any Turing Machine M with alphabet $\{1\}$, the 1-place function STEP_M such that

$$STEP_M(n) = \begin{cases} [\mathbf{M}(s,i,\mathbf{a})] & \text{if } n = [(s,i,\mathbf{a})] \text{ and} \\ & \mathbf{M}(s,i,\mathbf{a}) \text{ is defined.} \\ 0 & \text{otherwise} \end{cases}$$

3. For any Turing Machine M with alphabet $\{1\}$ the 1-place relation $COMP_M$ where

$$COMP_M(n) \Leftrightarrow n \text{ is the code of a computation } M$$

***Proof*:**
We use $EQUAL$, $Element$, $Length$, $Prime$, EXP, $MULT$, SUM, $DIFF$, $\chi_{\mathbb{N}^*}$ and the conventions on bounded sums and products from example 3.2.11 and example 3.2.17, $\chi_{\{a\}}$ from lemma 3.2.14(2), and **TapePos**, **TapePosSeq** from proposition 3.3.4. For a tape position code $n = [(s, i, \mathbf{a})]$, definition 3.3.1 gives $s = Element(n, 1)$, $i = Element(n, 2)$ and $a_c = Element(n, c + 3)$ for every cell c, and every marked cell c has $c + 3 \leq Length(n)$.

1. Write $s_n = Element(n, 1)$ and $i_n = Element(n, 2)$, let

   $$A(j, n, c) = MULT\big(EQUAL(c, i_n), j\big) + MULT\big(\chi_{\{0\}}(EQUAL(c, i_n)), Element(n, c+3)\big)$$

   and let $G(j, w, t, n)$ be the product of $\chi_{\mathbf{TapePos}}(n)$, $\chi_{\{0\}}(DIFF(j, 1))$, $\chi_{\{0\}}(w) + \chi_{\{2\}}(w)$, $\chi_{\mathbb{N}^*}(i_n + w)$ and $\chi_{\mathbb{N}^*}(t)$. Then

   $$ENTRY(j, w, t, n) = G(j, w, t, n) \cdot EXP(2, t) \cdot EXP(3, DIFF(i_n + w, 1)) \cdot \prod_{c=0}^{Length(n)+i_n} EXP(Prime(c + 3), A(j, n, c))$$

   is primitive recursive. The factor G is 1 exactly when n is a tape position code, $j \leq 1$, $w \in \{0, 2\}$, $i_n + w - 1 \geq 0$ and $t \geq 1$, which are the conditions of the statement, and it is 0 otherwise. When it is 1, the exponent of 3 is $i_n + w - 1$, the exponent $A(j, n, c)$ is j for $c = i_n$ and a_c for $c \neq i_n$, which is the cell c of $\mathbf{a}'$, and the product runs over every marked cell of $\mathbf{a}'$, all of which are at most $Length(n) + i_n$. So the value is $[(t, i_n + w - 1, \mathbf{a}')]$.

2. The machine M has finitely many instructions. For each $(s_0, b) \in \mathrm{Dom}(M)$ write $M(s_0, b) = (x_1, x_2, x_3)$, and let

   $$I(n, s_0, b) = EQUAL(Element(n, 1), s_0) \cdot EQUAL(Element(n, Element(n, 2) + 3), b)$$

   For a tape position code $n = [(s, i, \mathbf{a})]$, this is 1 exactly when $(s, a_i) = (s_0, b)$. Let

   $$STEP_M(n) = \sum_{(s_0,b)\in \mathrm{Dom}(M)} I(n, s_0, b) \cdot ENTRY(x_1, x_2 + 1, x_3, n)$$

   a finite sum of compositions of primitive recursive functions. If $n = [(s, i, \mathbf{a})]$, at most one term is nonzero, the one with $(s_0, b) = (s, a_i)$. If $M(s, a_i) = (x_1, x_2, x_3)$ is defined, this term is $ENTRY(x_1, x_2 + 1, x_3, n)$, which by (1) is the code $[(x_3, i + x_2, \mathbf{a}')]$ of the successor position $\mathbf{M}(s, i, \mathbf{a})$ of definition 3.1.5(1) when $i + x_2 \geq 0$, and 0 when the successor would leave the tape and so has no code. If $M(s, a_i)$ is undefined, every term is 0, and if n is not a tape position code, every term is 0 because of the factor $\chi_{\mathbf{TapePos}}(n)$ in $ENTRY$. This is $STEP_M$ as described in the statement, with the value 0 for a successor that leaves the tape.

3. Let $D_M(q) = \sum_{(s_0,b)\in \mathrm{Dom}(M)} I(q, s_0, b)$, which for a tape position code q is 1 exactly when M has an instruction at the position coded by q, and 0 otherwise. Writing e_l for $Element(n, l)$, let

   $$\chi_{COMP_M}(n) = \chi_{\mathbf{TapePosSeq}}(n) \cdot \chi_{\mathbb{N}^*}(Length(n)) \cdot EQUAL(Element(e_1, 1), 1) \cdot \chi_{\{0\}}(Element(e_1, 2)) \cdot \chi_{\{0\}}\big(D_M(e_{Length(n)})\big) \cdot \prod_{l=1}^{Length(n) \dot{-} 1} EQUAL\big(STEP_M(e_l), e_{l+1}\big)$$

This is primitive recursive. Its first two factors are 1 exactly when n codes a nonempty sequence $q_1, \ldots, q_r$ of tape positions, and the next two exactly when $q_1 = (1, 0, \mathbf{a})$ for some tape $\mathbf{a}$. The factor $\chi_{\{0\}}(D_M(e_{Length(n)}))$ is 1 exactly when M has no instruction at q_r, which is the halting condition of definition 3.1.5(3). As tape position codes are at least 2, by (2) each factor of the product is 1 exactly when $q_{l+1} = \mathbf{M}(q_l)$, so the product is 1 exactly when $q_1, \ldots, q_r$ is a partial computation (definition 3.1.5(2)). Hence $\chi_{COMP_M}(n) = 1$ exactly when n is the code of a computation of M, completing the proof.

Finally, the last important result to show that total Turing computable functions are recursive is this next recursive function that requires unbounded mineralization

Proposition 3.3.6: Simulation Function

Let M be a Turing machine with alphabet $\{1\}$. Then the 1-place (partial) function SIM_M is recursive, where

$$SIM_M(n) = [(t, j, \mathbf{b})]$$

if $n = [(1, 0, \mathbf{a})]$ for some input tape a and M eventually halts in position $(t, j, \mathbf{b})$, on input $\mathbf{a}$

Note that $SIM_M(n)$ may be undefined if $n \neq [(1, 0, \mathbf{a})]$ for an input $\mathbf{a}$ or if M does not eventually halt on input $\mathbf{a}$.

***Proof*:**
Define the 2-place relation

$$R(n, c) \quad \Leftrightarrow \quad COMP_M(c) \text{ and } Element(c, 1) = n$$

If $c = 2^{[t_1]}3^{[t_2]} \cdots p_r^{[t_r]}$ is the code of a sequence of tape positions as in definition 3.3.3, then $Element(c, 1) = [t_1]$, so $R(n, c)$ holds exactly when c codes a computation of M whose first tape position has code n. Its characteristic function

$$\chi_R(n, c) = \chi_{COMP_M}(c) \cdot \chi_{\{0\}}(AbsDiff(Element(c, 1), n))$$

is primitive recursive, with $COMP_M$ from proposition 3.3.5(3), $Element$ from example 3.2.17(11), $AbsDiff$ from example 3.2.11(8), and $\chi_{\{0\}}$ from lemma 3.2.14(2). The function $g(n, c) = \chi_{\{0\}}(\chi_R(n, c))$ is then primitive recursive, and $g(n, c) = 0$ exactly when $R(n, c)$ holds. The function g need not be regular, and by the second part of definition 3.2.28 the unbounded minimalization $c(n) = \mu m[g(n, m) = 0]$ is a 1-place partial recursive function. Set

$$SIM_M(n) = Element(c(n),\ Length(c(n)))$$

which is again partial recursive, being a composition with the primitive recursive functions $Element$ and $Length$ (example 3.2.17(10)).

Suppose $n = [(1, 0, \mathbf{a})]$ for an input tape $\mathbf{a}$ on which M eventually halts. Then there is a computation $p_1, \ldots, p_r$ with respect to M with $p_1 = (1, 0, \mathbf{a})$, by definition 3.1.5(3), and its code witnesses that some c satisfies $R(n, c)$. Hence $c(n)$ is defined, and it codes a sequence $q_1, \ldots, q_v$ which by $R(n, c(n))$ is a computation of M with $[q_1] = n = [p_1]$. Distinct tape positions have distinct codes, as noted after definition 3.3.3, so $q_1 = p_1$. Successive entries of a computation advance by the uniquely determined successor position, by definition 3.1.5(2), so $q_l = p_l$ for all $l \leq \min(r, v)$. If $v < r$, then $\mathbf{M}(q_v) = \mathbf{M}(p_v)$ is defined and $q_1, \ldots, q_v$ is not a computation.

Symmetrically $r < v$ is impossible. So $v = r$ and $q_l = p_l$ throughout. The exponents $[p_l] \geq 2$ are positive, so $Length(c(n)) = r$ and

$$SIM_M(n) = Element(c(n), r) = [p_r] = [(t, j, \mathbf{b})]$$

is the code of the halting position, as required.

If n is not of the form $[(1, 0, \mathbf{a})]$, or M does not halt on $\mathbf{a}$, then no c satisfies $R(n, c)$: any c with $COMP_M(c)$ codes a computation, whose first position is of the form $(1, 0, \mathbf{a}')$ and on whose input tape $\mathbf{a}'$ the machine M halts. Hence the minimalization, and with it $SIM_M(n)$, is undefined, in agreement with the statement.

Hence, we now see that given a Turing machine M and an encoded input take n, we can deduce what the Turing machine will output, given that it halts on the input. The final step we need is simply to insure that encoding and decoding a tape position is indeed recursive:

Lemma 3.3.7: More Primitive Recursive Functions

The following functions are primitive recursive

1. For any fixed $k \geq 1$, $CODE_k(n_1, n_2, ..., n_k) = [(1, 0, 01^{n_1+1}0...01^{n_k+1})]$
2. $DECODE(t) = n$ if $t = [(s, i, 01^{n+1})]$ (and anything else otherwise)

***Proof*:**

1. For $k = 1$: the tape 01^{n+1} has $a_0 = 0$, then $a_j = 1$ for $1 \leq j \leq n + 1$, and every later cell blank, so by definition 3.3.1

$$CODE_1(n) = [(1, 0, 01^{n+1})] = 2^1 3^0 \prod_{j=1}^{n+1} p_{j+3} = 2 \prod_{i=0}^{n} Prime(i + 4)$$

where $Prime(\ell) = p_\ell$ is primitive recursive by example 3.2.17(8). The bounded product is primitive recursive by lemma 3.2.14(1), so $CODE_1$ is primitive recursive.

Suppose $CODE_k$ is primitive recursive, and write $c = CODE_k(n_1, \ldots, n_k)$. The tape coded by c ends in a marked cell, so its last cell has index $Length(c) - 3$: cell j corresponds to the prime p_{j+3} in definition 3.3.1, and $Length(c)$ is the index of the largest prime dividing c by example 3.2.17(10). The tape of $CODE_{k+1}(n_1, \ldots, n_k, m)$ appends to it one blank cell followed by $m + 1$ marked cells, whose primes are $p_{Length(c)+2}, \ldots, p_{Length(c)+m+2}$, the blank cell contributing exponent 0. Hence

$$CODE_{k+1}(n_1, \ldots, n_k, m) = c \cdot \prod_{i=0}^{m} Prime\big(Length(c) + 2 + i\big)$$

a composition of primitive recursive functions, primitive recursive by the same two results. As k is fixed, finitely many steps of this form reach every $CODE_k$.

2. If $t = [(s, i, 01^{n+1})]$, then by definition 3.3.1 the exponent of the cell prime p_{j+3} in t is 1 for the marked cells $j = 1, \ldots, n + 1$ and 0 for every other cell, while the exponents of 2

and 3 are s and i. The sum

$$D(t) = \sum_{\ell=0}^{Length(t)} \chi_{\{1\}}(Element(t, \ell + 3))$$

ranges over the cell primes $p_3, \ldots, p_{Length(t)+3}$, which include every prime dividing t beyond $p_2 = 3$, and counts those cell exponents equal to 1. So $D(t) = n + 1$, and setting

$$DECODE(t) = PRED(D(t))$$

gives $DECODE(t) = n$ on codes of the stated form, with $\chi_{\{1\}}$ from lemma 3.2.14(2), *Element* from example 3.2.17(11), the bounded sum from example 3.2.11(2), and *PRED* from example 3.2.11(6). Hence *DECODE* is primitive recursive, and on inputs not of the stated form it returns some value, which the statement permits.

Theorem 3.3.8: Turing Computable, Then Recursive

Let f be a total k-place Turing computable function. Then f is recursive.

***Proof*:**

Let M compute f (definition 3.2.1). For $\vec{n} = (n_1, \ldots, n_k) \in \mathbb{N}^k$ write $\mathbf{a}_{\vec{n}}$ for the input tape $01^{n_1+1}0 \cdots 01^{n_k+1}$. As f is total, M halts on every $\mathbf{a}_{\vec{n}}$ with output tape $01^{f(\vec{n})+1}$, so there is a computation $p_1, \ldots, p_r$ of M with $p_1 = (1, 0, \mathbf{a}_{\vec{n}})$ and $p_r = (t, j, 01^{f(\vec{n})+1})$ for some t and j (definition 3.1.5).

Let the $(k + 1)$-place relation $R(\vec{n}, c)$ hold when $COMP_M(c)$ holds and $Element(c, 1) = CODE_k(\vec{n})$. Its characteristic function

$$\chi_R(\vec{n}, c) = \chi_{COMP_M}(c) \cdot \chi_{\{0\}}\big(AbsDiff(Element(c, 1), CODE_k(\vec{n}))\big)$$

is primitive recursive, with $COMP_M$ from proposition 3.3.5(3), *Element* from example 3.2.17(11), *AbsDiff* from example 3.2.11(8), $\chi_{\{0\}}$ from lemma 3.2.14(2) and $CODE_k$ from lemma 3.3.7(1). Hence $G = \chi_{\{0\}} \circ \chi_R$ is a primitive recursive $(k + 1)$-place function that vanishes exactly on R. It is regular (definition 3.2.24): for each $\vec{n}$ the code $c_0 = [p_1, \ldots, p_r]$ of the computation above satisfies $COMP_M(c_0)$ and $Element(c_0, 1) = [p_1] = CODE_k(\vec{n})$, so $G(\vec{n}, c_0) = 0$. By the first sense of definition 3.2.28, the function $C(\vec{n}) = \mu c[G(\vec{n}, c) = 0]$ is recursive, and so is the composition

$$F(\vec{n}) = DECODE\big(Element(C(\vec{n}),\ Length(C(\vec{n})))\big)$$

with *Length* from example 3.2.17(10) and *DECODE* from lemma 3.3.7(2).

It remains to show $F = f$. Fix $\vec{n}$. By $R(\vec{n}, C(\vec{n}))$, the number $C(\vec{n})$ codes a computation $q_1, \ldots, q_v$ of M with $[q_1] = CODE_k(\vec{n}) = [p_1]$, so $q_1 = p_1$, since distinct tape positions have distinct codes (as noted after definition 3.3.3). By definition 3.1.5, each entry of a computation after the first is the successor position of the previous entry, so $q_l = p_l$ for all $l \leq \min(r, v)$. If $v < r$, then $\mathbf{M}(q_v) = \mathbf{M}(p_v)$ is defined and $q_1, \ldots, q_v$ is not a computation, and if $r < v$, then $\mathbf{M}(p_r) = \mathbf{M}(q_r)$ is defined and $p_1, \ldots, p_r$ is not a computation. Hence $v = r$ and $C(\vec{n}) = [p_1, \ldots, p_r]$. The exponents $[p_l]$ are at least 2, so $Length(C(\vec{n})) = r$ and

$Element(C(\vec{n}), r) = [p_r] = [(t, j, 01^{f(\vec{n})+1})]$. Therefore $F(\vec{n}) = f(\vec{n})$ by the defining property of $DECODE$, and $f = F$ is recursive, as we sought to show.

Hence, since recursive implies Turing computable by theorem 3.2.30, we see that for total functions we get

Turing Computable if and only if recursive

3.3.1 Universal Turing Machine and Halting Problem

We can go a little further than what we did above: we can have a recursive function that can simulate *any* Turing machine. Since every recursive function can be computed by a Turing machine, this essentially gives us a *Universal Turing machine*

Definition 3.3.9: Universal Turing Machine (UTM)

A *universal Turing machine* U is a Turing machine such that when given (an appropriate description of) a Turing machine M and an input tape $\mathbf{a}$, it will simulate the computations of M with input $\mathbf{a}$ and gives the output of M on $\mathbf{a}$.

Definition 3.3.10: Encoding Turing Machine

Let M be a Turing machine, hence a partial function as defined in definition 3.1.3. Then if $M(a, b) = (x_1^{(a,b)}, x_2^{(a,b)}, x_3^{(a,b)})$, then:

$$[M] = p_1^{\left(2^{x_1^{(1,0)}} 3^{x_2^{(1,0)}+1} 5^{x_3^{(1,0)}}\right)} p_2^{\left(2^{x_1^{(1,1)}} 3^{x_2^{(1,1)}+1} 5^{x_3^{(1,1)}}\right)} \cdots p_{2n}^{\left(2^{x_1^{(n,1)}} 3^{x_2^{(n,1)}+1} 5^{x_3^{(n,1)}}\right)}$$

where each p_i represents one of the $2n$ possible inputs in the order of $(1, 0), (1, 1), (2, 0), (2, 1), ...$, the exponent represents the possible output,and is zero .

Proposition 3.3.11: Generalized STEP And COMP

The following 2-place function and 2-place relation are primitive recursive:

1.
$$STEP(m, n) = \begin{cases} [\mathbf{M}(s, i, \mathbf{a})] & \text{if } m = [M] \text{ for some machine } M, \\ & \text{if } n = [(s, i, \mathbf{a})] \text{ and } \mathbf{M}(s, i, \mathbf{a}) \text{ is defined.} \\ 0 & \text{otherwise} \end{cases}$$

2.
$$COMP(m, n) \quad \Leftrightarrow \quad m = [M] \text{ for some Turing machine } M \\ \text{and } n \text{ is the code of a computation of } M$$

***Proof*:**
Throughout, *Element*, *Power*, *Length*, EXP, $EQUAL$ and $CONCAT$ are the primitive recursive functions of example 3.2.17 and example 3.2.11, $n \dot{-} m$ is $DIFF(n,m)$, the functions $\chi_{\{a\}}$ are those of lemma 3.2.14(2), and $\chi_{\mathbb{N}^*} = \chi_{\{0\}} \circ \chi_{\{0\}}$ tests being nonzero as in example 3.2.17(2). A bounded sum starting at index 1, $\sum_{i=1}^{b} G(\ldots, i)$, is read as $\sum_{i=0}^{b} \chi_{\mathbb{N}^*}(i) \cdot G(\ldots, i)$, and a bounded product $\prod_{i=1}^{b} G(\ldots, i)$ as $\prod_{i=0}^{b} \big(\chi_{\{0\}}(i) + \chi_{\mathbb{N}^*}(i) \cdot G(\ldots, i)\big)$. Both modified terms are compositions of G with SUM, $Mult$ and the characteristic functions, so the sum and product stay primitive recursive when G is. By example 3.2.17(9) and (10), $Power(x, 0) = 0$ and $Length(0) = 0$, because the bounds $k \leq m$ and $\ell \leq n$ in those definitions leave only the value 0 at the argument 0. Every use of *Power* or *Length* below either applies to a positive argument or sits alongside a guard factor that vanishes on the offending inputs, so these values never matter.

In definition 3.3.10 the instruction of M at the input (s, b), with $s \geq 1$ and $b \in \{0, 1\}$, is stored as the exponent of p_{2s+b-1}, the position of (s, b) in the order $(1,0), (1,1), (2,0), (2,1), \ldots$, and the exponent is $2^{x_1} 3^{x_2+1} 5^{x_3}$ if $M(s, b) = (x_1, x_2, x_3)$ is defined and 0 if not. Since $x_3 \geq 1$, a stored instruction is a positive number, indeed one divisible by 5. A number e is a stored instruction of some machine exactly when $e > 0$, e has no prime factors beyond $2, 3, 5$, its exponent of 2 is at most 1, its exponent of 3 is 0 or 2, and its exponent of 5 is at least 1:

$$\begin{aligned} valid(e) = {} & \chi_{\mathbb{N}^*}(e) \\ & \cdot EQUAL\big(e,\ EXP(2, Power(2,e)) \cdot EXP(3, Power(3,e)) \cdot EXP(5, Power(5,e))\big) \\ & \cdot \chi_{\{0\}}(Power(2,e) \dot{-} 1) \cdot \big(\chi_{\{0\}}(Power(3,e)) + \chi_{\{2\}}(Power(3,e))\big) \\ & \cdot \chi_{\mathbb{N}^*}(Power(5,e)) \end{aligned}$$

is primitive recursive, the two summands never being 1 at once. A number m is $[M]$ for some Turing machine M exactly when every exponent in m is 0 or a stored instruction: given such an m, define $M(s, b)$ from the exponent of p_{2s+b-1} wherever that exponent is positive, and take the number of states large enough to cover every state mentioned. Hence the relation TM, that m codes a Turing machine, has the primitive recursive characteristic function

$$\chi_{TM}(m) = \chi_{\mathbb{N}^*}(m) \cdot \prod_{i=1}^{Length(m)} \Big(\chi_{\{0\}}(Element(m,i)) + valid(Element(m,i))\Big)$$

For $n = [(s, i, \mathbf{a})]$ the state is $s = Element(n, 1)$, the scanned cell is $i = Element(n, 2)$, and the scanned value is $a_i = Element(n, i + 3)$, by definition 3.3.1. So the stored instruction of the machine coded by m at the current input of the position coded by n is

$$e(m,n) = Element\big(m,\ 2 \cdot Element(n,1) + Element(n,\ Element(n,2) + 3) \dot{-} 1\big)$$

a composition of primitive recursive functions.

1. Abbreviating $e = e(m, n)$, define

$$\begin{aligned} STEP(m,n) = {} & \chi_{TM}(m) \cdot \chi_{\mathbf{TapePos}}(n) \cdot \chi_{\mathbb{N}^*}(e) \\ & \cdot ENTRY\big(Power(2,e),\ Power(3,e),\ Power(5,e),\ n\big) \end{aligned}$$

with **TapePos** from proposition 3.3.4(1) and $ENTRY$ from proposition 3.3.5(1). This is primitive recursive. If $m = [M]$ and $n = [(s, i, \mathbf{a})]$ with $M(s, a_i) = (x_1, x_2, x_3)$ defined,

then $e(m,n) = 2^{x_1}3^{x_2+1}5^{x_3} > 0$, all three guards equal 1, and

$$ENTRY(x_1,\ x_2+1,\ x_3,\ n) = [(x_3,\ i+x_2,\ \mathbf{a}')] = [\mathbf{M}(s,i,\mathbf{a})]$$

by comparing the case description of $ENTRY$ with definition 3.1.5(1), provided $i+x_2 \geq 0$. If m is not a machine code, or n is not a position code, or the instruction is undefined, a guard vanishes and $STEP(m,n) = 0$. If the instruction is defined but $i+x_2 < 0$, then $ENTRY$ returns 0 by its condition $i+w-1 \geq 0$. The successor is then not a tape position and has no code, and returning 0 is the reading of the statement already used for $STEP_M$ in proposition 3.3.5(2).

2. Define

$$\begin{aligned}\chi_{COMP}(m,n) = {} & \chi_{TM}(m)\cdot\chi_{\mathbf{TapePosSeq}}(n)\cdot\chi_{\mathbb{N}^*}(Length(n)) \\ & \cdot\chi_{\{1\}}\big(Element(Element(n,1),\ 1)\big)\cdot\chi_{\{0\}}\big(Element(Element(n,1),\ 2)\big) \\ & \cdot\prod_{l=1}^{Length(n)\dot{-}1} EQUAL\big(STEP(m,\ Element(n,l)),\ Element(n,l+1)\big) \\ & \cdot\chi_{\{0\}}\big(e(m,\ Element(n,Length(n)))\big)\end{aligned}$$

with **TapePosSeq** from proposition 3.3.4(2). This is primitive recursive. The first three factors require that m code a machine M and that n code a nonempty sequence of tape positions $q_1,\ldots,q_r$. The next two require that q_1 have state 1 and scanned cell 0, as definition 3.1.5(3) prescribes for a computation, so $q_1 = (1,0,\mathbf{a})$ for its tape $\mathbf{a}$. Each factor of the product requires $STEP(m,[q_l]) = [q_{l+1}]$. Position codes are at least 2 while $STEP$ returns 0 on undefined or off-tape steps, so this holds exactly when $q_{l+1} = \mathbf{M}(q_l)$ is a legal successor position (definition 3.1.5(1) and (2)), making $q_1,\ldots,q_r$ a partial computation that neither halts nor crashes before its last entry. The final factor requires that M store no instruction at the last position, which is the halting condition of definition 3.1.5(3): the run ends because $\mathbf{M}(q_r)$ is undefined, not because the scanner would run off the end of the tape. Hence $\chi_{COMP}(m,n) = 1$ exactly when $m = [M]$ for a machine M and n is the code of a computation of M, as the statement requires.

Proposition 3.3.12: Universal Turing Machine Simulation

Define the 2-place partial function SIM by

$$SIM(m,n) = [(t,j,\mathbf{b})]$$

if $m = [M]$ for some Turing machine M, $n = [(1,0,\mathbf{a})]$ for some input tape $\mathbf{a}$, and M eventually halts in position $(t,j,\mathbf{b})$ on input $\mathbf{a}$, with $SIM(m,n)$ undefined otherwise. Then SIM is recursive.

***Proof*:**
Define the 3-place relation

$$R(m,n,c) \quad\Leftrightarrow\quad COMP(m,c) \text{ and } Element(c,1) = n$$

whose characteristic function

$$\chi_R(m,n,c) = \chi_{COMP}(m,c) \cdot \chi_{\{0\}}(AbsDiff(Element(c,1),n))$$

is primitive recursive by proposition 3.3.11(2), with *Element* from example 3.2.17(11), *AbsDiff* from example 3.2.11(8), and $\chi_{\{0\}}$ from lemma 3.2.14(2). The primitive recursive function $g = \chi_{\{0\}} \circ \chi_R$ vanishes exactly on R. It need not be regular, and by the second part of definition 3.2.28 the unbounded minimalization $c(m,n) = \mu d[g(m,n,d) = 0]$ is a 2-place partial recursive function. Set

$$SIM(m,n) = Element\big(c(m,n),\ Length(c(m,n))\big)$$

a partial recursive function.

Suppose $m = [M]$, $n = [(1,0,\mathbf{a})]$, and M eventually halts on $\mathbf{a}$ in position $(t,j,\mathbf{b})$. The computation $p_1,\ldots,p_r$ of M with input tape $\mathbf{a}$ then exists and its code satisfies $R(m,n,\cdot)$, so $c(m,n)$ is defined and codes a computation $q_1,\ldots,q_v$ of M with $[q_1] = n = [p_1]$. As in the proof of proposition 3.3.6: $q_1 = p_1$ since position codes are injective, $q_l = p_l$ for all l since successor positions are uniquely determined, and $v = r$ since a computation halts at its last entry, so neither sequence properly extends the other. Hence

$$SIM(m,n) = Element(c(m,n),\ r) = [p_r] = [(t,j,\mathbf{b})]$$

If instead m is not a machine code, or n is not the code of a starting position, or M does not halt on $\mathbf{a}$, then no c satisfies $R(m,n,c)$, so the minimalization and with it $SIM(m,n)$ is undefined, as the statement requires.

As a consequence, there is a Turing machine U which can simulate any Turing machine M. Conversely, there is a recursive function f which can compute any other recursive functions.

So far, we have been working with the assumption that our machines will halt. Recall that a Tuing machine was was defined to be a partial function, where a lack of state represents the machine halting. Can we determine when a Turing machine will halt? For example, can we always determine when a Turing machine has an infinite loop? This is called the *Halting Problem*, that is:

Given a Turing machine M and an input tape $\mathbf{a}$, is there a method to determine whether or not M eventually halts on input $\mathbf{a}$?

With the definition of a universal Turing machine, we can ask this question more formally:

Definition 3.3.13: The Halting Problem

Is there a Turing Machine T which, for any Turing machine M with alphabet $\{1\}$ and tape $\mathbf{a}$, for M, halts on input

$$\underline{0}1^{[M]+1}01^{[(1,0,\mathbf{a})]+1}$$

with output $\underline{0}11$ if M halts on input $\mathbf{a}$ and with output $\underline{0}1$ if M does not halt on input $\mathbf{a}$?

If you know Church's Thesis, this is equivalent to saying that it is true. We shall not go over in details at the moment, but the answer to the Halting problem as stated above is No.

3.3.2 Recursively enumerable sets

We end off this chapter on computable and recursive functions by taking a slightly different approach to looking at recursive sets. We have defined a set P to be recursive if χ_P is recursive, that is the domain of a recursive function (in particular the characteristic function). What if instead we looked at the image of general recursive function? Will it always be recursive? We explore this here:

Definition 3.3.14: Recursively Enumerable Sets

A subset (i.e. a 1-place relation) $P \subseteq \mathbb{N}$ is *recursively enumerable* (or *r.e.* for short) if there is 1-place recursive function f such that

$$\text{im}(f) = P = \{f(n) \mid n \in \mathbb{N}\}$$

There are many recursively enumerable sets you may come up with: any constant is naturally recursively enumerable, so is the entire $\mathbb{N}$ ($\text{Im}(\pi^1_1)$), and any finite set is naturally recursively enumerable. proper infinite subsets may also be recursively enumerable, for example all even integers are given by:

$$f(n) = Mult(S(S(O(n))), n)$$

However, there do exist recursively enumerable sets that are not recursive:

Proposition 3.3.15: Properties Of Recursively Enumerable Sets

1. If P is a 1-place recursive relation, then P is recursively enumerable (the converse is not always the case)
2. $P \subseteq \mathbb{N}$ is recursive if and only if both P and $\mathbb{N} \setminus P$ are recursively enumerable
3. $P \subseteq \mathbb{N}$ is recursively enumerable if and only if there is a 1-place partial recursive function g such that $P = dom(g) = \{n \mid g(n) \text{ is defined }\}$

***Proof*:**

1. Let P be a 1-place relation. Then, choosing some $p \in P$ we can define $f(n) = n\chi_P(n) + p\chi_{\neg P}(n)$, where evidently $\text{im}(f) = P$ showing that it's recursively enumerable
2. If P is recursive, then we showed in example 3.2.17 that $\neg P = \mathbb{N} \setminus P$ is recursive, hence they are recursively enumerable. To show the converse, Do the following: If $f(1) = n$, stop, if not then if $g(1) = n$, stop, otherwise if $f(2) = n$, stop, and so on and so forth. Since f and g are both recursive, and $\text{im}(f) \cup \text{im}(g) = \mathbb{N}$, it is guaranteed that at some point we shall find a k such that either $f(k) = n$ or $g(k) = n$. Given this, if H function such that

$$H(t) = \begin{cases} f(t/2) & t = 2l \\ g((t-1)/2) & t = 2l+1 \end{cases}$$

Then

$$\chi_P(n) = div(2, \mu t[|H(t) - n| = 0])$$

is recursive. Note how it was important to use unbounded mineralization for this, would it be possible to prove this without it?

3. Suppose first that P is recursively enumerable, so that there is a 1-place recursive function f with $\text{im}(f) = P$ as in definition 3.3.14. The 2-place function $h(n,t) = AbsDiff(f(t),n)$ is recursive, being a composition of f, projections, and the primitive recursive function $AbsDiff$ of example 3.2.11(8). Define

$$g(n) = \mu t[h(n,t) = 0] = \mu t[AbsDiff(f(t),n) = 0]$$

The function h need not be regular, and by the second part of definition 3.2.28 its unbounded minimalization g is a 1-place partial recursive function. Now $g(n)$ is defined if and only if there is some t with $f(t) = n$, that is, if and only if $n \in \text{im}(f) = P$. Hence $P = \text{Dom}(g)$.

Conversely, suppose $P = \text{Dom}(g)$ for some 1-place partial recursive function g. Suppose first that $P \neq \emptyset$ and fix some $p \in P$. The build-up of g in the second part of definition 3.2.28 may apply unbounded minimalization to inner functions that are partial, and for these definition 3.2.23 needs a gloss. That definition is written for a total inner function, and it declares the minimalization defined exactly when a witness exists. For a partial inner function the two readings can diverge, in exactly one case: when the least witness sits above an argument at which the inner function is undefined. For partial inner functions, and only for them, we adopt the search-termination reading: $\mu m[h(n_1,...,n_k,m) = 0]$ is defined, with value m, exactly when there is an m such that $h(n_1,...,n_k,i)$ is defined for every $i \leq m$ and m is the least witness. In the first direction above the inner function h is total, so definition 3.2.23 applies there as written.

We claim there is a Turing machine M that computes g and halts on the input tape $\underline{0}1^{n+1}$ exactly when $n \in \text{Dom}(g)$. The halting claim is not part of definition 3.2.1, which constrains a machine only on $\text{Dom}(g)$, and it follows by induction on the construction of g from the initial functions. The machines for the initial functions compute them and always halt. The machines of proposition 3.2.6 and proposition 3.2.12 compute compositions and primitive recursions, and they halt exactly when every subcomputation they run halts. None of the machines assembled here ever moves the scanner off the left end of the tape, so halting is the only way any of their runs can terminate. For an unbounded minimalization we reuse the search machine from the proof of proposition 3.2.27, which tests $m = 0, 1, 2, ...$ in order. That proposition proves halting only for regular inner functions, so we take its construction without its halting guarantee and argue directly: by the induction hypothesis each test run halts exactly when the tested value is defined, so the search machine halts exactly when the search reaches a witness, that is, exactly where the minimalization is defined under the reading fixed above, and on halting it outputs that witness. Hence $n \in P$ exactly when some number codes a computation of M starting on input n. Recall that the code of a sequence of tape positions $t_1, t_2, ..., t_k$ is $2^{[t_1]}3^{[t_2]} \cdots p_k^{[t_k]}$ by definition 3.3.3, so $POWER(2,c)$ recovers $[t_1]$ from the code c. Recall also the primitive recursive function $CODE_1$ of lemma 3.3.7, where $CODE_1(n)$ is the code of the starting position on input n, and the primitive recursive relation $COMP_M$ of proposition 3.3.5. Define the 2-place relation

$$R(n,c) \Leftrightarrow COMP_M(c) \text{ and } POWER(2,c) = CODE_1(n)$$

so that $R(n,c)$ holds exactly when c codes a computation of M whose first tape position is the starting position on input n. Its characteristic function

$$\chi_R(n,c) = \chi_{COMP_M}(c) \cdot \chi_{\{0\}}(AbsDiff(POWER(2,c), CODE_1(n)))$$

is primitive recursive, being a product of compositions of primitive recursive functions, where $POWER$ is example 3.2.17(9) and $\chi_{\{0\}}$ is lemma 3.2.14(2). By example 3.2.17(9),

$POWER(n, 0) = 0$, since the bound $k \leq m$ of its definition allows only $k = 0$ when $m = 0$. The value is harmless: for $c = 0$ the factor $\chi_{COMP_M}(0)$ vanishes, since 0 is not the code of any sequence of tape positions, every such code being the positive number $2^{[t_1]} \cdots p_k^{[t_k]}$, so $R(n, 0)$ is false. Since a computation halts by definition, some c with $R(n, c)$ exists if and only if M halts on input n, that is, if and only if $n \in \mathrm{Dom}(g) = P$. As in (1), define the total 2-place function

$$F(n, c) = n\chi_R(n, c) + p\chi_{\neg R}(n, c)$$

which is recursive, where $\chi_{\neg R} = \chi_{\{0\}} \circ \chi_R$ as in example 3.2.17(2). Every value of F lies in P: it is either p, or an n for which $R(n, c)$ holds, and then $n \in \mathrm{Dom}(g) = P$. Conversely, if $n \in P$, then M halts on input n, the finitely many tape positions of that computation have a code c, and $F(n, c) = n$. Hence $\mathrm{im}(F) = P$. Define the 1-place function

$$f(t) = F(POWER(2, t), POWER(3, t))$$

which compresses the two arguments of F into a single prime-power code, the device of theorem 3.2.32. Since $POWER$ and F are total, f is a total 1-place recursive function. Every value of f is a value of F and so lies in P. For $n \in P$, pick c with $R(n, c)$ and take $t = 2^n 3^c$, so that $POWER(2, t) = n$, $POWER(3, t) = c$, and $f(t) = F(n, c) = n$. Hence $\mathrm{im}(f) = P$, and P is recursively enumerable.

For the remaining case $P = \emptyset$, note that the 2-place function $e(n, t) = S(O(t))$ is recursive and never 0, so $\mu t[e(n, t) = 0]$ is a partial recursive function with empty domain, while definition 3.3.14 asks for a total function with image P and the image of a total function is nonempty. The empty set is thus the domain of a partial recursive function but is not the image of any total recursive function. We adopt the standard convention that the empty set counts as recursively enumerable, and with that convention both directions of the equivalence hold for $P = \emptyset$ as well.

3.4 Summary

1. **Tape**: an infinite sequence $\mathbf{a} = a_0 a_1 ...$ such that $a_i \in \{0, 1\}$. If $a_i = 0$ for all i, then $\mathbf{a}$ is blank.
2. **Tape Position**: a triple (s, i, a) where
   (a) s will stand for *state* and we will require $s > 0$.
   (b) i will be *index*.
   (c) a will be the tape

   we would visualize a tape position like so:

   $$(2, 4, \mathbf{a}) = 2 : 010\underline{1}1001011011$$

3. **Turing Machine**: A partial function: $M : \mathrm{Dom}(M) \to \{0, 1\} \times \{-1, 1\} \times \{1, ..., n\}$ with domain subset of

   $$\{1, ..., n\} \times \{0, 1\}$$

   The input represent the current state it's in and what input it's reading on a current tape position, and the output represents what it should override in the current tape position, move either left or right, and go to the next state. Turing machines are usually visualized like so:

M	0	1
1	$1R2$	$0R1$
2	$0L2$	

4. **Computation**: A Turing machine M would, given a tape position, output the information for the successor tape position. Essentially, if (s, i, a) is a tape position and $M(s, a_i) = (b, d, t)$ then the successor tape position is $(t, i + d, a')$ where $a'_i = b$ and $a'_j = a_j$ for the rest of the positions. Then a partial computation with respect to M is a sequence of tape positions p_1... such that each tape position after p_1 are successor tape positions. A partial computation $p_1, p_2, ..., p_k$ is a computation if $p_1 = (1, 0, a)$ (a not necessarily blank) and $M(p_k)$ is undefined (i.e., it halts or terminates)

5. **Computable Function**: Let $f : \mathbb{N}^k \to \mathbb{N}$ be a partial function with domain $\text{Dom}(f)$. Then f is *Turing computable* if for each tuple in $\text{Dom}(f)$, when in unary representation

$$\underline{0}1^{n_1+1}01^{n_2+1}...01^{n_k+1}$$

will halt with output tape

$$\underline{0}1^{f(n_1,n_2,...,n_k)+1}$$

Note that not all functions are computable (like the one deduce in the busy beaver competition). Composition of Turing computable functions is Turing computable.

6. **Initial functions and primitive recursion** We know build up to characterizing all computable functions. They will be something called *recursive* which we know build to define. First, we have our building-block functions: O, S, π^k_i, which are all computable. Next, we have primitive recursion which have a base case, and for the inductive case, through the use of initial functions or previously established primitive recursive functions, it has as input all the inputs of your functions, the step of the induction, and the output of the previous step, and will output the next value. Note that a relation is primitive recursive if the characteristic function on the set is primitive recursive.

7. **Examples**: example 3.2.11, lemma 3.2.14 and example 3.2.17

8. **k-place to 1-place**: Any k-place function or relation can be converted to a 1-place function or relation, and vice-versa. Hence, we may without loss of generality work with 1-place functions/relations

9. **Akerman's Function**: This is a function which is computable but grows faster than any primitive-recursive function can grow, and hence is not primitive recursive

10. **unbounded Minimalization**: given a $k + 1$-place function, the unbounded minimalization is the function f where $f(n_1, n_2, ..., n_k) = m$ when $g(n_1, n_2, ..., n_k, m) = 0$. and m is the smallest such function. We require the extra condition that this is always defined, so we either make the domain a partial domain or make guarantee it by saying it's a property of the function that unbounded minimalization is well-defined; such a function is called a *regular function*. Note that bounded minimalization (bounded by a primitive recursive function) is primitive recursive. Unbounded minimalization on regular functions is computable

11. **Recursive function**: finite composition of initial functions, use of primitive recursion, and unbounded minimization is recursive function. If recursive, then computable. We know show that if computable then recursive.

12. **Code of position** Any tape position is given a numerical representation:

$$[(s, i, \mathbf{a})] = 2^s 3^i 5^{a_0} 7^{a_1} 11^{a_2} \cdots p_{n+3}^{a_n}$$

13. **Code of sequence of tape positions**

$$[t_1 t_2 ... t_n] = 2^{[t_1]} 3^{[t_2]} 5^{[t_3]} \cdots p_n^{[t_n]}$$

14. **Code for Turing Machine** see definition 3.3.10

15. **TapePose, TapePosSeq** relations that detect whether a number represents a tape position or a sequence of tape positions. these are both *primitive recursive*

16. **Entry, Step, Comp**: Entry takes in tape position and it's next moves, and gives the corresponding code of the successor tape position. Step takes in a code for a position and output the code of a Turing machine inputting at that position, and $Comp_M(n)$ is true if n is the code of a computation for M.

17. **Sim**: The recursive function Sim_M takes in a code of position $(1, 0, \mathbf{a})$ where $\mathbf{a}$ is blank and gives the result of the computation via Turing Machine M.

18. **Code, Decode**: Code takes in some finite sequence of natural numbers and represents the code of the unitary representation of it, and decode tasks in a code of a output tape position (i.e. after the computation finished) and gives back the actual number.

19. **Computable, then recursive** If a total function f is computable, then it is recursive

20. **Universal Turing Machine (UTM)** a Turing machine that simulates a Turing machine and gives the output of the simulated Turing machine. We can show that a recursive function exists that can simulate any Turing machine, hence there is a Universal Turing machine (and similarly, a recursive function that "has" all recursive functions as part of it)

21. **Recursively Enumerable Sets**: The image of a recursive function is a recursively enumerable sets

22. **Prop of recursively enumerable sets** 1-place recursive, then recursively enumerable. P recursive if and only if P and $\mathbb{N} \setminus P$ recursively enumerable. Recursively enumerable if and only if also domain of partial recursive function.

4

Incompleteness

Computability makes two questions about a set Σ of axioms precise. The first is Hilbert's:

> *Entscheidungsproblem*: Given a reasonable set Σ of formulas of a first-order language $\mathcal{L}$ and a formula φ of $\mathcal{L}$, is there an effective method for determining whether or not $\Sigma \vdash \varphi$?

The second asks whether Σ proves or refutes every sentence of its language. This chapter answers the second question for the axioms of arithmetic, and the first is not settled in this book. Gödel's first incompleteness theorem (theorem 4.3.3) shows that a consistent recursive set Σ of sentences of $\mathcal{L}_{NT}$ that deduces the nine axioms $\mathcal{A}$ of definition 4.2.1 leaves some sentence neither proved nor refuted. The second (theorem 4.3.10) shows that such a Σ that also deduces the induction axioms of Peano Arithmetic, and whose set of codes is primitive recursive, does not deduce the sentence $\mathrm{Con}(\Sigma)$ of definition 4.3.8, which expresses the consistency of Σ. The completeness theorem (theorem 2.6.31) is a different statement: for every set Σ of sentences and every sentence σ, $\Sigma \models \sigma$ implies $\Sigma \vdash \sigma$.

The chapter first codes the formulas and deductions of $\mathcal{L}_{NT}$ by natural numbers and shows that the relations involved are recursive (section 4.1), which makes "a computable set of axioms" precise. It then shows that every recursive function is representable in the theory of $\mathcal{A}$ by a formula of $\mathcal{L}_{NT}$ (section 4.2). Combining the two gives a sentence that asserts its own unprovability, and the incompleteness theorems follow.

4.1 Deductions are Recursive

The completeness theorem (theorem 2.6.31) is a property of the deductive system. The word also names a property of a set of sentences:

Definition 4.1.1: Completeness For Set of Sentences

A set of sentences Σ of a first-order language $\mathcal{L}$ is said to be *complete* if for every sentence τ of $\mathcal{L}$ either $\Sigma \vdash \tau$ or $\Sigma \vdash \neg\tau$.

In terms of the maximally consistent sets of chapter 2:

Proposition 4.1.2: Complete iff Maximally Consistent

A consistent set of sentences Σ of a first-order language $\mathcal{L}$ is complete if and only if the theory

$$\mathrm{Th}(\Sigma) = \{\tau \mid \tau \text{ is a sentence of } \mathcal{L} \text{ and } \Sigma \vdash \tau\}$$

is maximally consistent

***Proof*:**
Deductions are those of definition 2.6.7, and a set of sentences is maximally consistent if it is consistent and becomes inconsistent when any sentence outside it is added. Every member of $\mathrm{Th}(\Sigma)$ is deduced by Σ, so by proposition 2.6.10(4) every formula that $\mathrm{Th}(\Sigma)$ deduces is deduced by Σ.

Suppose Σ is complete. A deduction of some $\neg(\psi \to \psi)$ from $\mathrm{Th}(\Sigma)$ would give $\Sigma \vdash \neg(\psi \to \psi)$, so $\mathrm{Th}(\Sigma)$ is consistent because Σ is. Let τ be a sentence with $\tau \notin \mathrm{Th}(\Sigma)$. Then $\Sigma \nvdash \tau$, so $\Sigma \vdash \neg\tau$ by definition 4.1.1, and $\neg\tau \in \mathrm{Th}(\Sigma)$, as $\neg\tau$ is a sentence. No truth assignment satisfies $\{A_0, \neg A_0\}$, so $\{A_0, \neg A_0\} \models \neg(A_0 \to A_0)$, and lemma 2.6.8 gives $\{\tau, \neg\tau\} \vdash \neg(\tau \to \tau)$. By proposition 2.6.10(3), $\mathrm{Th}(\Sigma) \cup \{\tau\} \vdash \neg(\tau \to \tau)$, so $\mathrm{Th}(\Sigma) \cup \{\tau\}$ is inconsistent. Hence $\mathrm{Th}(\Sigma)$ is maximally consistent.

Conversely, suppose $\mathrm{Th}(\Sigma)$ is maximally consistent, and let τ be a sentence with $\Sigma \nvdash \tau$. Then $\tau \notin \mathrm{Th}(\Sigma)$, so $\neg\tau \in \mathrm{Th}(\Sigma)$ by proposition 2.6.21(2), that is, $\Sigma \vdash \neg\tau$. Hence Σ is complete.

By proposition 2.6.17(1) and theorem 2.6.31, this $\mathrm{Th}(\Sigma)$ is the theory of definition 1.6.6, defined there through $\models$. It differs from the theory $\mathrm{Th}(\mathfrak{M}) = \{\tau \mid \tau \text{ is a sentence and } \mathfrak{M} \models \tau\}$ of a structure (proposition 2.6.18).

Questions about completeness become questions about recursive functions once the symbols, formulas and deductions of $\mathcal{L}_{NT}$ are coded by natural numbers, as tape positions and Turing machines were coded in definition 3.3.1 and definition 3.3.10. The coding is due to Gödel.

Definition 4.1.3: Gödel Code

The symbols of $\mathcal{L}_{NT}$ are the parentheses, the connectives $\neg$ and $\to$, the quantifier $\forall$, the equality symbol, the variables $v_0, v_1, v_2, \ldots$ of definition 1.1.1, and the symbols $0, S, +, \cdot, E, <$ of definition 1.2.1. Each symbol s of $\mathcal{L}_{NT}$ receives the positive integer $[s]$, called the *Gödel code* of s, as follows:

1. $[(] = 1$ and $[)] = 2$
2. $[\neg] = 3$ and $[\to] = 4$
3. $[\forall] = 5$
4. $[=] = 6$
5. $[v_k] = k + 13$ for every $k \geq 0$
6. $[0] = 7$
7. $[S] = 8$
8. $[+] = 9$, $[\cdot] = 10$, $[E] = 11$
9. $[<] = 12$

The twelve symbols other than the variables receive the codes 1 to 12, and the variables receive the codes $13, 14, 15, \ldots$. So every positive integer is the code of exactly one symbol, and the coding is a bijection from the symbols of $\mathcal{L}_{NT}$ onto the positive integers.

Definition 4.1.4: Gödel Codes Of Sequences

Let $s_1 s_2 \cdots s_k$ be a sequence of symbols of $\mathcal{L}_{NT}$. Then the *Gödel code* of this sequence is

$$[s_1 \cdots s_k] = p_1^{[s_1]} \cdots p_k^{[s_k]}$$

where p_i is the ith prime number. The *Gödel code* of a sequence $\sigma_1 \cdots \sigma_\ell$ of sequences of symbols is $[\sigma_1 \cdots \sigma_\ell] = p_1^{[\sigma_1]} \cdots p_\ell^{[\sigma_\ell]}$.

Every symbol code is positive, so by unique factorization a code determines the sequence it codes.

Example 4.1.5: Gödel Coding

1. The official form of $\forall v_1 \; v_1 \cdot S0 = v_1$ is $(\forall v_1)(= \cdot v_1 S0 v_1)$. Its code is

$$\begin{aligned}[(\forall v_1)(= \cdot v_1 S0 v_1)] &= 2^{[(]}3^{[\forall]}5^{[v_1]}7^{[)]}11^{[(]}13^{[=]}17^{[\cdot]}19^{[v_1]}23^{[S]}29^{[0]}31^{[v_1]}37^{[)]} \\ &= 2^1 3^5 5^{14} 7^2 11^1 13^6 17^{10} 19^{14} 23^8 29^7 31^{14} 37^2\end{aligned}$$

   a number with 98 decimal digits.

2. The sequence of formulas

$$\begin{aligned} &= 00 \\ &(= 00 \to = S0S0) \\ &= S0S0 \end{aligned}$$

is written $0 = 0$, then $0 = 0 \to S0 = S0$, then $S0 = S0$ in ordinary notation. Its code is

$$2^{[=00]}3^{[(=00\to=S0S0)]}5^{[=S0S0]}$$

where

$$[= 00] = 2^6 3^7 5^7 \qquad [= S0S0] = 2^6 3^8 5^7 7^8 11^7$$
$$[(= 00 \to = S0S0)] = 2^1 3^6 5^7 7^7 11^4 13^6 17^8 19^7 23^8 29^7 31^2$$

A number n may be at once the code of a symbol, of a sequence of symbols and of a sequence of sequences of symbols, and the context determines which is meant. Codes of compound expressions are computed from the codes of their parts by $CONCAT$ (example 3.2.17(13)):

Lemma 4.1.6: Codes Of Compound Expressions

For $c, k, m, m_1, m_2 \in \mathbb{N}$ let

$$\begin{aligned} Neg(m) &= CONCAT\big(CONCAT(2^1 3^3, m),\ 2^2\big) \\ Imp(m_1, m_2) &= CONCAT\Big(CONCAT\big(CONCAT(CONCAT(2^1, m_1),\ 2^4),\ m_2\big),\ 2^2\Big) \\ All(k, m) &= CONCAT\big(CONCAT(2^1 3^5 5^{k+13} 7^2 11^1, m),\ 2^2\big) \\ Op(c, m_1, m_2) &= CONCAT\big(CONCAT(2^c, m_1),\ m_2\big) \end{aligned}$$

These functions are primitive recursive. Moreover:

1. if $\sigma = s_1 \cdots s_k$ and τ are nonempty sequences of symbols of $\mathcal{L}_{NT}$, then $LENGTH([\sigma]) = k$, $ELEMENT([\sigma], i) = [s_i]$ for $1 \le i \le k$, $CONCAT([\sigma], [\tau]) = [\sigma\tau]$ and $CONCAT(1, [\sigma]) = [\sigma]$
2. for formulas α, β, terms t, t_1, t_2, a variable v_k and a symbol s with $[s] = c$,

$$Neg([\alpha]) = [(\neg\alpha)] \qquad Imp([\alpha], [\beta]) = [(\alpha \to \beta)] \qquad All(k, [\alpha]) = [(\forall v_k)(\alpha)]$$
$$CONCAT(2^8, [t]) = [St] \qquad Op(c, [t_1], [t_2]) = [st_1t_2]$$

3. if a sequence σ of symbols is a segment of a sequence τ, that is τ is $\rho\sigma\rho'$ for some sequences ρ, ρ', and σ is not τ, then $[\sigma] < [\tau]$

***Proof*:**
The four functions are compositions of $CONCAT$ (example 3.2.17(13)) with EXP, $MULT$, SUM (example 3.2.11(4), (3), (1)) and constants, so they are primitive recursive.

1. By definition 4.1.4, $[\sigma] = p_1^{[s_1]} \cdots p_k^{[s_k]}$, and every exponent is at least 1 by definition 4.1.3. So p_k is the last prime dividing $[\sigma]$, and $LENGTH([\sigma]) = k$ by example 3.2.17(10). By unique factorization and example 3.2.17(11), $ELEMENT([\sigma], i) = [s_i]$. Then example 3.2.17(13) gives $CONCAT([\sigma], [\tau]) = [\sigma\tau]$. As $LENGTH(1) = 0$ by example 3.2.17(10), the formula for $CONCAT$ there gives $CONCAT(1, [\sigma]) = \prod_{t=1}^{k} EXP(Prime(t), [s_t]) = [\sigma]$.

2. By definition 4.1.3, the one-symbol sequences (,), $\to$, S and s have the codes 2^1, 2^2, 2^4, 2^8 and 2^c, and the sequences ($\neg$ and $(\forall v_k)($ have the codes $2^1 3^3$ and $2^1 3^5 5^{k+13} 7^2 11^1$. Each identity is (1) applied to the concatenation that forms the expression on the right. For example $(\alpha \to \beta)$ is the concatenation of (, α, $\to$, β and).

3. Let τ be $u_1 \cdots u_n$ and σ be $u_a \cdots u_b$, with $1 \le a \le b \le n$ and $(a, b) \ne (1, n)$. As $p_j \le p_{a+j-1}$,
$$[\sigma] = \prod_{j=1}^{b-a+1} p_j^{[u_{a+j-1}]} \le \prod_{j=a}^{b} p_j^{[u_j]}$$
and the right side divides $[\tau]$. If $a > 1$, the inequality is strict, since $p_1 < p_a$ and $[u_a] \ge 1$. If $a = 1$, then $b < n$, and $[\tau]$ is at least $p_n^{[u_n]} \ge 2$ times the right side. In both cases $[\sigma] < [\tau]$.

The relations that recognize codes are defined by recursions in which the value at n depends on the values at all smaller arguments. The list of earlier values is coded by a single number:

Lemma 4.1.7: Course-Of-Values Recursion

Let $k \ge 0$. For a $(k+1)$-place function f write
$$\tilde{f}(\vec{n}, m) = \prod_{i=0}^{m-1} p_{i+1}^{f(\vec{n}, i)}$$
for $\vec{n} = (n_1, \ldots, n_k) \in \mathbb{N}^k$ and $m \in \mathbb{N}$, with $\tilde{f}(\vec{n}, 0) = 1$. Then $f(\vec{n}, i) = ELEMENT(\tilde{f}(\vec{n}, m), i+1)$ whenever $i < m$. If g is a primitive recursive $(k+2)$-place function, then exactly one $(k+1)$-place function f satisfies
$$f(\vec{n}, m) = g\big(\vec{n}, m, \tilde{f}(\vec{n}, m)\big) \qquad \text{for all } \vec{n} \text{ and } m$$
and this f is primitive recursive.

***Proof*:**
The number $\tilde{f}(\vec{n}, m)$ is at least 1, and by unique factorization the exponent of p_{i+1} in it is $f(\vec{n}, i)$ for $i < m$. This exponent is $ELEMENT(\tilde{f}(\vec{n}, m), i+1)$ by example 3.2.17(11), which is the first claim. The number $\tilde{f}(\vec{n}, m)$ depends only on the values $f(\vec{n}, i)$ with $i < m$, so the display determines $f(\vec{n}, m)$ by strong induction on m, and f is unique.

The function $\tilde{f}$ satisfies
$$\tilde{f}(\vec{n}, 0) = 1 \qquad \tilde{f}(\vec{n}, m+1) = MULT\Big(\tilde{f}(\vec{n}, m),\ EXP\big(Prime(m+1),\ g(\vec{n}, m, \tilde{f}(\vec{n}, m))\big)\Big)$$

For $k \geq 1$ this is primitive recursion (definition 3.2.10) from the constant function 1, which is $S \circ O \circ \pi_1^k$, and the $(k+2)$-place function $(\vec{n}, m, w) \mapsto MULT(w, EXP(Prime(S(m)), g(\vec{n}, m, w)))$, a composition of primitive recursive functions by example 3.2.11(3), (4) and example 3.2.17(8). For $k = 0$, the same recursion with an extra variable u carried along unchanged gives a primitive recursive 2-place function $T(u, m)$, and $\tilde{f}(m) = T(m, m)$, as in example 3.2.11(5). In both cases $\tilde{f}$ is primitive recursive, and so is $f(\vec{n}, m) = ELEMENT(\tilde{f}(\vec{n}, m+1), m+1)$, a composition of primitive recursive functions.

Codes of terms, formulas, sentences and logical axioms can then be recognized, and substitution can be computed, by primitive recursive functions:

Proposition 4.1.8: Relations To Manipulate Gödel Codes

The following relations and function are primitive recursive.

1. $Term(n) \Leftrightarrow n = [t]$ for some term t of $\mathcal{L}_{NT}$
2. $Formula(n) \Leftrightarrow n = [\varphi]$ for some formula φ of $\mathcal{L}_{NT}$
3. $Sentence(n) \Leftrightarrow n = [\sigma]$ for some sentence σ of $\mathcal{L}_{NT}$
4. $Logical(n) \Leftrightarrow n = [\gamma]$ for some logical axiom γ of $\mathcal{L}_{NT}$ (definition 2.6.4)
5. $Free(n, k) \Leftrightarrow n = [\varphi]$ for a formula φ of $\mathcal{L}_{NT}$ in which v_k is free
6. $Subst(n, k, m) = [\varphi_t^{v_k}]$ if $n = [\varphi]$ for a formula φ and $m = [t]$ for a term t of $\mathcal{L}_{NT}$, and $Subst(n, k, m) = 0$ otherwise

***Proof*:**
Relations are handled through their characteristic functions (definition 3.2.16), and the following closure facts are used throughout.

(a) Negations, disjunctions and conjunctions of primitive recursive relations are primitive recursive by example 3.2.17(2), (3) and (4). So are $EQUAL$ (example 3.2.17(5)) and the relations $x \leq y$ and $x < y$, whose characteristic functions are $\chi_{\{0\}}(DIFF(x, y))$ and $\chi_{\mathbb{N}^*}(DIFF(y, x))$, with $\chi_{\{0\}}$ from lemma 3.2.14(2) and $DIFF$ from example 3.2.11(7).

(b) Let $R(\vec{n}, i)$ be a primitive recursive relation and b a primitive recursive function. The relations $\exists i < b(\vec{n})\, R(\vec{n}, i)$ and $\forall i < b(\vec{n})\, R(\vec{n}, i)$ have the characteristic functions

$$\chi_{\mathbb{N}^*}\Big(\sum_{i=0}^{b(\vec{n})} \chi_{\mathbb{N}^*}(DIFF(b(\vec{n}), i)) \cdot \chi_R(\vec{n}, i)\Big)$$

$$\chi_{\{0\}}\Big(\sum_{i=0}^{b(\vec{n})} \chi_{\mathbb{N}^*}(DIFF(b(\vec{n}), i)) \cdot \chi_{\{0\}}(\chi_R(\vec{n}, i))\Big)$$

which are primitive recursive by example 3.2.11(2) and the remark opening example 3.2.17 on composing a bounded sum with a primitive recursive upper limit.

(c) If $R_1, \ldots, R_r$ are pairwise disjoint primitive recursive relations and $h_0, \ldots, h_r$ are primitive

recursive functions, then the function equal to h_j on R_j and to h_0 outside $R_1 \cup \cdots \cup R_r$ is $\sum_{j=1}^{r} \chi_{R_j} h_j + \chi_{\{0\}}\big(\sum_{j=1}^{r} \chi_{R_j}\big) h_0$, which is primitive recursive.

(d) If $R(\vec{n}, i)$ and b are primitive recursive and, for each $\vec{n}$, at most one $i < b(\vec{n})$ has $R(\vec{n}, i)$, then the function sending $\vec{n}$ to that i, and to 0 if there is none, is $\sum_{i=0}^{b(\vec{n})} i \cdot \chi_{\mathbb{N}^*}(DIFF(b(\vec{n}), i)) \cdot \chi_R(\vec{n}, i)$, which is primitive recursive.

(e) Let f be a $(k+1)$-place function, and suppose that $f(\vec{n}, m)$ is obtained from $\vec{n}$, m and the values $f(\vec{n}, i)$ with $i < m$ by composition with primitive recursive functions and by (a) to (d). Replacing each such value $f(\vec{n}, i)$ by $ELEMENT(w, i+1)$ gives a $(k+2)$-place function $g(\vec{n}, m, w)$, primitive recursive by the same constructions, and $f(\vec{n}, m) = g(\vec{n}, m, \tilde{f}(\vec{n}, m))$ by the first claim of lemma 4.1.7. By the uniqueness in that lemma, f is primitive recursive. This applies in particular to $f = \chi_R$ for a relation R.

The functions Neg, Imp, All and Op are those of lemma 4.1.6, and a number codes at most one sequence of symbols. Let n code a term or a formula. Every term or formula named below as a part of it is a segment of it other than the whole, so its code is less than n by lemma 4.1.6(3). A variable v_j occurring in it has $j < n$, since $p^{j+13} | n$ for some prime p. So every existential quantifier below may be bounded by n through (b) without losing a witness, and the values of a recursively described function are used only at arguments below n, as (e) requires.

Step 1: terms. By definition 1.1.3 and definition 1.2.1, a term of $\mathcal{L}_{NT}$ is 0, a variable, St for a term t, or ft_1t_2 for a symbol f among $+$, $\cdot$, E and terms t_1, t_2. The symbols 0, v_j, S, $+$, $\cdot$, E have the codes 7, $j+13$, 8, 9, 10, 11. By lemma 4.1.6(2), $Term(n)$ holds exactly when one of the following holds:

- $n = 2^7$, or $n = 2^c$ for $c = ELEMENT(n, 1) \geq 13$
- $n = CONCAT(2^8, m)$ for some $m < n$ with $Term(m)$
- $n = Op(c, m_1, m_2)$ for some c with $9 \leq c \leq 11$ and some $m_1, m_2 < n$ with $Term(m_1)$ and $Term(m_2)$

If n codes a term, then the term has one of the four forms, and the codes of its parts witness one condition. Conversely, each condition makes n the code of 0 or of v_{c-13}, of St with $m = [t]$, or of ft_1t_2 with $[f] = c$ and $m_i = [t_i]$, and these are terms. By (a), (b) and (e), $Term$ is primitive recursive.

Step 2: formulas. By definition 1.1.5 and definition 1.2.1, whose only relation symbol is the binary $<$, a formula of $\mathcal{L}_{NT}$ is $= t_1t_2$ or $< t_1t_2$ for terms t_1, t_2, or $(\neg\alpha)$, $(\alpha \to \beta)$ or $(\forall v_j)(\alpha)$ for formulas α, β and a variable v_j. Let $Atomic(n)$ hold when $n = Op(c, m_1, m_2)$ for some $c \in \{6, 12\}$ and some $m_1, m_2 < n$ with $Term(m_1)$ and $Term(m_2)$. It is primitive recursive by Step 1, (a) and (b), and it holds exactly when n codes an atomic formula, as $[=] = 6$ and $[<] = 12$. So $Formula(n)$ holds exactly when one of the following holds:

- $Atomic(n)$
- $n = Neg(m)$ for some $m < n$ with $Formula(m)$
- $n = Imp(m_1, m_2)$ for some $m_1, m_2 < n$ with $Formula(m_1)$ and $Formula(m_2)$

- $n = All(j, m)$ for some $j, m < n$ with $Formula(m)$

If n codes a formula, the codes of its parts witness one condition, and conversely each condition makes n the code of a formula, by lemma 4.1.6(2). By (e), $Formula$ is primitive recursive.

Step 3: free variables and sentences. By theorem 1.1.8, a formula satisfies exactly one condition of definition 1.1.5, in one way, so the parts named in the conditions of Step 2 are determined by n. Following definition 1.2.2, and as v_k is the only symbol with code $k + 13$, $Free(n, k)$ holds exactly when $Formula(n)$ holds and one of the following holds:

- $Atomic(n)$, and $ELEMENT(n, i) = k + 13$ for some i with $1 \leq i \leq LENGTH(n)$
- $n = Neg(m)$ for some $m < n$ with $Formula(m)$ and $Free(m, k)$
- $n = Imp(m_1, m_2)$ for some $m_1, m_2 < n$ with $Formula(m_1)$ and $Formula(m_2)$, and $Free(m_1, k)$ or $Free(m_2, k)$
- $n = All(j, m)$ for some $j, m < n$ with $Formula(m)$, $j \neq k$ and $Free(m, k)$

By lemma 4.1.6(1), the first condition says that v_k occurs in the atomic formula coded by n. By (e), applied to $(k, n) \mapsto \chi_{Free}(n, k)$, the relation $Free$ is primitive recursive. By definition 1.2.2, a variable free in a formula occurs in it, so $Sentence(n)$ holds exactly when $Formula(n)$ holds and $Free(n, k)$ fails for every $k < n$. Hence $Sentence$ is primitive recursive by (a) and (b).

Step 4: substitution. For $n, k, m, i \in \mathbb{N}$ let $P(n, k, m, i)$ be m if $ELEMENT(n, i) = k + 13$ and $2^{ELEMENT(n,i)}$ otherwise, primitive recursive by (c). Let $H(n, k, m, 0) = 1$ and $H(n, k, m, j + 1) = CONCAT(H(n, k, m, j), P(n, k, m, j + 1))$, a primitive recursion (definition 3.2.10), and let $Rep(n, k, m) = H(n, k, m, LENGTH(n))$. Let $n = [s_1 \cdots s_\ell]$ and $m = [t]$ for a term t. By induction on j with lemma 4.1.6(1), $H(n, k, m, j)$ is the code of the sequence obtained from $s_1 \cdots s_j$ by replacing each occurrence of v_k with t, for $1 \leq j \leq \ell$. So $Rep(n, k, m)$ codes the sequence obtained from the one coded by n by replacing every occurrence of v_k with t. For an atomic formula φ, the formula $\varphi_t^{v_k}$ is this sequence, by definition 1.5.1 and definition 1.5.3, as recorded at the start of the proof of lemma 1.5.7.

By (d), (b) and theorem 1.1.8, the following functions are primitive recursive, each taking the value 0 when no such part exists:

- $N(n)$ is the $m < n$ with $Formula(m)$ and $n = Neg(m)$
- $L(n)$ is the $m_1 < n$ for which some $m_2 < n$ has $Formula(m_1)$, $Formula(m_2)$ and $n = Imp(m_1, m_2)$, and $R(n)$ is that m_2
- $J(n)$ is the $j < n$ for which some $m < n$ has $Formula(m)$ and $n = All(j, m)$, and $A(n)$ is that m

Following definition 1.5.3, $Subst(n, k, m)$ is 0 unless $Formula(n)$ and $Term(m)$ hold, and otherwise it is

- $Rep(n, k, m)$ if $Atomic(n)$

- $Neg(Subst(N(n), k, m))$ if $Formula(N(n))$ and $n = Neg(N(n))$
- $Imp(Subst(L(n), k, m), Subst(R(n), k, m))$ if $Formula(L(n))$, $Formula(R(n))$ and $n = Imp(L(n), R(n))$
- n if $Formula(A(n))$, $n = All(J(n), A(n))$ and $J(n) = k$
- $All(J(n), Subst(A(n), k, m))$ if $Formula(A(n))$, $n = All(J(n), A(n))$ and $J(n) \neq k$

The cases are disjoint by theorem 1.1.8, and the values of $Subst$ on the right are taken at $N(n), L(n), R(n), A(n) < n$. By (c) and (e), applied to $(k, m, n) \mapsto Subst(n, k, m)$, the function $Subst$ is primitive recursive.

Step 5: substitutability. Let $Occ(m, j)$ hold when $ELEMENT(m, i) = j + 13$ for some i with $1 \leq i \leq LENGTH(m)$, that is, by lemma 4.1.6(1), when v_j occurs in the sequence coded by m. Let $Sbl(n, k, m)$ say that n codes a formula φ, m codes a term t, and t is substitutable for v_k in φ. Following definition 1.5.5, $Sbl(n, k, m)$ holds exactly when $Formula(n)$ and $Term(m)$ hold and one of the following holds:

- $Atomic(n)$
- $n = Neg(m')$ for some $m' < n$ with $Formula(m')$ and $Sbl(m', k, m)$
- $n = Imp(m_1, m_2)$ for some $m_1, m_2 < n$ with $Formula(m_1)$, $Formula(m_2)$, $Sbl(m_1, k, m)$ and $Sbl(m_2, k, m)$
- $n = All(j, m')$ for some $j, m' < n$ with $Formula(m')$, and either $Free(n, k)$ fails, or $Occ(m, j)$ fails and $Sbl(m', k, m)$ holds

By (e), applied to $(k, m, n) \mapsto \chi_{Sbl}(n, k, m)$, the relation Sbl is primitive recursive.

Step 6: logical axioms. In each condition below, a, b, c are codes of formulas, so the condition includes $Formula(a)$, $Formula(b)$ and $Formula(c)$ for the codes it names. By definition 2.6.4 and lemma 4.1.6(2), n codes an instance of one of the schemas **A1** to **A8** exactly when one of the following holds for some numbers a, b, c, i, j, k, m below n:

A1 $n = Imp(a, Imp(b, a))$

A2 $n = Imp(Imp(a, Imp(b, c)), Imp(Imp(a, b), Imp(a, c)))$

A3 $n = Imp(Imp(Neg(b), Neg(a)), Imp(Imp(Neg(b), a), b))$

A4 $n = Imp(All(k, a), Subst(a, k, m))$ and $Sbl(a, k, m)$

A5 $n = Imp(All(k, Imp(a, b)), Imp(All(k, a), All(k, b)))$

A6 $n = Imp(a, All(k, a))$ and $Free(a, k)$ fails

A7 $n = Op(6, 2^{k+13}, 2^{k+13})$

A8 $n = Imp(Op(6, 2^{i+13}, 2^{j+13}), Imp(a, b))$, $Atomic(a)$, $LENGTH(b) = LENGTH(a)$, and every l with $1 \leq l \leq LENGTH(a)$ has $ELEMENT(b, l) = ELEMENT(a, l)$, or $ELEMENT(a, l) = i + 13$ and $ELEMENT(b, l) = j + 13$

In **A8**, replacing an occurrence of the one-symbol sequence v_i by v_j keeps the length and every other position, so by lemma 4.1.6(1) the condition says that b codes the result of replacing some occurrences of v_i by v_j in the atomic formula coded by a. The bound $m < n$ in **A4** loses no instance. If v_k is free in the formula α coded by a, then the term t coded by m is a segment of $\alpha_t^{v_k}$, by induction on α through definition 1.5.3: in the atomic case every occurrence of v_k is replaced by t, in the cases of $\neg$ and $\rightarrow$ the variable v_k is free in a part, and in the case $(\forall v_j)(\beta)$ the index j is not k and the substitution passes to β. If v_k is not free in α, then $\alpha_t^{v_k}$ is α for every term t by lemma 1.5.7(1), and the same instance arises from the term v_k, which is substitutable for v_k by lemma 1.5.7(1) and has the code $2^{k+13} < n$. So the relation $Inst(n)$, which says that n codes an instance of **A1** to **A8**, is primitive recursive by (a), (b) and Steps 1 to 5. By definition 2.6.4, the logical axioms are the generalizations $\forall x_1 \cdots \forall x_r \gamma$, $r \geq 0$, of these instances, and for $r \geq 1$ such a generalization is $(\forall x_1)(\gamma')$ for the generalization $\gamma' = \forall x_2 \cdots \forall x_r \gamma$. So $Logical(n)$ holds exactly when $Inst(n)$ holds or $n = All(j, m)$ for some $j, m < n$ with $Logical(m)$, and $Logical$ is primitive recursive by (e).

With these relations, the deductions from a set of sentences can be recognized once the set itself is computable:

Definition 4.1.9: Recursive Set

A set of formulas Δ of $\mathcal{L}_{NT}$ is *recursive* if the set of Gödel codes of its formulas

$$[\Delta] = \{[\delta] \mid \delta \in \Delta\}$$

is a recursive subset of $\mathbb{N}$, that is, a recursive 1-place relation. The set Δ is *recursively enumerable* if $[\Delta]$ is recursively enumerable (definition 3.3.14).

Proposition 4.1.10: Relations To Recursive Set

Let Δ be a recursive set of sentences of $\mathcal{L}_{NT}$. Then the following relations are recursive:

1. $Premise_\Delta(n) \Leftrightarrow n = [\beta]$ for some formula β of $\mathcal{L}_{NT}$ which is either a logical axiom or in Δ
2. $Formulas(n) \Leftrightarrow n = [\varphi_1 \varphi_2 \cdots \varphi_k]$ for some sequence $\varphi_1, \varphi_2, \ldots, \varphi_k$ of formulas of $\mathcal{L}_{NT}$
3. $Inference(n, i, j) \Leftrightarrow n = [\varphi_1 \varphi_2 \cdots \varphi_k]$ for some sequence $\varphi_1, \varphi_2, \ldots, \varphi_k$ of formulas of $\mathcal{L}_{NT}$, $1 \leq i, j \leq k$, and φ_k follows from φ_i and φ_j by Modus Ponens
4. $Deduction_\Delta(n) \Leftrightarrow n = [\varphi_1 \varphi_2 \cdots \varphi_k]$ for a deduction $\varphi_1, \varphi_2, \ldots, \varphi_k$ from Δ in $\mathcal{L}_{NT}$
5. $Conclusion_\Delta(n, m) \Leftrightarrow n = [\varphi_1 \varphi_2 \cdots \varphi_k]$ for a deduction $\varphi_1, \varphi_2, \ldots, \varphi_k$ from Δ in $\mathcal{L}_{NT}$ and $m = [\varphi_k]$

***Proof*:**

Throughout, $Element$, $Length$, $SUBSEQ$, $EQUAL$ and $DIFF$ (written $n \dot{-} m$) are the primitive recursive functions of example 3.2.17 and example 3.2.11, $\chi_{\{a\}}$ and the bounded products are those of lemma 3.2.14, the bounded sums are example 3.2.11(2), $\chi_{\mathbb{N}^*} = \chi_{\{0\}} \circ \chi_{\{0\}}$ tests being nonzero, and a bounded sum or product starting at index 1 is read as in the proof of proposition 3.3.11. The

bounded sum and product constructions apply verbatim to recursive ingredients, definition 3.2.28 closing recursive functions under primitive recursion and composition.

1. A number n is the code of a formula which is a logical axiom or a member of Δ exactly when $Logical(n)$ holds or $n \in [\Delta]$, so

$$\chi_{Premise_\Delta}(n) = \chi_{\mathbb{N}^*}\big(\chi_{Logical}(n) + \chi_{[\Delta]}(n)\big)$$

   Here $\chi_{Logical}$ is primitive recursive by proposition 4.1.8(4), and $\chi_{[\Delta]}$ is recursive by definition 4.1.9 since Δ is a recursive set. Hence $Premise_\Delta$ is recursive.

2. A number n codes a nonempty sequence of formulas exactly when $n \geq 2$ and the exponent of each p_i with $1 \leq i \leq Length(n)$ codes a formula. The code of a formula is at least 2, so $\chi_{Formula}(0) = 0$ and a missing prime is detected through its zero exponent. Hence

$$\chi_{Formulas}(n) = \chi_{\mathbb{N}^*}(n \dot{-} 1) \cdot \prod_{i=1}^{Length(n)} \chi_{Formula}(Element(n, i))$$

   with $\chi_{Formula}$ primitive recursive by proposition 4.1.8(2). So $Formulas$ is primitive recursive, hence recursive.

3. By lemma 4.1.6(2), the primitive recursive function Imp satisfies $Imp([\alpha], [\gamma]) = [(\alpha \to \gamma)]$ for formulas α, γ. The formula φ_k follows from φ_i and φ_j by Modus Ponens exactly when one of φ_i, φ_j is the implication whose antecedent is the other and whose consequent is φ_k. Hence, writing $b = Element(n, Length(n))$ for the code of φ_k,

$$\begin{aligned}\chi_{Inference}(n, i, j) = \chi_{Formulas}(n) &\cdot \chi_{\mathbb{N}^*}(i) \cdot \chi_{\mathbb{N}^*}(j) \\ &\cdot \chi_{\{0\}}(i \dot{-} Length(n)) \cdot \chi_{\{0\}}(j \dot{-} Length(n)) \\ &\cdot \chi_{\mathbb{N}^*}\big(EQUAL\big(Element(n, j),\ Imp(Element(n, i),\ b)\big) \\ &\qquad + EQUAL\big(Element(n, i),\ Imp(Element(n, j),\ b)\big)\big)\end{aligned}$$

   is primitive recursive, hence recursive.

4. By definition 2.6.7, with Modus Ponens as the only rule of inference, n codes a deduction from Δ exactly when n codes a nonempty sequence of formulas $\varphi_1, \ldots, \varphi_k$ in which each φ_l is a logical axiom or a member of Δ, or follows from two strictly earlier entries by Modus Ponens. The prefix $\varphi_1, \ldots, \varphi_l$ has code $SUBSEQ(n, 1, l)$ by example 3.2.17(12), and its last formula is φ_l, so

$$\begin{aligned}\chi_{Deduction_\Delta}(n) = \chi_{Formulas}(n) &\cdot \prod_{l=1}^{Length(n)} \chi_{\mathbb{N}^*}\Big(\chi_{Premise_\Delta}(Element(n, l)) \\ &\qquad + \sum_{i=1}^{l \dot{-} 1} \sum_{j=1}^{l \dot{-} 1} \chi_{Inference}\big(SUBSEQ(n, 1, l),\ i,\ j\big)\Big)\end{aligned}$$

   The lth factor equals 1 exactly when φ_l is justified, the inner sums ranging over the strictly earlier indices $i, j \leq l - 1$. All ingredients are recursive, so $Deduction_\Delta$ is recursive.

5. The conclusion of the deduction coded by n is its last formula. Its code is the value $Element(n, Length(n))$, so

$$\chi_{Conclusion_\Delta}(n,m) = \chi_{Deduction_\Delta}(n) \cdot EQUAL\big(m,\ Element(n, Length(n))\big)$$

is recursive.

If $[\Delta]$ is primitive recursive, the same constructions show that all five relations are primitive recursive, since $\chi_{[\Delta]}$ is the only ingredient that is not primitive recursive in general. The theorems of a recursive set of sentences can be listed by a recursive function, and decided when the set is complete:

Theorem 4.1.11: Computability And Completeness

Let Δ be a recursive set of sentences of $\mathcal{L}_{NT}$. Then $[Th(\Delta)]$ is

1. recursively enumerable, and
2. recursive if Δ is complete

***Proof*:**
Here $Th(\Delta)$ is the set of sentences τ with $\Delta \vdash \tau$, and $Element$, $Length$, $EQUAL$, $MULT$ and $\chi_{\{0\}}$ are the primitive recursive functions of example 3.2.17, example 3.2.11 and lemma 3.2.14(2). Recursive functions are closed under composition by definition 3.2.28.

1. Let $\sigma_0 :\equiv \forall v_1(v_1 = v_1)$, a generalization of an instance of **A7** and hence a logical axiom (definition 2.6.4), so $\sigma_0 \in Th(\Delta)$. Let

$$K(n) = \chi_{Deduction_\Delta}(n) \cdot \chi_{Sentence}(Element(n, Length(n)))$$

with $Deduction_\Delta$ recursive by proposition 4.1.10(4) and $Sentence$ primitive recursive by proposition 4.1.8(3), and let

$$F(n) = K(n) \cdot Element(n, Length(n)) + \chi_{\{0\}}(K(n)) \cdot [\sigma_0]$$

a total recursive 1-place function. If n codes a deduction $\varphi_1, \ldots, \varphi_k$ from Δ, then its exponents are codes of formulas, which are at least 2, so $Length(n) = k$ and $Element(n,k) = [\varphi_k]$. Hence $K(n) = 1$ exactly when n codes a deduction from Δ whose last formula is a sentence, and then $F(n)$ is the code of that sentence, which lies in $Th(\Delta)$. Otherwise $F(n) = [\sigma_0]$. So every value of F lies in $[Th(\Delta)]$, and conversely, if $\tau \in Th(\Delta)$, the code n of a deduction of τ from Δ has $F(n) = [\tau]$. Thus $[Th(\Delta)] = \mathrm{im}(F)$ is recursively enumerable (definition 3.3.14).

2. Let Δ be complete. If Δ is inconsistent, then Δ deduces every formula by proposition 2.6.17(3), so $[Th(\Delta)]$ is the set of codes of sentences, which is recursive by proposition 4.1.8(3). Suppose Δ is consistent, and let τ be a sentence. At least one of τ, $\neg\tau$ lies in $Th(\Delta)$, by completeness. At most one does: from $\{A_0, \neg A_0\} \models \neg(A_0 \to A_0)$, lemma 2.6.8 and proposition 2.6.10(4) would give $\Delta \vdash \neg(\tau \to \tau)$. By lemma 4.1.6(2), the primitive recursive function Neg satisfies $Neg([\tau]) = [(\neg\tau)]$. Let F be as in (1) and let

$$G(n,t) = MULT\Big(\chi_{Sentence}(n),\ \chi_{\{0\}}\big(EQUAL(F(t),n) + EQUAL(F(t), Neg(n))\big)\Big)$$

which vanishes exactly when n is not the code of a sentence or $F(t) \in \{n, Neg(n)\}$. It is regular (definition 3.2.24): if n is not the code of a sentence, then $G(n,0) = 0$, and if $n = [\tau]$, then $[\tau]$ or $[\neg\tau]$ lies in $[Th(\Delta)] = \text{im}(F)$. So $T(n) = \mu t[G(n,t) = 0]$ is recursive in the first sense of definition 3.2.28, and so is

$$\chi(n) = \chi_{Sentence}(n) \cdot EQUAL(F(T(n)), n)$$

We show $\chi = \chi_{[Th(\Delta)]}$. If n is not the code of a sentence, both are 0. Let $n = [\tau]$. If $\tau \in Th(\Delta)$, then $F(T(n))$ is n or $[\neg\tau]$, and it is not $[\neg\tau]$, since that code lies in $\text{im}(F) = [Th(\Delta)]$ only if $\neg\tau \in Th(\Delta)$, so $\chi(n) = 1$. If $\tau \notin Th(\Delta)$, then $n \notin \text{im}(F)$, so $F(T(n)) \neq n$ and $\chi(n) = 0$. Hence $[Th(\Delta)]$ is recursive, completing the proof.

4.2 Recursive Functions are Representable in $\mathcal{A}$

This section fixes a finite set $\mathcal{A}$ of axioms about the natural numbers and shows that every recursive function is representable in the theory of $\mathcal{A}$: a formula of $\mathcal{L}_{NT}$ defines it, and $\mathcal{A}$ deduces enough instances of that formula to determine every value. Representability then passes to every set Σ of sentences with $\Sigma \vdash \mathcal{A}$ (proposition 4.2.22).

The language of number theory $\mathcal{L}_{NT}$ of definition 1.2.1 has the constant 0, the unary function symbol S, the binary function symbols $+$, $\cdot$ and E, and the binary relation symbol $<$. Its standard structure is

$$\mathfrak{N} = (\mathbb{N}, 0, S, +, \cdot, E, <)$$

Section 4.1 codes formulas of $\mathcal{L}_{NT}$ by natural numbers and manipulates the codes with recursive functions. The complementary direction expresses natural numbers and recursive functions by formulas of $\mathcal{L}_{NT}$. It rests on a set of axioms that proves enough about successor, addition, multiplication and exponentiation:

Definition 4.2.1: Non-Logical Axioms Of Number Theory ($\mathcal{A}$)

Let $\mathcal{A}$ be the following set of sentences of $\mathcal{L}_{NT}$, written out in official form:

1. $(\forall v_0)((\neg = Sv_0 0))$
2. $(\forall v_0)(((\neg = v_0 0) \to (\neg(\forall v_1)((\neg = Sv_1 v_0)))))$
3. $(\forall v_0)((\forall v_1)((= Sv_0 Sv_1 \to = v_0 v_1)))$
4. $(\forall v_0)(= +v_0 0 v_0)$
5. $(\forall v_0)((\forall v_1)(= +v_0 Sv_1 S + v_0 v_1))$
6. $(\forall v_0)(= \cdot v_0 00)$
7. $(\forall v_0)((\forall v_1)(= \cdot v_0 Sv_1 + \cdot v_0 v_1 v_0))$
8. $(\forall v_0)(= Ev_0 0S0)$
9. $(\forall v_0)((\forall v_1)(= Ev_0 Sv_1 \cdot Ev_0 v_1 v_0))$

Read informally, the axioms say:

1. for every n, $n + 1 \neq 0$
2. for every $n \neq 0$ there is a k with $k + 1 = n$
3. for all n and k, $n + 1 = k + 1$ implies $n = k$
4. for every n, $n + 0 = n$
5. for all n and k, $n + (k + 1) = (n + k) + 1$
6. for every n, $n \cdot 0 = 0$
7. for all n and k, $n \cdot (k + 1) = (n \cdot k) + n$
8. for every n, $n^0 = 1$
9. for all n and k, $n^{k+1} = (n^k) \cdot n$

Proposition 4.2.2: Natural Number Axioms Are True

Let $\mathfrak{N} = (\mathbb{N}, 0, S, +, \cdot, E, <)$ be the structure consisting of the natural numbers and the usual functions. Then

$$\mathfrak{N} \models \mathcal{A}$$

***Proof*:**
Each member of $\mathcal{A}$ is a sentence, so it suffices to fix an arbitrary assignment s into $\mathfrak{N}$ and verify $\mathfrak{N} \models \sigma[s]$ for each of the nine axioms σ. As s is arbitrary, $\mathfrak{N} \models \mathcal{A}$ then holds by definition 1.4.11. In each case we unwind definition 1.4.5: the clause for $\forall$ passes to the modifications $s[v_0|a]$, and then $s[v_0|a][v_1|b]$, over all $a, b \in \mathbb{N}$, the clause for $\neg$ negates, the clause for $\rightarrow$ requires the consequent whenever the antecedent holds, and the atomic clause compares the values that the term assignment function of definition 1.4.4 gives to the two sides, interpreting S, $+$, $\cdot$, E as the usual successor, sum, product and power on $\mathbb{N}$, and $<$, which no axiom mentions, as the usual order. The nine axioms so reduce to the following statements, quantified over all $a, b \in \mathbb{N}$.

1. $a + 1 \neq 0$. True: successors are positive.
2. If $a \neq 0$, then it is not the case that every $b \in \mathbb{N}$ has $b + 1 \neq a$. True: a nonzero natural number a has the predecessor $b = a - 1$, and $b + 1 = a$.
3. If $a + 1 = b + 1$, then $a = b$. True: the successor operation on $\mathbb{N}$ is injective.
4. $a + 0 = a$.
5. $a + (b + 1) = (a + b) + 1$.
6. $a \cdot 0 = 0$.
7. $a \cdot (b + 1) = a \cdot b + a$.
8. $a^0 = 1$ (with $0^0 = 1$, the value the usual recursion for exponentiation on $\mathbb{N}$ assigns).
9. $a^{b+1} = a^b \cdot a$.

Items (4) to (9) are the defining recursions of addition, multiplication and exponentiation on $\mathbb{N}$, and items (1) to (3) are basic properties of the successor. Hence every axiom is satisfied, and $\mathfrak{N} \models \mathcal{A}$.

The axioms of $\mathcal{A}$ do not deduce every sentence true in $\mathfrak{N}$. For instance, let φ be $\neg(x = Sx)$. The instance $(\varphi_0^x \wedge \forall x(\varphi \to \varphi_{Sx}^x)) \to \forall x\varphi$ of induction is true in $\mathfrak{N}$, but it is false in the following model $\mathfrak{M}$ of $\mathcal{A}$. Its universe is $\mathbb{N} \cup \{\infty\}$, it agrees with $\mathfrak{N}$ on $\mathbb{N}$ (including $<$, which holds of no pair involving ∞), and $S(\infty) = \infty$, $x + y = \infty$ whenever $\infty \in \{x, y\}$, $x \cdot y = 0$ whenever $0 \in \{x, y\}$ and $x \cdot y = \infty$ for the other pairs involving ∞, and $E(x, 0) = 1$ for every x, $E(\infty, b) = \infty$ for $b \geq 1$, $E(0, \infty) = 0$, $E(1, \infty) = 1$ and $E(x, \infty) = \infty$ for $x \geq 2$ and for $x = \infty$. At arguments in $\mathbb{N}$ every axiom holds as in $\mathfrak{N}$ (proposition 4.2.2), and at arguments involving ∞ each side of each axiom is computed from these values: for instance $\infty = S(\infty)$ is the successor that axiom (2) asks for, $x \cdot S(\infty) = x \cdot \infty = x \cdot \infty + x$ for every x, and $E(x, S(\infty)) = E(x, \infty) = E(x, \infty) \cdot x$ for every x. In $\mathfrak{M}$ the premise of the instance holds, as $S(0) \neq 0$, $S(n) \neq S(S(n))$ for $n \in \mathbb{N}$, and φ fails at ∞ so that the implication holds there. The conclusion $\forall x\varphi$ fails at ∞. So $\mathcal{A}$ does not deduce the instance, by proposition 2.6.17(1). Neither $\mathcal{L}_{NT}$ nor $\mathcal{A}$ is as small as possible: E is definable from $+$ and $\cdot$.

For $m \in \mathbb{N}$, write $S^m 0$ for the term consisting of m symbols S followed by 0, the *numeral* of m. Its value in $\mathfrak{N}$ is m:

Lemma 4.2.3: Repeated Successor Function

For every $m \in \mathbb{N}$ and every assignment function s for $\mathfrak{N}$,

$$\overline{s}(S^m 0) = m$$

***Proof*:**
We use induction on m and definition 1.4.4. The term $S^0 0$ is 0, and $\overline{s}(0) = 0^{\mathfrak{N}} = 0$. The term $S^{m+1}0$ is S followed by $S^m 0$, so $\overline{s}(S^{m+1}0) = S^{\mathfrak{N}}(\overline{s}(S^m 0)) = S^{\mathfrak{N}}(m) = m + 1$ by the induction hypothesis, as claimed.

For a formula φ and $n_1, \ldots, n_k \in \mathbb{N}$, write

$$\varphi(S^{n_1}0, \ldots, S^{n_k}0) :\equiv \varphi_{S^{n_1}0 \cdots S^{n_k}0}^{v_1 \cdots v_k}$$

for the formula obtained by substituting $S^{n_1}0$ for v_1, then $S^{n_2}0$ for v_2, and so on up to v_k. When the free variables of φ are among $v_1, \ldots, v_{k+1}$, the same formula is also written $\varphi(S^{n_1}0, \ldots, S^{n_k}0, v_{k+1})$, as v_{k+1} may still be free in it. A numeral contains no variable, and its substitution is evaluated in any structure as follows:

Lemma 4.2.4: Numerals In Structures

Let $\mathfrak{M}$ be an $\mathcal{L}_{NT}$-structure.

1. For $a \in \mathbb{N}$, the element $\overline{s}(S^a 0)$ of $|\mathfrak{M}|$ is the same for every assignment s into $\mathfrak{M}$. Writing $\hat{a}$ for it, $\hat{0} = 0^{\mathfrak{M}}$ and $\widehat{a+1} = S^{\mathfrak{M}}(\hat{a})$, and $\hat{a} = a$ when $\mathfrak{M}$ is $\mathfrak{N}$.

2. Let the free variables of a formula φ be among $v_1, \ldots, v_j$, and let $e_1, \ldots, e_j \in |\mathfrak{M}|$. Whether $\mathfrak{M} \models \varphi[s(v_1|e_1)\cdots(v_j|e_j)]$ holds is the same for every assignment s into $\mathfrak{M}$, and $\mathfrak{M} \models \varphi[e_1, \ldots, e_j]$ denotes that it holds.

3. Let x be a variable, $a \in \mathbb{N}$ and φ a formula. Then $S^a 0$ is substitutable for x in φ, every variable free in $\varphi^x_{S^a 0}$ is free in φ and is not x, and $\mathfrak{M} \models \varphi^x_{S^a 0}[s]$ exactly when $\mathfrak{M} \models \varphi[s(x|\hat{a})]$, for every assignment s into $\mathfrak{M}$.

4. Let the free variables of φ be among $v_1, \ldots, v_j$, let $i \leq j$ and $a_1, \ldots, a_i \in \mathbb{N}$. Then the free variables of $\varphi(S^{a_1}0, \ldots, S^{a_i}0)$ are among $v_{i+1}, \ldots, v_j$, and $\mathfrak{M} \models \varphi(S^{a_1}0, \ldots, S^{a_i}0)[s]$ exactly when $\mathfrak{M} \models \varphi[s(v_1|\hat{a}_1)\cdots(v_i|\hat{a}_i)]$, for every assignment s. For $i = j$, the formula $\varphi(S^{a_1}0, \ldots, S^{a_j}0)$ is a sentence, true in $\mathfrak{M}$ exactly when $\mathfrak{M} \models \varphi[\hat{a}_1, \ldots, \hat{a}_j]$.

***Proof*:**

1. No variable occurs in $S^a 0$, so its value is the same under every assignment by lemma 1.4.8. By definition 1.4.4, $\overline{s}(0) = 0^{\mathfrak{M}}$ and $\overline{s}(SS^a 0) = S^{\mathfrak{M}}(\overline{s}(S^a 0))$. In $\mathfrak{N}$, $\hat{a} = a$ by lemma 4.2.3.

2. Two assignments of the form $s(v_1|e_1)\cdots(v_j|e_j)$ agree on $v_1, \ldots, v_j$, hence on the free variables of φ, so proposition 1.4.9 applies.

3. The term $S^a 0$ is closed, so it is substitutable for x in φ by lemma 2.6.27(1), and lemma 1.5.7(3) gives the claim on free variables. By lemma 2.4.1, $\mathfrak{M} \models \varphi^x_{S^a 0}[s]$ exactly when $\mathfrak{M} \models \varphi[s(x|\overline{s}(S^a 0))]$, and $\overline{s}(S^a 0) = \hat{a}$ by (1).

4. The formula $\varphi(S^{a_1}0, \ldots, S^{a_i}0)$ is $\varphi(S^{a_1}0, \ldots, S^{a_{i-1}}0)$ with $S^{a_i}0$ substituted for v_i. By induction on i with (3), its free variables are among $v_{i+1}, \ldots, v_j$, and for every assignment s, $\mathfrak{M} \models \varphi(S^{a_1}0, \ldots, S^{a_i}0)[s]$ holds exactly when $\mathfrak{M} \models \varphi(S^{a_1}0, \ldots, S^{a_{i-1}}0)[s(v_i|\hat{a}_i)]$, that is, by the induction hypothesis at the assignment $s(v_i|\hat{a}_i)$, exactly when

$$\mathfrak{M} \models \varphi[s(v_i|\hat{a}_i)(v_1|\hat{a}_1)\cdots(v_{i-1}|\hat{a}_{i-1})]$$

   As $v_1, \ldots, v_i$ are distinct, this assignment is $s(v_1|\hat{a}_1)\cdots(v_i|\hat{a}_i)$. For $i = j$ the formula has no free variable, so it is a sentence (definition 1.2.4), and by (2), corollary 1.4.10 and definition 1.4.11(4) it is true in $\mathfrak{M}$ exactly when $\mathfrak{M} \models \varphi[\hat{a}_1, \ldots, \hat{a}_j]$.

In a model of $\mathcal{A}$, the elements named by numerals obey the arithmetic of $\mathbb{N}$. No axiom of $\mathcal{A}$ mentions $<$, so a model of $\mathcal{A}$ may interpret $<$ as any binary relation. The relation $\prec$ of the following lemma, defined from $+$, is the order that the axioms determine at the elements named by numerals.

Lemma 4.2.5: Arithmetic In Models Of $\mathcal{A}$

Let $\mathfrak{M}$ be a model of $\mathcal{A}$, write $0, S, +, \cdot, E$ for $0^{\mathfrak{M}}, S^{\mathfrak{M}}, +^{\mathfrak{M}}, \cdot^{\mathfrak{M}}, E^{\mathfrak{M}}$, and let $\hat{a}$ be as in lemma 4.2.4(1). For terms t, u write $t \prec u$ for the formula $\exists w(w + St = u)$, where w is a variable occurring in neither t nor u, and for $x, y \in |\mathfrak{M}|$ write $x \prec y$ if $e + S(x) = y$ for some $e \in |\mathfrak{M}|$. Then the following hold for all $x, y \in |\mathfrak{M}|$ and $a, b \in \mathbb{N}$.

1. $S(x) \neq 0$, every $x \neq 0$ is $S(z)$ for some $z \in |\mathfrak{M}|$, $S(x) = S(y)$ only if $x = y$, $x + 0 = x$, $x + S(y) = S(x + y)$, $x \cdot 0 = 0$, $x \cdot S(y) = (x \cdot y) + x$, $E(x, 0) = S(0)$ and $E(x, S(y)) = E(x, y) \cdot x$
2. $\hat{a} + \hat{b} = \widehat{a + b}$, $\hat{a} \cdot \hat{b} = \widehat{ab}$, $E(\hat{a}, \hat{b}) = \widehat{a^b}$ and $S(x) + \hat{b} = S(x + \hat{b})$
3. $\hat{a} = \hat{b}$ only if $a = b$
4. if $y + x = \hat{b}$, then $x = \hat{i}$ and $y = \widehat{b - i}$ for some $i \leq b$
5. $\mathfrak{M} \models (t \prec u)[s]$ exactly when $\overline{s}(t) \prec \overline{s}(u)$, for every assignment s into $\mathfrak{M}$, and in $\mathfrak{N}$ the relation $x \prec y$ is $x < y$
6. $x \prec \hat{b}$ exactly when $x = \hat{i}$ for some $i < b$
7. $x \prec \hat{b}$, or $x = \hat{b}$, or $\hat{b} \prec x$

Moreover, $\mathcal{A} \vdash \neg(S^a 0 = S^b 0)$ for all $a, b \in \mathbb{N}$ with $a \neq b$.

***Proof*:**

1. Each item is one of the nine axioms of definition 4.2.1 unwound at an assignment through definition 1.4.5, as in the proof of proposition 4.2.2, with the operations of $\mathfrak{M}$ in place of those of $\mathfrak{N}$. Write $S^j(x)$ for the j-fold application of S to x, so that $\hat{a} = S^a(0)$ by lemma 4.2.4(1).
2. Each identity is proved for all a and x by induction on b, with (1) and the induction hypothesis. First, $\hat{a} + \hat{0} = \hat{a}$, and $\hat{a} + \widehat{b + 1} = \hat{a} + S(\hat{b}) = S(\hat{a} + \hat{b}) = S(\widehat{a + b}) = \widehat{a + b + 1}$. Then $\hat{a} \cdot \hat{0} = \hat{0}$, and $\hat{a} \cdot \widehat{b + 1} = \hat{a} \cdot \hat{b} + \hat{a} = \widehat{ab} + \hat{a} = \widehat{ab + a}$ by the first identity. Then $E(\hat{a}, \hat{0}) = S(\hat{0}) = \hat{1}$, and $E(\hat{a}, \widehat{b + 1}) = E(\hat{a}, \hat{b}) \cdot \hat{a} = \widehat{a^b} \cdot \hat{a} = \widehat{a^{b+1}}$ by the second identity. Last, $S(x) + \hat{0} = S(x) = S(x + \hat{0})$, and $S(x) + \widehat{b + 1} = S(S(x) + \hat{b}) = S(S(x + \hat{b})) = S(x + \widehat{b + 1})$.
3. Suppose $a < b$ and $\hat{a} = \hat{b}$, that is, $S^a(0) = S^b(0)$. Cancelling S a total of a times by (1) gives $0 = S^{b-a}(0) = S(S^{b-a-1}(0))$, which (1) excludes. By symmetry, $\hat{a} = \hat{b}$ forces $a = b$.
4. Induction on b. If $x = 0$, then $y = y + 0 = \hat{b}$, and $i = 0$ works. Otherwise $x = S(x')$ for some x' by (1), and $\hat{b} = y + S(x') = S(y + x')$. By (1) this excludes $b = 0$, so $\hat{b} = S(\widehat{b - 1})$ and $y + x' = \widehat{b - 1}$. The induction hypothesis gives $x' = \hat{i}'$ and $y = \widehat{b - 1 - i'}$ for some $i' \leq b - 1$, so $i = i' + 1$ works, as $x = S(\hat{i}') = \widehat{i' + 1}$.
5. By proposition 1.4.13, $\mathfrak{M} \models (t \prec u)[s]$ exactly when $\mathfrak{M} \models (w + St = u)[s(w|e)]$ for some $e \in |\mathfrak{M}|$. The variable w occurs in neither t nor u, so t and u have the same values under

$s(w|e)$ as under s by lemma 1.4.8. By the equality clause and definition 1.4.4, the condition says $e + S(\overline{s}(t)) = \overline{s}(u)$. In $\mathfrak{N}$, $e + x + 1 = y$ for some e exactly when $x < y$.

6. If $e + S(x) = \hat{b}$, then (4) gives $S(x) = \hat{j}$ for some $j \leq b$. Here $j \neq 0$, as $S(x) \neq 0 = \hat{0}$, so $S(x) = S(\widehat{j-1})$ and $x = \widehat{j-1}$ by (1), with $j - 1 < b$. Conversely, for $i < b$, (2) gives $\widehat{b-i-1} + S(\hat{i}) = \widehat{b-i-1} + \widehat{i+1} = \hat{b}$, so $\hat{i} \prec \hat{b}$.

7. Induction on b. For $b = 0$, either $x = 0 = \hat{0}$, or $x = S(z)$ for some z by (1), and then $z + S(\hat{0}) = S(z + \hat{0}) = S(z) = x$, so $\hat{0} \prec x$. Suppose (7) holds for b. If $x \prec \hat{b}$, then $x = \hat{i}$ with $i < b$ by (6), so $x \prec \widehat{b+1}$ by (6). If $x = \hat{b}$, then $x \prec \widehat{b+1}$ by (6). Let $\hat{b} \prec x$, say $e + S(\hat{b}) = x$. If $e = 0$, then $x = \hat{0} + \widehat{b+1} = \widehat{b+1}$ by (2). Otherwise $e = S(e')$ for some e' by (1), and (1) and (2) give

$$x = S(e') + S(\hat{b}) = S(S(e') + \hat{b}) = S(S(e' + \hat{b})) = S(e' + \widehat{b+1}) = e' + S(\widehat{b+1})$$

so $\widehat{b+1} \prec x$.

For the last claim, let $a \neq b$. In every model $\mathfrak{M}$ of $\mathcal{A}$, (3) gives $\hat{a} \neq \hat{b}$, so $\mathfrak{M} \not\models (S^a 0 = S^b 0)$ by the equality clause of definition 1.4.5 and lemma 4.2.4(1), and $\mathfrak{M} \models \neg(S^a 0 = S^b 0)$ by proposition 1.4.14(1). Hence $\mathcal{A} \vdash \neg(S^a 0 = S^b 0)$ by theorem 2.6.31.

For a variable x and $b \geq 1$, the sentences

$$\forall x\big(x \prec S^b 0 \leftrightarrow (x = 0 \vee x = S0 \vee \cdots \vee x = S^{b-1}0)\big) \qquad \forall x\big(x \prec S^b 0 \vee x = S^b 0 \vee S^b 0 \prec x\big)$$

are true in every model of $\mathcal{A}$, by lemma 4.2.5(5), (6) and (7), proposition 1.4.6(1) and (3), and the quantifier and equality clauses of definition 1.4.5, so $\mathcal{A}$ deduces them by theorem 2.6.31. The second sentence is deducible for $b = 0$ as well.

With numerals as names for the natural numbers, a function is represented by a formula when the axioms deduce, for each input, which value the formula takes:

Definition 4.2.6: Representable Function

Let Σ be a set of sentences of $\mathcal{L}_{NT}$ and f a k-place function. A formula φ of $\mathcal{L}_{NT}$ whose free variables are among $v_1, \ldots, v_{k+1}$ *represents* f in $\mathrm{Th}(\Sigma)$ if

$$\Sigma \vdash \forall v_{k+1}\big(\varphi(S^{n_1}0, \ldots, S^{n_k}0, v_{k+1}) \leftrightarrow v_{k+1} = S^{f(n_1,\ldots,n_k)}0\big)$$

for all $n_1, \ldots, n_k \in \mathbb{N}$. The function f is *representable* in $\mathrm{Th}(\Sigma)$ if some formula represents it.

The displayed formula is a sentence by lemma 4.2.4(4), so the condition says that it lies in $\mathrm{Th}(\Sigma) = \{\tau \mid \Sigma \vdash \tau\}$. It makes Σ prove that, at the inputs $S^{n_1}0, \ldots, S^{n_k}0$, the formula φ holds at the value $S^{f(n_1,\ldots,n_k)}0$ and at no other value of v_{k+1}. This uniqueness is what lets representing formulas be composed (proposition 4.2.15). The cases of interest are $\Sigma = \mathcal{A}$ and sets Σ with $\Sigma \vdash \mathcal{A}$.

Example 4.2.7: Representable Functions

Each of the following formulas represents the given function in $\mathrm{Th}(\Sigma)$, for every set Σ of sentences of $\mathcal{L}_{NT}$, in particular in $\mathrm{Th}(\mathcal{A})$.

1. The constant function $c_3^1(n) = 3$ is represented by $v_2 = S^3 0$. The variable v_1 does not occur in this formula, so substituting $S^n 0$ for v_1 leaves it unchanged by lemma 1.5.7(1), and the condition of definition 4.2.6 is $\Sigma \vdash \forall v_2(v_2 = S^3 0 \leftrightarrow v_2 = S^3 0)$ for every n.

2. The zero function $O(n) = 0$ is represented by $v_2 = 0$, for the same reason with $S^0 0$, which is 0, in place of $S^3 0$.

3. The successor function $S(n) = n + 1$ is represented by $v_2 = Sv_1$. Substituting $S^n 0$ for v_1 gives $v_2 = SS^n 0$, and $SS^n 0$ is the term $S^{n+1}0$, so the condition is $\Sigma \vdash \forall v_2(v_2 = S^{n+1}0 \leftrightarrow v_2 = S^{n+1}0)$.

4. The projection π_i^k is represented by $v_{k+1} = v_i$. Substituting $S^{n_1}0, \ldots, S^{n_k}0$ for $v_1, \ldots, v_k$ gives $v_{k+1} = S^{n_i}0$, the substitutions for the variables other than v_i leaving the formula unchanged by lemma 1.5.7(1). So the condition is $\Sigma \vdash \forall v_{k+1}(v_{k+1} = S^{n_i}0 \leftrightarrow v_{k+1} = S^{n_i}0)$.

Each condition has the form $\Sigma \vdash \forall v(\chi \leftrightarrow \chi)$. The formula $\chi \leftrightarrow \chi$ replaces A_0 by χ in $A_0 \leftrightarrow A_0$, which every truth assignment makes true by proposition 1.7.4(5). So $\vdash \chi \leftrightarrow \chi$ by lemma 2.6.8, then $\vdash \forall v(\chi \leftrightarrow \chi)$ by theorem 2.6.12 with no premises, and $\Sigma \vdash \forall v(\chi \leftrightarrow \chi)$ by proposition 2.6.10(3).

Definition 4.2.8: Representable Relation

Let Σ be a set of sentences of $\mathcal{L}_{NT}$ and $P \subseteq \mathbb{N}^k$ a k-place relation. A formula ψ of $\mathcal{L}_{NT}$ whose free variables are among $v_1, \ldots, v_k$ *represents* P in $\mathrm{Th}(\Sigma)$ if, for all $n_1, \ldots, n_k \in \mathbb{N}$,

$$\begin{aligned} P(n_1, \ldots, n_k) \quad &\Rightarrow \quad \Sigma \vdash \psi(S^{n_1}0, \ldots, S^{n_k}0) \\ \text{not } P(n_1, \ldots, n_k) \quad &\Rightarrow \quad \Sigma \vdash \neg\psi(S^{n_1}0, \ldots, S^{n_k}0) \end{aligned}$$

The relation P is *representable* in $\mathrm{Th}(\Sigma)$ if some formula represents it.

So Σ deduces the instances of ψ at the members of P and refutes its instances at the other tuples. When Σ is consistent, $P(n_1, \ldots, n_k)$ holds exactly when $\Sigma \vdash \psi(S^{n_1}0, \ldots, S^{n_k}0)$ (proposition 4.2.22(1)).

Example 4.2.9: Representable Relation

1. The formula $v_1 = S^3 0$ represents the 1-place relation $P = \{3\}$ in $\mathrm{Th}(\mathcal{A})$. Substituting $S^n 0$ for v_1 gives $S^n 0 = S^3 0$. For $n = 3$ this sentence is deducible by lemma 2.6.27(2), hence deducible from $\mathcal{A}$ by proposition 2.6.10(3), and for $n \neq 3$, $\mathcal{A} \vdash \neg(S^n 0 = S^3 0)$ by lemma 4.2.5. The formula $v_2 = S^3 0$ does not represent P, as its free variable is not among v_1. The formula $v_1 = S^3 0$ does not represent the function of example 4.2.7(1): for $n = 3$ the sentence $\forall v_2(S^3 0 = S^3 0 \leftrightarrow v_2 = S^3 0)$ that definition 4.2.6 requires is false in $\mathfrak{N}$, at an assignment sending v_2 to 0, so $\mathcal{A}$ does not deduce it, by proposition 2.6.17(1) and proposition 4.2.2.

2. Let $P = 2\mathbb{N}$. The formula $\psi :\equiv \exists v_2(v_1 = S^2 0 \cdot (v_2 \cdot S0))$ represents P in $\mathrm{Th}(\mathcal{A})$. By lemma 4.2.11(2) below it is enough to show, for a model $\mathfrak{M}$ of $\mathcal{A}$ and $n \in \mathbb{N}$, that $\mathfrak{M} \models \psi[\hat{n}]$ exactly when n is even. By proposition 1.4.13, the equality clause of definition 1.4.5, definition 1.4.4 and lemma 4.2.4(1), $\mathfrak{M} \models \psi[\hat{n}]$ exactly when $\hat{n} = \hat{2} \cdot (e \cdot \hat{1})$ for some $e \in |\mathfrak{M}|$. If $n = 2m$, then $e = \hat{m}$ works, as $\hat{2} \cdot (\hat{m} \cdot \hat{1}) = \hat{2} \cdot \hat{m} = \hat{n}$ by lemma 4.2.5(2). For odd n, induction on n shows that no $d \in |\mathfrak{M}|$ has $\hat{2} \cdot d = \hat{n}$, which covers $d = e \cdot \hat{1}$. By lemma 4.2.5(1), $\hat{2} \cdot 0 = 0 = \hat{0}$, which is not $\hat{n}$ by lemma 4.2.5(3), as $n \neq 0$. Any other d is $S(y)$ for some

y, and then $\hat{2} \cdot d = (\hat{2} \cdot y) + \hat{2}$ by lemma 4.2.5(1). If this were $\hat{n}$, then lemma 4.2.5(4) and (3) would give $2 \leq n$ and $\hat{2} \cdot y = \widehat{n-2}$. For $n = 1$ the first fails, and for $n \geq 3$ the second contradicts the induction hypothesis at the odd number $n - 2$.

By soundness and the completeness theorem, a sentence is deducible from Σ exactly when it is true in every model of Σ. So representability can be checked in models, at the elements named by numerals:

Definition 4.2.10: Faithful Formula

A formula φ of $\mathcal{L}_{NT}$ whose free variables are among $v_1, \ldots, v_{k+1}$ is *faithful* for a k-place function g if

(F1) for all $a_1, \ldots, a_k, b \in \mathbb{N}$, $\mathfrak{N} \models \varphi[a_1, \ldots, a_k, b]$ exactly when $g(a_1, \ldots, a_k) = b$, and

(F2) for all $a_1, \ldots, a_k \in \mathbb{N}$ and every model $\mathfrak{M}$ of $\mathcal{A}$, the element $\hat{b}$ with $b = g(a_1, \ldots, a_k)$ is the only $e \in |\mathfrak{M}|$ with $\mathfrak{M} \models \varphi[\hat{a}_1, \ldots, \hat{a}_k, e]$.

Condition (F1) says that φ defines the graph of g in $\mathfrak{N}$, and (F2) says that in every model of $\mathcal{A}$, at the inputs $\hat{a}_1, \ldots, \hat{a}_k$, the formula φ holds at the value $g(\widehat{a_1, \ldots, a_k})$ and nowhere else. As $\mathfrak{N}$ is a model of $\mathcal{A}$ (proposition 4.2.2) in which $\hat{a} = a$ (lemma 4.2.4(1)), condition (F1) is the case $\mathfrak{M} = \mathfrak{N}$ of (F2).

Lemma 4.2.11: Representability In Models

Let Σ be a set of sentences of $\mathcal{L}_{NT}$.

1. A formula φ whose free variables are among $v_1, \ldots, v_{k+1}$ represents a k-place function f in $\mathrm{Th}(\Sigma)$ if and only if, for every model $\mathfrak{M}$ of Σ, all $a_1, \ldots, a_k \in \mathbb{N}$ and every $e \in |\mathfrak{M}|$,

$$\mathfrak{M} \models \varphi[\hat{a}_1, \ldots, \hat{a}_k, e] \quad \text{exactly when} \quad e = \hat{b}, \text{ where } b = f(a_1, \ldots, a_k)$$

2. A formula ψ whose free variables are among $v_1, \ldots, v_k$ represents a k-place relation P in $\mathrm{Th}(\Sigma)$ if and only if, for every model $\mathfrak{M}$ of Σ and all $a_1, \ldots, a_k \in \mathbb{N}$, $\mathfrak{M} \models \psi[\hat{a}_1, \ldots, \hat{a}_k]$ exactly when $P(a_1, \ldots, a_k)$.

3. A formula is faithful for a k-place function g if and only if it represents g in $\mathrm{Th}(\mathcal{A})$.

***Proof*:**
A sentence σ satisfies $\Sigma \vdash \sigma$ exactly when every model of Σ satisfies σ: one direction is proposition 2.6.17(1) with definition 1.6.1, and the other is theorem 2.6.31. Write $\vec{a}$ for $(a_1, \ldots, a_k)$.

1. Fix $\vec{a}$, let $b = f(\vec{a})$, and let τ be the sentence $\forall v_{k+1}(\varphi(S^{a_1}0, \ldots, S^{a_k}0, v_{k+1}) \leftrightarrow v_{k+1} = S^b 0)$ of definition 4.2.6. Let $\mathfrak{M}$ be a model of Σ and s an assignment into $\mathfrak{M}$. By the quantifier clause of definition 1.4.5 and proposition 1.4.6(3), $\mathfrak{M} \models \tau[s]$ exactly when, for every $e \in |\mathfrak{M}|$, $\mathfrak{M} \models \varphi(S^{a_1}0, \ldots, S^{a_k}0, v_{k+1})[s(v_{k+1}|e)]$ holds exactly when $\mathfrak{M} \models (v_{k+1} = S^b 0)[s(v_{k+1}|e)]$ does. By lemma 4.2.4(4) and (2) the first is $\mathfrak{M} \models \varphi[\hat{a}_1, \ldots, \hat{a}_k, e]$, and by the equality clause and lemma 4.2.4(1) the second is $e = \hat{b}$. As τ is a sentence, $\mathfrak{M} \models \tau$ exactly when $\mathfrak{M} \models \tau[s]$, by corollary 1.4.10 and definition 1.4.11(4). Hence $\Sigma \vdash \tau$ exactly when the displayed condition holds at $\vec{a}$ for every model of Σ and every e. Ranging over all $\vec{a}$ gives

(1).

2. By lemma 4.2.4(4), $\psi(S^{a_1}0, \ldots, S^{a_k}0)$ is a sentence, true in a model $\mathfrak{M}$ exactly when $\mathfrak{M} \models \psi[\hat{a}_1, \ldots, \hat{a}_k]$, and by proposition 1.4.14(1) its negation is true in $\mathfrak{M}$ exactly when $\mathfrak{M} \not\models \psi[\hat{a}_1, \ldots, \hat{a}_k]$. So the first clause of definition 4.2.8 says that $P(\vec{a})$ implies $\mathfrak{M} \models \psi[\hat{a}_1, \ldots, \hat{a}_k]$ for every model $\mathfrak{M}$ of Σ, and the second says that the failure of $P(\vec{a})$ implies $\mathfrak{M} \not\models \psi[\hat{a}_1, \ldots, \hat{a}_k]$ for every such $\mathfrak{M}$. Together they say that $\mathfrak{M} \models \psi[\hat{a}_1, \ldots, \hat{a}_k]$ exactly when $P(\vec{a})$.

3. By (1) with $\Sigma = \mathcal{A}$, condition (F2) of definition 4.2.10 holds exactly when the formula represents g in $\mathrm{Th}(\mathcal{A})$, and (F1) is the case $\mathfrak{M} = \mathfrak{N}$ of (F2), as $\mathfrak{N} \models \mathcal{A}$ by proposition 4.2.2 and $\hat{a} = a$ in $\mathfrak{N}$ by lemma 4.2.4(1).

Building representing formulas from given ones renames their variables. A variable that does not occur in a formula can replace a free variable, and the result is evaluated by the substitution lemma:

Lemma 4.2.12: Substituting A Fresh Variable

Let φ be a formula and x, z variables such that z does not occur in φ.

1. The variable z is substitutable for x in φ.
2. Every variable free in φ^x_z is z, or is a variable other than x that is free in φ.
3. For every structure $\mathfrak{A}$ and every assignment s into $\mathfrak{A}$, $\mathfrak{A} \models \varphi^x_z[s]$ exactly when $\mathfrak{A} \models \varphi[s(x|s(z))]$.

***Proof*:**
Items (1) and (2) are proved by induction on φ (definition 1.1.5).

1. An atomic formula admits every substitution by definition 1.5.5, and the cases of $\neg$ and $\to$ follow from the induction hypothesis. Let $\varphi :\equiv (\forall y)(\alpha)$. As z does not occur in φ, the variable y is not z, so y does not occur in the term z, and z is substitutable for x in α by the induction hypothesis. So the second alternative of the quantifier clause of definition 1.5.5 applies.

2. For an atomic formula, φ^x_z is φ with every occurrence of x replaced by z, by definition 1.5.1 and definition 1.5.3, and the variables free in an atomic formula are those occurring in it (definition 1.2.2). If $\varphi :\equiv (\neg\alpha)$ or $\varphi :\equiv (\alpha \to \beta)$, then φ^x_z is $(\neg\alpha^x_z)$ or $(\alpha^x_z \to \beta^x_z)$, a variable free in it is free in α^x_z or in β^x_z, and the induction hypothesis applies, a variable free in α or in β being free in φ. Let $\varphi :\equiv (\forall y)(\alpha)$. If y is x, then φ^x_z is φ, in which x is not free. Otherwise φ^x_z is $(\forall y)(\alpha^x_z)$, and a variable v free in it is not y and is free in α^x_z. By the induction hypothesis v is z, or v is not x and is free in α, hence free in φ.

3. By (1) and lemma 2.4.1, $\mathfrak{A} \models \varphi^x_z[s]$ exactly when $\mathfrak{A} \models \varphi[s(x|\overline{s}(z))]$, and $\overline{s}(z) = s(z)$ by definition 1.4.4.

With $\prec$, a formula can select the least witness of a represented relation:

Lemma 4.2.13: Least Witnesses

Let R be a $(k+1)$-place relation such that every $(a_1, \ldots, a_k) \in \mathbb{N}^k$ has some $b \in \mathbb{N}$ with $R(a_1, \ldots, a_k, b)$, and let ψ represent R in $\mathrm{Th}(\mathcal{A})$. Let z be a variable other than $v_1, \ldots, v_{k+1}$ that does not occur in ψ, and let

$$\psi_{\min} :\equiv \psi \wedge \forall z\big(z \prec v_{k+1} \to \neg \psi_z^{v_{k+1}}\big)$$

Then $\psi_{\min}$ represents in $\mathrm{Th}(\mathcal{A})$ the k-place function sending $(a_1, \ldots, a_k)$ to the least b with $R(a_1, \ldots, a_k, b)$.

***Proof*:**

By lemma 4.2.12(2), the free variables of $\psi_z^{v_{k+1}}$ are among $v_1, \ldots, v_k, z$, so those of $\psi_{\min}$ are among $v_1, \ldots, v_{k+1}$. Let $\mathfrak{M}$ be a model of $\mathcal{A}$, let $a_1, \ldots, a_k \in \mathbb{N}$, write $\vec{a}$ for $(a_1, \ldots, a_k)$ and $\hat{a}$ for $\hat{a}_1, \ldots, \hat{a}_k$, and let b be the least number with $R(\vec{a}, b)$. By lemma 4.2.11(1) it suffices to show that, for $e \in |\mathfrak{M}|$, $\mathfrak{M} \models \psi_{\min}[\hat{a}, e]$ exactly when $e = \hat{b}$.

Let s be an assignment sending $v_1, \ldots, v_k, v_{k+1}$ to $\hat{a}_1, \ldots, \hat{a}_k, e$. By proposition 1.4.6(2), $\mathfrak{M} \models \psi_{\min}[s]$ exactly when $\mathfrak{M} \models \psi[s]$ and the second conjunct holds at s. By the quantifier, implication and negation clauses of definition 1.4.5, the second conjunct holds at s exactly when every $x \in |\mathfrak{M}|$ with $\mathfrak{M} \models (z \prec v_{k+1})[s(z|x)]$ has $\mathfrak{M} \not\models \psi_z^{v_{k+1}}[s(z|x)]$. By lemma 4.2.5(5), $\mathfrak{M} \models (z \prec v_{k+1})[s(z|x)]$ says $x \prec e$. By lemma 4.2.12(3), $\mathfrak{M} \models \psi_z^{v_{k+1}}[s(z|x)]$ exactly when $\mathfrak{M} \models \psi[s(z|x)(v_{k+1}|x)]$, and as z does not occur in ψ, this is $\mathfrak{M} \models \psi[\hat{a}, x]$ by proposition 1.4.9. Hence

$$\mathfrak{M} \models \psi_{\min}[\hat{a}, e] \quad \text{exactly when} \quad \mathfrak{M} \models \psi[\hat{a}, e] \text{ and } \mathfrak{M} \not\models \psi[\hat{a}, x] \text{ for every } x \prec e$$

By lemma 4.2.11(2), $\mathfrak{M} \models \psi[\hat{a}, \hat{c}]$ exactly when $R(\vec{a}, c)$, for $c \in \mathbb{N}$.

The element $\hat{b}$ satisfies the right side of the display: $R(\vec{a}, b)$ gives $\mathfrak{M} \models \psi[\hat{a}, \hat{b}]$, and if $x \prec \hat{b}$, then $x = \hat{i}$ for some $i < b$ by lemma 4.2.5(6), so $R(\vec{a}, i)$ fails by the choice of b and $\mathfrak{M} \not\models \psi[\hat{a}, \hat{i}]$. Conversely, let e satisfy the right side. By lemma 4.2.5(7), $e \prec \hat{b}$, or $e = \hat{b}$, or $\hat{b} \prec e$. If $e \prec \hat{b}$, then $e = \hat{i}$ for some $i < b$ by lemma 4.2.5(6), and $\mathfrak{M} \models \psi[\hat{a}, \hat{i}]$ gives $R(\vec{a}, i)$, contradicting the choice of b. If $\hat{b} \prec e$, then the right side with $x = \hat{b}$ gives $\mathfrak{M} \not\models \psi[\hat{a}, \hat{b}]$, contradicting $R(\vec{a}, b)$. Hence $e = \hat{b}$.

Proposition 4.2.14: Relating Representable Functions and Relations

Let f be a k-place function and $P \subseteq \mathbb{N}^k$ a k-place relation.

1. The function f is representable in $\mathrm{Th}(\mathcal{A})$ if and only if the $(k+1)$-place relation P_f defined by
$$P_f(n_1, \ldots, n_k, n_{k+1}) \Leftrightarrow f(n_1, \ldots, n_k) = n_{k+1}$$
is representable in $\mathrm{Th}(\mathcal{A})$. Every formula that represents f in $\mathrm{Th}(\mathcal{A})$ represents P_f.

2. The relation P is representable in $\mathrm{Th}(\mathcal{A})$ if and only if its characteristic function χ_P is. If φ represents χ_P in $\mathrm{Th}(\mathcal{A})$, then $\varphi_{S0}^{v_{k+1}}$ represents P.

3. If ψ represents P in $\mathrm{Th}(\mathcal{A})$, then $\neg\psi$ represents $\mathbb{N}^k \setminus P$. In particular P is representable in $\mathrm{Th}(\mathcal{A})$ exactly when its complement is.

***Proof*:**
Throughout, $\mathfrak{M}$ is a model of $\mathcal{A}$, $a_1, \ldots, a_k \in \mathbb{N}$, $\vec{a} = (a_1, \ldots, a_k)$, and $\hat{a}$ abbreviates $\hat{a}_1, \ldots, \hat{a}_k$. Each claim is checked through lemma 4.2.11.

1. Let φ represent f, and let $b \in \mathbb{N}$. By lemma 4.2.11(1), $\mathfrak{M} \models \varphi[\hat{a}, \hat{b}]$ exactly when $\hat{b} = \widehat{f(\vec{a})}$, that is, by lemma 4.2.5(3), exactly when $P_f(\vec{a}, b)$. So φ represents P_f by lemma 4.2.11(2). Conversely, let ψ represent P_f. For every $\vec{a}$, the only b with $P_f(\vec{a}, b)$ is $f(\vec{a})$, which is therefore the least such b. So by lemma 4.2.13, for a variable z as there, the formula $\psi_{\min}$ represents f.

2. Let φ represent χ_P. By lemma 4.2.4(3), the free variables of $\varphi_{S0}^{v_{k+1}}$ are among $v_1, \ldots, v_k$, and $\mathfrak{M} \models \varphi_{S0}^{v_{k+1}}[\hat{a}]$ exactly when $\mathfrak{M} \models \varphi[\hat{a}, \hat{1}]$. By lemma 4.2.11(1) and lemma 4.2.5(3), this holds exactly when $\chi_P(\vec{a}) = 1$, that is, when $P(\vec{a})$. So $\varphi_{S0}^{v_{k+1}}$ represents P by lemma 4.2.11(2). Conversely, let ψ represent P, and let
$$\varphi :\equiv (\psi \wedge v_{k+1} = S0) \vee ((\neg\psi) \wedge v_{k+1} = 0)$$
whose free variables are among $v_1, \ldots, v_{k+1}$. Let $e \in |\mathfrak{M}|$. As v_{k+1} is not free in ψ, proposition 1.4.9 evaluates ψ at the assignments of $\mathfrak{M} \models \varphi[\hat{a}, e]$ as $\mathfrak{M} \models \psi[\hat{a}]$. By proposition 1.4.6(1) and (2), the negation and equality clauses of definition 1.4.5, and lemma 4.2.4(1), $\mathfrak{M} \models \varphi[\hat{a}, e]$ exactly when either $\mathfrak{M} \models \psi[\hat{a}]$ and $e = \hat{1}$, or $\mathfrak{M} \not\models \psi[\hat{a}]$ and $e = \hat{0}$. By lemma 4.2.11(2), the first alternative is $P(\vec{a})$ and $e = \hat{1}$, and the second is the failure of $P(\vec{a})$ and $e = \hat{0}$. In both cases $\mathfrak{M} \models \varphi[\hat{a}, e]$ exactly when $e = \widehat{\chi_P(\vec{a})}$, so φ represents χ_P by lemma 4.2.11(1).

3. By the negation clause of definition 1.4.5, $\mathfrak{M} \models (\neg\psi)[\hat{a}]$ exactly when $\mathfrak{M} \not\models \psi[\hat{a}]$, which by lemma 4.2.11(2) holds exactly when $P(\vec{a})$ fails. So $\neg\psi$ represents $\mathbb{N}^k \setminus P$ by lemma 4.2.11(2). Applied to $\mathbb{N}^k \setminus P$, whose complement is P, this gives the last claim.

A recursive function is built from the initial functions, which are representable by example 4.2.7, by composition, primitive recursion and the unbounded minimalization of regular functions (definition 3.2.28). Each of the three constructions preserves representability in $\mathrm{Th}(\mathcal{A})$.

Proposition 4.2.15: Composition Of Representable Is Representable

Let $g_1, g_2, \ldots, g_m$ be k-place functions and h an m-place function, all representable in $\mathrm{Th}(\mathcal{A})$. Then $f = h \circ (g_1, g_2, \ldots, g_m)$ is a k-place function representable in $\mathrm{Th}(\mathcal{A})$.

***Proof*:**
Let $\psi_1, \ldots, \psi_m$ represent $g_1, \ldots, g_m$ and ψ_h represent h in $\mathrm{Th}(\mathcal{A})$, so that the free variables of each ψ_j are among $v_1, \ldots, v_{k+1}$ and those of ψ_h among $v_1, \ldots, v_{m+1}$. Let $n_0 = \max(k, m)$, and choose distinct variables $u_1, \ldots, u_{m+1}$, other than $v_1, \ldots, v_{n_0+1}$, that occur in none of these formulas. Let $\psi'_j :\equiv (\psi_j)^{v_{k+1}}_{u_j}$, let ψ'_h arise from ψ_h by substituting $u_1, \ldots, u_{m+1}$ for $v_1, \ldots, v_{m+1}$ one at a time, and let

$$\varphi :\equiv \exists u_1 \cdots \exists u_{m+1} \big(\psi'_1 \wedge \cdots \wedge \psi'_m \wedge \psi'_h \wedge u_{m+1} = v_{k+1}\big)$$

At each substitution the variable substituted does not occur in the formula reached so far, so lemma 4.2.12 applies at each stage. By its item (2), the free variables of ψ'_j are among $v_1, \ldots, v_k, u_j$, and those of ψ'_h among $u_1, \ldots, u_{m+1}$, so the free variables of φ are among $v_1, \ldots, v_{k+1}$. By its item (3), at an assignment s the formula ψ'_j holds exactly when ψ_j holds at $s(v_{k+1}|s(u_j))$, and ψ'_h holds exactly when ψ_h holds at the assignment that agrees with s except that it sends each v_i, $i \leq m+1$, to $s(u_i)$.

Let $\mathfrak{M}$ be a model of $\mathcal{A}$, let $a_1, \ldots, a_k \in \mathbb{N}$, write $\hat{a}$ for $\hat{a}_1, \ldots, \hat{a}_k$, and let $c_j = g_j(a_1, \ldots, a_k)$ and $b = f(a_1, \ldots, a_k) = h(c_1, \ldots, c_m)$. By proposition 1.4.13, proposition 1.4.6(2), the equality clause and proposition 1.4.9, $\mathfrak{M} \models \varphi[\hat{a}, e]$ exactly when there are $e_1, \ldots, e_{m+1} \in |\mathfrak{M}|$ with $\mathfrak{M} \models \psi_j[\hat{a}, e_j]$ for every $j \leq m$, $\mathfrak{M} \models \psi_h[e_1, \ldots, e_{m+1}]$ and $e_{m+1} = e$. By lemma 4.2.11(1) for ψ_j, $\mathfrak{M} \models \psi_j[\hat{a}, e_j]$ exactly when $e_j = \hat{c}_j$. With $e_j = \hat{c}_j$ for $j \leq m$, lemma 4.2.11(1) for ψ_h gives $\mathfrak{M} \models \psi_h[\hat{c}_1, \ldots, \hat{c}_m, e_{m+1}]$ exactly when $e_{m+1} = \hat{b}$. Hence $\mathfrak{M} \models \varphi[\hat{a}, e]$ exactly when $e = \hat{b}$, and φ represents f in $\mathrm{Th}(\mathcal{A})$ by lemma 4.2.11(1).

Proposition 4.2.16: Representable Functions And Unbounded Minimalization

Let g be a $(k+1)$-place regular function which is representable in $\mathrm{Th}(\mathcal{A})$. Then the unbounded minimalization of g is a k-place function representable in $\mathrm{Th}(\mathcal{A})$.

***Proof*:**
Let φ_g represent g in $\mathrm{Th}(\mathcal{A})$, and let R be the $(k+1)$-place relation with $R(a_1, \ldots, a_k, y)$ exactly when $g(a_1, \ldots, a_k, y) = 0$. Let $\psi :\equiv (\varphi_g)^{v_{k+2}}_0$, whose free variables are among $v_1, \ldots, v_{k+1}$ by lemma 4.2.4(3). Let $\mathfrak{M}$ be a model of $\mathcal{A}$, $a_1, \ldots, a_k, y \in \mathbb{N}$, and $\hat{a}$ the list $\hat{a}_1, \ldots, \hat{a}_k$. The term 0 is the numeral $S^0 0$, so by lemma 4.2.4(3), $\mathfrak{M} \models \psi[\hat{a}, \hat{y}]$ exactly when $\mathfrak{M} \models \varphi_g[\hat{a}, \hat{y}, \hat{0}]$. By lemma 4.2.11(1) this holds exactly when $\hat{0} = g(a_1, \ldots, a_k, y)$, that is, by lemma 4.2.5(3), when $R(a_1, \ldots, a_k, y)$. So ψ represents R by lemma 4.2.11(2). As g is regular, every $(a_1, \ldots, a_k)$ has some y with $R(a_1, \ldots, a_k, y)$ (definition 3.2.24), and the least such y is $\mu y[g(a_1, \ldots, a_k, y) = 0]$ (definition 3.2.23). By lemma 4.2.13, the formula $\psi_{\min}$ represents the unbounded minimalization of g in $\mathrm{Th}(\mathcal{A})$.

Primitive recursion defines $f(\vec{a}, x+1)$ from the earlier value $f(\vec{a}, x)$, and a formula representing f needs the whole list $f(\vec{a}, 0), \ldots, f(\vec{a}, x)$. Gödel coded such a list by two numbers, through remainders:

Definition 4.2.17: Gödel's β-Function

For $c, d, i \in \mathbb{N}$, let $\beta(c, d, i)$ be the remainder of c on division by $(i+1)d+1$, that is, the unique r with $0 \leq r < (i+1)d+1$ and $c = l((i+1)d+1) + r$ for some $l \in \mathbb{N}$. The 3-place function β is *Gödel's β-function.*

Lemma 4.2.18: The β-Function Codes Finite Sequences

For all $x \in \mathbb{N}$ and $a_0, \ldots, a_x \in \mathbb{N}$ there are $c, d \in \mathbb{N}$ with $\beta(c, d, i) = a_i$ for every $i \leq x$.

***Proof*:**
Let $\ell = \max(x, a_0, \ldots, a_x) + 1$, $d = \ell!$ and $m_i = (i+1)d+1$ for $i \leq x$. For $i < j \leq x$ the numbers m_i and m_j have no common prime factor: such a prime p would divide $m_j - m_i = (j-i)d$ but not d, since $p \big| (i+1)d+1$. By unique factorization $p \big| j - i$, so $p \leq x < \ell$, which forces $p \big| \ell! = d$, a contradiction. Let $N_0 = m_0 m_1 \cdots m_x$. If $0 \leq e < e' < N_0$ and e, e' leave the same remainder on division by every m_i, then every m_i divides $e' - e$. No prime divides two of the m_i, so for each prime p the power of p in N_0 is its power in a single m_i, which divides $e' - e$. By unique factorization N_0 divides $e' - e$, which is impossible as $0 < e' - e < N_0$. Thus the map sending e to its tuple of remainders modulo $m_0, \ldots, m_x$ is injective from $\{0, \ldots, N_0 - 1\}$ to the set of tuples $(r_0, \ldots, r_x)$ with $0 \leq r_i < m_i$. Both sets have N_0 elements, so the map is onto. As $a_i < \ell \leq d < m_i$, some c has remainder a_i modulo m_i for every $i \leq x$, that is, $\beta(c, d, i) = a_i$.

Proposition 4.2.19: Primitive Recursive, Then Representable

Let g be a k-place function and h a $(k+2)$-place function, with $k \geq 1$, both representable in $\mathrm{Th}(\mathcal{A})$. Then the $(k+1)$-place function f obtained from g and h by primitive recursion is representable in $\mathrm{Th}(\mathcal{A})$.

***Proof*:**
Write $\vec{a}$ for $(a_1, \ldots, a_k) \in \mathbb{N}^k$ and $\hat{a}$ for $\hat{a}_1, \ldots, \hat{a}_k$, so that $f(\vec{a}, 0) = g(\vec{a})$ and $f(\vec{a}, x+1) = h(\vec{a}, x, f(\vec{a}, x))$. Let φ_g and φ_h represent g and h in $\mathrm{Th}(\mathcal{A})$, with free variables among $v_1, \ldots, v_{k+1}$ and $v_1, \ldots, v_{k+3}$. Choose distinct variables $u_1, \ldots, u_6$, q, w and z, other than $v_1, \ldots, v_{k+3}$, that occur in neither φ_g nor φ_h. Every formula $t \prec t'$ below is $\exists w(w + St = t')$, and w occurs in none of the terms involved. Throughout, $\mathfrak{M}$ is a model of $\mathcal{A}$.

Step 1: the remainder formula. For terms t_1, t_2, t_3, t_4 in which neither q nor w occurs, let

$$B(t_1, t_2, t_3, t_4) :\equiv \exists q\big(t_1 = q \cdot S(St_3 \cdot t_2) + t_4 \wedge t_4 \prec S(St_3 \cdot t_2)\big)$$

Let s be an assignment into $\mathfrak{M}$ with $\overline{s}(t_1) = \hat{c}$, $\overline{s}(t_2) = \hat{d}$ and $\overline{s}(t_3) = \hat{i}$ for some $c, d, i \in \mathbb{N}$. We show that $\mathfrak{M} \models B(t_1, t_2, t_3, t_4)[s]$ exactly when $\overline{s}(t_4) = \hat{r}$ for $r = \beta(c, d, i)$. Let $\mu = (i+1)d+1$. By lemma 4.2.4(1) and lemma 4.2.5(2), the term $S(St_3 \cdot t_2)$ has the value $S(\widehat{i+1} \cdot \hat{d}) = \hat{\mu}$. As q occurs in none of the four terms, proposition 1.4.13, proposition 1.4.6(2), the equality clause of

definition 1.4.5, lemma 1.4.8 and lemma 4.2.5(5) show that $\mathfrak{M} \models B(t_1, t_2, t_3, t_4)[s]$ exactly when some $e \in |\mathfrak{M}|$ has

$$\hat{c} = e \cdot \hat{\mu} + \overline{s}(t_4) \qquad \text{and} \qquad \overline{s}(t_4) \prec \hat{\mu}$$

If $\overline{s}(t_4) = \hat{r}$, write $c = l\mu + r$ as in definition 4.2.17. Then $e = \hat{l}$ satisfies the first condition by lemma 4.2.5(2), and the second holds by lemma 4.2.5(6), as $r < \mu$. Conversely, suppose some e satisfies both. By lemma 4.2.5(6), $\overline{s}(t_4) = \hat{r_0}$ for some $r_0 < \mu$. By lemma 4.2.5(4) and (3), $r_0 \leq c$ and $e \cdot \hat{\mu} = \widehat{c - r_0}$. As $\mu \geq 1$, we have $\hat{\mu} = S(\widehat{\mu - 1})$, and lemma 4.2.5(1) gives $e \cdot \hat{\mu} = e \cdot \widehat{\mu - 1} + e$, so $e = \hat{l}$ for some $l \in \mathbb{N}$ by lemma 4.2.5(4). Then $\widehat{l\mu} = \widehat{c - r_0}$ by lemma 4.2.5(2), and $l\mu = c - r_0$ by lemma 4.2.5(3). So $c = l\mu + r_0$ with $0 \leq r_0 < \mu$, and $r_0 = \beta(c, d, i)$ by definition 4.2.17.

Step 2: the recursion formula. For $\vec{a} \in \mathbb{N}^k$ and $x, c, d \in \mathbb{N}$, let $\mathit{Good}(\vec{a}, x, c, d)$ hold when $\beta(c, d, 0) = g(\vec{a})$ and $\beta(c, d, i+1) = h(\vec{a}, i, \beta(c, d, i))$ for every $i < x$. Let $\varphi'_g :\equiv (\varphi_g)_{u_4}^{v_{k+1}}$, let φ'_h arise from φ_h by substituting u_3, u_4, u_5 for $v_{k+1}, v_{k+2}, v_{k+3}$ one at a time, and let

$$\begin{aligned} G :\equiv\ & \exists u_4 \big(B(u_1, u_2, 0, u_4) \wedge \varphi'_g \big) \\ & \wedge \forall u_3 \big(u_3 \prec v_{k+1} \to \exists u_4\, \exists u_5\, (B(u_1, u_2, u_3, u_4) \wedge B(u_1, u_2, Su_3, u_5) \wedge \varphi'_h) \big) \end{aligned}$$

At each substitution the variable substituted does not occur in the formula reached so far, so by lemma 4.2.12(2) the free variables of G are among $v_1, \ldots, v_{k+1}, u_1, u_2$. Let s be an assignment into $\mathfrak{M}$ sending $v_1, \ldots, v_k$ to $\hat{a}_1, \ldots, \hat{a}_k$, v_{k+1} to $\hat{x}$, u_1 to $\hat{c}$ and u_2 to $\hat{d}$. We show that $\mathfrak{M} \models G[s]$ exactly when $\mathit{Good}(\vec{a}, x, c, d)$.

By proposition 1.4.6(2) and proposition 1.4.13, the first conjunct holds at s exactly when some $e \in |\mathfrak{M}|$ satisfies $B(u_1, u_2, 0, u_4)$ and φ'_g at $s(u_4|e)$. By Step 1, with $\overline{s}(0) = \hat{0}$, the first says $e = \widehat{\beta(c, d, 0)}$. By lemma 4.2.12(3) and proposition 1.4.9, the second says $\mathfrak{M} \models \varphi_g[\hat{a}, e]$, that is, $e = \widehat{g(\vec{a})}$ by lemma 4.2.11(1). So the first conjunct holds exactly when $\beta(c, d, 0) = g(\vec{a})$, by lemma 4.2.5(3).

By the quantifier and implication clauses of definition 1.4.5 and lemma 4.2.5(5), the second conjunct holds at s exactly when every $e_3 \in |\mathfrak{M}|$ with $e_3 \prec \hat{x}$ has elements $e_4, e_5 \in |\mathfrak{M}|$ satisfying the three formulas in the parenthesis at $s(u_3|e_3)(u_4|e_4)(u_5|e_5)$. By lemma 4.2.5(6), these e_3 are the elements $\hat{i}$ with $i < x$. Fix such an i. The term Su_3 has the value $S(\hat{i}) = \widehat{i+1}$, so by Step 1 the two instances of B say $e_4 = \widehat{\beta(c, d, i)}$ and $e_5 = \beta(c, d, \widehat{i+1})$. By lemma 4.2.12(3), proposition 1.4.9 and lemma 4.2.11(1), applied at the inputs $\hat{a}_1, \ldots, \hat{a}_k, \hat{i}, e_4$, which are named by numerals, the formula φ'_h then says $e_5 = h(\vec{a}, \hat{i}, \widehat{\beta(c, d, i)})$. So the second conjunct holds exactly when $\beta(c, d, i+1) = h(\vec{a}, i, \beta(c, d, i))$ for every $i < x$, by lemma 4.2.5(3).

Step 3: the least bound. For $\vec{a}$ and $x, y \in \mathbb{N}$, let $R'(\vec{a}, x, y)$ hold when $\mathit{Good}(\vec{a}, x, c, d)$ for some $c, d \leq y$, and let

$$\rho :\equiv \exists u_1 \big(u_1 \prec Sv_{k+2} \wedge \exists u_2 (u_2 \prec Sv_{k+2} \wedge G) \big)$$

whose free variables are among $v_1, \ldots, v_{k+2}$. Consider an assignment sending each v_i with $i \leq k$ to $\hat{a}_i$, and v_{k+1}, v_{k+2} to $\hat{x}$, $\hat{y}$. There the term Sv_{k+2} has the value $\widehat{y+1}$, so by lemma 4.2.5(5) and (6) the two quantifiers range over the elements $\hat{c}$ and $\hat{d}$ with $c, d \leq y$. As v_{k+2} is not free in G, Step 2 gives $\mathfrak{M} \models \rho[\hat{a}, \hat{x}, \hat{y}]$ exactly when $R'(\vec{a}, x, y)$. So ρ represents R' in $\mathrm{Th}(\mathcal{A})$ by

lemma 4.2.11(2). For all $\vec{a}$ and x, lemma 4.2.18 gives c, d with $\beta(c, d, i) = f(\vec{a}, i)$ for every $i \leq x$. Then $\mathit{Good}(\vec{a}, x, c, d)$ holds by the recursion equations of f, so $R'(\vec{a}, x, c + d)$. By lemma 4.2.13, with $k + 1$ in place of k and the variable z, the formula $\rho_{\min}$ represents in $\mathrm{Th}(\mathcal{A})$ the function sending $(\vec{a}, x)$ to the least y with $R'(\vec{a}, x, y)$. Write $y^*(\vec{a}, x)$ for this least y.

Step 4: the representing formula. If $\mathit{Good}(\vec{a}, x, c, d)$, then $\beta(c, d, i) = f(\vec{a}, i)$ for every $i \leq x$, by induction on i: $\beta(c, d, 0) = g(\vec{a}) = f(\vec{a}, 0)$, and for $i < x$, $\beta(c, d, i{+}1) = h(\vec{a}, i, f(\vec{a}, i)) = f(\vec{a}, i{+}1)$. Let

$$\varphi :\equiv \exists u_6 \Big((\rho_{\min})_{u_6}^{v_{k+2}} \wedge \exists u_1 \big(u_1 \prec S u_6 \wedge \exists u_2 (u_2 \prec S u_6 \wedge G \wedge B(u_1, u_2, v_{k+1}, v_{k+2})) \big) \Big)$$

The variable u_6 does not occur in $\rho_{\min}$, so by lemma 4.2.12(2) the free variables of φ are among $v_1, \ldots, v_{k+2}$. Fix $\vec{a}$ and x, let $y^* = y^*(\vec{a}, x)$, and let $e \in |\mathfrak{M}|$. By proposition 1.4.13, proposition 1.4.6(2), lemma 4.2.12(3), proposition 1.4.9 and lemma 4.2.5(5), $\mathfrak{M} \models \varphi[\hat{a}, \hat{x}, e]$ exactly when some $y \in |\mathfrak{M}|$ has $\mathfrak{M} \models \rho_{\min}[\hat{a}, \hat{x}, y]$, and some $c', d' \in |\mathfrak{M}|$ with $c' \prec S(y)$ and $d' \prec S(y)$ satisfy G and $B(u_1, u_2, v_{k+1}, v_{k+2})$ at an assignment sending $v_1, \ldots, v_{k+2}$ to $\hat{a}_1, \ldots, \hat{a}_k, \hat{x}, e$ and u_1, u_2 to c', d'. By Step 3 and lemma 4.2.11(1), $y = \widehat{y^*}$, so $S(y) = \widehat{y^* + 1}$, and by lemma 4.2.5(6) the elements c', d' are $\hat{c}, \hat{d}$ with $c, d \leq y^*$. By Steps 2 and 1 the condition then says that $\mathit{Good}(\vec{a}, x, c, d)$ and $e = \widehat{\beta(c, d, x)}$ for some $c, d \leq y^*$. Such c, d exist, as $R'(\vec{a}, x, y^*)$ holds, and every such pair has $\beta(c, d, x) = f(\vec{a}, x)$. Hence $\mathfrak{M} \models \varphi[\hat{a}, \hat{x}, e]$ exactly when $e = \widehat{f(\vec{a}, x)}$, and φ represents f in $\mathrm{Th}(\mathcal{A})$ by lemma 4.2.11(1).

The initial functions and the three constructions give every recursive function:

Lemma 4.2.20: Recursive Functions Have Faithful Formulas

Every recursive function has a faithful formula.

***Proof*:**
A recursive function f is recursive in the first sense of definition 3.2.28, so it is the last term of a finite sequence of functions, each of which is an initial function (definition 3.2.8) or arises from earlier ones by composition, by primitive recursion, or by the unbounded minimalization of a regular function. We show by strong induction along this sequence that each of its functions is representable in $\mathrm{Th}(\mathcal{A})$. The initial functions are representable by example 4.2.7(2), (3) and (4). A composition of representable functions is representable by proposition 4.2.15, a function obtained by primitive recursion from representable functions is representable by proposition 4.2.19, and the unbounded minimalization of a representable regular function is representable by proposition 4.2.16. So some formula represents f in $\mathrm{Th}(\mathcal{A})$, and it is faithful for f by lemma 4.2.11(3).

Theorem 4.2.21: Recursive, Then Representable

Let f be a recursive function. Then every faithful formula for f (definition 4.2.10) represents f in $\mathrm{Th}(\mathcal{A})$. In particular f is representable in $\mathrm{Th}(\mathcal{A})$

***Proof*:**
By lemma 4.2.11(3), every faithful formula for f represents f in $\mathrm{Th}(\mathcal{A})$, and by lemma 4.2.20 there is such a formula.

In particular, each recursive relation on codes from section 4.1, such as *Sentence* and, for a recursive set Δ of sentences, $Conclusion_\Delta$, is represented in $\mathrm{Th}(\mathcal{A})$ by a formula of $\mathcal{L}_{NT}$, by theorem 4.2.21 applied to its characteristic function and proposition 4.2.14(2).

An inconsistent set of sentences represents every function and every relation, so representability carries information only for consistent sets. It passes from $\mathrm{Th}(\Gamma)$ to $\mathrm{Th}(\Sigma)$ whenever $\Sigma \vdash \Gamma$:

Proposition 4.2.22: Representability And Consistency

Let Σ and Γ be sets of sentences of $\mathcal{L}_{NT}$.

1. If Σ is inconsistent, then every function and every relation is representable in $\mathrm{Th}(\Sigma)$. If Σ is consistent, then a formula ψ representing a k-place relation P in $\mathrm{Th}(\Sigma)$ satisfies $P(n_1, \ldots, n_k)$ exactly when $\Sigma \vdash \psi(S^{n_1}0, \ldots, S^{n_k}0)$. If Σ is consistent and $\Sigma \vdash \mathcal{A}$, then a formula φ representing a k-place function f in $\mathrm{Th}(\Sigma)$ satisfies

$$f(n_1, \ldots, n_k) = m \quad \Leftrightarrow \quad \Sigma \vdash \varphi(S^{n_1}0, \ldots, S^{n_k}0, S^m 0)$$

   for all $n_1, \ldots, n_k, m \in \mathbb{N}$.

2. If $\Sigma \vdash \Gamma$, then every function and every relation representable in $\mathrm{Th}(\Gamma)$ is representable in $\mathrm{Th}(\Sigma)$, by the same formula.

3. If $\Sigma \vdash \mathcal{A}$, then every function and every relation representable in $\mathrm{Th}(\mathcal{A})$ is representable in $\mathrm{Th}(\Sigma)$.

***Proof*:**
Write $\vec{n}$ for $(n_1, \ldots, n_k)$ and $\hat{n}$ for $\hat{n}_1, \ldots, \hat{n}_k$.

1. If Σ is inconsistent, then Σ deduces every formula by proposition 2.6.17(3), in particular every sentence that definition 4.2.6 or definition 4.2.8 requires of a formula with the right free variables, such as $v_{k+1} = v_{k+1}$ or $v_1 = v_1$. Let Σ be consistent. Then Σ has a model $\mathfrak{M}$ by theorem 2.6.30. If $P(\vec{n})$, then $\Sigma \vdash \psi(S^{n_1}0, \ldots, S^{n_k}0)$ by definition 4.2.8. If $P(\vec{n})$ fails, then $\Sigma \vdash \neg\psi(S^{n_1}0, \ldots, S^{n_k}0)$, so $\mathfrak{M} \models \neg\psi(S^{n_1}0, \ldots, S^{n_k}0)$ by proposition 2.6.17(1), and $\mathfrak{M} \not\models \psi(S^{n_1}0, \ldots, S^{n_k}0)$ by proposition 1.4.14(1). Then $\Sigma \nvdash \psi(S^{n_1}0, \ldots, S^{n_k}0)$, by proposition 2.6.17(1) again.

   Let also $\Sigma \vdash \mathcal{A}$, and let φ represent f in $\mathrm{Th}(\Sigma)$. If $f(\vec{n}) = m$, then by lemma 4.2.11(1) every model $\mathfrak{M}'$ of Σ has $\mathfrak{M}' \models \varphi[\hat{n}, \hat{m}]$, that is $\mathfrak{M}' \models \varphi(S^{n_1}0, \ldots, S^{n_k}0, S^m 0)$ by lemma 4.2.4(4). So $\Sigma \models \varphi(S^{n_1}0, \ldots, S^{n_k}0, S^m 0)$ (definition 1.6.1), and Σ deduces this sentence by theorem 2.6.31. Conversely, if $\Sigma \vdash \varphi(S^{n_1}0, \ldots, S^{n_k}0, S^m 0)$, then $\mathfrak{M} \models \varphi[\hat{n}, \hat{m}]$ by proposition 2.6.17(1) and lemma 4.2.4(4), so $\hat{m} = \widehat{f(\vec{n})}$ by lemma 4.2.11(1). As $\Sigma \vdash \mathcal{A}$, the model $\mathfrak{M}$ of Σ is a model of $\mathcal{A}$ by proposition 2.6.17(1), and $f(\vec{n}) = m$ by lemma 4.2.5(3).

2. Each sentence that definition 4.2.6 or definition 4.2.8 requires Γ to deduce is deduced by Σ as well, by proposition 2.6.10(4) with $\Sigma \vdash \Gamma$.

3. This is (2) with $\Gamma = \mathcal{A}$.

4.3 Incompleteness Theorems

By section 4.1 and section 4.2, recursive relations on codes of formulas, such as being the code of a deduction from a recursive set of sentences, are represented in $\mathrm{Th}(\mathcal{A})$ by formulas of $\mathcal{L}_{NT}$. The Fixed-Point lemma, lemma 4.3.1(3), then gives sentences that $\mathcal{A}$ proves equivalent to statements about their own codes.

Lemma 4.3.1: Incompleteness Theorem Lemmas I

1. The function
$$Sub(n,k) = \begin{cases} [\varphi(S^k 0)] & \text{if } n = [\varphi] \text{ for a formula } \varphi \text{ of } \mathcal{L}_{NT} \\ & \text{with no free variable other than } v_1 \\ 0 & \text{otherwise} \end{cases}$$
is primitive recursive, and hence representable in $\mathrm{Th}(\mathcal{A})$
2. The set $[\mathcal{A}]$ is primitive recursive, so $\mathcal{A}$ is a recursive set of sentences of $\mathcal{L}_{NT}$
3. (Fixed-Point lemma) Let φ be a formula of $\mathcal{L}_{NT}$ with no free variable other than v_1. Then there is a sentence σ of $\mathcal{L}_{NT}$ such that
$$\mathcal{A} \vdash \sigma \leftrightarrow \varphi(S^{[\sigma]}0)$$

***Proof*:**

1. Let $N(k) = [S^k 0]$ for $k \in \mathbb{N}$. By definition 4.1.3 and definition 4.1.4, $N(0) = [0] = 2^7$, and as $S^{k+1}0$ is S followed by $S^k 0$, $N(k+1) = CONCAT(2^8, N(k))$ by lemma 4.1.6(2). With a variable u carried along unchanged, as in example 3.2.11(5), the 2-place function T with $T(u,0) = 2^7$ and $T(u,k+1) = CONCAT(2^8, T(u,k))$ arises by primitive recursion from a constant function and $CONCAT$ (example 3.2.17(13)), and $N(k) = T(k,k)$. So N is primitive recursive.

   Let $O(n)$ hold when $Formula(n)$ holds and $Free(n,j)$ fails for every $j < n$ other than 1. A variable v_j free in the formula coded by n occurs in it (definition 1.2.2), so $p^{j+13} \big| n$ for some prime p, and $j < n$. Hence $O(n)$ holds exactly when n codes a formula with no free variable other than v_1, and O is primitive recursive by proposition 4.1.8(2), (5) and the closure facts (a) and (b) in its proof. For such an $n = [\varphi]$, the formula $\varphi(S^k 0)$ is $\varphi^{v_1}_{S^k 0}$, whose code is $Subst(n, 1, N(k))$ by proposition 4.1.8(6). Hence
   $$Sub(n,k) = MULT\big(\chi_O(n),\ Subst(n,1,N(k))\big)$$
   a composition of primitive recursive functions, with $MULT$ from example 3.2.11(3). A primitive recursive function is recursive, the build-up of definition 3.2.13 being a special case of that in definition 3.2.28, so Sub is representable in $\mathrm{Th}(\mathcal{A})$ by theorem 4.2.21.

2. By definition 4.2.1, $\mathcal{A}$ consists of exactly nine sentences $\alpha_1, \alpha_2, \ldots, \alpha_9$ of $\mathcal{L}_{NT}$. Each α_i is a finite sequence of symbols of $\mathcal{L}_{NT}$, so it has a Gödel code $[\alpha_i]$ by definition 4.1.4, and the nine codes are distinct: each symbol code in definition 4.1.3 is a positive integer, so by unique factorization the sequence code $p_1^{[s_1]} \cdots p_k^{[s_k]}$ determines its length and every exponent, and the nine axioms are distinct sequences of symbols. Hence

$$[\mathcal{A}] = \{[\alpha_i] \mid 1 \leq i \leq 9\}$$

is a finite subset of $\mathbb{N}$, with characteristic function

$$\chi_{[\mathcal{A}]}(n) = \sum_{i=1}^{9} \chi_{\{[\alpha_i]\}}(n)$$

where the sum takes only the values 0 and 1 because the nine codes are distinct. Each $\chi_{\{[\alpha_i]\}}$ is primitive recursive by lemma 3.2.14(2), and the nine-fold sum is built from these and SUM of example 3.2.11(1) by finitely many compositions, which definition 3.2.13 permits. Hence $\chi_{[\mathcal{A}]}$ is primitive recursive, and it is recursive, the build-up of definition 3.2.13 being a special case of that in definition 3.2.28. So $[\mathcal{A}]$ is a recursive subset of $\mathbb{N}$, and by definition 4.1.9 the set $\mathcal{A}$ is a recursive set of sentences of $\mathcal{L}_{NT}$.

3. By (1), some formula ψ represents Sub in $\mathrm{Th}(\mathcal{A})$. Its free variables are among v_1, v_2, v_3, and by definition 4.2.6, for all $n, k \in \mathbb{N}$,

$$\mathcal{A} \vdash \forall v_3 \big(\psi(S^n 0, S^k 0, v_3) \leftrightarrow v_3 = S^{Sub(n,k)} 0\big)$$

Choose distinct variables w and z, other than v_1, v_2, v_3, that occur in neither φ nor ψ. Let $\varphi' :\equiv \varphi_z^{v_1}$, let ψ' arise from ψ by substituting w for v_2 and then z for v_3, and let

$$\alpha :\equiv \exists z \, \exists w \big(w = v_1 \wedge (\varphi' \wedge \psi')\big)$$

At each substitution the variable substituted does not occur in the formula reached so far, so lemma 4.2.12 applies at each stage. By its item (2), the free variables of φ' are among z and those of ψ' are among v_1, w, z, so α has no free variable other than v_1. By its item (3), at an assignment s the formula φ' holds exactly when φ holds at $s(v_1|s(z))$, and ψ' holds exactly when ψ holds at $s(v_2|s(w))(v_3|s(z))$.

Let $\mathfrak{M}$ be a model of $\mathcal{A}$ and $e \in |\mathfrak{M}|$. By proposition 1.4.13, proposition 1.4.6(2), the equality clause of definition 1.4.5, the previous paragraph and proposition 1.4.9, $\mathfrak{M} \models \alpha[e]$ holds exactly when some $d \in |\mathfrak{M}|$ has $\mathfrak{M} \models \varphi[d]$ and $\mathfrak{M} \models \psi[e, e, d]$, in the notation of lemma 4.2.4(2). Let $k = [\alpha]$, let $\sigma :\equiv \alpha(S^k 0)$, a sentence by lemma 4.2.4(4), and let $m = [\sigma]$, so that $m = Sub(k, k)$. At $e = \hat{k}$, lemma 4.2.11(1) gives $\mathfrak{M} \models \psi[\hat{k}, \hat{k}, d]$ exactly when $d = \hat{m}$. So $\mathfrak{M} \models \alpha[\hat{k}]$ exactly when $\mathfrak{M} \models \varphi[\hat{m}]$. By lemma 4.2.4(4), the first says that $\mathfrak{M} \models \sigma$ and the second that $\mathfrak{M} \models \varphi(S^m 0)$, and both are sentences, so $\mathfrak{M} \models \sigma \leftrightarrow \varphi(S^m 0)$ by proposition 1.4.14(5). This holds in every model of $\mathcal{A}$, so $\mathcal{A} \vdash \sigma \leftrightarrow \varphi(S^{[\sigma]} 0)$ by definition 1.6.1 and theorem 2.6.31.

If v_1 is free in φ, the sentence σ of the Fixed-Point lemma is a different sentence from $\varphi(S^{[\sigma]} 0)$. For every $k \in \mathbb{N}$, the numeral $S^k 0$ is then a segment of $\varphi(S^k 0)$, by the induction on formulas in Step 6 of the proof of proposition 4.1.8, and its code $p_1^8 \cdots p_k^8 p_{k+1}^7$, a product of $k+1$ factors that are each at

least 2, exceeds k. So $[\varphi(S^k 0)] > k$ by lemma 4.1.6(3), and no k has $[\varphi(S^k 0)] = k$. The lemma is also called the *diagonal lemma*: its proof substitutes into the formula α the numeral of the code of α, that is, it evaluates Sub at the diagonal pair (k, k), as Cantor's diagonal argument evaluates the kth sequence at the place k.

Applied to the negation of a formula that represents $[\mathrm{Th}(\Sigma)]$, the Fixed-Point lemma gives a sentence that $\mathcal{A}$ proves equivalent to the statement that it is not a theorem of Σ (lemma 4.3.2). This is the liar sentence, "this sentence is false", with deducibility in place of truth. Applied to the negation of a formula that defines truth in $\mathfrak{N}$, the lemma gives the liar sentence itself (theorem 4.3.13). Boolos gave a proof of the first incompleteness theorem that formalizes Berry's paradox, about the least number not nameable in fewer than a given number of symbols [2, §6.5].

Lemma 4.3.2: Theorems Are Not Recursive

Let Σ be a consistent set of sentences of $\mathcal{L}_{NT}$ such that $\Sigma \vdash \mathcal{A}$. Then the set $[\mathrm{Th}(\Sigma)]$ of codes of the sentences that Σ deduces is not recursive.

***Proof*:**
Here $\mathrm{Th}(\Sigma)$ is the set of sentences τ of $\mathcal{L}_{NT}$ with $\Sigma \vdash \tau$, as in proposition 4.1.2. Suppose, for a contradiction, that the 1-place relation $[\mathrm{Th}(\Sigma)]$ is recursive (definition 3.2.29). By theorem 4.2.21, applied to its characteristic function, and proposition 4.2.14(2), some formula ψ represents it in $\mathrm{Th}(\mathcal{A})$, and the same formula represents it in $\mathrm{Th}(\Sigma)$ by proposition 4.2.22(2) with $\Gamma = \mathcal{A}$. By definition 4.2.8, no variable other than v_1 is free in ψ, nor in $\neg\psi$. The Fixed-Point lemma, lemma 4.3.1(3), applied to $\neg\psi$, gives a sentence σ with $\mathcal{A} \vdash \sigma \leftrightarrow \neg\theta$, where $\theta :\equiv \psi(S^{[\sigma]}0)$, as $(\neg\psi)(S^{[\sigma]}0)$ is $(\neg\theta)$ by definition 1.5.3. By proposition 2.6.10(4), Σ deduces this biconditional. Every truth assignment that gives $A_0 \leftrightarrow (\neg A_1)$ the value T gives $A_0 \to (\neg A_1)$ and $(\neg A_1) \to A_0$ the value T, by proposition 1.7.4, so lemma 2.6.8 and proposition 2.6.10(4) give

$$\Sigma \vdash \sigma \to \neg\theta \qquad \text{and} \qquad \Sigma \vdash \neg\theta \to \sigma$$

A code determines the sequence it codes (the remark after definition 4.1.4), so $[\sigma] \in [\mathrm{Th}(\Sigma)]$ exactly when $\Sigma \vdash \sigma$.

Suppose $\Sigma \vdash \sigma$. Then $\Sigma \vdash \theta$ by the first clause of definition 4.2.8, and $\Sigma \vdash \neg\theta$ by the first display and proposition 2.6.10(2). No truth assignment gives A_0 and $\neg A_0$ the value T, so $\{A_0, \neg A_0\} \models \neg(A_0 \to A_0)$, and lemma 2.6.8 with proposition 2.6.10(4) gives $\Sigma \vdash \neg(\theta \to \theta)$. So Σ is inconsistent, contrary to hypothesis. Suppose $\Sigma \nvdash \sigma$. Then $[\sigma] \notin [\mathrm{Th}(\Sigma)]$, so $\Sigma \vdash \neg\theta$ by the second clause of definition 4.2.8, and $\Sigma \vdash \sigma$ by the second display and proposition 2.6.10(2), a contradiction. Hence $[\mathrm{Th}(\Sigma)]$ is not recursive.

Theorem 4.3.3: Gödel's First Incompleteness Theorem

Let Σ be a consistent recursive set of sentences of $\mathcal{L}_{NT}$ such that $\Sigma \vdash \mathcal{A}$. Then Σ is *not* complete

***Proof*:**
If Σ were complete, then $[\mathrm{Th}(\Sigma)]$ would be recursive by theorem 4.1.11(2), as Σ is recursive. This contradicts lemma 4.3.2, as Σ is consistent and $\Sigma \vdash \mathcal{A}$.

So for a consistent recursive Σ with $\Sigma \vdash \mathcal{A}$, a recursive function lists the codes of the theorems of Σ (theorem 4.1.11(1)), no recursive function decides them (lemma 4.3.2), and some sentence is neither deduced nor refuted by Σ. The proof does not exhibit such a sentence, as it argues by contradiction from lemma 4.3.2.

Corollary 4.3.4: Consequences Of First Incompleteness Theorem

1. Let Γ be a complete set of sentences of $\mathcal{L}_{NT}$ such that $\Gamma \cup \mathcal{A}$ is consistent. Then Γ is *not* recursive
2. Let Δ be a recursive set of sentences of $\mathcal{L}_{NT}$ such that $\Delta \cup \mathcal{A}$ is consistent. Then Δ is *not* complete
3. The theory of $\mathfrak{N}$

$$\mathrm{Th}(\mathfrak{N}) = \{\sigma \mid \sigma \text{ is a sentence of } \mathcal{L}_{NT} \text{ and } \mathfrak{N} \models \sigma \}$$

is *not* recursive

***Proof*:**
For a set Γ of sentences, $\Gamma \cup \mathcal{A} \vdash \mathcal{A}$ through one-line deductions, and every sentence that Γ deduces is deduced by $\Gamma \cup \mathcal{A}$, by proposition 2.6.10(3). If Γ is recursive, then so is $\Gamma \cup \mathcal{A}$: the characteristic function of $[\Gamma \cup \mathcal{A}] = [\Gamma] \cup [\mathcal{A}]$ is $\chi_{\mathbb{N}^*} \circ (\chi_{[\Gamma]} + \chi_{[\mathcal{A}]})$, where $\chi_{\mathbb{N}^*} = \chi_{\{0\}} \circ \chi_{\{0\}}$ with $\chi_{\{0\}}$ from lemma 3.2.14(2), and it is recursive by lemma 4.3.1(2), example 3.2.11(1) and definition 3.2.28.

1. Suppose Γ is recursive. Then $\Gamma \cup \mathcal{A}$ is a consistent recursive set of sentences with $\Gamma \cup \mathcal{A} \vdash \mathcal{A}$, and it is complete, as Γ is. This contradicts theorem 4.3.3.
2. By theorem 4.3.3, applied to the consistent recursive set $\Delta \cup \mathcal{A}$, some sentence τ has $\Delta \cup \mathcal{A} \nvdash \tau$ and $\Delta \cup \mathcal{A} \nvdash \neg\tau$. Then $\Delta \nvdash \tau$ and $\Delta \nvdash \neg\tau$, so Δ is not complete.
3. The set $\mathrm{Th}(\mathfrak{N})$ is consistent by proposition 2.6.17(2), as $\mathfrak{N} \models \mathrm{Th}(\mathfrak{N})$, and $\mathrm{Th}(\mathfrak{N}) \vdash \mathcal{A}$, as $\mathcal{A} \subseteq \mathrm{Th}(\mathfrak{N})$ by proposition 4.2.2. A sentence that $\mathrm{Th}(\mathfrak{N})$ deduces is true in $\mathfrak{N}$ by proposition 2.6.17(1), so the sentences deduced by $\mathrm{Th}(\mathfrak{N})$ are the members of $\mathrm{Th}(\mathfrak{N})$. By lemma 4.3.2, the set of their codes, $[\mathrm{Th}(\mathfrak{N})]$, is not recursive, that is, $\mathrm{Th}(\mathfrak{N})$ is not recursive (definition 4.1.9).

Bilaniuk notes that the proof of the first incompleteness theorem can be carried out for other first-order languages and recursive sets of axioms that code and deduce enough facts about arithmetic, the axioms of Zermelo-Fraenkel set theory among them [1, Ch. 18].

A recursive set Σ of sentences is consistent exactly when no number codes a deduction from Σ of $\neg(0 = 0 \to 0 = 0)$, and this condition is expressed by a sentence of $\mathcal{L}_{NT}$:

Lemma 4.3.5: Expressing Consistency

Let Σ be a recursive set of sentences of $\mathcal{L}_{NT}$, let $C(m,c)$ be the relation that c is the code of a deduction from Σ whose conclusion has code m, and let $m_0 = [\neg(0=0 \to 0=0)]$. Then χ_C is recursive, so it has a faithful formula (lemma 4.2.20), and for every faithful formula φ_C for χ_C the sentence

$$\mathrm{Con}(\Sigma) :\equiv \forall v_2 \neg\big(((\varphi_C)^{v_3}_{S0})^{v_1}_{S^{m_0}0}\big)$$

of $\mathcal{L}_{NT}$ has the property that Σ is consistent if and only if $\mathfrak{N} \models \mathrm{Con}(\Sigma)$

***Proof*:**
Let $\theta :\equiv (0=0)$, so that $m_0 = [\neg(\theta \to \theta)]$. The set Σ is inconsistent exactly when $\Sigma \vdash \neg(\theta \to \theta)$: an inconsistent set deduces every formula by proposition 2.6.17(3), and a deduction of $\neg(\theta \to \theta)$ witnesses inconsistency. As Σ is recursive, $Conclusion_\Sigma$ is recursive by proposition 4.1.10(5). The relation $C(m,c)$ holds exactly when $Conclusion_\Sigma(c,m)$ does, so χ_C is the composition of $\chi_{Conclusion_\Sigma}$ with the projections π^2_2 and π^2_1, and it is recursive by definition 3.2.28 and definition 3.2.29. By lemma 4.2.20 it has a faithful formula. Let φ_C be one, with free variables among v_1, v_2, v_3. Substituting the numerals $S0$ and $S^{m_0}0$ for v_3 and v_1 leaves at most v_2 free, so $\mathrm{Con}(\Sigma)$ is a sentence. Numerals are substitutable for every variable by lemma 2.6.27(1), so by the quantifier and negation clauses of definition 1.4.5, lemma 2.4.1 and lemma 4.2.3, $\mathfrak{N} \models \mathrm{Con}(\Sigma)$ holds exactly when $\mathfrak{N} \not\models \varphi_C[m_0, c, 1]$ for every $c \in \mathbb{N}$, that is, by (F1) of definition 4.2.10, exactly when $\chi_C(m_0, c) \neq 1$ for every c. This says that no number codes a deduction from Σ whose last formula has the code m_0, that is, $\Sigma \nvdash \neg(\theta \to \theta)$, as distinct formulas have distinct codes. Hence $\mathfrak{N} \models \mathrm{Con}(\Sigma)$ exactly when Σ is consistent, as claimed.

The axioms of $\mathcal{A}$ do not deduce every instance of induction in the sense below: the model $\mathfrak{M}$ described after proposition 4.2.2 satisfies $\mathcal{A}$ and falsifies the instance for $\varphi :\equiv \neg(x = Sx)$.

Definition 4.3.6: Peano Arithmetic

Let φ be a formula of $\mathcal{L}_{NT}$ and x a variable such that no variable other than x is free in φ. The formula

$$\big(\varphi^x_0 \wedge \forall x(\varphi \to \varphi^x_{Sx})\big) \to \forall x \varphi$$

is an *instance of induction*. The set PA of axioms of *Peano Arithmetic* consists of the nine sentences of $\mathcal{A}$ (definition 4.2.1) and all instances of induction.

Proposition 4.3.7: Peano Arithmetic Is True And Primitive Recursive

Every instance of induction is a sentence, $\mathfrak{N} \models \mathrm{PA}$, the set PA is consistent, and the set [PA] of codes of its members is primitive recursive.

***Proof*:**
Let φ and x be as in definition 4.3.6.

Step 1: the instances are sentences. The term Sx is substitutable for x in φ, and every variable free in φ^x_{Sx} is x or is free in φ. Both claims hold for every formula φ, by induction on φ

(definition 1.1.5). For an atomic formula, φ^x_{Sx} is φ with every occurrence of x replaced by Sx (definition 1.5.1, definition 1.5.3), the atomic case of definition 1.5.5 applies, and the variables free in an atomic formula are those occurring in it (definition 1.2.2). The cases of $\neg$ and $\to$ follow from the induction hypothesis. Let $\varphi :\equiv (\forall y)(\beta)$. If y is x, then x is not free in φ, the first alternative of the quantifier clause of definition 1.5.5 applies, and φ^x_{Sx} is φ. Otherwise y does not occur in Sx, the term Sx is substitutable for x in β by the induction hypothesis, and φ^x_{Sx} is $(\forall y)(\beta^x_{Sx})$, whose free variables are those of β^x_{Sx} other than y, hence x or free in φ. Now let no variable other than x be free in φ. Then no variable is free in φ^x_0, by lemma 4.2.4(3) with $0 = S^0 0$, and none other than x in φ^x_{Sx}, so by definition 1.2.2 no variable is free in $\forall x(\varphi \to \varphi^x_{Sx})$ or in $\forall x\varphi$. As $\wedge$ is an abbreviation built from $\neg$ and $\to$ (definition 1.1.5), the instance of induction is a sentence.

Step 2: truth in $\mathfrak{N}$. Let s be an assignment into $\mathfrak{N}$ with $\mathfrak{N} \models (\varphi^x_0 \wedge \forall x(\varphi \to \varphi^x_{Sx}))[s]$. By proposition 1.4.6(2), $\mathfrak{N} \models \varphi^x_0[s]$ and $\mathfrak{N} \models \forall x(\varphi \to \varphi^x_{Sx})[s]$. By lemma 4.2.4(3) and (1), the first gives $\mathfrak{N} \models \varphi[s(x|0)]$. Let $n \in \mathbb{N}$ with $\mathfrak{N} \models \varphi[s(x|n)]$. By the quantifier and implication clauses of definition 1.4.5, $\mathfrak{N} \models \varphi^x_{Sx}[s(x|n)]$, and by Step 1 and lemma 2.4.1, with the value $n+1$ of Sx at $s(x|n)$ (definition 1.4.4), this says $\mathfrak{N} \models \varphi[s(x|n+1)]$. By induction on n, $\mathfrak{N} \models \varphi[s(x|n)]$ for every $n \in \mathbb{N}$, so $\mathfrak{N} \models (\forall x\varphi)[s]$. By the implication clause, every assignment satisfies the instance of induction, so $\mathfrak{N}$ satisfies it (definition 1.4.11). With proposition 4.2.2, $\mathfrak{N} \models \mathrm{PA}$, and PA is consistent by proposition 2.6.17(2).

Step 3: codes. Let $Conj(a,b) = Neg(Imp(a, Neg(b)))$, a primitive recursive function with $Conj([\alpha],[\beta]) = [(\alpha \wedge \beta)]$ by lemma 4.1.6(2) and the abbreviation $(\alpha \wedge \beta) :\equiv (\neg(\alpha \to (\neg\beta)))$ of definition 1.1.5. The terms 0 and Sv_k have the codes 2^7 and $2^8 3^{k+13}$ (definition 4.1.3, definition 4.1.4). By lemma 4.1.6(2) and proposition 4.1.8(6), the instance of induction for φ with code m and $x = v_k$ has the code

$$Ind(m,k) = Imp\Big(Conj\big(Subst(m,k,2^7),\ All(k, Imp(m, Subst(m,k,2^8 3^{k+13})))\big),\ All(k,m)\Big)$$

a primitive recursive function of m and k. Let $I(n)$ hold when there are $m,k < n$ such that $Formula(m)$ holds, $Free(m,j)$ fails for every $j < m$ other than k, and $n = Ind(m,k)$. If n codes an instance of induction, then the formula φ is a segment of it other than the whole, so $m < n$ by lemma 4.1.6(3), the variable v_k occurs in it, so $p^{k+13} \big| n$ for some prime p and $k < n$, and each variable v_j free in φ occurs in φ, so $j < m$ in the same way. So $I(n)$ holds exactly when n codes an instance of induction, and I is primitive recursive by proposition 4.1.8(2), (5) and the closure facts (a) and (b) in its proof. As [PA] is $[\mathcal{A}]$ together with the numbers n with $I(n)$, $\chi_{[\mathrm{PA}]} = \chi_{\mathbb{N}^*} \circ (\chi_{[\mathcal{A}]} + \chi_I)$, with $\chi_{\mathbb{N}^*} = \chi_{\{0\}} \circ \chi_{\{0\}}$, and it is primitive recursive by lemma 4.3.1(2), lemma 3.2.14(2) and example 3.2.11(1).

Not every faithful formula for χ_C gives a consistency sentence that Σ fails to deduce: an example follows theorem 4.3.10.

Definition 4.3.8: Provability Formula And Consistency Sentence

Let Σ be a set of sentences of $\mathcal{L}_{NT}$ whose set of codes $[\Sigma]$ is primitive recursive, and fix a construction of $\chi_{[\Sigma]}$ from the initial functions by finitely many compositions and primitive recursions (definition 3.2.13). Let C and m_0 be as in lemma 4.3.5. The proofs of proposition 4.1.8, proposition 4.1.10 and lemma 4.3.5, with the remark after proposition 4.1.10, extend this construction to such a construction of χ_C. Along it, the proof of lemma 4.2.20, taking at each choice of variables those of least index that it allows, builds a formula φ_C^Σ, which is faithful for χ_C by lemma 4.2.11(3). The *provability formula* of Σ is

$$\mathrm{Prov}_\Sigma :\equiv \exists v_2\, (\varphi_C^\Sigma)_{S0}^{v_3}$$

and the *consistency sentence* $\mathrm{Con}(\Sigma)$ of Σ is the sentence of lemma 4.3.5 for $\varphi_C = \varphi_C^\Sigma$. For a sentence α, write $\mathrm{Pr}_\Sigma(\alpha)$ for $\mathrm{Prov}_\Sigma(S^{[\alpha]}0)$.

By lemma 4.2.4(3) and (4), no variable other than v_1 is free in Prov_Σ, and $\mathrm{Pr}_\Sigma(\alpha)$ is a sentence. As $\exists v_2$ abbreviates $\neg\forall v_2\neg$ and v_1 is not v_2, definition 1.5.3 shows that $\mathrm{Pr}_\Sigma(\neg(0=0\to 0=0))$, which is $\mathrm{Prov}_\Sigma(S^{m_0}0)$, is the formula $(\neg\,\mathrm{Con}(\Sigma))$.

Theorem 4.3.9: Derivability Conditions

Let Σ be as in definition 4.3.8, and let α and β be sentences of $\mathcal{L}_{NT}$.

(D1) If $\Sigma\vdash\mathcal{A}$ and $\Sigma\vdash\alpha$, then $\Sigma\vdash\mathrm{Pr}_\Sigma(\alpha)$.

(D2) If $\Sigma\vdash\mathrm{PA}$, then $\Sigma\vdash\mathrm{Pr}_\Sigma(\alpha)\to\mathrm{Pr}_\Sigma(\mathrm{Pr}_\Sigma(\alpha))$.

(D3) If $\Sigma\vdash\mathrm{PA}$, then $\Sigma\vdash\mathrm{Pr}_\Sigma((\alpha\to\beta))\to(\mathrm{Pr}_\Sigma(\alpha)\to\mathrm{Pr}_\Sigma(\beta))$.

Conditions (D2) and (D3) are deferred: their proofs carry out inside Σ the computations of section 4.1 and section 4.2 for all elements of the models of Σ, including the elements not named by numerals, which takes many pages and lies outside the scope of this book.

***Proof Key ideas*:**
Condition (D1). Let c be the code of a deduction of α from Σ, so that $C([\alpha],c)$ holds. The formula φ_C^Σ represents χ_C in $\mathrm{Th}(\mathcal{A})$ by lemma 4.2.11(3), so $\chi :\equiv (\varphi_C^\Sigma)_{S0}^{v_3}$ represents C in $\mathrm{Th}(\mathcal{A})$ by proposition 4.2.14(2). Let $\mathfrak{M}$ be a model of $\mathcal{A}$. By lemma 4.2.11(2), $\mathfrak{M}\models\chi[\widehat{[\alpha]},\hat{c}]$, so $\mathfrak{M}\models(\exists v_2\,\chi)[\widehat{[\alpha]}]$ by proposition 1.4.13, and $\mathfrak{M}\models\mathrm{Pr}_\Sigma(\alpha)$ by lemma 4.2.4(4). Hence $\mathcal{A}\vdash\mathrm{Pr}_\Sigma(\alpha)$ by theorem 2.6.31, and $\Sigma\vdash\mathrm{Pr}_\Sigma(\alpha)$ by proposition 2.6.10(4), as $\Sigma\vdash\mathcal{A}$.

Conditions (D2) and (D3). Condition (D3) states inside Σ that deductions from Σ are closed under Modus Ponens: a deduction of $\alpha\to\beta$, followed by a deduction of α and then by β, is a deduction of β. Condition (D2) states inside Σ that (D1) holds for α. Their proofs deduce from Σ the universal closures of the recursion equations of the functions that the formulas built in definition 4.3.8 represent, and they use the instances of induction in PA, which $\mathcal{A}$ does not deduce. Leary and Kristiansen state (D1) to (D3) without proof for their axioms of Peano Arithmetic, which also govern $<$, and for their own provability formula, and refer to Smoryński's chapter [4] for the details. They also state the second incompleteness theorem for every consistent

set of axioms that extends their Peano Arithmetic and is represented by a formula built from atomic formulas and their negations by $\wedge$, $\vee$, bounded quantifiers and $\exists$ [2, Definition 4.2.2 and §6.6]. The book takes (D2) and (D3) for Prov_Σ on that authority.

Theorem 4.3.10: Gödel's Second Incompleteness Theorem

Let Σ be a consistent set of sentences of $\mathcal{L}_{NT}$ such that $\Sigma \vdash \mathrm{PA}$ and $[\Sigma]$ is primitive recursive, and let $\mathrm{Con}(\Sigma)$ be its consistency sentence (definition 4.3.8) for a fixed construction of $\chi_{[\Sigma]}$. Then $\Sigma \nvdash \mathrm{Con}(\Sigma)$.

***Proof*:**
Write Pr for Pr_Σ and $\bot$ for the sentence $\neg(0=0 \to 0=0)$, so that $m_0 = [\bot]$ and $\mathrm{Pr}(\bot)$ is the formula $(\neg\,\mathrm{Con}(\Sigma))$, as shown after definition 4.3.8. Every member of $\mathcal{A}$ is a member of PA, so $\Sigma \vdash \mathcal{A}$, and all three conditions of theorem 4.3.9 hold. Propositional steps use lemma 2.6.8 with proposition 2.6.10(4): if Σ deduces $\gamma_1^*, \ldots, \gamma_n^*$, in the notation of lemma 2.6.8, and every truth assignment that gives $\gamma_1, \ldots, \gamma_n$ the value T gives φ the value T, then $\Sigma \vdash \varphi^*$.

Step 1: the fixed point. No variable other than v_1 is free in $\neg\,\mathrm{Prov}_\Sigma$, so the Fixed-Point lemma, lemma 4.3.1(3), gives a sentence σ with $\mathcal{A} \vdash \sigma \leftrightarrow \neg\,\mathrm{Pr}(\sigma)$, as $(\neg\,\mathrm{Prov}_\Sigma)(S^{[\sigma]}0)$ is $(\neg\,\mathrm{Pr}(\sigma))$ by definition 1.5.3. By proposition 2.6.10(4), Σ deduces this biconditional. Every truth assignment that gives $A_0 \leftrightarrow (\neg A_1)$ the value T gives $A_0 \to (\neg A_1)$ and $(\neg A_1) \to A_0$ the value T (proposition 1.7.4), so

$$\text{(a)}\quad \Sigma \vdash \sigma \to \neg\,\mathrm{Pr}(\sigma) \qquad\qquad \text{(b)}\quad \Sigma \vdash \neg\,\mathrm{Pr}(\sigma) \to \sigma$$

Step 2: $\Sigma \vdash \mathrm{Pr}(\sigma) \to \mathrm{Pr}(\bot)$. Write π for the sentence $\mathrm{Pr}(\sigma)$. By (a) and (D1), $\Sigma \vdash \mathrm{Pr}((\sigma \to \neg\pi))$, and (D3) with proposition 2.6.10(2) gives (c) below. Condition (D2) for σ is (d).

$$\text{(c)}\quad \Sigma \vdash \pi \to \mathrm{Pr}(\neg\pi) \qquad\qquad \text{(d)}\quad \Sigma \vdash \pi \to \mathrm{Pr}(\pi)$$

Every truth assignment gives $A_0 \to (\neg A_0 \to A_1)$ the value T, so $\vdash \pi \to (\neg\pi \to \bot)$ by lemma 2.6.8, Σ deduces this sentence by proposition 2.6.10(3), and $\Sigma \vdash \mathrm{Pr}((\pi \to (\neg\pi \to \bot)))$ by (D1). So (D3) with proposition 2.6.10(2) gives (e), and (f) is an instance of (D3):

$$\text{(e)}\quad \Sigma \vdash \mathrm{Pr}(\pi) \to \mathrm{Pr}((\neg\pi \to \bot)) \qquad\qquad \text{(f)}\quad \Sigma \vdash \mathrm{Pr}((\neg\pi \to \bot)) \to (\mathrm{Pr}(\neg\pi) \to \mathrm{Pr}(\bot))$$

Every truth assignment that gives $A_0 \to A_1$, $A_1 \to A_2$, $A_2 \to (A_3 \to A_4)$ and $A_0 \to A_3$ the value T gives $A_0 \to A_4$ the value T: if A_0 has the value T, then so do A_1, A_2, A_3 and then A_4. Applied to (d), (e), (f) and (c), with π, $\mathrm{Pr}(\pi)$, $\mathrm{Pr}((\neg\pi \to \bot))$, $\mathrm{Pr}(\neg\pi)$ and $\mathrm{Pr}(\bot)$ in place of $A_0, \ldots, A_4$, this gives $\Sigma \vdash \pi \to \mathrm{Pr}(\bot)$.

Step 3: $\Sigma \vdash \mathrm{Con}(\Sigma) \to \sigma$. By Step 2, $\Sigma \vdash \pi \to \neg\,\mathrm{Con}(\Sigma)$, and $\Sigma \vdash \neg\pi \to \sigma$ by (b). Every truth assignment that gives $A_0 \to (\neg A_1)$ and $(\neg A_0) \to A_2$ the value T gives $A_1 \to A_2$ the value T: if A_1 has the value T, then A_0 has the value F, and A_2 the value T. Hence $\Sigma \vdash \mathrm{Con}(\Sigma) \to \sigma$.

Step 4: conclusion. Suppose, for a contradiction, that $\Sigma \vdash \mathrm{Con}(\Sigma)$. By Step 3 and proposition 2.6.10(2), $\Sigma \vdash \sigma$. Then $\Sigma \vdash \pi$ by (D1), and $\Sigma \vdash \neg\pi$ by (a) and proposition 2.6.10(2).

> No truth assignment gives A_0 and $\neg A_0$ the value T, so $\Sigma \vdash \neg(\pi \to \pi)$ by lemma 2.6.8 and proposition 2.6.10(4), and Σ is inconsistent, contrary to hypothesis. Hence $\Sigma \nvdash \mathrm{Con}(\Sigma)$.

An inconsistent set deduces every formula (proposition 2.6.17(3)), $\mathrm{Con}(\Sigma)$ included. So for a set Σ of sentences with $\Sigma \vdash \mathrm{PA}$ and $[\Sigma]$ primitive recursive, $\Sigma \vdash \mathrm{Con}(\Sigma)$ holds exactly when Σ is inconsistent, while $\mathfrak{N} \models \mathrm{Con}(\Sigma)$ holds exactly when Σ is consistent (lemma 4.3.5). By proposition 4.3.7, the set PA satisfies the hypotheses of the theorem, so $\mathrm{PA} \nvdash \mathrm{Con}(\mathrm{PA})$.

The theorem concerns the consistency sentence built from φ_C^Σ, and its conclusion fails for the sentences built from some other faithful formulas for χ_C. Feferman showed that Peano Arithmetic deduces a consistency sentence built from another formula representing its set of axioms [2, §6.6]. A shorter example: let Σ be a consistent recursive set of sentences with $\Sigma \vdash \mathcal{A}$, let φ_C be faithful for χ_C, write χ for $(\varphi_C)_{S0}^{v_3}$ and η for $(v_1 = S^{m_0}0 \wedge \chi)$, and let

$$\varphi'_C :\equiv (\varphi_C \wedge \neg\eta) \vee (v_3 = 0 \wedge \eta)$$

whose free variables are among v_1, v_2, v_3. Let $\mathfrak{M}$ be a model of $\mathcal{A}$ and $m, c \in \mathbb{N}$. The formula η fails at $(\hat{m}, \hat{c})$: for $m \neq m_0$ because $\hat{m} \neq \widehat{m_0}$ (lemma 4.2.5(3)), and for $m = m_0$ because, by lemma 4.2.4(3), (F2) of definition 4.2.10 and lemma 4.2.5(3), χ holds at $(\widehat{m_0}, \hat{c})$ only if $\chi_C(m_0, c) = 1$, that is, only if c codes a deduction of $\neg(0 = 0 \to 0 = 0)$ from Σ, and the consistent set Σ has no such deduction. By proposition 1.4.6(1) and (2) and the negation clause of definition 1.4.5, φ'_C then holds at $(\hat{m}, \hat{c}, e)$ exactly when φ_C does, so φ'_C satisfies (F2), and (F1) as its case $\mathfrak{M} = \mathfrak{N}$. As v_3 is not free in χ (lemma 4.2.4(3)), definition 1.5.3 and lemma 1.5.7(1) show that the sentence $\mathrm{Con}'(\Sigma)$ of lemma 4.3.5 for φ'_C is $\forall v_2 \neg\rho$, where ρ is $(\chi' \wedge \neg(\epsilon \wedge \chi')) \vee (S0 = 0 \wedge (\epsilon \wedge \chi'))$ for $\chi' :\equiv \chi_{S^{m_0}0}^{v_1}$ and $\epsilon :\equiv (S^{m_0}0 = S^{m_0}0)$. At every assignment into a model of $\mathcal{A}$, the formula ϵ holds and $S0 = 0$ fails, by the equality clause and lemma 4.2.5(1), so ρ fails by proposition 1.4.6(1) and (2) and the negation clause. Hence $\mathrm{Con}'(\Sigma)$ is true in every model of $\mathcal{A}$, and $\Sigma \vdash \mathrm{Con}'(\Sigma)$ by theorem 2.6.31 and proposition 2.6.10(4). The proof of (D1) uses only that φ_C^Σ is faithful, so it applies to φ'_C, and by Steps 1 to 4 of the proof of the theorem, (D2) or (D3) fails for the provability formula built from φ'_C.

A relation on $\mathbb{N}$ is definable when some formula of $\mathcal{L}_{NT}$ holds in $\mathfrak{N}$ exactly at its members:

Definition 4.3.11: Definable

Let P be a k-place relation in $\mathfrak{N}$. Then P is said to be *definable* if there is a formula φ of $\mathcal{L}_{NT}$ with at most $v_1, v_2, ..., v_k$ as free variables such that

$$P(n_1, n_2, ..., n_k) \Leftrightarrow \mathfrak{N} \models \varphi[s(v_1|n_1)...(v_k|n_k)]$$

for every assignment s of $\mathfrak{N}$. The formula φ is said to *define* P in $\mathfrak{N}$.

Definability concerns truth in the structure $\mathfrak{N}$, and representability concerns deductions from a set of axioms.

Proposition 4.3.12: Representable And Definable

Let P be a k-place relation which is representable in $\mathrm{Th}(\mathcal{A})$. Then P is *definable* in $\mathfrak{N}$

***Proof*:**
Let ψ represent P in $\mathrm{Th}(\mathcal{A})$, so that its free variables are among $v_1, \ldots, v_k$ (definition 4.2.8). As $\mathfrak{N}$ is a model of $\mathcal{A}$ (proposition 4.2.2) in which $\hat{a} = a$ for every $a \in \mathbb{N}$ (lemma 4.2.4(1)), lemma 4.2.11(2) gives, for all $n_1, \ldots, n_k \in \mathbb{N}$, that $\mathfrak{N} \models \psi[n_1, \ldots, n_k]$ exactly when $P(n_1, \ldots, n_k)$. By lemma 4.2.4(2), $\mathfrak{N} \models \psi[n_1, \ldots, n_k]$ says that $\mathfrak{N} \models \psi[s(v_1|n_1) \cdots (v_k|n_k)]$ for every assignment s into $\mathfrak{N}$. Hence ψ defines P in $\mathfrak{N}$ (definition 4.3.11).

Every recursive relation is representable in $\mathrm{Th}(\mathcal{A})$, by theorem 4.2.21 and proposition 4.2.14(2). Conversely, let ψ represent a 1-place relation P in $\mathrm{Th}(\mathcal{A})$, so that no variable other than v_1 is free in ψ (definition 4.2.8). The set $\mathcal{A}$ is consistent by proposition 2.6.17(2), as $\mathfrak{N} \models \mathcal{A}$, and $\neg\psi$ represents the complement of P by proposition 4.2.14(3). So by proposition 4.2.22(1), applied to ψ and to $\neg\psi$, $P(n)$ holds exactly when $\mathcal{A} \vdash \psi(S^n 0)$ and fails exactly when $\mathcal{A} \vdash \neg\psi(S^n 0)$, and for each $n \in \mathbb{N}$ exactly one of these two sentences is deduced by $\mathcal{A}$. Their codes are $Sub([\psi], n)$ and $Neg(Sub([\psi], n))$, by lemma 4.3.1(1) and lemma 4.1.6(2). By theorem 4.1.11(1) for $\Delta = \mathcal{A}$, which is recursive by lemma 4.3.1(2), and definition 3.3.14, there is a recursive 1-place function F with $\mathrm{im}(F) = [\mathrm{Th}(\mathcal{A})]$. The function

$$g(n,t) = \chi_{\{0\}}\big(EQUAL(F(t), Sub([\psi], n)) + EQUAL(F(t), Neg(Sub([\psi], n)))\big)$$

is recursive and regular (definition 3.2.24), so $T(n) = \mu t[g(n,t) = 0]$ is recursive (definition 3.2.28). The value $F(T(n))$ is the code of whichever of the two sentences $\mathcal{A}$ deduces, so

$$\chi_P(n) = EQUAL\big(F(T(n)),\ Sub([\psi], n)\big)$$

and P is recursive. So a 1-place relation is representable in $\mathrm{Th}(\mathcal{A})$ exactly when it is recursive.

The converse of proposition 4.3.12 fails. The set $[\mathrm{Th}(\mathcal{A})]$ is not recursive, by lemma 4.3.2 with $\Sigma = \mathcal{A}$, which is consistent and deduces $\mathcal{A}$ through one-line deductions, so it is not representable in $\mathrm{Th}(\mathcal{A})$. It is definable in $\mathfrak{N}$. By theorem 4.2.21, some formula φ_F represents F in $\mathrm{Th}(\mathcal{A})$, and by proposition 4.2.14(1) and proposition 4.3.12 it defines in $\mathfrak{N}$ the relation that holds of (t, n) when $F(t) = n$. Choose distinct variables w and z, other than v_1 and v_2, that do not occur in φ_F, and let

$$\delta :\equiv \exists z\, \exists w\big(w = v_1 \wedge ((\varphi_F)_z^{v_1})_w^{v_2}\big)$$

By lemma 4.2.12(2), applied to both substitutions, no variable other than v_1 is free in δ, and by its item (3), proposition 1.4.13, proposition 1.4.6(2), the equality clause of definition 1.4.5 and proposition 1.4.9, $\mathfrak{N} \models \delta[s(v_1|n)]$ holds exactly when some $t \in \mathbb{N}$ has $\mathfrak{N} \models \varphi_F[t, n]$, that is, $F(t) = n$. So δ defines $\mathrm{im}(F) = [\mathrm{Th}(\mathcal{A})]$ in $\mathfrak{N}$.

Truth in $\mathfrak{N}$, a property of sentences, corresponds through their codes to the set of natural numbers

$$[\mathrm{Th}(\mathfrak{N})] = \{[\sigma] \mid \sigma \text{ is a sentence of } \mathcal{L}_{NT} \text{ and } \mathfrak{N} \models \sigma\}$$

Theorem 4.3.13: Tarski's Undefinability Theorem

The set $[\mathrm{Th}(\mathfrak{N})]$ is *not* definable in $\mathfrak{N}$

***Proof*:**
Suppose, for a contradiction, that a formula φ with no free variable other than v_1 defines $[\mathrm{Th}(\mathfrak{N})]$ in $\mathfrak{N}$ (definition 4.3.11). The Fixed-Point lemma, lemma 4.3.1(3), applied to $\neg\varphi$, gives a sentence σ with

$$\mathcal{A} \vdash \sigma \leftrightarrow \neg\varphi(S^{[\sigma]}0)$$

where $(\neg\varphi)(S^{[\sigma]}0)$ is $(\neg\varphi(S^{[\sigma]}0))$ by definition 1.5.3. By proposition 2.6.17(1) and proposition 4.2.2, $\mathfrak{N}$ satisfies this sentence, so by proposition 1.4.14(5) and (1), $\mathfrak{N} \models \sigma$ exactly when $\mathfrak{N} \not\models \varphi(S^{[\sigma]}0)$. By lemma 4.2.4(4), (2) and (1), $\mathfrak{N} \models \varphi(S^{[\sigma]}0)$ exactly when $\mathfrak{N} \models \varphi[s(v_1|[\sigma])]$ for every assignment s, that is, as φ defines $[\mathrm{Th}(\mathfrak{N})]$, exactly when $[\sigma] \in [\mathrm{Th}(\mathfrak{N})]$. A code determines the sequence it codes (the remark after definition 4.1.4), so $[\sigma] \in [\mathrm{Th}(\mathfrak{N})]$ exactly when $\mathfrak{N} \models \sigma$. Hence $\mathfrak{N} \models \sigma$ exactly when $\mathfrak{N} \not\models \sigma$, a contradiction, and $[\mathrm{Th}(\mathfrak{N})]$ is not definable in $\mathfrak{N}$.

4.4 Summary

1. *Coding* (definition 4.1.3, definition 4.1.4): the twelve symbols other than the variables have the codes 1 to 12, the variable v_k has the code $k + 13$, and a sequence $s_1 \cdots s_k$ of symbols has the code $p_1^{[s_1]} \cdots p_k^{[s_k]}$, which determines it.

2. *Relations on codes* (proposition 4.1.8, proposition 4.1.10): $Term$, $Formula$, $Sentence$, $Logical$, $Free$ and $Subst$ are primitive recursive, and for a recursive set Δ of sentences, $Premise_\Delta$, $Formulas$, $Inference$, $Deduction_\Delta$ and $Conclusion_\Delta$ are recursive.

3. *Theorems of a recursive set* Δ (theorem 4.1.11): $[\mathrm{Th}(\Delta)]$ is recursively enumerable, and recursive if Δ is complete. A consistent set of sentences is complete exactly when its set of theorems is maximally consistent (proposition 4.1.2).

4. *Axioms* (definition 4.2.1, definition 4.3.6): $\mathcal{A}$ consists of nine sentences about 0, S, $+$, $\cdot$ and E, and PA adds the instances of induction. Both sets are true in $\mathfrak{N}$ and have primitive recursive sets of codes (proposition 4.2.2, lemma 4.3.1(2), proposition 4.3.7). In a model of $\mathcal{A}$, the elements named by numerals obey the arithmetic of $\mathbb{N}$ (lemma 4.2.5).

5. *Representability* (definition 4.2.6, definition 4.2.8): a formula represents a function in $\mathrm{Th}(\Sigma)$ when Σ deduces, for each input, that the value is the only element satisfying it, and a relation when Σ deduces its instances at members and refutes them at the other tuples. Both are checked in models of Σ (lemma 4.2.11). A relation and its characteristic function are representable together (proposition 4.2.14), and representability passes from $\mathrm{Th}(\Gamma)$ to $\mathrm{Th}(\Sigma)$ when $\Sigma \vdash \Gamma$ (proposition 4.2.22).

6. *Recursive, then representable* (theorem 4.2.21): every recursive function is representable in $\mathrm{Th}(\mathcal{A})$, through proposition 4.2.15, proposition 4.2.19, with Gödel's β-function (definition 4.2.17), and proposition 4.2.16. A 1-place relation is representable in $\mathrm{Th}(\mathcal{A})$ exactly when it is recursive (the remark after proposition 4.3.12).

7. *Fixed points, incompleteness and undefinability*, by hypotheses:

Result	Hypotheses	Conclusion
lemma 4.3.1(3)	no free variable other than v_1 in φ	$\mathcal{A} \vdash \sigma \leftrightarrow \varphi(S^{\lceil\sigma\rceil}0)$ for some sentence σ
lemma 4.3.2	Σ consistent, $\Sigma \vdash \mathcal{A}$	$\lceil\mathrm{Th}(\Sigma)\rceil$ is not recursive
theorem 4.3.3	Σ consistent and recursive, $\Sigma \vdash \mathcal{A}$	Σ is not complete
corollary 4.3.4(3)	none	$\mathrm{Th}(\mathfrak{N})$ is not recursive
lemma 4.3.5	Σ recursive, φ_C faithful for χ_C	Σ is consistent exactly when $\mathfrak{N} \models \mathrm{Con}(\Sigma)$
theorem 4.3.9	$\lceil\Sigma\rceil$ primitive recursive, $\Sigma \vdash$ PA	(D1) to (D3), with (D2) and (D3) taken on the authority of [2, §6.6]
theorem 4.3.10	Σ consistent, $\lceil\Sigma\rceil$ primitive recursive, $\Sigma \vdash$ PA	$\Sigma \nvdash \mathrm{Con}(\Sigma)$ for the sentence of definition 4.3.8
proposition 4.3.12	P representable in $\mathrm{Th}(\mathcal{A})$	P is definable in $\mathfrak{N}$
theorem 4.3.13	none	$\lceil\mathrm{Th}(\mathfrak{N})\rceil$ is not definable in $\mathfrak{N}$

A

Algebrization

I really want to add some about this: algebrization. Quines name shows up!!!!
Also, this: duality between logic and geometry.

Bibliography

[1] Stefan Bilaniuk. *A Problem Course in Mathematical Logic*. Version 1.6. Peterborough, Ontario: Trent University, 2003.

[2] Christopher C. Leary and Lars Kristiansen. *A Friendly Introduction to Mathematical Logic*. 2nd ed. Geneseo, NY: Milne Library, SUNY Geneseo, 2015. ISBN: 978-1-942341-07-9.

[3] T. Rado. "On Non-Computable Functions". In: *Bell System Technical Journal* 41.3 (May 1962), pp. 877–884. DOI: 10.1002/j.1538-7305.1962.tb00480.x. URL: https://doi.org/10.1002/j.1538-7305.1962.tb00480.x.

[4] Craig Smoryński. "The Incompleteness Theorems". In: *Handbook of Mathematical Logic*. Ed. by Jon Barwise. Vol. 90. Studies in Logic and the Foundations of Mathematics. Amsterdam: North-Holland, 1977, pp. 821–865. DOI: 10.1016/S0049-237X(08)71123-6.

[5] The bbchallenge Collaboration et al. *Determination of the Fifth Busy Beaver Value*. 2025. DOI: 10.48550/arxiv.2509.12337. URL: https://arxiv.org/abs/2509.12337. Pre-published.